



SiFive Tenstorrent X280 MC Manual

21G3.04.00

Copyright © 2021–2022 by SiFive, Inc. All rights reserved.

CONFIDENTIAL INFORMATION

SiFive Tenstorrent X280 MC Manual

Proprietary Notice

Copyright © 2021–2022 by SiFive, Inc. All rights reserved.

Information in this document is provided “as is,” with all faults.

SiFive expressly disclaims all warranties, representations, and conditions of any kind, whether express or implied, including, but not limited to, the implied warranties or conditions of merchantability, fitness for a particular purpose and non-infringement.

SiFive does not assume any liability rising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation indirect, incidental, special, exemplary, or consequential damages.

SiFive reserves the right to make changes without further notice to any products herein.

Contents

List of Tables	19
List of Figures	28
1 Introduction	34
1.1 About this Document	35
1.2 About this Release	35
1.3 Tenstorrent X280 MC Overview	35
1.4 X280 RISC-V Cores	36
1.5 Memory System	37
1.6 Interrupts	37
1.7 Debug Support	38
1.8 Compliance	38
2 List of Abbreviations and Terms	39
3 X280 RISC-V Core	41
3.1 Supported Privilege Modes	42
3.2 Instruction Memory System	42
3.2.1 Execution Memory Space	42
3.2.2 L1 Instruction Cache	42
3.2.3 Cache Maintenance	43
3.2.4 Coherence with Higher Level Caches	43
3.2.5 Instruction Fetch Unit	43
3.2.6 Branch Prediction	44
3.3 Execution Pipeline	45
3.4 Data Memory System	46
3.4.1 L1 Data Cache	47
3.4.2 Cache Maintenance Operations	47

3.4.3	L1 Data Cache Coherency	47
3.4.4	Coherence with Higher Level Caches.....	48
3.5	Atomic Memory Operations.....	48
3.6	Floating-Point Unit (FPU).....	48
3.7	Vector Unit	49
3.7.1	Chaining Vector Instructions.....	50
3.7.2	Arithmetic Sequencer	50
3.7.3	Vector Memory System.....	51
3.7.4	Vector-Scalar Communication	52
3.8	Virtual Memory Support.....	52
3.8.1	Address and Page Table Formats	53
3.8.1.1	Page Table Configurations.....	55
3.8.2	Supervisor Address Translation and Protection Register (satp).....	56
3.8.3	Supervisor Memory-Management Fence Instruction (SFENCE.VMA)	57
3.8.4	Scenarios that Require SFENCE.VMA Instruction.....	59
3.8.4.1	Speculation	59
3.8.4.2	ASID Usage for Supervisor Software	59
3.8.5	Trap Virtual Memory	60
3.8.6	Virtual Address Translation Process.....	60
3.8.7	Virtual-to-Physical Address Mapping Example	61
3.8.8	MMU at Reset.....	63
3.9	Physical Memory Protection (PMP).....	63
3.9.1	PMP Functional Description	63
3.9.2	PMP Region Locking	64
3.9.3	PMP Registers.....	64
3.9.3.1	PMP Configuration Registers.....	64
3.9.3.2	PMP Address Registers	66
3.9.4	PMP and PMA	66
3.9.5	PMP Programming Overview	66
3.9.5.1	PMP Programming Example.....	66
3.9.5.2	PMP Access Scenarios	67
3.9.6	PMP and Paging	68
3.9.7	PMP Limitations	68

3.9.8	Behavior for Regions without PMP Protection	68
3.9.9	Cache Flush Behavior on PMP Protected Region	69
3.10	Hardware Performance Monitor	69
3.10.1	Performance Monitoring Counters Reset Behavior	69
3.10.2	Fixed-Function Performance Monitoring Counters	69
3.10.2.1	Fixed-Function Cycle Counter (mcycle)	70
3.10.2.2	Fixed-Function Instructions-Retired Counter (minstret).....	70
3.10.3	Event-Programmable Performance Monitoring Counters	70
3.10.4	Event Selector Registers	70
3.10.4.1	Counter Overflow and Event Filters	70
3.10.4.2	Performance Counter Overflow Operation	71
3.10.4.3	The Scountovf CSR	72
3.10.4.4	Disabling Counters in Debug Mode.....	72
3.10.5	Event Selector Encodings	72
3.10.5.1	Combining Events.....	74
3.10.6	Counter-Enable Registers	74
3.11	Ports	75
3.11.1	Front Port.....	75
3.11.2	Memory Port	75
3.11.3	Peripheral Port	75
3.11.4	System Port	76
4	Physical Memory Attributes and Memory Map	77
4.1	Physical Memory Attributes Overview	77
4.2	Memory Map	79
5	Programmer's Model.....	82
5.1	Base Instruction Formats	82
5.2	RV64I Base Integer Instruction Set	83
5.2.1	R-Type (Register-Based) Integer Instructions	84
5.2.2	I-Type Integer Instructions	85
5.2.3	I-Type Load Instructions.....	86
5.2.4	S-Type Store Instructions	87

5.2.5	Unconditional Jumps	88
5.2.6	Conditional Branches.....	89
5.2.7	Upper-Immediate Instructions.....	90
5.2.8	Memory Ordering Operations	90
5.2.9	Environment Call and Breakpoints	91
5.2.10	NOP Instruction.....	91
5.3	M Extension: Multiplication Operations.....	91
5.3.1	Division Operations	92
5.4	A Extension: Atomic Operations	92
5.4.1	Atomic Load-Reserve and Store-Conditional Instructions	92
5.4.2	Atomic Memory Operations (AMOs)	93
5.5	F Extension: Single-Precision Floating-Point Instructions	94
5.5.1	Floating-Point Control and Status Registers	94
5.5.2	Rounding Modes	95
5.5.3	Single-Precision Floating-Point Load and Store Instructions	95
5.5.4	Single-Precision Floating-Point Computational Instructions	96
5.5.5	Single-Precision Floating-Point Conversion and Move Instructions.....	96
5.5.5.1	Single-Precision Floating-Point Conversion Instructions	96
5.5.5.2	Single-Precision Floating-Point-to-Floating-Point Sign-Injection Instructions	97
5.5.5.3	Single-Precision Floating-Point Move Instructions.....	98
5.5.6	Single-Precision Floating-Point Compare Instructions	99
5.5.6.1	Single-Precision Floating-Point Classify Instruction	99
5.6	D Extension: Double-Precision Floating-Point Instructions	100
5.6.1	Double-Precision Floating-Point Load and Store Instructions.....	100
5.6.2	Double-Precision Floating-Point Computational Instructions	101
5.6.3	Double-Precision Floating-Point Conversion and Move Instructions	102
5.6.3.1	Double-Precision Floating-Point Conversion Instructions	102
5.6.3.2	Double-Precision Floating-Point-to-Floating-Point Sign-Injection Instructions	104
5.6.3.3	Double-Precision Floating-Point Move Instructions.....	104
5.6.4	Double-Precision Floating-Point Compare Instructions.....	105
5.6.5	Double-Precision Floating-Point Classify Instruction	106

5.7	Zfh Extension: Half-Precision Floating-Point Instructions	106
5.7.1	Half-Precision Floating-Point Load and Store Instructions	106
5.7.2	Half-Precision Floating-Point Computational Instructions.....	107
5.7.3	Half-Precision Floating-Point Conversion and Move Instructions.....	108
5.7.3.1	Half-Precision Floating-Point Conversion Instructions.....	108
5.7.3.2	Half-Precision Floating-Point-to-Floating-Point Sign-Injection Instructions	110
5.7.3.3	Half-Precision Floating-Point Move Instructions	110
5.7.4	Half-Precision Floating-Point Compare Instructions	111
5.7.5	Half-Precision Floating-Point Classify Instruction	112
5.8	V Extension: Vector Instructions	112
5.8.1	Vector Registers.....	112
5.8.2	Vector Context Status in mstatus	113
5.8.3	Vector Type Register (vtype)	113
5.8.3.1	Vector Selected Element Width (vsew[2:0])	114
5.8.3.2	Vector Register Grouping (vlmul[2:0]).....	114
5.8.3.3	Vector Tail Agnostic and Vector Mask Agnostic (vta and vma).....	115
5.8.3.4	Vector Type Illegal (vill)	116
5.8.4	Vector Length Register (vl).....	116
5.8.5	Vector Byte Length (vlenb)	116
5.8.6	Vector Start Index CSR (vstart)	116
5.8.7	Vector Fixed-Point Rounding Mode Register (vxrm).....	117
5.8.8	Vector Fixed-Point Saturation Flag (vxsat)	117
5.8.9	Vector Control and Status Register (vcsr)	117
5.8.10	Vector Instruction Formats.....	118
5.8.10.1	VMEM Format	118
5.8.10.2	VALU Format.....	118
5.8.10.3	VCFG Format	119
5.8.11	Configuration-Setting Instructions.....	120
5.8.11.1	vsetvli and vsetvl Instructions.....	120
5.8.12	Vector Loads and Stores	121
5.8.12.1	Vector Load/Store Instruction Encoding.....	121
5.8.12.2	Vector Load/Store Addressing Modes	122

5.8.12.3	Vector Load/Store Width Encoding	124
5.8.12.4	VL* Unit-Stride Instructions	125
5.8.12.5	VLS* Strided Instructions	125
5.8.12.6	VL* Indexed Instructions	126
5.8.12.7	VS* Unit-Stride Instructions	126
5.8.12.8	VSS* Strided Instructions	126
5.8.12.9	VS* Indexed Instructions	127
5.8.12.10	Vector Load/Store Segment Instructions	127
5.8.12.10.1	Vector Unit-Stride Segment Loads and Stores	128
5.8.12.10.2	Vector Strided Segment Loads and Stores	129
5.8.12.10.3	Vector Indexed Segment Loads and Stores	130
5.8.12.11	Vector Load/Store Whole Register Instructions	131
5.8.13	Vector Memory Alignment Constraints	131
5.8.14	Vector Arithmetic Instruction Formats	132
5.8.14.1	Vector Arithmetic Instruction Encoding	132
5.8.14.2	Widening Vector Arithmetic Instructions	134
5.8.14.3	Narrowing Vector Arithmetic Instructions	134
5.8.15	Vector Integer Arithmetic Instructions	134
5.8.15.1	Vector Single-Width Integer Add and Subtract	135
5.8.15.2	Vector Widening Integer Add/Subtract	135
5.8.15.3	Vector Integer Extension	136
5.8.15.4	Vector Integer Add-with-Carry / Subtract-with-Borrow Instructions	136
5.8.15.5	Vector Bitwise Logical Instructions	138
5.8.15.6	Vector Single-Width Bit Shift Instructions	138
5.8.15.7	Vector Narrowing Integer Right Shift Instructions	139
5.8.15.8	Vector Integer Comparison Instructions	139
5.8.15.9	Vector Integer Min/Max Instructions	140
5.8.15.10	Vector Single-Width Integer Multiply Instructions	141
5.8.15.11	Vector Integer Divide Instructions	141
5.8.15.12	Vector Widening Integer Multiply Instructions	141
5.8.15.13	Vector Single-Width Integer Multiply-Add Instructions	142
5.8.15.14	Vector Widening Integer Multiply-Add Instructions	142
5.8.15.15	Vector Integer Merge Instructions	143

5.8.15.16	Vector Integer Move Instructions.....	143
5.8.16	Vector Fixed-Point Arithmetic Instructions	143
5.8.16.1	Vector Single-Width Saturating Add and Subtract.....	144
5.8.16.2	Vector Single-Width Averaging Add and Subtract.....	144
5.8.16.3	Vector Single-Width Fractional Multiply with Rounding and Saturation ..	145
5.8.16.4	Vector Single-Width Scaling Shift Instructions.....	145
5.8.16.5	Vector Narrowing Fixed-Point Clip Instructions.....	145
5.8.17	Vector Floating-Point Instructions	146
5.8.17.1	Vector Floating-Point Exception Flags.....	146
5.8.17.2	Vector Single-Width Floating-Point Add/Subtract Instructions	147
5.8.17.3	Vector Widening Floating-Point Add/Subtract Instructions	147
5.8.17.4	Vector Single-Width Floating-Point Multiply/Divide Instructions	147
5.8.17.5	Vector Single-Width Floating-Point Fused Multiply-Add Instructions	147
5.8.17.6	Vector Widening Floating-Point Fused Multiply-Add Instructions	148
5.8.17.7	Vector Floating-Point Square-Root Instruction	149
5.8.17.8	Vector Floating-Point Reciprocal Estimate Instruction	149
5.8.17.9	Vector Floating-Point Reciprocal Square-Root Estimate Instruction	149
5.8.17.10	Vector Floating-Point MIN/MAX Instructions.....	149
5.8.17.11	Vector Floating-Point Sign-Injection Instructions	149
5.8.17.12	Vector Floating-Point Compare Instructions	150
5.8.17.13	Vector Floating-Point Classify Instruction.....	151
5.8.17.14	Vector Floating-Point Merge Instruction	151
5.8.17.15	Vector Floating-Point Move Instruction	151
5.8.17.16	Single-Width Floating-Point/Integer Type-Convert Instructions.....	152
5.8.17.17	Widening Floating-Point/Integer Type-Convert Instructions	152
5.8.17.18	Narrowing Floating-Point/Integer Type-Convert Instructions	152
5.8.18	Vector Reduction Operations	153
5.8.18.1	Vector Single-Width Integer Reduction Instructions	153
5.8.18.2	Vector Widening Integer Reduction Instructions	154
5.8.18.3	Vector Single-Width Floating-Point Reduction Instructions	154
5.8.18.4	Vector Widening Floating-Point Reduction Instructions	154
5.8.19	Vector Mask Instructions	154
5.8.19.1	Vector Mask-Register Logical Instructions	155

5.8.19.2	Vector Count Population in Mask (vcpop.m)	156
5.8.19.3	Find-First-Set Mask Bit (vfirst)	156
5.8.19.4	Set-Before-First Mask Bit (vmsbf.m)	157
5.8.19.5	Set-Including-First Mask Bit (vmsif.m)	157
5.8.19.6	Set-Only-First Mask Bit (vmsof.m)	158
5.8.19.7	Using Vector Mask Instructions	159
5.8.19.8	Vector Iota Instruction.....	160
5.8.19.9	Vector Element Index Instruction	162
5.8.20	Vector Permutation Instructions	163
5.8.20.1	Integer Scalar Move Instructions	163
5.8.20.2	Floating-Point Scalar Move Instructions	163
5.8.20.3	Vector Slide Instructions	163
5.8.20.3.1	Vector Slideup Instructions	164
5.8.20.3.2	Vector Slidedown Instructions.....	164
5.8.20.3.3	Vector Slide1up.....	165
5.8.20.3.4	Vector Slide1down Instruction.....	165
5.8.20.4	Vector Register Gather Instructions	166
5.8.20.5	Vector Compress Instruction	167
5.8.20.6	Whole Vector Register Move	168
5.8.21	Xsfvqmacccqq Extension: Int8 Matrix Multiplication Instructions.....	169
5.8.22	Xsfvfnrclipxfqf Extension: FP32-to-int8 Ranged Clip Instructions.....	170
5.8.23	Xsfvfhbfbmin Extension: Bfloat16 Conversion Instructions.....	170
5.9	C Extension: Compressed Instructions.....	171
5.9.1	Compressed 16-bit Instruction Formats	171
5.9.2	Stack-Pointed-Based Loads and Stores	172
5.9.3	Register-Based Loads and Stores.....	173
5.9.4	Control Transfer Instructions.....	174
5.9.5	Integer Computational Instructions	175
5.9.5.1	Integer Constant-Generation Instructions	175
5.9.5.2	Integer Register-Immediate Operations.....	175
5.9.5.3	Integer Register-Register Operations.....	177
5.9.5.4	Defined Illegal Instruction	178
5.10	Zba Extension: Address Calculation Instructions	178

5.10.1	Address Calculation Instructions	178
5.10.2	Add/Shift with Prefix Zero-Extend Instructions	179
5.11	Zbb Extension: Basic Bit Manipulation Instructions	179
5.11.1	Count Leading/Trailing Zeroes Instructions.....	179
5.11.2	Count Population Instructions	180
5.11.3	Logic-With-Negate Instructions	180
5.11.4	Comparison Instructions.....	181
5.11.5	Sign-Extend and Zero-Extend Instructions	181
5.11.6	Bitwise Rotation Instructions.....	182
5.11.7	OR Combine Instruction	183
5.11.8	Byte-Reverse Instruction	183
5.12	Zicsr Extension: Control and Status Register Instructions	183
5.12.1	Control and Status Registers	185
5.12.2	Defined CSRs	185
5.12.3	CSR Access Ordering.....	189
5.12.4	SiFive RISC-V Implementation Version Registers.....	189
5.12.4.1	Machine Vendor ID Register (mvendorid)	190
5.12.4.2	Machine Architecture ID Register (marchid).....	190
5.12.4.3	Machine Implementation ID Register (mimpid).....	190
5.12.4.4	Reading Implementation Version Registers	191
5.12.5	RISC-V Status Registers.....	192
5.12.5.1	Machine Status Register (mstatus)	192
5.12.5.2	Supervisor Status Register (sstatus).....	192
5.12.6	Custom CSRs	193
5.13	Base Counters and Timers.....	193
5.13.1	Timer Register.....	194
5.13.2	Timer API	194
5.13.2.1	Functions	195
5.14	Privileged Instructions	195
5.14.1	Machine-Mode Privileged Instructions	195
5.14.1.1	Environment Call and Breakpoint	195
5.14.1.2	Trap-Return Instructions	196
5.14.1.3	Wait for Interrupt	196

5.14.2	Supervisor-Mode Privileged Instructions	197
5.14.2.1	Supervisor Memory-Management Fence Instruction.....	197
5.15	ABI - Register File Usage and Calling Conventions	197
5.15.1	RISC-V Assembly.....	199
5.15.2	Assembler to Machine Code.....	199
5.15.3	Calling a Function (Calling Convention)	201
5.15.3.1	Nested Functions.....	202
5.16	Memory Ordering - FENCE Instructions	203
5.17	Boot Flow	204
5.18	Linker File	205
5.18.1	Linker File Symbols	206
5.18.1.1	Generated Linker Symbols.....	206
5.19	RISC-V Compiler Flags	208
5.19.1	arch, abi, and mtune.....	208
5.19.1.1	-march.....	208
5.19.1.2	-mabi	209
5.19.1.3	arch/abi Combinations	209
5.20	Compilation Process	211
5.21	Large Code Model Workarounds	211
5.21.1	RISC-V Code Model Summary	212
5.21.2	Enabling the Compact Code Model	212
5.21.2.1	Makefile Update.....	213
5.21.2.2	Assembly File Updates.....	213
5.22	Pipeline Hazards.....	213
5.22.1	Read-After-Write Hazards	214
5.22.2	Write-After-Write Hazards.....	214
5.23	Reading CSRs.....	214
6	Custom Instructions and CSRs.....	216
6.1	SiFive Custom Instructions	216
6.1.1	CFLUSH.D.L1	216
6.1.2	CDISCARD.D.L1.....	216
6.1.3	CEASE.....	217

6.1.4	PAUSE	217
6.2	SiFive Custom CSRs.....	217
6.2.1	Branch Prediction Mode CSR	217
6.2.2	SiFive Feature Disable CSR	218
6.2.3	Power Dial CSR	219
6.3	Other Custom Instructions and CSRs	220
7	Interrupts and Exceptions.....	221
7.1	Interrupt Concepts	221
7.2	Exception Concepts	221
7.3	Trap Concepts	223
7.4	Interrupt Block Diagram	224
7.5	Local Interrupts.....	225
7.6	Interrupt Operation	226
7.6.1	Interrupt Entry and Exit	226
7.7	Interrupt Control and Status Registers	226
7.7.1	Machine Status Register (mstatus).....	227
7.7.2	Machine Trap Vector (mtvec).....	227
7.7.2.1	Mode Direct	228
7.7.2.2	Mode Vectored	228
7.7.3	Machine Interrupt Enable (mie).....	229
7.7.4	Machine Interrupt Pending (mip).....	229
7.7.5	Machine Cause (mcause)	230
7.7.6	Minimum Interrupt Configuration	231
7.8	Supervisor Mode Interrupts.....	232
7.8.1	Delegation Registers (mideleg and medeleg)	232
7.8.2	Supervisor Status Register (sstatus)	233
7.8.3	Supervisor Interrupt Enable Register (sie)	234
7.8.4	Supervisor Interrupt Pending (sip).....	234
7.8.5	Supervisor Cause Register (scause).....	235
7.8.6	Supervisor Trap Vector (stvec)	236
7.8.7	Delegated Interrupt Handling	237
7.9	Interrupt Priorities	238

7.10	Interrupt Latency	238
7.11	Non-Maskable Interrupt.....	239
7.11.1	Handler Addresses	239
7.11.2	RNMI CSRs	239
7.11.3	MNRET Instruction	240
7.11.4	RNMI Operation	240
8	Core-Local Interruptor (CLINT)	242
8.1	CLINT Priorities and Preemption	242
8.2	CLINT Vector Table	243
8.3	CLINT Interrupt Sources	245
8.4	CLINT Interrupt Attribute	245
8.5	CLINT Memory Map	246
8.6	Register Descriptions	246
8.6.1	MSIP Register.....	246
8.6.2	Timer Registers.....	247
8.7	Supervisor Mode Delegation	247
9	Platform-Level Interrupt Controller (PLIC)	248
9.1	Memory Map	249
9.2	Interrupt Sources	253
9.3	Interrupt Priorities	253
9.4	Interrupt Pending Bits.....	254
9.5	Interrupt Enables	255
9.6	PLIC Clock Gate Disable	256
9.7	Priority Thresholds	257
9.8	Interrupt Claim Process	257
9.9	Interrupt Completion.....	257
9.10	Example PLIC Interrupt Handler	258
10	TileLink Error Device	260
11	Bus-Error Unit	261

11.1	Bus-Error Unit Overview	261
11.2	Memory Map.....	261
11.3	Reportable Errors	262
11.4	Functional Description	262
11.4.1	BEU Global Interrupt.....	263
11.4.2	BEU Local Interrupt.....	263
11.4.3	Global Interrupt Configuration	263
12	Level 2 Prefetcher	264
12.1	Operation	264
12.2	Retiring Streams	265
12.3	Page Boundaries	265
12.4	Memory Map	266
12.5	Control Registers	266
12.6	L2 Prefetcher Initialization.....	267
13	Private Level 2 Cache	268
13.1	Functional Description	269
13.1.1	Coherence and Ordering.....	269
13.1.2	Cache Maintenance.....	270
13.1.2.1	Cache Maintenance Operation Completion	270
13.1.3	Error Correction Codes (ECC)	270
13.1.4	Private L2 Performance Monitor.....	271
13.1.4.1	PL2PM Event Selector Registers (pmEventSelectX).....	271
13.1.4.2	PL2PM Counter Client Filter CSR (pmClientFilter).....	271
13.1.4.3	PL2 Performance Monitor Counters (pmEventCounterX)	272
13.1.4.4	Event Selector Encodings.....	272
13.2	Memory Map	276
13.3	Register Descriptions	277
13.3.1	PL2 Cache Configuration Register (Config).....	277
13.3.2	ECC Error Injection Register (ECCInjectError).....	278
13.3.3	ECC Error Injection Done Register (EccInjectErrorDone).....	278
13.3.4	PL2 Cache Flush Register (Cflush64)	278

13.3.5	Flush Count Register (FlushCount)	280
13.3.6	PL2 Configuration Register (pL2Config).....	280
14	Level 3 Cache Controller	282
14.1	Level 3 Cache Controller Overview	282
14.2	Functional Description	282
14.2.1	Way Enable and the L3 Loosely-Integrated Memory (L3 LIM)	283
14.2.2	Way Masking and Locking	284
14.2.3	L3 Zero Device	285
14.2.4	L3 Features Access Summary	287
14.2.5	L3 Performance Monitor	287
14.2.5.1	L3PM Event Selector Registers (pmEventSelectX)	287
14.2.5.2	L3PM Counter Client Filter CSR (pmClientFilterX)	288
14.2.5.3	L3 Performance Monitor Counters (pmEventCounterX)	288
14.2.5.4	Event Selector Encodings	288
14.2.5.5	Setting up the pmClientFilterX Register	290
14.2.5.6	Programming the pmEventSelect registers	291
14.2.6	Error Correction Codes (ECC)	291
14.2.7	Coherence	292
14.3	Memory Map	292
14.4	Register Descriptions	294
14.4.1	L3 Cache Configuration Register (Config)	294
14.4.2	Way Enable Register (wayEnable)	295
14.4.3	ECC Error Injection Register (ECCInjectError)	295
14.4.4	ECC Directory Fix Registers (DirECCFix*)	296
14.4.5	ECC Directory Fail Registers (DirECCFail*)	296
14.4.6	ECC Data Fix Registers (DatECCFix*)	296
14.4.7	ECC Data Fail Registers (DatECCFail*)	296
14.4.8	L3 Cache ECC Error Injection and Correction	297
14.4.9	Cache Flush Registers (Flush*)	297
14.4.10	Way Mask Registers (wayMask*)	298
14.4.11	L3 Cache Feature Disable Register (FeatureDisable)	300
14.5	Procedure to Flush the L3 Cache	301

14.5.1	Flushing a Single Index+Way.....	301
14.5.2	Flushing the Entire L3 Cache.....	301
15	Power Management.....	303
15.1	Power Modes	303
15.2	Run Mode	303
15.2.1	Power Control	303
15.2.1.1	Core Clock	303
15.2.1.2	Power Dial	303
15.3	WFI Clock Gate Mode	304
15.3.1	WFI Wake Up.....	304
15.4	CEASE Instruction for Power Down	305
15.5	Subsystem LowPower Controller (SLPC)	305
15.6	L3 Cache Clock Gating	306
15.7	Hardware Reset.....	306
15.8	Early Boot Flow.....	307
15.9	Interrupt State During Early Boot	307
15.10	Other Boot Time Considerations	308
15.11	Power-Down Flow	309
16	Debug	311
16.1	Debug Module	311
16.2	Debug and Trigger Registers.....	315
16.2.1	Debug Control and Status Register (dcsr)	315
16.2.2	Debug PC (dpc)	316
16.2.3	Debug Scratch (dscratch).....	316
16.2.4	Trigger Select Register (tselect)	317
16.2.5	Trigger Data Registers (tdata1-3).....	317
16.3	Breakpoints	318
16.3.1	Breakpoint Match Control Register (mcontrol6).....	318
16.3.2	Breakpoint Match Address Register (maddress)	321
16.3.3	Breakpoint Execution	321
16.3.4	Sharing Breakpoints Between Debug and Machine Mode	322

16.4	Debug Memory Map	322
16.4.1	Debug RAM and Program Buffer (0x300–0x3FF)	322
16.4.2	Debug ROM (0x800–0xFFF)	322
16.4.3	Debug Flags (0x100–0x110, 0x400–0x7FF)	322
16.4.4	Safe Address	323
16.5	Instruction Trace Interface.....	323
16.6	Debug Module Interface.....	324
16.6.1	Debug Module Status Register (dmstatus)	326
16.6.2	Debug Module Control Register (dmcontrol)	328
16.6.3	Hart Info Register (hartinfo)	329
16.6.4	Hart Array Window Register (hawindow)	330
16.6.5	Abstract Control and Status Register (abstractcs)	331
16.6.6	Abstract Command Register (command)	332
16.6.7	Abstract Command Autoexec Register (abstractauto)	332
16.6.8	Debug Module Control and Status 2 Register (dmcs2)	332
16.6.9	Abstract Commands	333
16.6.10	Multi-core Synchronization	335
16.6.10.1	Hart Array.....	335
16.6.10.2	Halt Groups and External Triggers	335
16.6.10.3	Using External Triggers	337
16.6.11	System Bus Access	337
16.7	Debug Module Operational Sequences	338
16.7.1	Entering Debug Mode	338
16.7.2	Exiting Debug Mode	338
16.8	Debug and Core Complex Power Gating	339
16.8.1	Individual Core Debug Availability	339
16.8.2	Core Complex Debug Availability	339
17	SiFive Trace System	340
17.1	Introduction	340
17.2	Trace Control Interface	341
17.3	Instruction Trace Encoder	342
17.3.1	Ingress Port	342

17.3.2	In-Circuit Trace Message	343
17.3.3	TE Trace Packer	344
17.3.4	Register Interface	345
17.4	Instrumentation Trace Component (ITC).....	350
17.4.1	ITC Register Interface	350
17.4.2	Using the ITC	351
17.5	Trace Encoder Timestamp Unit	352
17.5.1	Trace Encoder Timestamp Register Interface	352
17.6	External Triggers.....	353
17.6.1	Register Descriptions.....	355
17.6.2	Using External Trigger Inputs.....	356
17.6.3	Using External Trigger Outputs	356
17.6.4	Code Examples.....	357
17.7	Core Watchpoints.....	357
17.7.1	Register Descriptions.....	357
17.8	Watchpoint Precedence	358
17.9	PC Sampling Unit.....	359
17.9.1	Register Descriptions.....	359
17.10	Trace Funnel	362
17.10.1	Register Descriptions	363
17.10.2	Trace Funnel Timestamp Unit	367
17.10.2.1	Trace Funnel Timestamp Register Interface.....	368
17.11	Nexus Messages.....	369
17.11.1	Nexus Message Formats	369
17.11.2	Address Compression	372
17.11.3	Timestamp Compression	372
17.11.4	Nexus Transmission Protocol.....	373
17.12	PIB Trace Sink	373
17.12.1	Register Descriptions	374
17.12.2	Programming Recommendations	375
17.12.3	Calibration Mode	375
17.12.4	SWT Manchester Protocol.....	376
17.12.5	SWT UART Protocol	376

17.12.6	PIB Parallel Protocol	377
17.13	Nexus Branch History Trace Encoder	378
17.13.1	History Buffer	378
17.13.2	Return-Address Stack Optimization	379
17.13.3	Repeat Branch Optimization	380
17.13.4	Sync Messages	381
17.13.5	HTM Nexus Message Formats	382
18	Error Correction Codes (ECC)	384
18.1	ECC Configuration	384
18.1.1	ECC Initialization	385
18.2	ECC Interrupt Handling and Error Injection	385
18.3	Hardware Operation Upon ECC Error	385
A	SiFive Core Complex Configuration Options	387
A.1	X280 Series	387
B	SiFive RISC-V Implementation Registers	390
B.1	Machine Architecture ID Register (marchid)	390
B.2	Machine Implementation ID Register (mimpid)	391
C	SiFive Custom CSRs	392
D	Floating-Point Unit Instruction Timing	393
D.1	X280 Floating-Point Instruction Timing	393
E	Revision History	397
F	Knowledge Base Articles	399
	References	400

Tables

Table 1	Tenstorrent X280 MC Feature Set.....	34
Table 2	RISC-V Specification Compliance	38
Table 3	Abbreviations and Terms.....	40
Table 4	X280 Feature Set	41
Table 5	Executable Memory Regions for the Tenstorrent X280 MC	42
Table 6	X280 Instruction Latency.....	46
Table 7	PTE Configuration Bits.....	55
Table 8	PTE Encoding fields	56
Table 9	Supervisor Address Translation and Protection Register	57
Table 10	pmpXcfg Bitfield Description	65
Table 11	pmpaddrX Encoding Examples for A=NAPOT	66
Table 12	mhpmeventX Register Bit Layout.....	71
Table 13	mhpmeventN Register	73
Table 14	Physical Memory Attributes for External Regions.....	78
Table 15	Physical Memory Attributes for Internal Regions.....	78
Table 16	Tenstorrent X280 MC Memory Map. Physical Memory Attributes: R –Read, W –Write, X –Execute, I –Instruction Cacheable, D –Data Cacheable, A –Atomics.....	80
Table 17	Base Instruction Formats	82
Table 18	R-Type Integer Instructions.....	84
Table 19	R-Type Integer Instruction Description	84
Table 20	I-Type Integer Instructions	85
Table 21	I-Type Integer Instruction Description	86
Table 22	I-Type Load Instructions	87
Table 23	I-Type Load Instruction Description	87
Table 24	S-Type Store Instructions	88
Table 25	S-Type Store Instruction Description	88
Table 26	J-Type Instruction Description.....	89
Table 27	B-Type Instructions	89
Table 28	B-Type Instruction Description	89

Table 29	RISC-V Base Instruction to Assembly Pseudoinstruction Example	90
Table 30	Multiplication Operation Description	91
Table 31	Division Operation Description	92
Table 32	Atomic Load-Reserve and Store-Conditional Instruction Description.....	93
Table 33	Atomic Memory Operation Description.....	94
Table 34	Accrued Exception Flags.....	94
Table 35	Floating-Point Rounding Modes	95
Table 36	Single-Precision FP Load and Store Instructions Description	95
Table 37	Single-Precision FP Computational Instructions Description	96
Table 38	Single-Precision FP Conversion Instructions Description.....	97
Table 39	Single-Precision FP to FP Sign-Injection Instructions Description.....	98
Table 40	RISC-V Base Instruction to Assembly Pseudoinstruction Example	98
Table 41	Single-Precision FP Move Instructions Description	98
Table 42	Single-Precision FP Compare Instructions Description	99
Table 43	Single-Precision FP Classify Instruction Description	99
Table 44	Floating-Point Number Classes.....	100
Table 45	Double-Precision FP Load and Store Instructions Description.....	100
Table 46	Double-Precision FP Computational Instructions Description	101
Table 47	Double-Precision FP Conversion Instructions Description	103
Table 48	Double-Precision FP to FP Sign-Injection Instructions Description	104
Table 49	RISC-V Base Instruction to Assembly Pseudoinstruction Example	104
Table 50	Double-Precision FP Move Instructions Description	105
Table 51	Double-Precision FP Compare Instructions Description.....	105
Table 52	Double-Precision FP Classify Instruction Description	106
Table 53	Half-Precision FP Load and Store Instructions Description	106
Table 54	Half-Precision FP Computational Instructions Description	107
Table 55	Half-Precision FP Conversion Instructions Description	109
Table 56	Half-Precision FP to FP Sign-Injection Instructions Description	110
Table 57	RISC-V Base Instruction to Assembly Pseudoinstruction Example	110
Table 58	Half-Precision FP Move Instructions Description.....	111
Table 59	Half-Precision FP Compare Instructions Description	111
Table 60	Half-Precision FP Classify Instruction Description.....	112
Table 61	Vector CSRs	112

Table 62	Vector Type Register	113
Table 63	Encoding of vseg.....	114
Table 64	Setting Vector Register Groups	114
Table 65	Encoding of vta and vma.....	115
Table 66	Encoding of vxrm.....	117
Table 67	Vector Control and Status Register	117
Table 68	vsetvli and vsetvl Instructions Description.....	120
Table 69	Encoding of AVL in vsetvli and vsetvl Instructions	120
Table 70	VMEM Format Instruction Fields	122
Table 71	Encoding of mop for Loads.....	123
Table 72	Encoding of mop for Stores	123
Table 73	Encoding of lumop.....	123
Table 74	Encoding of sumop.....	123
Table 75	Encoding of Vector Loads and Stores width	124
Table 76	Encoding of nf	128
Table 77	Unit-Stride Segmented Load/Store Instructions Description	129
Table 78	funct3 Fields for Arithmetic Instructions.....	132
Table 79	Compare Instructions in Assembly Format	151
Table 80	Single-Width Floating-Point/Integer Type-Convert Instructions Description.....	152
Table 81	Widening Floating-Point/Integer Type-Convert Instructions Description.....	152
Table 82	Narrowing Floating-Point/Integer Type-Convert Instructions.....	153
Table 83	Vector Mask Logical Instructions	156
Table 84	Stack-Pointed-Based Load Instruction Description.....	172
Table 85	Stack-Pointed-Based Store Instruction Description	173
Table 86	Register-Based Load Instruction Description	173
Table 87	Register-Based Store Instruction Description	174
Table 88	Unconditional Jump Instruction Description	174
Table 89	Unconditional Control Transfer Instruction Description	174
Table 90	Conditional Control Transfer Instruction Description	175
Table 91	Integer Constant-Generation Instruction Description	175
Table 92	Integer Register-Immediate Operation Description.....	176
Table 93	Integer Register-Immediate Operation Description (cont.)	176

Table 94	Integer Register-Immediate Operation Description (cont.)	176
Table 95	Integer Register-Immediate Operation Description (cont.)	176
Table 96	Integer Register-Immediate Operation Description (cont.)	177
Table 97	Integer Register-Register Operation Description	177
Table 98	Integer Register-Register Operation Description (cont.).....	177
Table 99	Address Calculation Instructions Description	178
Table 100	Add/Shift with Prefix Zero-Extend Instructions Description	179
Table 101	Count Leading/Trailing Zeroes Instructions Description	180
Table 102	Count Population Instructions Description	180
Table 103	Logic-With-Negate Instructions Description	181
Table 104	Comparison Instructions Description	181
Table 105	Sign- and Zero-Extend Instructions	182
Table 106	Bitwise Rotation Instructions Description.....	183
Table 107	OR Combine Instruction Description	183
Table 108	Byte-Reverse Instruction Description	183
Table 109	Control and Status Register Instruction Description.....	184
Table 110	CSR Reads and Writes	185
Table 111	User Mode CSRs	186
Table 112	Supervisor Mode CSRs	187
Table 113	Machine Mode CSRs	188
Table 114	Debug Mode Registers.....	189
Table 115	Core Generator Encoding of marchid	190
Table 116	Generator Release Encoding of mimpid	191
Table 117	Machine Status Register (partial).....	192
Table 118	Supervisor Status Register (partial)	193
Table 119	Timer and Counter Pseudoinstruction Description	194
Table 120	Timer and Counter CSRs	194
Table 121	RISC-V Registers	198
Table 122	RISC-V Assembly and C Examples	199
Table 123	RISC-V Code Model Table.....	212
Table 124	Branch Prediction Mode CSR	218
Table 125	SiFive Feature Disable CSR	219
Table 126	SiFive Feature Disable CSR Usage	219

Table 127	Exception Priority	222
Table 128	Summary of Exception and Interrupt CSRs	223
Table 129	Machine Status Register (partial)	227
Table 130	Encoding of RISC-V Privilege Levels	227
Table 131	Machine Trap Vector Register	228
Table 132	Encoding of <code>mtvec.MODE</code>	228
Table 133	Machine Interrupt Enable Register	229
Table 134	Machine Interrupt Pending Register	230
Table 135	Machine Cause Register	230
Table 136	<code>mcause</code> Exception Codes	231
Table 137	Machine Interrupt Delegation Register	233
Table 138	Machine Exception Delegation Register	233
Table 139	Supervisor Status Register (partial)	234
Table 140	Supervisor Interrupt Enable Register	234
Table 141	Supervisor Interrupt Pending Register	235
Table 142	Supervisor Cause Register	235
Table 143	<code>scause</code> Exception Codes	236
Table 144	Supervisor Trap Vector Register	237
Table 145	Encoding of <code>stvec.MODE</code>	237
Table 146	RNMI CSRs	239
Table 148	<code>mnstatus</code> CSR Fields	240
Table 149	Tenstorrent X280 MC Interrupt IDs	245
Table 150	CLINT Memory Map	246
Table 151	PLIC Memory Map	250
Table 152	PLIC Interrupt Source Mapping	253
Table 153	Mapping of <code>global_interrupts</code> Signal Bits to PLIC Interrupt ID	253
Table 154	PLIC Interrupt Priority Register	254
Table 155	PLIC Interrupt Pending Register 1	254
Table 156	PLIC Interrupt Pending Register 5	255
Table 157	PLIC Interrupt Enable Register 1 for Hart 0 M-Mode	255
Table 158	PLIC Interrupt Enable Register 5 for Hart 0 M-Mode	255
Table 159	PLIC Interrupt Enable Register 1 for Hart 0 S-Mode	256
Table 160	PLIC Interrupt Enable Register 5 for Hart 0 S-Mode	256

Table 161	PLIC Clock Gate Disable Register	256
Table 162	PLIC Interrupt Priority Threshold Register for Hart 0 M-Mode.....	257
Table 163	PLIC Interrupt Priority Threshold Register for Hart 0 S-Mode	257
Table 164	PLIC Claim/Complete Register for Hart 0 M-Mode	258
Table 165	PLIC Claim/Complete Register for Hart 0 S-Mode.....	258
Table 166	Bus-Error Unit Memory Map	262
Table 167	Bus-Error Unit Error Events	262
Table 168	L2 Prefetcher Instances	264
Table 169	L2 Prefetcher Memory Map	266
Table 170	Basic Control Register	266
Table 171	Additional Control Register	267
Table 172	Private L2 Cache Instances	269
Table 173	Private L2 Cache Feature Set.....	269
Table 174	PL2 Performance Monitor Event Selectors	271
Table 175	PL2 Performance Monitor Counter Client Filter	272
Table 176	Master IDs in the Private L2 Cache	272
Table 177	PL2 Performance Monitor Counters.....	272
Table 178	PL2PM pmEventSelect Register	274
Table 179	Private L2 Cache Memory Map	277
Table 180	PL2 Cache Configuration Register	277
Table 181	ECC Error Injection Register.....	278
Table 182	ECC Error Injection Done Register	278
Table 183	PL2 Cache Flush Register.....	279
Table 184	Cache Flush Commands	279
Table 185	PL2 Flush Count Register.....	280
Table 186	PL2 Configuration 0 Register	281
Table 187	L3 Features Access Summary	287
Table 188	L3 Performance Monitor Event Selectors	287
Table 189	L3 Performance Monitor Counter Client Filter	288
Table 190	L3 Performance Monitor Counters.....	288
Table 191	L3PM pmEventSelect Register.....	289
Table 192	L3 Cache Controller Memory Map	293
Table 193	L3 Cache Configuration Register.....	295

Table 194	Way Enable Register	295
Table 195	ECC Error Injection Register	295
Table 196	Way Mask 0 Register	298
Table 197	Master IDs in the L3 Cache Controller	299
Table 198	L3 Cache Feature Disable Register	300
Table 199	Power Dial CSR	304
Table 200	CorePower State Memory Map	305
Table 201	CorePower State Register	305
Table 202	Debug Module Memory Map Seen from the Debug Module Interface	312
Table 203	Debug Module Memory Map from the Perspective of the Core	314
Table 204	Debug and Trigger Registers	315
Table 205	Debug Control and Status Register	316
Table 206	Trigger Select Register	317
Table 207	Trigger Data Register 1	317
Table 208	Trigger Data Registers 2 and 3	318
Table 209	Trigger CSRs When Used as Breakpoints	318
Table 210	Breakpoint Match Control Register	319
Table 211	NAPOT Size Encoding	320
Table 212	Encoding of priv	323
Table 213	SiFive Instruction Trace Interface Fields	324
Table 214	Debug Module Interface Signals	325
Table 215	Debug Module Status Register	327
Table 216	Debug Module Control Register	329
Table 217	Hart Info Register	330
Table 218	Abstract Control and Status Register	331
Table 219	Abstract Command Register	332
Table 220	Abstract Command Autoexec Register	332
Table 221	Debug Module Control and Status 2 Register	333
Table 222	Debug Abstract Commands	334
Table 223	Abstract Command Example for 32-bit Block Write	335
Table 224	System Bus vs. Program Buffer Comparison	338
Table 225	Trace Encoder Memory Map	342
Table 226	Core to Trace Encoder Interface	343

Table 227	CKSRC Description	344
Table 228	Trace Encoder Control Register	345
Table 229	Trace Encoder Implementation Register	349
Table 230	Instrumentation Trace Enable Register	350
Table 231	Instrumentation Trigger Enable Register	350
Table 232	Instrumentation Trace Stimulus Register	351
Table 233	Trace Encoder Timestamp Control Register	352
Table 234	Trace Encoder Timestamp Lower Bits Register	353
Table 235	Trace Encoder Timestamp Upper Bits Register	353
Table 236	External Trigger Input Control Register	355
Table 237	External Trigger Output Register	356
Table 238	Watchpoint Action Identifiers.....	357
Table 239	Core Watchpoint Control Register	358
Table 240	PC Sample Control Register	359
Table 241	PC Capture and Sample Implementation Register	360
Table 242	PC Capture Register	360
Table 243	PC Capture High Register	361
Table 244	PC Sample Register	361
Table 245	PC Sample High Register	362
Table 246	Trace Funnel Control Register	364
Table 247	Trace Funnel Implementation Register.....	365
Table 248	Trace Funnel Sink Base Register	365
Table 249	Trace Funnel Sink Base High Bits Register	366
Table 250	Trace Funnel Sink Limit Register.....	366
Table 251	Trace Funnel Sink Write Pointer Register	366
Table 252	Trace Funnel SRAM Sink Access Pointer Register	367
Table 253	Trace Funnel SRAM Sink Data Register	367
Table 254	Trace Funnel Timestamp Control Register	368
Table 255	Trace Funnel Timestamp Lower Bits Register.....	368
Table 256	Trace Funnel Timestamp Upper Bits Register.....	369
Table 257	Direct Branch Message Description.....	369
Table 258	Indirect Branch Message Description.....	369
Table 259	Direct Branch with Sync Message Description	370

Table 260	Indirect Branch with Sync Message Description	370
Table 261	Program Trace Synchronization Message Description	370
Table 262	Error Message Description	371
Table 263	Program Trace Correlation Message Description	371
Table 264	In-Circuit Trace Message Description (Reports PC)	371
Table 265	Data Acquisition Message (from ITC) Description	371
Table 266	Ownership Trace Message (from ITC) Description	372
Table 267	Address Compression Example	372
Table 268	Timestamp Compression Description	373
Table 269	Nexus Transmission Protocol Description	373
Table 270	PIB Sink Control Register	374
Table 271	PIB Port Width Configuration Description	375
Table 272	Calibration Mode Description	376
Table 273	SWT Manchester Protocol Description	376
Table 274	RAS Optimization Example	379
Table 275	Repeat Branch Optimization Example 1	380
Table 276	Repeat Branch Optimization Example 2	380
Table 277	Sync Messages	381
Table 278	Indirect Branch History Message	382
Table 279	Indirect Branch History with Sync Message	382
Table 280	Resource Full Message	383
Table 281	Program Trace Correlation Message (HTM format)	383
Table 282	Memory Protection Summary	384
Table 283	Core Generator Encoding of marchid	390
Table 284	Generator Release Encoding of mimpid	391
Table 285	SiFive Custom CSRs	392
Table 286	X280 Single-Precision FPU Instruction Latency and Repeat Rates	394
Table 287	X280 Double-Precision FPU Instruction Latency and Repeat Rates	395
Table 288	X280 Half-Precision FPU Instruction Latency and Repeat Rates	396
Table 289	Tenstorrent X280 MC Manual Revision History	397

Figures

Figure 1	X280 Series Block Diagram	36
Figure 2	X280 Execution Pipeline.....	45
Figure 3	Vector Unit Pipeline	49
Figure 4	TLB Update Flow.....	53
Figure 5	Sv48 Virtual Address	54
Figure 6	Sv48 Physical Address.....	54
Figure 7	Sv48 PTE Format	54
Figure 8	SFENCE.VMA Instruction	58
Figure 9	Linux User Application Memory Map Example	61
Figure 10	Hardware Table Walk Example.....	62
Figure 11	RV64 pmpcfg0 Register.....	64
Figure 12	RV64 pmpcfg2 Register.....	64
Figure 13	RV64 pmpXcfg bitfield	65
Figure 14	RV64 pmpaddrX Register.....	66
Figure 15	PMP Example Block Diagram	67
Figure 16	Event Selector Fields	70
Figure 17	Counter-Enable Register (mcounteren)	74
Figure 18	R-Type.....	82
Figure 19	I-Type	83
Figure 20	S-Type.....	83
Figure 21	B-Type.....	83
Figure 22	U-Type.....	83
Figure 23	J-Type	83
Figure 24	ADD Instruction Example.....	84
Figure 25	ADDI Instruction Example.....	86
Figure 26	LW Instruction Example	87
Figure 27	Store Instructions.....	88
Figure 28	SW Instruction Example	88
Figure 29	JAL Instruction.....	88

Figure 30	JALR Instruction	89
Figure 31	Branch Instructions	89
Figure 32	Upper-Immediate Instructions	90
Figure 33	FENCE Instructions	90
Figure 34	NOP Instructions	91
Figure 35	Multiplication Operations	91
Figure 36	Division Operations	92
Figure 37	Atomic Operations	92
Figure 38	Atomic Memory Operations	93
Figure 39	Floating-Point Control and Status Register	94
Figure 40	Single-Precision FP Load Instruction	95
Figure 41	Single-Precision FP Store Instruction	95
Figure 42	Single-Precision FP Computational Instructions	96
Figure 43	Single-Precision FP Fused Computational Instructions	96
Figure 44	Single-Precision FP to Integer and Integer to FP Conversion Instructions	96
Figure 45	Single-Precision FP to FP Sign-Injection Instructions	97
Figure 46	Single-Precision FP Move Instructions	98
Figure 47	Single-Precision FP Compare Instructions	99
Figure 48	Single-Precision FP Classify Instruction	99
Figure 49	Double-Precision FP Load Instruction	100
Figure 50	Double-Precision FP Store Instruction	100
Figure 51	Double-Precision FP Computational Instructions	101
Figure 52	Double-Precision FP Fused Computational Instructions	101
Figure 53	Double-Precision FP to Integer and Integer to FP Conversion Instructions	102
Figure 54	Double-Precision to Single-Precision and Single-Precision to Double-Precision FP Conversion Instructions	102
Figure 55	Double-Precision FP to FP Sign-Injection Instructions	104
Figure 56	Double-Precision FP Move Instructions	104
Figure 57	Double-Precision FP Compare Instructions	105
Figure 58	Double-Precision FP Classify Instruction	106
Figure 59	Half-Precision FP Load Instruction	106
Figure 60	Half-Precision FP Store Instruction	106
Figure 61	Half-Precision FP Computational Instructions	107

Figure 62	Half-Precision FP Fused Computational Instructions	107
Figure 63	Half-Precision FP to Integer and Integer to FP Conversion Instructions.....	108
Figure 64	Half-Precision to Single-Precision and Single-Precision to Half-Precision FP Conversion Instructions	108
Figure 65	Half-Precision to Double-Precision and Double-Precision to Half-Precision FP Conversion Instructions	108
Figure 66	Half-Precision FP to FP Sign-Injection Instructions.....	110
Figure 67	Half-Precision FP Move Instructions	110
Figure 68	Half-Precision FP Compare Instructions.....	111
Figure 69	Half-Precision FP Classify Instruction	112
Figure 70	Vector Register VLEN	113
Figure 71	VL Unit Stride Instruction Format.....	118
Figure 72	VLS Strided Instruction Format	118
Figure 73	VLX Indexed Instruction Format	118
Figure 74	VS Unit-Stride Instruction Format	118
Figure 75	VSS Strided Instruction Format	118
Figure 76	VSX Indexed Instruction Format.....	118
Figure 77	OPIVV Instruction Format.....	119
Figure 78	OPFVV Instruction Format.....	119
Figure 79	OPMVV Instruction Format	119
Figure 80	OPIVI Instruction Format	119
Figure 81	OPIVX Instruction Format.....	119
Figure 82	OPFVF Instruction Format.....	119
Figure 83	OPMVX Instruction Format	119
Figure 84	vsetvli Instruction Format.....	119
Figure 85	vsetv1 Instruction Format	119
Figure 86	vsetvli Instruction.....	120
Figure 87	vsetv1 Instruction.....	120
Figure 88	Vector Unit-Stride Load Instruction	125
Figure 89	Vector Unit-Stride Fault-Only-First Load Instruction.....	125
Figure 90	Vector Strided Load Instruction	125
Figure 91	Vector Indexed-Unordered Load Instruction.....	126
Figure 92	Vector Indexed-Ordered Load Instruction	126

Figure 93	Vector Unit-Stride Store Instruction	126
Figure 94	Vector Strided Store Instruction	126
Figure 95	Vector Ordered-Indexed Store Instruction.....	127
Figure 96	Vector Unordered-Indexed Store Instruction	127
Figure 97	Vector Load Whole Register Instruction	131
Figure 98	Vector Store Whole Register Instruction	131
Figure 99	OPIVV Instruction	132
Figure 100	OPFVV Instruction	132
Figure 101	OPMVV Instruction.....	132
Figure 102	OPIVI Instruction.....	132
Figure 103	OPIVX Instruction	132
Figure 104	OPFVF Instruction	132
Figure 105	OPMVX Instruction.....	132
Figure 106	CR Format - Register	171
Figure 107	CI Format - Immediate.....	171
Figure 108	CSS Format - Stack-relative Store	171
Figure 109	CIW Format - Wide Immediate	171
Figure 110	CL Format - Load	171
Figure 111	CS Format - Store	171
Figure 112	CA Format - Arithmetic	172
Figure 113	CJ Format - Jump	172
Figure 114	Stack-Pointed-Based Loads	172
Figure 115	Stack-Pointed-Based Stores	172
Figure 116	Register-Based Loads	173
Figure 117	Register-Based Stores.....	173
Figure 118	Unconditional Jump Instructions	174
Figure 119	Unconditional Control Transfer Instructions.....	174
Figure 120	Conditional Control Transfer Instructions	175
Figure 121	Integer Constant-Generation Instructions	175
Figure 122	Integer Register-Immediate Operations	175
Figure 123	Integer Register-Immediate Operations (cont.).....	176
Figure 124	Integer Register-Immediate Operations (cont.).....	176
Figure 125	Integer Register-Immediate Operations (cont.).....	176

Figure 126	Integer Register-Immediate Operations (cont.).....	177
Figure 127	Integer Register-Register Operations	177
Figure 128	Integer Register-Register Operations (cont.).....	177
Figure 129	Defined Illegal Instruction.....	178
Figure 130	Address Calculation Instructions.....	178
Figure 131	Address Calculation Instructions (cont.)	178
Figure 132	Add with Prefix Zero-Extend Instruction.....	179
Figure 133	Shift with Prefix Zero-Extend Instruction	179
Figure 134	Count Leading/Trailing Zeroes Instructions.....	179
Figure 135	Count Leading/Trailing Zeroes Instructions (cont.).....	179
Figure 136	Count Population Instruction	180
Figure 137	Count Population Instruction (cont.)	180
Figure 138	Logic-With-Negate Instructions.....	180
Figure 139	Comparison Instructions	181
Figure 140	Sign-Extend Instructions	181
Figure 141	Zero-Extend Instruction	181
Figure 142	Bitwise Rotation Instructions	182
Figure 143	Bitwise Rotation Instructions (cont.)	182
Figure 144	Bitwise Rotation Instructions (cont.)	182
Figure 145	Bitwise Rotation Instructions (cont.)	182
Figure 146	Zicsr Instructions.....	184
Figure 147	Timer and Counter Pseudoinstructions.....	193
Figure 148	ECALL and EBREAK Instructions.....	196
Figure 149	Wait for Interrupt Instruction	196
Figure 150	Supervisor Memory-Management Fence Instruction	197
Figure 151	RISC-V Assembly Example	199
Figure 152	RISC-V Assembly to Machine Code	200
Figure 153	One RISC-V Instruction	201
Figure 154	Stack Memory during Function Calls.....	203
Figure 155	Tenstorrent X280 MC Interrupt Architecture Block Diagram	225
Figure 156	CLINT Block Diagram.....	242
Figure 157	CLINT Interrupts and Vector Table	243
Figure 158	CLINT Vector Table Example	244

Figure 159	CLINT Interrupt Attribute Example	245
Figure 160	PLIC Multi-Core Block Diagram	249
Figure 161	Bus-Error Unit Block Diagram	261
Figure 162	Cache Structure in the Tenstorrent X280 MC	268
Figure 163	Event Selector Fields	271
Figure 164	Organization of the SiFive L3 Cache Controller	283
Figure 165	Mapping of L3 Cache Ways to L3 LIM Addresses	284
Figure 166	Difference between L3 LIM and L3 Zero Device	286
Figure 167	Event Selector Fields	287
Figure 168	Debug External Triggers	336
Figure 169	Debug External Trigger Handshake	337
Figure 170	Trace Encoder Block Diagram	340
Figure 171	Trace Packer	345
Figure 172	Trace Encoder External Trigger Handshake	354
Figure 173	Trace Encoder External Triggers	354
Figure 174	Trace Funnel Block Diagram	363
Figure 175	SWT Manchester Protocol	376
Figure 176	SWT UART Protocol	377
Figure 177	PIB Parallel Protocol	377
Figure 178	PIB Parallel Protocol, pibRefCenter=1	378

Chapter 1

Introduction

The Tenstorrent X280 MC is enabled with support for the RISC-V Vector (RVV) ISA extension. The SiFive Tenstorrent X280 MC is guaranteed to be compatible with all applicable RISC-V standards, and this document should be read together with the official RISC-V user-level, privileged, and external debug architecture specifications.



A summary of features in the Tenstorrent X280 MC can be found in Table 1.

Tenstorrent X280 MC Feature Set	
Feature	Description
Number of Harts	4 Harts.
X280 Core	4 × X280 RISC-V cores.
PLIC Interrupts	127 Interrupt signals, which can be connected to off-core-complex devices.
PLIC Priority Levels	The PLIC supports 7 priority levels.
Level 3 Cache	2 MiB 16-way L3 Cache.
Hardware Breakpoints	4 hardware breakpoints.
Physical Memory Protection Unit	PMP with 8 regions and a minimum granularity of 4096 bytes.
Vector VLEN	512-bit
Vector SLEN	512-bit
Vector ELEN	64-bit
Vector Datapath Width	256-bit

Table 1: Tenstorrent X280 MC Feature Set

The Tenstorrent X280 MC also has a number of on-core-complex configurability options, allowing one to tune the design to a specific application. The configurable options are described in Appendix A.

1.1 About this Document

This document describes the functionality of the Tenstorrent X280 MC 21G3.04.00. To learn more about the production deliverables of the Tenstorrent X280 MC, consult the Tenstorrent X280 MC User Guide.

1.2 About this Release

This is a general release of the Tenstorrent X280 MC 21G3.04.00, with a supported life cycle as outlined in your SiFive Core IP contract. General releases are used for production tapeouts, with known errata fixed in subsequent release updates. Contact support@sifive.com if you have any questions.

1.3 Tenstorrent X280 MC Overview

The Tenstorrent X280 MC includes 4 × X280 64-bit RISC-V cores, along with the necessary functional units required to support the cores. These units include a Core-Local Interruptor (CLINT) to support local interrupts, a Platform-Level Interrupt Controller (PLIC) to support platform interrupts, physical memory protection, a Debug unit to support a JTAG-based debugger host connection, Nexus 5001 compliant instruction trace for non-invasive debug and performance profiling, and a local crossbar that integrates the various components together.

The Tenstorrent X280 MC memory system consists of a Data Cache and Instruction Cache, with coherent L1 caches, private L2 Cache, shared L3 Cache, and a directory based coherence manager. The Tenstorrent X280 MC also includes a Front Port, which allows external masters to be coherent with the L1 memory system and access to the TIMs, thereby removing the need to maintain coherence in software for any external agents.

All memories, including caches and TIMs, support Single Error Correction, Double Error Detection (SECEDED) ECC to provide improved reliability and address safety critical applications.

An overview of the SiFive X280 Series is shown in Figure 1. Refer to the docs/core_complex_configuration.txt file for a comprehensive summary of the Tenstorrent X280 MC configuration.

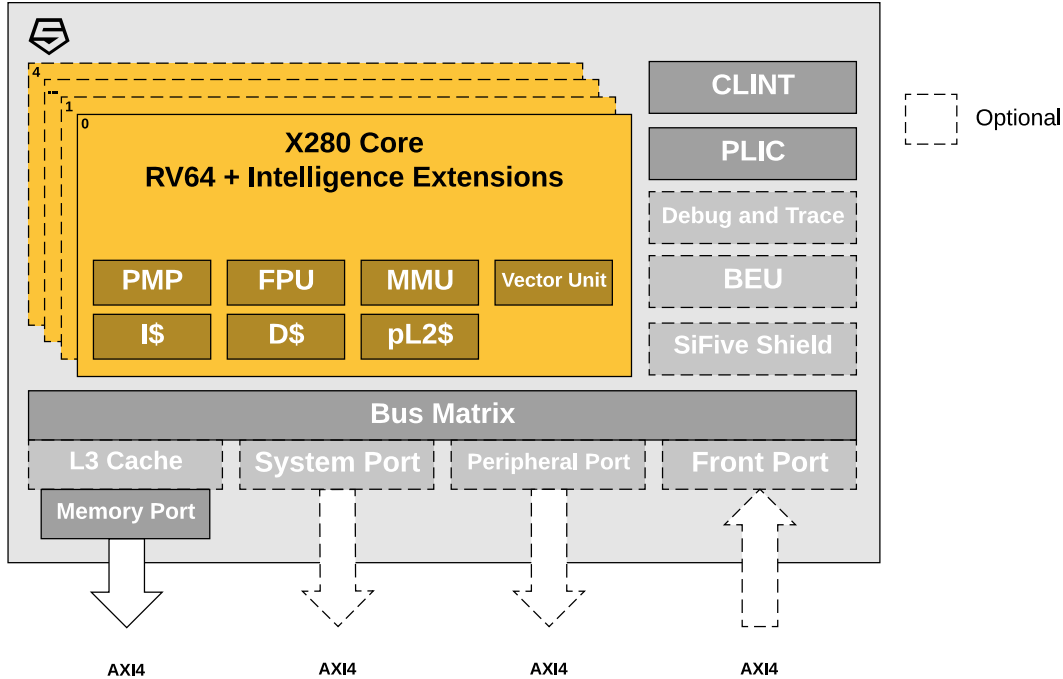


Figure 1: X280 Series Block Diagram

The Tenstorrent X280 MC memory map is detailed in Section 4.2, and the interfaces are described in full in the Tenstorrent X280 MC User Guide.

1.4 X280 RISC-V Cores

The Tenstorrent X280 MC includes four 64-bit X280 RISC-V cores, which each have a dual-issue, in-order execution pipeline, with a peak execution rate of two instructions per clock cycle. The SiFive X280 core is guaranteed to be compatible with all applicable RISC-V standards.

Each X280 core is configured to support the RV64I base ISA, as well as the Multiply (M), Atomic (A), Single-Precision Floating Point (F), Double-Precision Floating Point (D), Compressed (C), Vectors (V), CSR Instructions (Zicsr), Instruction-Fetch Fence (Zifencei), Address Calculation (Zba), Basic Bit Manipulation (Zbb), Half-Precision Floating Point (Zfh), and Count Overflow and Mode-Based Filtering (Sscofpmf) RISC-V extensions. This is captured by the RISC-V extension string: `RV64GCV_Zfh_Zba_Zbb_Sscofpmf`. The X280 also supports the SiFive Intelligence extensions:

1. Xsfvfhbfmin Extension: Bfloat16 Conversion Instructions
2. Xsfvfnrclipxfqf Extension: FP32-to-int8 Ranged Clip Instructions
3. Xsfvqmaccqoq Extension: Int8 Matrix Multiplication Instructions

These SiFive custom RISC-V extensions are specially designed to accelerate AI and machine learning applications. The base ISA and instruction extensions are described in Chapter 5.

The X280 also supports machine, supervisor, and user privilege modes, in conjunction with Physical Memory Protection (PMP), thereby allowing System-on-Chip (SoC) implementations to make the right area, power, and feature trade-offs.

The X280 core is designed to be feature rich, providing a very flexible memory system that includes L1 caches, standards-based configurable bus interfaces, and memory maps that provide a lot of flexibility for SoC integration.

The microarchitecture also incorporates a branch prediction unit that is composed of a 16-entry Branch Target Buffer (BTB), a 3.6 KiB-entry Branch History Table (BHT), a 6-entry Return Address Stack (RAS), 8-entry Indirect Jump Target Predictor (IJTP), and a 16-entry Return Instruction Predictor.

The X280 includes an IEEE 754-2008 compliant Floating-Point Unit.

The X280 cores are described in more detail in Chapter 3.

1.5 Memory System

The Tenstorrent X280 MC memory system has a Level 1 memory system optimized for high performance. The instruction subsystem consists of a 32 KiB, 2-way instruction cache. The data subsystem is comprised of a high performance 32 KiB, 4-way L1 data cache. In addition to the L1 memory system, the Tenstorrent X280 MC has a per hart 128 KiB, 8-way private L2 cache. The memory system is described in more detail in Chapter 3.

The Tenstorrent X280 MC also supports a shared, 2 MiB, 16-way L3 cache with 4 banks. The L3 Cache Controller is described in Chapter 14.

1.6 Interrupts

The Tenstorrent X280 MC provides the standard RISC-V M-mode timer and software interrupts via the Core-Local Interruptor (CLINT).

The Tenstorrent X280 MC also includes a RISC-V standard Platform-Level Interrupt Controller (PLIC), which supports 135 global interrupts with 7 priority levels pre-integrated with the on-core-complex peripherals.

Interrupts are described in Chapter 7. The CLINT is described in Chapter 8. The PLIC is described in Chapter 9.

1.7 Debug Support

The Tenstorrent X280 MC provides external debugger support over an industry-standard JTAG port, including 4 hardware-programmable breakpoints per hart.

Debug support is described in detail in Chapter 16, and the debug interface is described in the Tenstorrent X280 MC User Guide.

1.8 Compliance

The Tenstorrent X280 MC is compliant to the following versions of the various RISC-V specifications:

ISA	Version	Status
RV64I Base Integer Instruction Set	2.1	Ratified
Extensions	Version	Status
M Standard Extension for Integer Multiplication and Division	2.0	Ratified
A Standard Extension for Atomic Instruction	2.1	Ratified
F Standard Extension for Single-Precision Floating-Point	2.2	Ratified
D Standard Extension for Double-Precision Floating-Point	2.2	Ratified
C Standard Extension for Compressed Instruction	2.0	Ratified
V Standard Extension for Vector Operations	1.0	Frozen
Zicsr Standard Extension for Control and Status Register (CSR) Instructions	2.0	Ratified
Zifencei Standard Extension for Instruction-Fetch Fence	2.0	Ratified
Zfh Standard Extension for Half-Precision Floating-Point	0.1	Draft
Zba Standard Extension for Address Calculation	1.0	Ratified
Zbb Standard Extension for Basic Bit Manipulation	1.0	Ratified
Zvfh Standard Extension for Vector FP16 Support	0.1	Draft
Sscofpmf Standard Extension for Count Overflow and Mode-Based Filtering	0.1	Ratified
Privilege Mode	Version	Status
Machine-Level ISA	1.11	Ratified
User-Level ISA	1.11	Ratified
Supervisor-Level ISA	1.11	Ratified
Devices	Version	Status
The RISC-V Debug Specification	1.0	Frozen
RISC-V Platform-Level Interrupt Controller (PLIC) Specification	—	—

Table 2: RISC-V Specification Compliance

Chapter 2

List of Abbreviations and Terms

Term	Definition
BHT	Branch History Table
BTB	Branch Target Buffer
CLIC	Core-Local Interrupt Controller. Configures priorities and levels for core-local interrupts.
CLINT	Core-Local Interruptor. Generates per hart software and timer interrupts.
DLEN	Vector ALU and memory datapath width
DTIM	Data Tightly-Integrated Memory
ELEN	Vector parameter that defines the execution length
Hart	HARdware Thread
IJTP	Indirect-Jump Target Predictor
ITIM	Instruction Tightly-Integrated Memory
JTAG	Joint Test Action Group
LIM	Loosely-Integrated Memory. Used to describe memory space delivered in a SiFive Core Complex that is not tightly integrated to a CPU core.
LMUL	Vector register grouping factor
PLIC	Platform-Level Interrupt Controller. The global interrupt controller in a RISC-V system.
PMC	Power Management Controller
PMP	Physical Memory Protection
RAS	Return-Address Stack
RO	Used to describe a Read-Only register field
RS	Read/Set field. A register field that cannot be cleared by software, only reset will clear.
RVV	RISC-V Vector ISA
RW	Used to describe a Read/Write register field
RW1C	Used to describe a Read/Write-1-to-Clear register field
SEW	Vector parameter that defines the selected element width
SLEN	Vector parameter that specifies the striping distance
TileLink	A free and open interconnect standard originally developed at UC Berkeley
VLEN	Vector parameter that defines the number of bits in a single vector register
W1C	Used to describe a Write-1-to-Clear register field
WARL	Write-Any, Read-Legal field. A register field that can be written with any value, but returns only supported values when read.
WIRI	Writes-Ignored, Reads-Ignore field. A read-only register field reserved for future use. Writes to the field are ignored and reads should ignore the value returned.
WLRL	Write-Legal, Read-Legal field. A register field that should only be written with legal values and that only returns legal value if last written with a legal value.
WO	Used to describe a Write-Only register field
WPRI	Writes-Preserve, Reads-Ignore field. A register field that might contain unknown information. Reads should ignore the value returned, but writes to the whole register should preserve the original value.

Table 3: Abbreviations and Terms

Chapter 3

X280 RISC-V Core

This chapter describes the 64-bit X280 RISC-V processor core, instruction fetch and execution unit, L1, L2, and L3 memory systems, Physical Memory Protection unit, Hardware Performance Monitor, and external interfaces.

The X280 feature set is summarized in Table 4.

Feature	Description
ISA	<i>RV64GCV_Zfh_Zba_Zbb_Sscofpmf</i>
SiFive Custom Instruction Extension (SCIE)	Not Present
Privilege Modes	M, U, S
L1 Instruction Cache	32 KiB 2-way instruction cache
L1 Data Cache	32 KiB 4-way data cache
Private L2 (pL2) Cache	128 KiB 8-way pL2 cache with 2 banks
L3 Cache	2 MiB 16-way L3 cache with 4 banks
ECC Support	Single error correction, double error detection on the L1 data cache, pL2 cache, and L3 cache
Physical Memory Protection	8 regions with a granularity of 4096 bytes
Memory Management Unit	Sv48 virtual memory support with fully-associative 64-entry L1 data and instruction TLBs, and a direct-mapped 1024-entry unified TLB
Vector Unit	Present
Vector VLEN	512-bit
Vector SLEN	512-bit
Vector ELEN	64-bit
Vector Datapath Width	256-bit
SEW	8, 16, 32, 64
LMUL	1/8, 1/4, 1/2, 1, 2, 4, 8
EDIV	1

Table 4: X280 Feature Set

3.1 Supported Privilege Modes

The X280 supports the RISC-V user- and supervisor-level architectures, providing three levels of privilege: machine (M), user (U), and supervisor (S) modes.

M-mode provides low-level access to the entire machine and is the mode entered at reset. U-mode provides a mechanism to isolate application processes from each other and from trusted code running in M-mode. S-mode adds a number of additional CSRs and capabilities to provide further isolation for OS-level code.

See *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11* for more information on the privilege modes.

3.2 Instruction Memory System

This section describes the instruction memory system of the X280 core.

3.2.1 Execution Memory Space

The regions of executable memory consist of all directly addressable memory in the system. The memory includes any volatile or non-volatile memory located off the Core Complex ports, and includes the on-core-complex L3 LIM and L3 Zero Device.

Table 5 shows the executable regions of the Tenstorrent X280 MC.

Base	Top	Description
0x0800_0000	0x081F_FFFF	L3 LIM
0x0A00_0000	0x0A1F_FFFF	L3 Zero Device
0x2000_0000	0x2FFF_FFFF	Peripheral Port (256 MiB)
0x3000_0000	0x4000_2FFF_FFFF	System Port (64 TiB)
0x4000_3000_0000	0x7FFF_FFFF_FFFF	Memory Port (64.0 TiB)

Table 5: Executable Memory Regions for the Tenstorrent X280 MC

All executable regions are treated as instruction cacheable. There is no method to disable this behavior.

Trying to execute an instruction from a non-executable address results in an instruction access trap.

3.2.2 L1 Instruction Cache

The L1 instruction cache is a 32 KiB 2-way set-associative cache. It is virtually-indexed, physically-tagged with a line size of 64 bytes and is read/write-allocate with a random replacement policy. A cache line fill triggers a burst access outside of the Core Complex, starting with the first address of the cache line. There are no write-backs to memory from the instruction cache and it

is not kept coherent with rest of the platform memory system. In multi-core systems, the instruction caches are not kept coherent with each other.

Out of reset, all blocks of the instruction cache are invalidated. The access latency of the cache is one clock cycle. There is no way to disable the instruction cache and cache allocations begin immediately out of reset.

The L1 instruction cache has parity error protection support.

3.2.3 Cache Maintenance

The instruction cache supports the `FENCE.I` instruction, which invalidates the entire instruction cache, as described in Section 5.16. Writes to instruction memory from the core or another master must be synchronized with the instruction fetch stream by executing `FENCE.I`.

3.2.4 Coherence with Higher Level Caches

The L1 instruction cache is partially inclusive with the private L2 Cache (pL2), described in Chapter 13, and the L3 Cache, described in Chapter 14.

When a block of instruction memory is allocated to the L1 cache, it is also allocated to the pL2 and L3 caches if the access was from the Memory Port. Instruction accesses to all other ports will not allocate to the pL2 or L3 caches, only the L1 cache.

When a block is evicted from L1, it might still reside in pL2 or L3, which will reduce access time the next time the block is fetched.

If a hart modifies instruction memory (i.e., self-modifying code), then a `FENCE.I` instruction is required to synchronize the instruction and data streams. Even though `FENCE.I` targets the L1 instruction cache, no cache operation is required on the pL2 or L3 caches to maintain instruction coherency.

3.2.5 Instruction Fetch Unit

The X280 instruction fetch unit is responsible for keeping the pipeline fed with instructions from memory. The instruction fetch unit delivers up to 8 bytes of instructions per clock cycle to support superscalar instruction execution. Fetches are always word-aligned and there is a one-cycle penalty for branching to a 32-bit instruction that is not word-aligned.

The X280 implements the standard Compressed (C) extension to the RISC-V architecture, which allows for 16-bit RISC-V instructions. As four 16-bit instructions can be fetched per cycle, the instruction fetch unit can be idle when executing programs comprised mostly of compressed 16-bit instructions. This reduces memory accesses and power consumption.

All branches must be aligned to half-word addresses. Otherwise, the fetch generates an instruction address misaligned trap. Trying to fetch from a non-executable or unimplemented address results in an instruction access trap.

3.2.6 Branch Prediction

The X280 instruction fetch unit contains sophisticated predictive hardware to mitigate the performance impact of control hazards within the instruction stream. The instruction fetch unit is decoupled from the execution unit, so that correctly predicted control-flow events usually do not result in execution stalls.

- A 16-entry branch target buffer (BTB), which predicts the target of taken branches and direct jumps;
- A 3.6 KiB branch history table (BHT), which predicts the direction of conditional branches;
- An 8-entry indirect-jump target predictor (IJTP);
- A 6-entry return-address stack (RAS), which predicts the target of procedure returns.

The BHT is a correlating predictor that supports long branch histories. The BTB has one-cycle latency, so that correctly predicted branches and direct jumps result in no penalty, provided the target is 8-byte aligned.

Direct jumps that miss in the BTB result in a one-cycle fetch bubble. This event might not result in any execution stalls if the fetch queue is sufficiently full.

The BHT, IJTP, and RAS take precedence over the BTB. If these structures' predictions disagree with the BTB's prediction, a one-cycle fetch bubble results. Similar to direct jumps that miss in the BTB, the fetch bubble might not result in an execution stall.

Mispredicted branches usually incur a four-cycle penalty, but sometimes the branch resolves later in the execution pipeline and incurs a six-cycle penalty instead. Mispredicted indirect jumps incur a six-cycle penalty.

Branch prediction is enabled out of reset and cannot be disabled. However, instruction speculation, fetching before a prediction is confirmed, must be enabled in the Feature Disable CSR, described in Chapter 6.

As instruction speculation can occur at any point after it has been enabled, data cacheable regions of memory (i.e., DDR) must be able to respond to instruction fetches immediately after instruction speculation is enabled. If DDR initialization is not completed before instruction speculation is enabled, the memory system must return a decode error (DECERR) for accesses made to DDR. The fetch unit will ignore errors associated with speculative accesses and continue to operate normally.

The Branch Prediction Mode CSR, also described in Chapter 6, provides a means to customize the branch predictor behavior to trade average performance for more predictable execution time.

3.3 Execution Pipeline

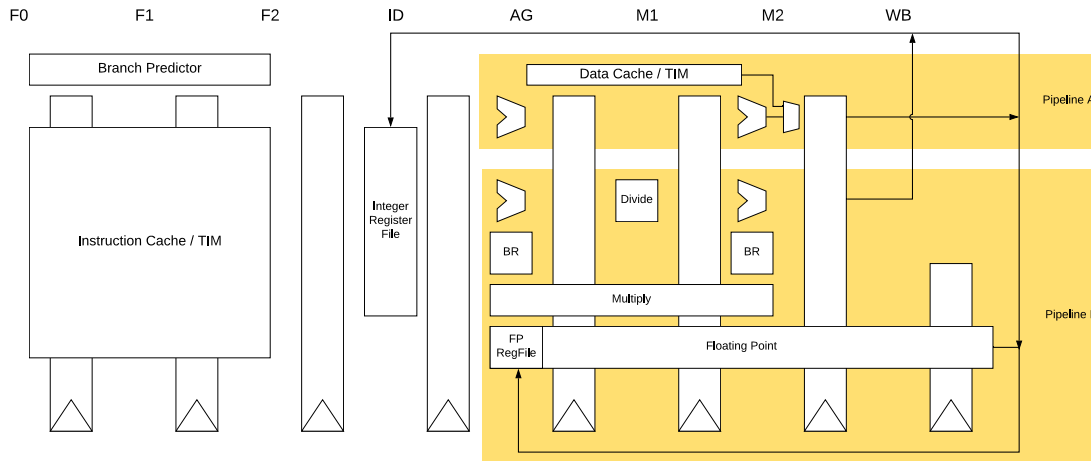


Figure 2: X280 Execution Pipeline

The X280 execution unit is a dual-issue, in-order pipeline. The pipeline comprises eight stages: three stages of instruction fetch (F0-F2), instruction decode (ID), address generation (AG), two stages of data memory access (M1 and M2), and register write-back (WB). The pipeline has a peak execution rate of two instructions per clock cycle, and is fully bypassed so that most instructions have a one-cycle result latency:

- Integer arithmetic and branch instructions can execute in either the AG or M2 pipeline stage. If such an instruction's operands are available when the instruction enters the AG stage, then it executes in AG; otherwise, it executes in M2.
- Loads produce their result in the M2 stage. There is no load-use delay for most integer instructions. However, effective addresses for memory accesses are always computed in the AG stage. Hence, loads, stores, and indirect jumps require their address operands to be ready when the instruction enters AG. If an address-generation operation depends upon a load from memory, then the load-use delay is two cycles.
- Integer multiplication instructions consume their operands in the AG stage and produce their results in the M2 stage. The integer multiplier is fully pipelined.
- Integer division instructions consume their operands in the AG stage. These instructions have between a six-cycle and 68-cycle result latency, depending on the operand values.
- CSR accesses execute in the M2 stage. CSR read data can be bypassed to most integer instructions with no delay. Most CSR writes flush the pipeline, which is a seven-cycle penalty.

Instruction	Latency
LW	Three-cycle latency, assuming cache hit ¹
LH, LHU, LB, LBU	Three-cycle latency, assuming cache hit ¹
CSR Reads	One-cycle latency ²
MUL, MULH, MULHU, MULHSU	Three-cycle latency
DIV, DIVU, REM, REMU	Between six-cycle to 68-cycle latency, depending on operand values ³
¹ Effective address not ready in AG stage. Load-to-use latency = load-to-use delay + 1 ² cycle latency = cycle delay + 1 ³ The latency of DIV, DIVU, REM, and REMU instructions can be determined by calculating: Latency = 2 cycles + $\log_2(\text{dividend}) - \log_2(\text{divisor}) + 1$ cycle if the input is negative + 1 cycle if the output is negative	

Table 6: X280 Instruction Latency

The pipeline has some register dependencies, where it interlocks on read-after-write and write-after-write hazards, so instructions may be scheduled to avoid stalls. Otherwise, the processor can have multiple outstanding memory-mapped I/O accesses, even to the same address. The pipeline implements a flexible dual-instruction-issue scheme. Provided there are no data hazards between a pair of instructions, the two instructions may issue in the same cycle, provided the following constraints are met:

- At most one instruction accesses data memory.
- At most one instruction is a branch or jump.
- At most one instruction is a floating-point arithmetic operation.
- At most one instruction is an integer multiplication or division operation.
- Neither instruction explicitly accesses a CSR.

See Appendix D for a complete list of floating-point unit instruction timings.

3.4 Data Memory System

The data memory system consists of on-core-complex data and the ports in the Tenstorrent X280 MC memory map, shown in Section 4.2. The on-core-complex data memory consists of a 32 KiB L1 data cache, 128 KiB private L2 cache, and 2 MiB L3 cache. The private L2 cache is described in Chapter 13. A design cannot have both data cache and DTIM.

Data accesses are classified as cacheable, for those targeting the Memory Port; or non-cacheable, for those targeting any other port in the Core Complex. Non-cacheable data accesses are collectively called memory-mapped I/O accesses, or MMIOs.

The X280 pipeline allows for multiple outstanding memory accesses, but only allows one outstanding cache line fill. The number of outstanding MMIOs are implementation dependent. Mis-aligned accesses are not allowed to any memory region and result in a trap to allow for software emulation.

3.4.1 L1 Data Cache

The L1 data cache is a 32 KiB 4-way set-associative cache. It is virtually-indexed, physically-tagged with a line size of 64 bytes and is read/write-allocate with a PLRU replacement policy. The cache operates in write-back mode; this means that if a cache line is dirty, it is written back to memory when evicted. Out of reset, all lines of the cache are invalidated.

The L1 data cache supports ECC protection, as described in Chapter 18.

The Memory Port address range is the only cacheable region of memory. A cache line fill triggers a burst access starting with the first address of the cache line. On a cache hit, the access latency is two clock cycles for words and double-words, and three clock cycles for smaller quantities. Stores are pipelined and commit on cycles where the data memory system is otherwise idle. Pending stores are stored in a buffer, which drains whenever there is an idle cycle or another store. Loads to addresses currently in the store pipeline result in a five-cycle penalty.

The data cache supports only one outstanding line fill. Once a cacheable access is made that misses, another cannot be issued until the line fill completes. However, other MMIOs can be issued before or after the line fill as long as there are no address or register hazards.

The data cache cannot be disabled and the properties of the Memory Port cannot be modified to prevent cacheable accesses.

3.4.2 Cache Maintenance Operations

The data cache supports `CFLUSH.D.L1` and `CDISCARD.D.L1`. The instruction `CFLUSH.D.L1` cleans and invalidates the specified line or all cache lines. The instruction `CDISCARD.D.L1` invalidates the specified line or all cache lines.

These custom instructions are further described in Chapter 6.

3.4.3 L1 Data Cache Coherency

All of the L1 data caches in the Core Complex are kept coherent with an integrated coherency manager. This is an automatic feature and cannot be disabled. The `CFLUSH.D.L1` and `CDISCARD.D.L1` instructions only affect the core that executed the instruction. They are not broadcast to all cores in the Complex.

3.4.4 Coherence with Higher Level Caches

The L1 data cache is inclusive with the private L2 Cache (pL2), described in Chapter 13, and the L3 Cache, described in Chapter 14.

When a block of data is allocated to the L1 cache, it is also allocated to the pL2 and L3 caches. When a block is evicted from the L1, the corresponding line in pL2 and L3 are then updated and marked dirty.

The custom instructions `CFLUSH.D.L1` and `CDISCARD.D.L1` only target the L1 data cache, and do not impact the pL2 and L3 caches. Flush capability of the pL2 Cache is described in Section 13.3.4. Flush capability of the L3 Cache Controller is described in Section 14.4.9.

3.5 Atomic Memory Operations

The X280 core supports the RISC-V standard Atomic (A) extension on the Memory Port, Peripheral Port, and internal memory regions.

Atomic instructions that target the Memory Port are implemented in the data cache and are not observable on the external data bus. The load-reserved (LR) and store-conditional (SC) instructions are special atomic instructions that are only supported in data cacheable regions. They will generate a precise access exception if targeted at uncacheable data regions.

Atomic memory operations are not supported on the System Port. Atomic operations that target the System Port will generate a precise access exception.

See Section 5.4 for more information on the instructions added by this extension.

3.6 Floating-Point Unit (FPU)

The X280 FPU provides full hardware support for the IEEE 754-2008 floating-point standard for 32-bit single-precision, 64-bit double-precision, and 16-bit half-precision arithmetic. The FPU includes a fully pipelined fused-multiply-add unit and an iterative divide and square-root unit, magnitude comparators, and float-to-integer conversion units, all with full hardware support for subnormals and all IEEE default values.

Section 5.5 describes the 32-bit single-precision instructions. Section 5.6 describes the 64-bit double-precision instructions. Section 5.7 describes the 16-bit half-precision instructions.

The FPU comes up disabled on reset. First initialize `fcsr` and `mstatus.FS` prior to executing any floating-point instructions. In the `freedom-metal` startup code, write `mstatus.FS[1:0]` to `0x1`.

3.7 Vector Unit

The vector unit enables hardware support for the RISC-V V extension, version 1.0-rc1 and the instructions described therein.

Vector operations are supported on 8-, 16-, 32-, and 64-bit integers and fixed-point numbers (i.e., ELEN=64), as well as on IEEE 754 binary16, binary32, and binary64 floating-point numbers. The datapath width, DLEN, is not an architecturally defined parameter, but it specifies the peak compute rate in bits/cycle for the vector unit and is an important attribute.

The V extension also comprises the Zvlssseg and Zvamo extensions, providing segmented loads and stores and atomic memory operations. The Zvediv and Zvqmac extensions are not part of the base RVV extension and are not included.

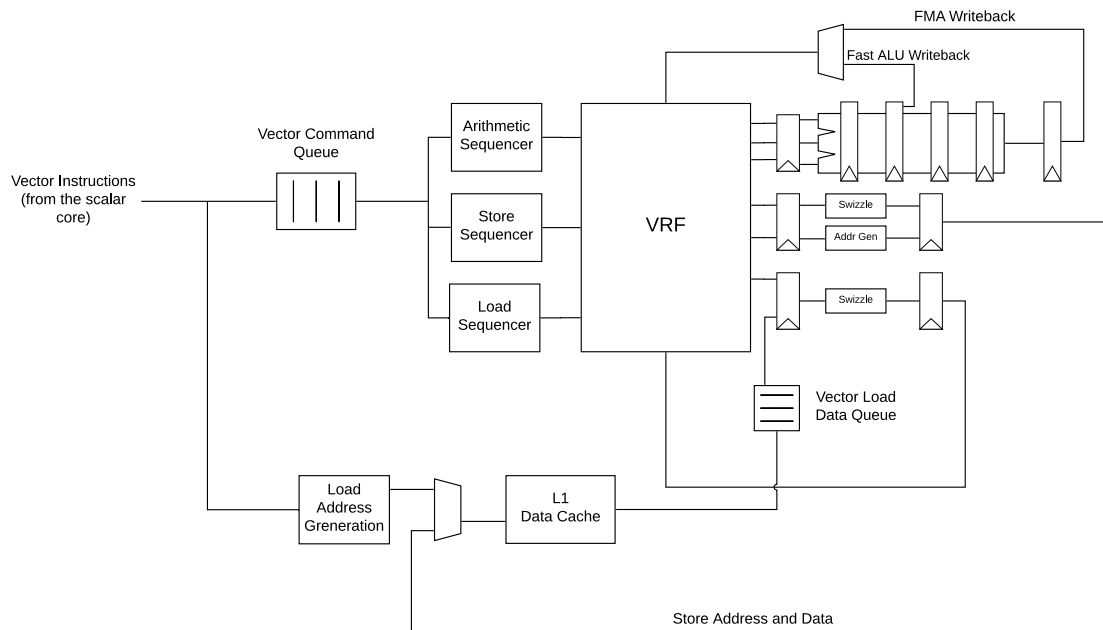


Figure 3: Vector Unit Pipeline

Figure 3 above shows the pipeline for the vector unit. The vector unit is decoupled from the scalar core. First, a vector command queue accepts instructions from the writeback stage of the scalar core and holds them until the vector unit is ready to dequeue them. Then, the vector unit dequeues the instructions from the command queue in-order as soon as the sequencer for that type of instruction is available.

A separate load-address generation unit accepts vector loads from the writeback stage of the scalar core and begins issuing requests to the memory system before the vector load reaches the head of the vector command queue.

The vector unit comprises three sequencers:

1. the load writeback sequencer

2. the store sequencer
3. the arithmetic sequencer (described in Section 3.7.2)

In addition to executing its class of vector instructions, each sequencer is responsible for performing hazard checks and register-file accesses.

3.7.1 Chaining Vector Instructions

Most vector instructions can be chained, so that a younger instruction that depends on an older instruction can begin executing its earlier elements before all elements of the older instruction have been processed. Chaining of WAR hazards (a.k.a. tailgating) and WAW hazards is also supported. Chaining can proceed once the first element of the older instruction has been produced, in accordance with the instruction latencies described in Section 3.7.2, Section 3.7.3 and Section 3.7.4. For example, the chaining latency for simple ALU ops is four cycles, meaning that to completely hide this latency to avoid stalls, back-to-back ALU ops require $vl * SEW / DLEN \geq 4$, or independent instructions need to be scheduled in between.

3.7.2 Arithmetic Sequencer

All vector instructions besides loads, stores, and `vsetvl{i}` execute in the arithmetic sequencer. The arithmetic sequencer has a datapath width `DLEN`.

Most instructions progress at a rate of `DLEN` bits per cycle, and therefore occupy the sequencer for about $vl * SEW / DLEN$ cycles, assuming no hazards. Exceptions include:

- `vrgather` and `vcompress` produce one element per cycle, regardless of `SEW`.
- Division and square-root instructions use a one-bit-at-a-time algorithm, but up to four elements can be processed at a time.
- Most reductions take a total of approx. $vl * SEW / DLEN + 5 * (4 + \log_2(DLEN / SEW))$ cycles.
- Ordered floating-point reductions take a total of approx. $5 * vl$ cycles.
- Narrowing instructions produce $DLEN / 2$ result bits per cycle.
- Widening instructions produce $DLEN * 2$ result bits per cycle.

Simple integer instructions (addition, subtraction, comparison) and floating-point comparisons have four-cycle latency. More complex integer instructions (shifts, multiplications, widening additions), fixed-point instructions, and floating-point instructions have eight-cycle latency.

The number of fixed-point and floating-point operations per cycle can be calculated with the following formula:

$$FLOPs/cycle = \frac{2 \times DLEN}{FloatSize}$$

Note

To hide the latency of a simple arithmetic instruction, it suffices to schedule one independent arithmetic instruction in between the arithmetic instruction that produces the result and the arithmetic instruction that consumes it, or to use LMUL ≥ 2 . To hide the latency of most complex arithmetic instructions, it suffices to schedule three independent arithmetic instructions in between, or to schedule one independent arithmetic instruction in between with LMUL = 2, or to use LMUL ≥ 4 .

3.7.3 Vector Memory System

In the Tenstorrent X280 MC, which has a cache-based memory system, vector memory accesses check the L1 data cache. The L1 data cache satisfies the request on a hit, but on a miss, the access is forwarded to the pL2 cache (i.e., a no-allocate policy).

Although most vector accesses are serviced by the pL2 cache, the vector unit is decoupled so that the pL2 cache latency is usually not exposed. Unit-strided and strided vector loads are decoupled from other vector instructions, and so their memory accesses can usually be issued well in advance of when a dependent instruction needs the data. The extent of decoupling depends on the fullness of the vector command queue. For strip-mined vector loops, this usually means that the pL2 cache latency will be exposed on the first loop iteration, but subsequent loop iterations' loads will be issued early enough that their pL2 cache latency will not be exposed.

- If the data has already been loaded from memory by the time a vector load reaches the head of the command queue, then the observable load latency is 4 cycles.
- Unit-stride loads and stores can operate at the full bandwidth of the memory pipe. There is usually a 1-cycle occupancy increase (regardless of vector length) if the base address is not aligned to the width of the memory port.
- Strided loads and stores operate at one element per cycle and should be scheduled accordingly.
- Indexed loads and stores operate at one element per cycle, and they stall the machine until all addresses have been generated, so they cannot be scheduled.
- Unit-stride segmented loads and stores are effectively converted into strided segment loads and stores.
- Strided segment loads and stores operate at up to one segment per cycle if the segment fits within one aligned memory beat.
- Indexed segment loads and stores operate at the same rate as strided ones, but they stall the machine until all addresses have been generated.

By default, up to 18 outstanding loads and stores are supported. Each outstanding access corresponds to a single naturally aligned DLEN-sized chunk of memory. Unit-strided loads and stores consume up to $(vl*SEW/DLEN + 1)$ outstanding accesses. Strided and indexed accesses consume one outstanding access per element (or more for segments that span a naturally aligned DLEN-sized chunk of memory).

3.7.4 Vector-Scalar Communication

Moving values from the scalar unit to the vector unit via registers is generally efficient. Moving values from the scalar unit to the vector unit via memory is also usually efficient, as long as recent older vector instructions are not accessing the same memory.

Moving values from the vector unit to the scalar unit incurs significant overhead. When executing optimized vector code, the scalar unit is often running far ahead of the vector unit, to the extent that the vector command queue allows. This decoupled design helps hide vector load latency, but it is only effective when the scalar core does not depend on values from the vector unit.

The best-case latency for vector-to-scalar register moves is about 10 cycles. Unfortunately, the best-case latency often does not apply, because the move instruction must first reach the head of the command queue before it can execute: that is, the current occupancy of the command queue affects the latency. It is advisable to schedule the vector-to-scalar move as far in advance of the use of the result as possible.

The best-case latency for vector-store to scalar-load dependences is about 16 cycles. A similar caveat applies: the scalar load cannot proceed until the vector store has completed, so it is advisable to schedule the vector store and scalar load as far apart as possible.

3.8 Virtual Memory Support

The X280 supports virtual memory through the use of a Memory Management Unit (MMU). The MMU supports the Sv48 address mode as described in *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*. SiFive's Sv48 implementation provides a 48-bit virtual address space using 47-bits of physical address space. Supported page sizes include 4 KiB pages, 2 MiB megapages, 1 GiB gigapages and 512 GiB terapages. The default Linux page size (PAGESIZE) is 4 KiB.

The translation lookaside buffers (TLBs) are address translation caches within the MMU. Translation is accomplished through page table entries (PTE) that reside in the TLB region. A hardware page-table walker refills the TLBs upon a miss. The PTE entries are fetched from a region defined by the root page table base address in the Supervisor Address Translation and Protection (satp) CSR. Each PTE contains the information necessary to translate the virtual memory address to a physical address on the design.

There are both level 1 and level 2 TLB entries. Level 1 entries contain separate instruction buffers (ITLB) and data buffers (DTLB) since they are accessed in different pipeline stages. The ITLB and DTLB each contain 64 entries, which are fully associative. Level 2 TLB entries are uni-

fied, and contain 1024 direct-mapped TLB entries. Level 2 TLB are all 4 KiB pages. A block diagram of the instruction and data memory access from the L2 into the MMU TLB is shown below.

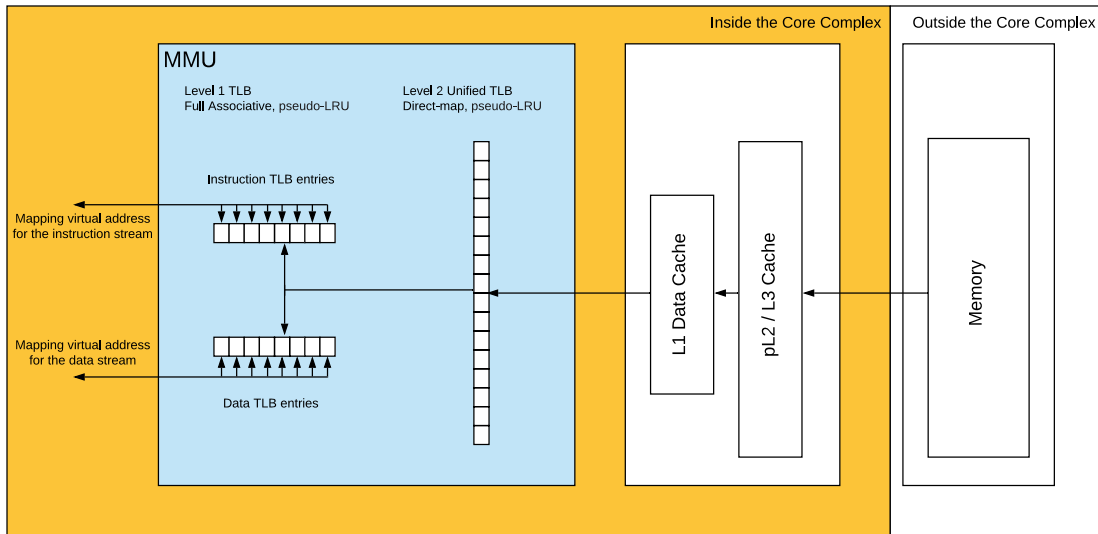


Figure 4: TLB Update Flow

Behaviors of the hardware are described below.

- When there is a TLB miss in the L1 ITLB or DTLB, the L2 unified TLB will populate the L1 TLB with the correct PTE, if it exists
- When there is a miss in both L1 and L2 TLB, a hardware page table walk will occur by the MMU to fill the TLB page table entry from the memory. The memory location where the hart will start fetching TLB page table entry from is determined by the physical page number (PPN) field in the Supervisor Address Translation and Protection (satp) CSR. The refill will occur from the data cache if it exists there, otherwise it will refill from the pL2 or L3 cache. If the pL2 or L3 cache does not contain the data, then it will be fetched from system memory.
- The L1 TLB page table entry replacement policy is pseudo-LRU; the L2 unified TLB page table entry is direct-mapped
- When L1 TLB entry is evicted, this entry is not updated in the L2 unified TLB
- When the L1 TLB entry is updated from L2, the entry will reside in L2 and will not be removed
- Executing the `SFENCE.VMA` instruction will invalidate both L1 and L2 TLB entries

3.8.1 Address and Page Table Formats

An Sv48 virtual address is partitioned as shown below. Note that address bits [63:48] of every instruction fetch, load, and store operation must be equal to bit 47, or else a page-fault exception will occur.

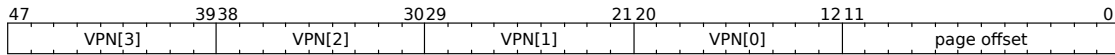


Figure 5: Sv48 Virtual Address

The 36-bit VPN is translated into a 44-bit PPN via a four-level page table, while the 12-bit page offset is untranslated.

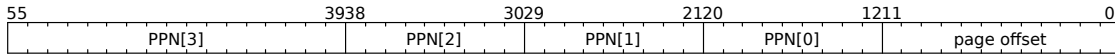


Figure 6: Sv48 Physical Address

Sv48 page tables contain 2^9 page table entries (PTEs), eight bytes each. A page table is exactly the size of a page and must always be aligned to a page boundary. As mentioned, `satp.PPN` holds the physical page number of the root page table. Any level of PTE may be a leaf PTE, and all page sizes (4 KiB, 2 MiB, 1 GiB, and 512 GiB) must be virtually and physically aligned to a boundary equal to its size. A page-fault exception is raised if the physical address is insufficiently aligned.

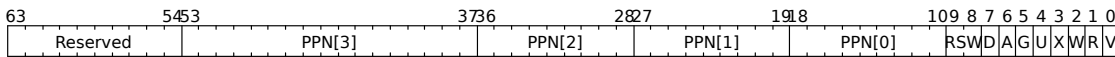


Figure 7: Sv48 PTE Format

A description of the PTE configuration bits can be found in Table 7.

Bits	Description
0	V: Valid 0x0 - Page table entry not valid 0x1 - Page table entry valid
1	R: Readable 0x0 - Page table entry not readable 0x1 - Page table entry readable
2	W: Writable 0x0 - Page table entry not writable 0x1 - Page table entry writable
3	X: Executable 0x0 - Page table entry not executable 0x1 - Page table entry executable
4	U: User mode access 0x0 - No access to user mode software 0x1 - Access granted to user mode software
5	G: Global mapping 0x0 - This mapping does not exist globally 0x1 - This mapping exists globally
6	A: Accessed 0x0 - Leaf page table entry has not been read, written, or fetched since the last time A was cleared 0x1 - Leaf page table entry has been read, written, or fetched since the last time A was cleared
7	D: Dirty 0x0 - The virtual page has not been written since the last time D was cleared 0x1 - The virtual page has been written since the last time D was cleared
[9:8]	RSW: Supervisor software use X - Open for supervisor software use

Table 7: PTE Configuration Bits

3.8.1.1 Page Table Configurations

Read, write, and execute permissions for Sv48 are summarized in Table 8. The value `PTE.V=1` indicates the PTE is valid, while `PTE.V=0` means all other bits in PTE are don't cares, and software can use these freely. The value `PTE.R=1` indicates the page is readable. Likewise, `PTE.W=1` indicates the page is writable, while `PTE.X=1` means the page is executable. When `PTE.V=0`, `PTE.R=0`, and `PTE.W=0`, this indicates the PTE is a pointer to the next level page table,

otherwise it is a leaf PTE. If a page is marked writable, it must also be marked readable. Combinations of `PTE.W=1` and `PTE.R=0` are not currently supported.

X	W	R	Meaning
0	0	0	Pointer to next level of page table
0	0	1	Read-only page
0	1	0	Reserved
0	1	1	Read-write page
1	0	0	Execute-only page
1	0	1	Read-execute page
1	1	0	Reserved
1	1	1	Read-write-execute page

Table 8: PTE Encoding fields

A fetch page-fault exception will occur if an instruction is fetched from a page that does not have execute permissions. A load page-fault exception will occur if a load or load-reserved instruction falls within a page without read permissions. A store page-fault exception will occur if a store, store-conditional, or AMO instruction falls within a page without write permissions.

The value `PTE.U=1` indicates the page is accessible to user mode. Supervisor mode software may also perform loads and stores to a page marked with `PTE.U=1`, but only if `sstatus.SUM=1`. The `sstatus.SUM` bit modifies the privilege of supervisor mode loads and stores to virtual memory. Supervisor mode software may not execute code on any page marked with `PTE.U=1`.

Two schemes to manage the A and D bits are permitted:

- When a virtual page is accessed and the A bit is clear, or is written and the D bit is clear, a page-fault exception is raised
- When a virtual page is accessed and the A bit is clear, or is written and the D bit is clear, the corresponding bit(s) are set in the PTE. The PTE update is atomic with respect to other accesses to the PTE, and memory access will not occur until the PTE update is visible globally.

For non-leaf PTEs, the D, A, and U bits are reserved for future use and must be cleared by software for forward compatibility.

It is important to note the X280 does not automatically set the accessed (A) and dirty (D) bits in a Sv48 Page Table Entry (PTE). Instead, the X280 MMU will raise a page fault exception for a read to a page with `PTE.A=0` or a write to a page with `PTE.D=0`.

3.8.2 Supervisor Address Translation and Protection Register (satp)

The `satp` register is a 64-bit read/write register used to control supervisor address translation and protection.

Supervisor Address Translation and Protection Register (satp)			
CSR	0x180		
Bits	Field Name	Attr.	Description
[43:0]	PPN	WARL	Holds the physical page number of the root page table, which is the supervisor physical address divided by 4 KiB
[59:44]	ASID	WARL	An address space identifier used to facilitate address-translation fences on a per-address-space basis
[63:60]	MODE	WARL	Determines the selected address-translation scheme. • 0x0 - Bare mode, no translation enabled • 0x8 - Page-based 38-bit virtual addressing (Sv39) • 0x9 - Page-based 48-bit virtual addressing (Sv48)

Table 9: Supervisor Address Translation and Protection Register

When `satp.MODE=0x0`, supervisor virtual addresses are equal to supervisor physical addresses, and there is no additional memory protection beyond the physical memory protection scheme described in Section 3.9. In this case, the remaining fields in `satp` have no effect.

Note that writing `satp` does not imply any ordering constraints between page-table updates and subsequent address translations. If the new address space's page tables have been modified, or if an ASID is reused, it may be necessary to execute an `SFENCE.VMA` instruction after writing `satp`, which will:

1. Synchronize page table writes and address translation hardware for higher privilege levels
2. Guarantee previous stores are ordered before all subsequent references from the hart to the memory management data structures
3. Flush L1 and L2 unified TLB entry

Note

Content from Section 3.8.3, Section 3.8.4, Section 3.8.5, and Section 3.8.6 are directly from *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*.

3.8.3 Supervisor Memory-Management Fence Instruction (SFENCE.VMA)

The supervisor memory-management fence instruction `SFENCE.VMA` is used to synchronize updates to in-memory memory-management data structures with current execution.

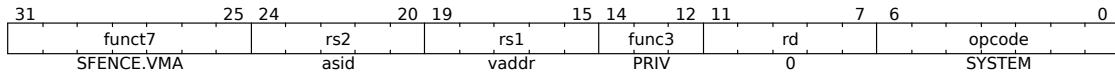


Figure 8: SFENCE.VMA Instruction

Instruction execution causes implicit reads and writes to these data structures; however, these implicit references are ordinarily not ordered with respect to explicit loads and stores. Executing an SFENCE.VMA instruction guarantees that any previous stores already visible to the current RISC-V hart are ordered before all subsequent implicit references from that hart to the memory-management data structures.

The SFENCE.VMA is used to flush any local hardware caches related to address translation. It is specified as a fence rather than a TLB flush to provide cleaner semantics with respect to which instructions are affected by the flush operation and to support a wider variety of dynamic caching structures and memory-management schemes. SFENCE.VMA is also used by higher privilege levels to synchronize page table writes and the address translation hardware.

SFENCE.VMA orders only the local hart's implicit references to the memory-management data structures.

Consequently, other harts must be notified separately when the memory-management data structures have been modified. One approach is to use 1) a local data fence to ensure local writes are visible globally, then 2) an interprocessor interrupt to the other thread, then 3) a local SFENCE.VMA in the interrupt handler of the remote thread, and finally 4) signal back to the originating thread that the operation is complete. This is, of course, the RISC-V analog to a TLB shutdown.

For the common case that the translation data structures have only been modified for a single address mapping (i.e., one page or superpage), rs1 can specify a virtual address within that mapping to affect a translation fence for that mapping only. Furthermore, for the common case that the translation data structures have only been modified for a single address-space identifier, rs2 can specify the address space. The behavior of SFENCE.VMA depends on rs1 and rs2 as follows:

- If rs1=x0 and rs2=x0, the fence orders all reads and writes made to any level of the page tables, for all address spaces
- If rs1=x0 and rs2≠x0, the fence orders all reads and writes made to any level of the page tables, but only for the address space identified by integer register rs2. Accesses to global mappings are not ordered.
- If rs1≠x0 and rs2=x0, the fence orders only reads and writes made to the leaf page table entry corresponding to the virtual address in rs1, for all address spaces
- If rs1≠x0 and rs2≠x0, the fence orders only reads and writes made to the leaf page table entry corresponding to the virtual address in rs1, for the address space identified by integer register rs2. Accesses to global mappings are not ordered.

When rs2≠x0, bits [SXLEN-1:ASIDMAX] of the value held in rs2 are reserved. Furthermore, if ASIDLEN<ASIDMAX, bits [ASIDMAX-1:ASIDLEN] of the value held in rs2 are ignored.

3.8.4 Scenarios that Require SFENCE.VMA Instruction

The following common situations typically require executing an SFENCE.VMA instruction:

- When software recycles an ASID (i.e., reassociates it with a different page table), it should first change satp to point to the new page table using the recycled ASID, then execute SFENCE.VMA with rs1=x0 and rs2 set to the recycled ASID. Alternatively, software can execute the same SFENCE.VMA instruction while a different ASID is loaded into satp, provided the next time satp is loaded with the recycled ASID, it is simultaneously loaded with the new page table.
- If the implementation does not provide ASIDs, or software chooses to always use ASID 0, then after every satp write, software should execute SFENCE.VMA with rs1=x0. In the common case that no global translations have been modified, rs2 should be set to a register other than x0 but which contains the value zero, so that global translations are not flushed.
- If software modifies a non-leaf PTE, it should execute SFENCE.VMA with rs1=x0. If any PTE along the traversal path had its G bit set, rs2 must be x0; otherwise, rs2 should be set to the ASID for which the translation is being modified.
- If software modifies a leaf PTE, it should execute SFENCE.VMA with rs1 set to a virtual address within the page. If any PTE along the traversal path had its G bit set, rs2 must be x0; otherwise, rs2 should be set to the ASID for which the translation is being modified.
- For the special cases of increasing the permissions on a leaf PTE and changing an invalid PTE to a valid leaf, software may choose to execute the SFENCE.VMA lazily. After modifying the PTE but before executing SFENCE.VMA, either the new or old permissions will be used. In the latter case, a page fault exception might occur, at which point software should execute SFENCE.VMA in accordance with the previous bullet point.

3.8.4.1 Speculation

The X280 will perform a speculative data access as a result of speculative ITLB refill. Changes in the satp register do not necessarily flush TLB entries. It is required to execute an SFENCE.VMA instruction after modifying page table entries in order to flush the cached translations. Exceptions only occur on accesses that are generated as a result of instruction execution, not access that are done speculatively.

3.8.4.2 ASID Usage for Supervisor Software

Supervisor software that uses ASIDs should use a nonzero ASID value to refer to the same address space across all harts in the supervisor execution environment (SEE) and should not use an ASID value of 0. If supervisor software does not use ASIDs, then the ASID field in the satp CSR should be set to 0.

3.8.5 Trap Virtual Memory

The `mstatus.TVM` (Trap Virtual Memory) bit supports intercepting supervisor virtual-memory management operations. When `TVM=1`, attempts to read or write the `satp` CSR or execute the `SFENCE.VMA` instruction while executing in S-mode will raise an illegal instruction exception. When `TVM=0`, these operations are permitted in supervisor mode. `TVM` is hard-wired to 0 when supervisor mode is not supported. The `TVM` mechanism improves virtualization efficiency by permitting guest operating systems to execute in supervisor mode, rather than classically virtualizing them in user mode. This approach obviates the need to trap accesses to most S-mode CSRs. Trapping `satp` accesses and the `SFENCE.VMA` instruction provides the hooks necessary to lazily populate shadow page tables.

3.8.6 Virtual Address Translation Process

For Sv48, `LEVELS` equals 4, and `PTESIZE` equals 8 in the steps below. A virtual address (`va`) is translated into a physical address (`pa`) as follows:

1. Let `a` be `satp.ppn × PAGESIZE`, and let `i = LEVELS - 1`.
2. Let `pte` be the value of the PTE at address `a + va.vpn[i] × PTESIZE`. If accessing `pte` violates a PMA or PMP check, raise an access exception corresponding to the original access type.
3. If `pte.v = 0`, or if `pte.r = 0` and `pte.w = 1`, stop and raise a page-fault exception corresponding to the original access type
4. Otherwise, the PTE is valid. If `pte.r = 1` or `pte.x = 1`, go to Step 5. Otherwise, this PTE is a pointer to the next level of the page table. Let `i = i - 1`. If `i < 0`, stop and raise a page-fault exception corresponding to the original access type. Otherwise, let `a = pte.ppn × PAGESIZE` and go to Step 2.
5. A leaf PTE has been found. Determine if the requested memory access is allowed by the `pte.r`, `pte.w`, `pte.x`, and `pte.u` bits, given the current privilege mode and the value of the `SUM` and `MXR` fields of the `mstatus` register. If not, stop and raise a page-fault exception corresponding to the original access type.
6. If `i > 0` and `pte.ppn[i-1:0] ≠ 0`, this is a misaligned superpage; stop and raise a page-fault exception corresponding to the original access type
7. If `pte.a = 0`, or if the memory access is a store and `pte.d = 0`, either raise a page-fault exception corresponding to the original access type, or:
 - a. Set `pte.a` to 1 and, if the memory access is a store, also set `pte.d` to 1
 - b. If this access violates a PMA or PMP check, raise an access exception corresponding to the original access type.
 - c. This update and the loading of `pte` in Step 2 must be atomic; in particular, no intervening store to the PTE may be perceived to have occurred in-between
8. The translation is successful. The translated physical address is given as follows:

- a. $pa.pgoff = va.pgoff$
- b. If $i > 0$, then this is a superpage translation and $pa.ppn[i-1:0] = va.vpn[i-1:0]$
- c. $pa.ppn[LEVELS-1:i] = pte.ppn[LEVELS-1:i]$

3.8.7 Virtual-to-Physical Address Mapping Example

The following figure is a high-level view of how a virtual address is mapped to a physical address for a Linux application. When the Linux kernel creates a process, it will allocate multiple pages of physical memory to store the code and data. TLB MMU is used to:

- Translate the virtual addresses to physical addresses
- Provide uniform virtual memory layout for a user application
- Protect user applications unauthorized access to other address space

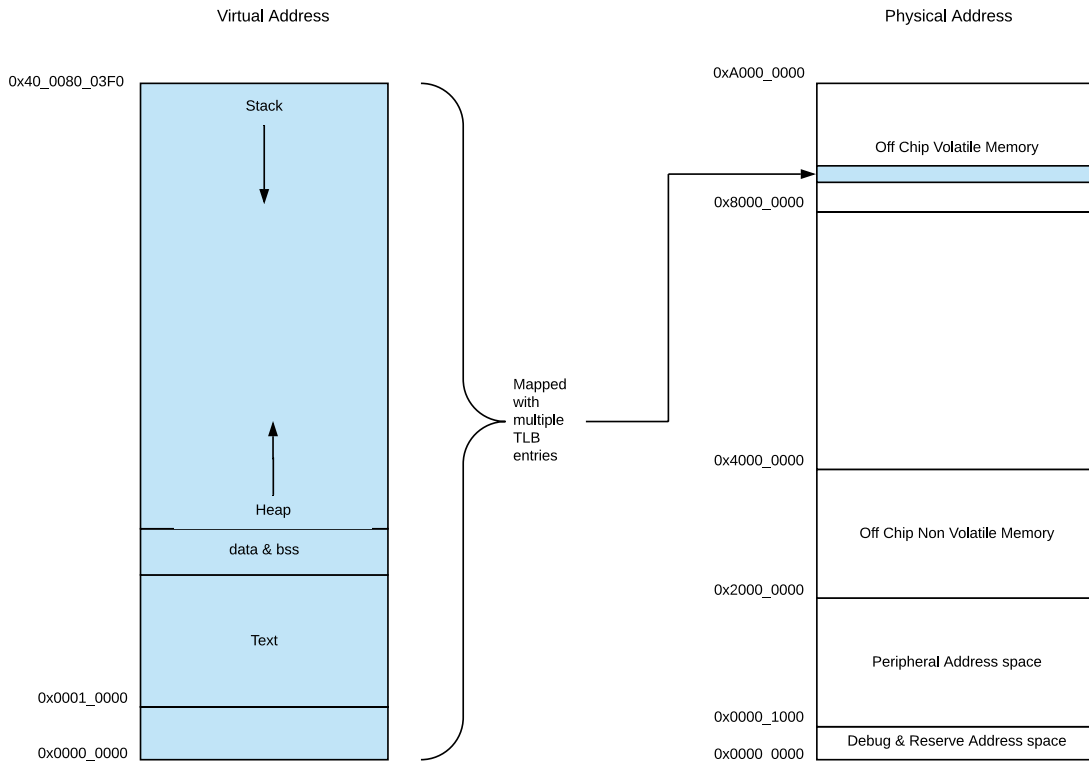


Figure 9: Linux User Application Memory Map Example

In this example, code beginning at virtual address `0x0001_0000` needs to be mapped to an address in off-chip volatile memory.

When the hart tries to execute instructions at this address, it needs to use a matched TLB page table entry to perform a virtual address to physical address translation. If it cannot be located in the L1 instruction TLB, or the L2 unified TLB, the hart will start hardware table walk from the

TLB page table base address. The page table base address is obtained by multiplying `satp.PPN` by the L2 PAGESIZE (4 KiB).

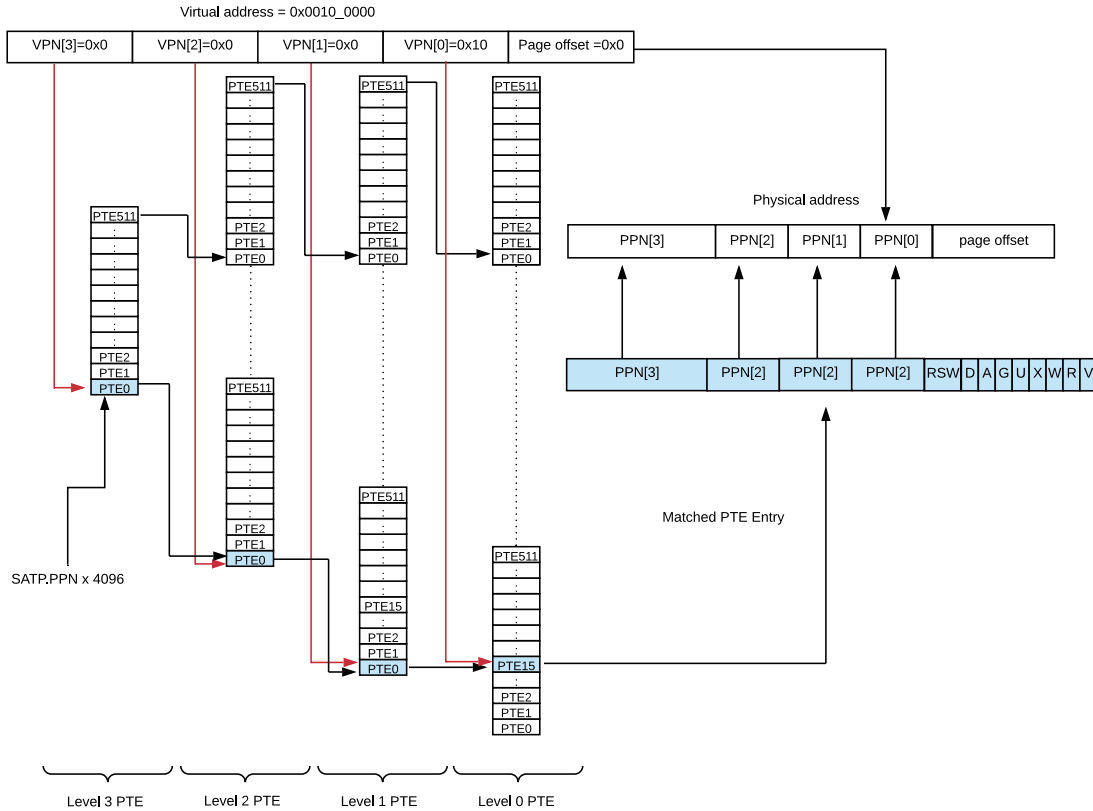


Figure 10: Hardware Table Walk Example

The TLB MMU will execute a page table walk in order to determine the correct mapping for a particular virtual address. Page table entries are pointers to the next level page table if the page is marked as not Readable ($R=0$), not Writable ($W=0$), and not Executable ($X=0$). Otherwise, it is a leaf PTE.

In this example, there are 4 levels of page table entries. The hart will start the hardware table walk from the level 3 page table entry. In the Sv48 scheme, there are 512 page table entries in level 3 page table entry. A hart can quickly locate the entry using `VPN[3]` number, in this case, entry 0. The hart will continue the hardware table walk to the L2 page table entry when the entry doesn't match.

There are 512 clusters of page table entries in L2 and each cluster also has 512 TLB page table entries. In this example, PPNs in the level 3 entry 0 is used to locate the right cluster in the L2 page entry. The hart will locate the entry using `VPN[2]` number, in this case, entry 0. The hart will continue a hardware table walk to L1 page table entry when this entry does not match.

There are 512×512 clusters of page table entries in L1 and each cluster also has 512 TLB page table entries. In this example, PPNs in the L2 entry 0 is used to locate the right cluster in the L1

page entry. The hart will locate the entry using `VPN[1]` number, in this case, entry 0. The hart will continue a hardware table walk to level 0 page table entry when this entry does not match.

At the level 0 page table entry, there are $512 \times 512 \times 512$ clusters of page table entries, and each cluster has 512 TLB page table entries. In this example, PPNs in the L1 entry 0 is used to locate the correct cluster in the level 0 page entry. The hart then finds the entry using the `VPN[0]` value, which in this case, corresponds to entry 15.

When there is a match in level 0 page table entry, the virtual address will map to a physical address. The physical page number is combined with the page offset to give the complete physical address.

3.8.8 MMU at Reset

The TLB MMU is disabled by default out of reset. All accessed regions have a 1:1 virtual-to-physical address mapping when the MMU is disabled. If the PMP is not yet enabled, all access permissions out of reset are determined by the static PMA values.

3.9 Physical Memory Protection (PMP)

Machine mode is the highest privilege level and by default has read, write, and execute permissions across the entire memory map of the device. However, privilege levels below machine mode do not have read, write, or execute permissions to any region of the device memory map unless it is specifically allowed by the PMP. For the lower privilege levels, the PMP may grant permissions to specific regions of the device's memory map, but it can also revoke permissions when in machine mode.

When programmed accordingly, the PMP will check every access when the hart is operating in supervisor or user modes. For machine mode, PMP checks do not occur unless the lock bit (L) is set in the `pmpcfgY` CSR for a particular region.

PMP checks also occur on loads and stores when the machine previous privilege level is supervisor or user (`mstatus.MPP=0x1` or `mstatus.MPP=0x0`), and the Modify Privilege bit is set (`mstatus.MPRV=1`). For virtual address translation, PMP checks are also applied to page table accesses in supervisor mode.

The X280 PMP supports 8 regions with a minimum region size of 4096 bytes.

This section describes how PMP concepts in the RISC-V architecture apply to the X280. For additional information on the PMP refer to *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*.

3.9.1 PMP Functional Description

The X280 PMP unit has 8 regions and a minimum granularity of 4096 bytes. Access to each region is controlled by an 8-bit `pmpXcfg` field and a corresponding `pmpaddrX` register. Overlap-

ping regions are permitted, where the lower numbered `pmpXcfg` and `pmpaddrX` registers take priority over higher numbered regions. The X280 PMP unit implements the architecturally defined `pmpcfgY` CSR `pmpcfg0`, supporting 8 regions. `pmpcfg2` is implemented, but hardwired to zero. Access to `pmpcfg1` or `pmpcfg3` results in an illegal instruction exception.

The PMP registers may only be programmed in M-mode. Ordinarily, the PMP unit enforces permissions on S-mode and U-mode accesses. However, locked regions (see Section 3.9.2) additionally enforce their permissions on M-mode.

3.9.2 PMP Region Locking

The PMP allows for region locking whereby, once a region is locked, further writes to the configuration and address registers are ignored. Locked PMP entries may only be unlocked with a system reset. A region may be locked by setting the L bit in the `pmpXcfg` register.

In addition to locking the PMP entry, the L bit indicates whether the R/W/X permissions are enforced on machine mode accesses. When the L bit is clear, the R/W/X permissions apply to S-mode and U-mode.

3.9.3 PMP Registers

Each PMP region is described by an 8-bit `pmpXcfg` field, used in association with a 64-bit `pmpaddrX` register that holds the base address of the protected region. The range of each region depends on the Addressing (A) mode described in the next section. The `pmpXcfg` fields reside within 64-bit `pmpcfgY` CSRs.

Each 8-bit `pmpXcfg` field includes a read, write, and execute bit, plus a two bit address-matching field A, and a Lock bit, L. Overlapping regions are permitted, where the lowest numbered PMP entry wins for that region.

3.9.3.1 PMP Configuration Registers

For RV64 architectures, `pmpcfg1` and `pmpcfg3` are not implemented. This reduces the footprint since `pmpcfg2` already contains configuration fields `pmp8cfg` through `pmp11cfg` for both RV32 and RV64.

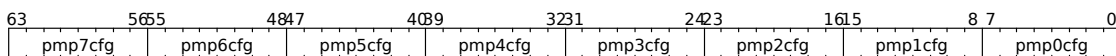


Figure 11: RV64 `pmpcfg0` Register

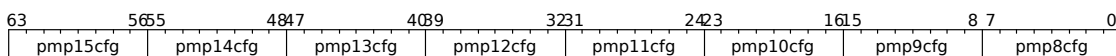


Figure 12: RV64 `pmpcfg2` Register

The `pmpcfgY` and `pmpaddrX` registers are only accessible via CSR specific instructions such as `csrr` for reads, and `csrw` for writes.

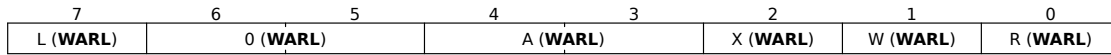


Figure 13: RV64 pmpXcfg bitfield

Bits	Description
0	R: Read Permissions 0x0 - No read permissions for this region 0x1 - Read permission granted for this region
1	W: Write Permissions 0x0 - No write permissions for this region 0x1 - Write permission granted for this region
2	X: Execute permissions 0x0 - No execute permissions for this region 0x1 - Execute permission granted for this region
[4:3]	A: Address matching mode 0x0 - PMP Entry disabled. No PMP protection applied for any privilege level. 0x1 - Top of range (TOR) region defined by two adjacent pmpaddr registers. The upper limit of region X is defined by pmpaddrX, and the base of the region is defined by pmpaddr(X-1). Address 'a' matches the region if $[pmpaddr(X-1) \leq a < pmpaddrX]$. If pmp0cfg defines a TOR region, then the base address of that region is 0x0, and pmpaddr0 defines the upper limit. Supports only a four byte granularity. 0x2 - Reserved 0x3 - Naturally aligned power-of-two region (NAPOT), ≥ 8 bytes. When this setting is programmed, the low bits of the pmpaddrX register encode the size, while the upper bits encode the base address right shifted by two. There is a zero bit in between, we will refer to as the least significant zero bit (LSZB).
7	L: Lock Bit 0x0 - PMP Entry Unlocked, no permission restrictions applied to machine mode. PMP entry only applies to S and U modes. 0x1 - PMP Entry Locked, permissions enforced for all privilege levels including machine mode. Writes to pmpXcfg and pmpcfigY are ignored and can only be cleared with system reset.
Note: The combination of R=0 and W=1 is not currently implemented.	

Table 10: pmpXcfg Bitfield Description

Out of reset, the PMP register fields A and L are set to 0. All other hart state is unspecified by *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*.

Some examples follow using NAPOT address mode.

Base Address	Region Size*	LSZB Position	pmpaddrX Value
0x4000_0000	8 B	0	(0x1000_0000 1'b0)
0x4000_0000	32 B	2	(0x1000_0000 3'b011)
0x4000_0000	4 KB	9	(0x1000_0000 10'b01_1111_1111)
0x4000_0000	64 KB	13	(0x1000_0000 14'b01_1111_1111_1111)
0x4000_0000	1 MB	17	(0x1000_0000 18'b01_1111_1111_1111_1111)
*Region size is $2^{(LSZB+3)}$.			

Table 11: pmpaddrX Encoding Examples for A=NAPOT

3.9.3.2 PMP Address Registers

The PMP has 8 address registers. Each address register pmpaddrX correlates to the respective pmpXcfg field. Each address register contains the base address of the protected region right shifted by two, for a minimum 4-byte alignment.

The maximum encoded address bits per *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11* are [55:2].

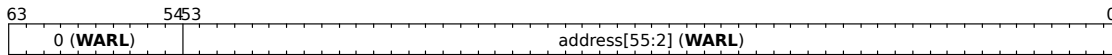


Figure 14: RV64 pmpaddrX Register

3.9.4 PMP and PMA

The PMP values are used in conjunction with the Physical Memory Attributes (PMAs) described in Section 4.1. Since the PMAs are static and not configurable, the PMP can only revoke read, write, or execute permissions to the PMA regions if those permissions already apply statically.

3.9.5 PMP Programming Overview

The PMP registers can only be programmed in machine mode. The pmpaddrX register should be first programmed with the base address of the protected region, right shifted by two. Then, the pmpcfgY register should be programmed with the properly configured 64-bit value containing each properly aligned 8-bit pmpXcfg field. Fields that are not used can be simply written to 0, marking them unused.

3.9.5.1 PMP Programming Example

The following example shows a machine mode only configuration where PMP permissions are applied to three regions of interest, and a fourth region covers the remaining memory map. Recall that lower numbered pmpXcfg and pmpaddrX registers take priority over higher numbered regions. This rule allows higher numbered PMP registers to have blanket coverage over the entire memory map while allowing lower numbered regions to apply permissions to specific regions of interest. The following example shows a 64 KB Flash region at base address 0x0, a

32 KB RAM region at base address 0x2000_0000, and finally a 4 KB peripheral region at base address base 0x3000_0000. The rest of the memory map is reserved space.

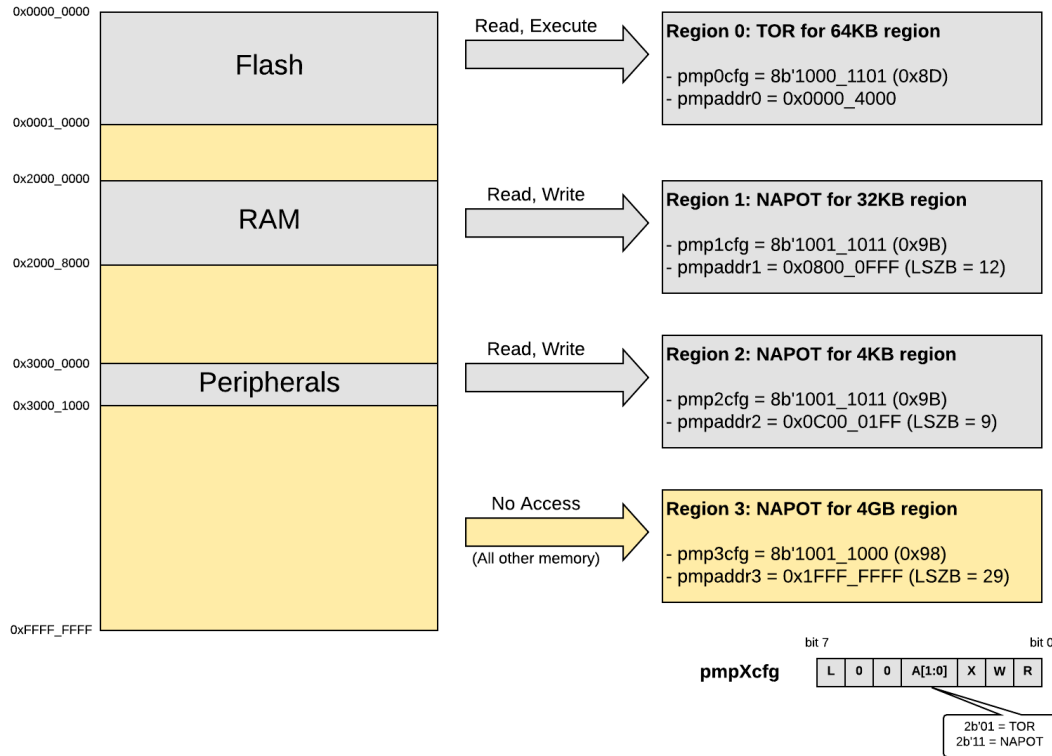


Figure 15: PMP Example Block Diagram

3.9.5.2 PMP Access Scenarios

The L, R, W, and X bits only determine if an access succeeds if all bytes of that access are covered by that PMP entry. For example, if a PMP entry is configured to match the four-byte range 0xC–0xF, then an 8-byte access to the range 0x8–0xF will fail, assuming that PMP entry is the highest-priority entry that matches those addresses.

While operating in machine mode when the lock bit is clear (L=0), if a PMP entry matches all bytes of an access, the access succeeds. If the lock bit is set (L=1) while in machine mode, then the access depends on the permissions set for that region. Similarly, while in Supervisor mode or User mode, the access depends on permissions set for that region.

Failed read or write accesses generate a load or store access exception, and an instruction access fault would occur on a failed instruction fetch. When an exception occurs while attempting to execute from a region without execute permissions, the fault occurs on the fetch and not the branch, so the mepc CSR will reflect the value of the targeted protected region, and not the address of the branch.

It is possible for a single instruction to generate multiple accesses, which may not be mutually atomic. If at least one access generated by an instruction fails, then an exception will occur. It might be possible that other accesses from a single instruction will succeed, with visible side effects. For example, references to virtual memory may be decomposed into multiple accesses.

On some implementations, misaligned loads, stores, and instruction fetches may also be decomposed into multiple accesses, some of which may succeed before an access exception occurs. In particular, a portion of a misaligned store that passes the PMP check may become visible, even if another portion fails the PMP check. The same behavior may manifest for floating-point stores wider than XLEN bits (e.g., the FSD instruction in RV32D), even when the store address is naturally aligned.

3.9.6 PMP and Paging

The Physical Memory Protection mechanism is designed to compose with the page-based virtual memory systems described in *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*. When paging is enabled, instructions that access virtual memory may result in multiple physical-memory accesses, including implicit references to the page tables. The PMP checks apply to all of these accesses. The effective privilege mode for implicit page-table accesses is supervisor mode.

Implementations with virtual memory are permitted to perform address translations speculatively and earlier than required by an explicit virtual-memory access. The PMP settings for the resulting physical address may be checked at any point between the address translation and the explicit virtual-memory access. A mis-predicted branch to a non-executable address range does not generate a trap. Hence, when the PMP settings are modified in a manner that affects either the physical memory that holds the page tables or the physical memory to which the page tables point, M-mode software must synchronize the PMP settings with the virtual memory system. This is accomplished by executing an SFENCE.VMA instruction with `rs1=x0` and `rs2=x0`, after the PMP CSRs are written.

If page-based virtual memory is not implemented, or when it is disabled, memory accesses check the PMP settings synchronously, so no fence is needed.

3.9.7 PMP Limitations

In a system containing multiple harts, each hart has its own PMP device. The PMP permissions on a hart cannot be applied to accesses from other harts in a multi-hart system. In addition, SiFive designs may contain a Front Port to allow external bus masters access to the full memory map of the system. The PMP cannot prevent access from external bus masters on the Front Port.

3.9.8 Behavior for Regions without PMP Protection

If a non-reserved region of the memory map does not have PMP permissions applied, then by default, supervisor or user mode accesses will fail, while machine mode access will be allowed.

Access to reserved regions within a device's memory map (an interrupt controller for example) will return 0x0 on reads, and writes will be ignored. Access to reserved regions outside of a device's memory map without PMP protection will result in a bus error. The bus error can generate an interrupt to the hart using the Bus-Error Unit (BEU). See Chapter 11 for more information.

3.9.9 Cache Flush Behavior on PMP Protected Region

When a line is brought into cache and the PMP is set up with the lock (L) bit asserted to protect a part of that line, a data cache flush instruction will generate a store access fault exception if the flush includes any part of the line that is protected. The cache flush instruction does an invalidate and write-back, so it is essentially trying to write back to the memory location that is protected. If a cache flush occurs on a part of the line that was not protected, the flush will succeed and not generate an exception. If a data cache flush is required without a write-back, use the cache discard instruction instead, as this will invalidate but not write back the line.

3.10 Hardware Performance Monitor

The X280 processor core supports a basic hardware performance monitoring (HPM) facility. The performance monitoring facility is divided into two classes of counters: fixed-function and event-programmable counters. These classes consist of a set of fixed counters and their counter-enable registers, as well as a set of event-programmable counters and their event selector registers. The registers are available to control the behavior of the counters. Performance monitoring can be useful for multiple purposes, from optimization to debug.

The X280 also supports the Sscopmf extension. This extension expands the hardware performance monitor capability of the X280.

3.10.1 Performance Monitoring Counters Reset Behavior

The `instret` and `cycle` counters are initialized to zero on system reset. The hardware performance monitor event counters are not initialized on system reset, and thus have an arbitrary value. Users can write desired values to the counter control and status registers (CSRs) to start counting at a given, known value.

3.10.2 Fixed-Function Performance Monitoring Counters

A fixed-function performance monitor counter is hardware wired to only count one specific event type. That is, they cannot be reconfigured with respect to the event type(s) they count. The only modification to the fixed-function performance monitoring counters that can be done is to enable or disable counting, and write the counter value itself.

The X280 processor core contains two fixed-function performance monitoring counters.

3.10.2.1 Fixed-Function Cycle Counter (`mcycle`)

The fixed-function performance monitoring counter `mcycle` holds a count of the number of clock cycles the hart has executed since some arbitrary time in the past. The `mcycle` counter is read-write and 64 bits wide. Reads of `mcycle` return all 64 bits of the `mcycle` CSR.

3.10.2.2 Fixed-Function Instructions-Retired Counter (`minstret`)

The fixed-function performance monitoring counter `minstret` holds a count of the number of instructions the hart has retired since some arbitrary time in the past. The `minstret` counter is read-write and 64 bits wide. Reads of `minstret` return all 64 bits of the `minstret` CSR.

3.10.3 Event-Programmable Performance Monitoring Counters

Complementing the fixed-function counters are a set of programmable event counters. The X280 HPM includes four additional event counters, `mhpmpcounter3`, `mhpmpcounter4`, `mhpmpcounter5`, and `mhpmpcounter6`. These programmable event counters are read-write and 64 bits wide. The hardware counters themselves are implemented as 40-bit counters on the X280 core series. These hardware counters can be written to in order to initialize the counter value.

3.10.4 Event Selector Registers

To control the event type to count, event selector CSRs `mhpmevent3`, `mhpmevent4`, `mhpmevent5`, and `mhpmevent6` are used to program the corresponding event counters. These event selector CSRs are 64-bit **WARL** registers.

The event selectors are partitioned into three fields; the lower 8 bits select an event class, a middle set of bits that form a mask of events in that class, with the upper 8-bits used for counter overflow and event filtering.

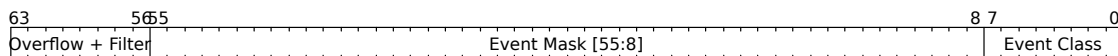


Figure 16: Event Selector Fields

The counter increments if the event corresponding to any set mask bit occurs. For example, if `mhpmevent3` is set to `0x4200`, then `mhpmpcounter3` will increment when either a load instruction or a conditional branch instruction retires. An event selector of 0 means "count nothing".

3.10.4.1 Counter Overflow and Event Filters

The upper 8-bits of the `mhpmevent` register are used for controlling a counter overflow interrupt, as well as mode-based event filtering. The bit layout is shown below:

Machine Hardware Performance Monitor Event Selection Register			
Bits	Name	Attr	Description
[7:0]	Class	WARL	Selects the Event Class to make available for counting
[55:8]	EventSel	WARL	Bit-mask of Event(s) to count
[57:56]	Reserved		
58	VUINH	WARL	Reserved
59	VSINH	WARL	Reserved
60	UINH	WARL	If set, counting of events in U-mode is inhibited
61	SINH	WARL	If set, counting of events in S-mode is inhibited
62	MINH	RW	If set, then counting of events in M-mode is inhibited
63	OF	RW	Overflow status and interrupt disable bit. Set by hardware when counter overflows.

Table 12: mhpmeventX Register Bit Layout

3.10.4.2 Performance Counter Overflow Operation

Each of the five xINH bits inhibits counting of event when the core is in privilege mode x. All zeroes in these fields results in counting of events in all modes.

The OF bit is set by hardware when the corresponding hpmcounterX overflows and remains set until written by software. Since hpmcounter values are unsigned, overflow is defined as incrementing the implemented lower 40-bits of the 64-bit register from all-ones to all-zeroes. Note that there is no loss of information after an overflow since the counter wraps around and keeps counting while the sticky OF bit remains set.

If an hpmcounter overflows while the associated OF bit is zero, then a *count overflow interrupt request* is generated. If an hpmcounter overflows while the associated OF bit is one, then no interrupt is generated. Consequently, the OF bit also functions as a count overflow interrupt disable for the associated hpmcounter.

Count overflow never results from writes to the mhpmcouterX or mhpmeventX registers. Overflow occurs only as a result of an event causing a counter to increment.

The counter overflow interrupt is a standard local interrupt that corresponds to bit 13 in the mip and mie registers. This interrupt is shared by up to 30 counters. The mip LCOFIP bit is the interrupt-pending bit for this interrupt and the mie LCOFIE bit is the interrupt-enable bit for this interrupt. LCOFI represents Local Count Overflow Interrupt. If S-mode is present, sip and sie include the corresponding bits for supervisor interrupt control and status.

Generation of a count overflow interrupt request by an hpmcounter sets the LCOFIP bit in the mip register and sets the associated OF bit. If S-mode is present, the mideleg register controls the delegation of this interrupt to S-mode, which sets the LCOFIP bit in the sip register. The LCOFIP bit is cleared by software after servicing the count overflow interrupt.

Multiple simultaneous interrupts destined for the same privilege mode are handled in the following decreasing priority order: MEI, MSI, MTI, SEI, SSI, STI, LCOFI.

Note that there are not separate overflow status and overflow interrupt enable bits. In practice, enabling overflow interrupt generation by clearing the OF bit is done in conjunction with initializing the counter to a starting value. Once a counter has overflowed, it and the OF bit must be reinitialized before another overflow interrupt can be generated. In other words, after an overflow has occurred, the OF bit is set and OF must be reset to zero.

Software can distinguish newly-overflowed counters which have yet to be serviced by an overflow interrupt handler from overflowed counters that have already been serviced (or that are configured not to generate an interrupt on overflow by maintaining a bit mask reflecting which counters are active and due to overflow eventually).

3.10.4.3 The Scountovf CSR

The `scountovf` CSR is a 32-bit read-only register that contains shadow copies of the OF bits in the 29 `mhpmevent` CSRs. Bit `X` in `scountovf` corresponds to the OF bit in `mhpmeventX`. This register enables supervisor-level overflow interrupt handler software to quickly determine which counter(s) have overflowed without needing to make an environment call to M-mode. The CSR address is `0xDA0`.

Read access to `scountovf` bit `X` is subject to the same `mcounteren` CSRs that mediate access to `hpmcounterX` by S-mode software. In S-mode, `scountovf` bit `X` is readable when `mcounteren` bit `X` is set and otherwise reads as zero.

3.10.4.4 Disabling Counters in Debug Mode

When set, the `dcsr.stopcount` bit stops counters while in debug mode. This is especially important for `mcycle` and `minstret` counters, since in debug mode, the core is executing ROM instructions in a tight loop. The Freedom Studio Performance Monitor View automatically sets the `dcsr.stopcount` bit.

3.10.5 Event Selector Encodings

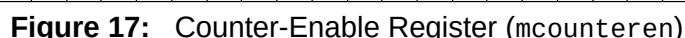
Table 13 describes the event selector encodings available. Events are categorized into classes based on the Event Class field encoded in `mhpmeventX[7:0]`. One or more events can be programmed by setting the respective Event Mask bit for a given event class. An event mask of zero means "count nothing". Multiple events bits set that match will cause the counter to increment any time any of the selected events occur.

Machine Hardware Performance Monitor Event Selection Register	
Instruction Commit Events , mhpmeventX[7:0]=Event Class=0x0	
Event Mask Bits	Description
8	Exception taken
9	Integer load instruction retired
10	Integer store instruction retired
11	Atomic memory operation retired
12	System instruction retired
13	Integer arithmetic instruction retired
14	Conditional branch retired
15	JAL instruction retired
16	JALR instruction retired
17	Integer multiplication instruction retired
18	Integer division instruction retired
19	Floating-point load instruction retired
20	Floating-point store instruction retired
21	Floating-point addition retired
22	Floating-point multiplication retired
23	Floating-point fused multiply-add retired
24	Floating-point division or square-root retired
25	Other floating-point instruction retired
Microarchitectural Events , mhpmeventX[7:0]=Event Class=0x1	
Event Mask Bits	Description
8	Address-generation interlock
9	Long-latency interlock
10	CSR read interlock
11	Instruction cache/ITIM busy
12	Data cache/DTIM busy
13	Branch direction misprediction
14	Branch/jump target misprediction
15	Pipeline flush from CSR write
16	Pipeline flush from other event
17	Integer multiplication interlock
18	Floating-point interlock
Memory System Events , mhpmeventX[7:0]=Event Class=0x2	
Event Mask Bits	Description
8	Instruction cache miss
9	Data cache miss or memory-mapped I/O access
10	Data cache write-back
11	Instruction TLB miss
12	Data TLB miss
13	UTLB miss

Table 13: mhpmeventN Register

3.10.5.1 Combining Events

3.10.6 Counter-Enable Registers



The settings in these registers only control accessibility. The act of reading or writing these enable registers does not affect the underlying counters, which continue to increment when not accessible. For example, this means that the shpmcounter CSR cannot be accessed in S-mode unless enabled xcounteren of a higher privilege level.

When any bit in the mcounteren register is clear, attempts to read the cycle, time, instruction retire, or hpmcounterX register while executing in S-mode will cause an illegal instruction exception. When one of these bits is set, access to the corresponding register is permitted in the next implemented privilege mode, S-mode.

The same bit positions in the `scounteren` register analogously control access to these registers while executing in U-mode. If S-mode is permitted to access a counter register and the corresponding bit is set in `scounteren`, then U-mode is also permitted to access that register.

mcounteren and scounteren are **WARL** registers. Any of the bits may contain a hardwired value of zero, indicating reads to the corresponding counter will cause an illegal instruction exception when executing in a less-privileged mode.

3.11 Ports

This section describes the Port interfaces to the X280 core.

3.11.1 Front Port

The Front Port can be used by external masters to read from and write into the memory system utilizing any port in the Core Complex.

If a Front Port access targets the Memory Port, a coherency manager is responsible for maintaining coherency with the L1, pL2, and L3 caches. A read access can be returned directly from the caches without generating an external bus access. If a write from the Front Port targets a location in the L1 data cache or pL2 cache, it results in the line being evicted and invalidated. The write will then allocate to the L3 cache.

Any Front Port access that targets the Memory Port and results in an L1 and pL2 cache miss will allocate to the L3 cache.

The Tenstorrent X280 MC User Guide describes the implementation details of the Front Port.

3.11.2 Memory Port

The Memory Port is used to interface with memory that offers the highest performance for the Tenstorrent X280 MC, such as DDR. It supports cacheable accesses for data and instructions.

Consult Section 4.1 for further information about the Memory Port and its Physical Memory Attributes.

See the Tenstorrent X280 MC User Guide for a description of the Memory Port implementation in the Tenstorrent X280 MC.

3.11.3 Peripheral Port

The Peripheral Port is used to interface with lower speed peripherals and also supports code execution. When a device is attached to the Peripheral Port, it is expected that there are no other masters connected to that device.

Consult Section 4.1 for further information about the Peripheral Port and its Physical Memory Attributes.

See the Tenstorrent X280 MC User Guide for a description of the Peripheral Port implementation in the Tenstorrent X280 MC.

3.11.4 System Port

The System Port is used to interface with lower performance memory, like SRAM, memory-mapped I/O (MMIO), and higher speed peripherals. The System Port also supports code execution.

Consult Section 4.1 for further information about the System Port and its Physical Memory Attributes.

See the Tenstorrent X280 MC User Guide for a description of the System Port implementation in the Tenstorrent X280 MC.

Chapter 4

Physical Memory Attributes and Memory Map

This chapter describes the Tenstorrent X280 MC physical memory attributes and memory map.

4.1 Physical Memory Attributes Overview

The memory map is divided into different regions covering on-core-complex memory, system memory, peripherals, and empty holes. Physical memory attributes (PMAs) describe the properties of the accesses that can be made to each region in the memory map. These properties encompass the type of access that may be performed: execute, read, or write. As well as other optional attributes related to the access, such as supported access size, alignment, atomic operations, and cacheability.

RISC-V utilizes a simpler approach than other processor architectures in defining the attributes of memory accesses. Instead of defining access characteristics in page table descriptors or memory protection logic, the properties are fixed for memory regions or may only be modified in platform-specific control registers. As most systems don't require the ability to modify PMAs, SiFive cores only support fixed PMAs, which are set at design time. This results in a simpler design with lower gate count and power savings, and an easier programming interface.

External memory map regions are accessed through a specific port type and that port type is used to define the PMAs. The port types are Memory, Peripheral, and System. Memory map regions defined for internal memory and internal control regions also have a predefined PMA based on the underlying contents of the region.

The assigned PMA properties and attributes for Tenstorrent X280 MC memory regions are shown in Table 14 and Table 15 for external and internal regions, respectively.

The configured memory regions of the Tenstorrent X280 MC are listed with their attributes in Table 16.

Port Type	Access Properties	Attributes
Memory Port	Read, Write, Execute	Atomics+LR/SC, Data Cacheable, Instruction Cacheable, Instruction Speculation
Peripheral Port	Read, Write, Execute	Atomics, Instruction Cacheable
System Port	Read, Write, Execute	Instruction Cacheable

Table 14: Physical Memory Attributes for External Regions

Region	Access Properties	Attributes
Bus-Error Unit	Read, Write	Atomics
CLINT	Read, Write	Atomics
Debug	None	N/A
Error Device	Read, Write, Execute	Atomics
L2 Prefetcher	Read, Write	Atomics
L3 Cache Controller	Read, Write	Atomics
L3 LIM	Read, Write, Execute	Atomics
L3 Zero Device	Read, Write, Execute	Atomics, Data Cacheable, Instruction Cacheable
PLIC	Read, Write	Atomics
Private L2 Cache	Read, Write	Atomics
SLPC	Read, Write	Atomics
Test Indicator	Read, Write	Atomics
Trace Encoder	Read, Write	Atomics
Trace Funnel	Read, Write	Atomics
Unmapped	None	N/A

Table 15: Physical Memory Attributes for Internal Regions

All memory map regions support word, half-word, and byte size data accesses.

Atomic access support enables the RISC-V standard Atomic (A) Extension for atomic instructions. These atomic instructions are further documented in Section 3.5 for the X280 core. The load-reserved (LR) and store-conditional (SC) instructions are only supported on the data cacheable region, marked in Table 14 with "Atomics+LR/SC".

No region supports unaligned accesses. An unaligned access will generate the appropriate trap: instruction address misaligned, load address misaligned, or store/AMO address misaligned.

The Physical Memory Protection unit is capable of controlling access properties based on address ranges, not ports. It has no control over the attributes of an address range, however.

Note

The Debug and Error Device regions have special behavior. The Debug region is reserved for use from a Debugger, and all accesses to it from the core in non-Debug mode will trap. The Error Device will also trap all accesses, as described in Chapter 10.

4.2 Memory Map

The memory map of the Tenstorrent X280 MC is shown in Table 16.

Base	Top	PMA	Description
0x0000_0000_0000	0x0000_0000_0FFF		Debug
0x0000_0000_1000	0x0000_0000_2FFF		Unmapped
0x0000_0000_3000	0x0000_0000_3FFF	RWX A	Error Device
0x0000_0000_4000	0x0000_0000_4FFF	RW A	Test Indicator
0x0000_0000_5000	0x0000_016F_FFFF		Unmapped
0x0000_0170_0000	0x0000_0170_0FFF	RW A	Hart 0 Bus-Error Unit
0x0000_0170_1000	0x0000_0170_1FFF	RW A	Hart 1 Bus-Error Unit
0x0000_0170_2000	0x0000_0170_2FFF	RW A	Hart 2 Bus-Error Unit
0x0000_0170_3000	0x0000_0170_3FFF	RW A	Hart 3 Bus-Error Unit
0x0000_0170_4000	0x0000_01FF_FFFF		Unmapped
0x0000_0200_0000	0x0000_0200_FFFF	RW A	CLINT
0x0000_0201_0000	0x0000_0201_3FFF	RW A	L3 Cache Controller
0x0000_0201_4000	0x0000_0202_FFFF		Unmapped
0x0000_0203_0000	0x0000_0203_1FFF	RW A	Hart 0 L2 Prefetcher
0x0000_0203_2000	0x0000_0203_3FFF	RW A	Hart 1 L2 Prefetcher
0x0000_0203_4000	0x0000_0203_5FFF	RW A	Hart 2 L2 Prefetcher
0x0000_0203_6000	0x0000_0203_7FFF	RW A	Hart 3 L2 Prefetcher
0x0000_0203_8000	0x0000_0300_7FFF		Unmapped
0x0000_0300_8000	0x0000_0300_8FFF	RW A	SLPC
0x0000_0300_9000	0x0000_07FF_FFFF		Unmapped
0x0000_0800_0000	0x0000_081F_FFFF	RWX A	L3 LIM
0x0000_0820_0000	0x0000_09FF_FFFF		Unmapped
0x0000_0A00_0000	0x0000_0A1F_FFFF	RWXIDA	L3 Zero Device
0x0000_0A20_0000	0x0000_0BFF_FFFF		Unmapped
0x0000_0C00_0000	0x0000_0FFF_FFFF	RW A	PLIC
0x0000_1000_0000	0x0000_1000_0FFF	RW A	Trace Encoder 0
0x0000_1000_1000	0x0000_1000_1FFF	RW A	Trace Encoder 1
0x0000_1000_2000	0x0000_1000_2FFF	RW A	Trace Encoder 2
0x0000_1000_3000	0x0000_1000_3FFF	RW A	Trace Encoder 3
0x0000_1000_4000	0x0000_1001_7FFF		Unmapped
0x0000_1001_8000	0x0000_1001_8FFF	RW A	Trace Funnel
0x0000_1001_9000	0x0000_1010_3FFF		Unmapped
0x0000_1010_4000	0x0000_1010_7FFF	RW A	Hart 0 Private L2 Cache
0x0000_1010_8000	0x0000_1010_BFFF	RW A	Hart 1 Private L2 Cache
0x0000_1010_C000	0x0000_1010_FFFF	RW A	Hart 2 Private L2 Cache
0x0000_1011_0000	0x0000_1011_3FFF	RW A	Hart 3 Private L2 Cache
0x0000_1011_4000	0x0000_1FFF_FFFF		Unmapped
0x0000_2000_0000	0x0000_2FFF_FFFF	RWXI A	Peripheral Port (256 MiB)
0x0000_3000_0000	0x4000_2FFF_FFFF	RWXI	System Port (64 TiB)
0x4000_3000_0000	0x7FFF_FFFF_FFFF	RWXIDA	Memory Port (64.0 TiB)

Table 16: Tenstorrent X280 MC Memory Map. Physical Memory Attributes: **R**–Read, **W**–Write, **X**–Execute, **I**–Instruction Cacheable, **D**–Data Cacheable, **A**–Atomics

Base	Top	PMA	Description
0x8000_0000_0000	0xFFFF_FFFF_FFFF		Unmapped

Table 16: Tenstorrent X280 MC Memory Map. Physical Memory Attributes: **R**–Read, **W**–Write, **X**–Execute, **I**–Instruction Cacheable, **D**–Data Cacheable, **A**–Atomics

Note

Every component that appears in Table 16 is accessible by any core in the Core Complex and via the Front Port.

Chapter 5

Programmer's Model

The Tenstorrent X280 MC implements the 64-bit RISC-V architecture. The following chapter provides a reference for programmers and an explanation of the extensions supported by *RV64GCV_Zfh_Zba_Zbb_Sscofpmf*.

This chapter contains a high-level discussion of the RISC-V instruction set architecture and additional resources which will assist software developers working with RISC-V products. The Tenstorrent X280 MC is an implementation of the RISC-V *RV64GCV_Zfh_Zba_Zbb_Sscofpmf* architecture, and is guaranteed to be compatible with all applicable RISC-V standards. *RV64GCV_Zfh_Zba_Zbb_Sscofpmf* can emulate almost any other RISC-V ISA extension.

5.1 Base Instruction Formats

RISC-V base instructions are fixed to 32 bits in length and must be aligned on a four-byte boundary in memory. RISC-V ISA keeps the source (*rs1* and *rs2*) and destination (*rd*) registers at the same position in all formats to simplify decoding, with the exception of the 5-bit immediates used in CSR instructions.

The various formats are described in Table 17 below.

Format	Description
R	Format for register-register arithmetic/logical operations.
I	Format for register-immediate ALU operations and loads.
S	Format for stores.
B	Format for branches.
U	Format for 20-bit upper immediate instructions.
J	Format for jumps.

Table 17: Base Instruction Formats

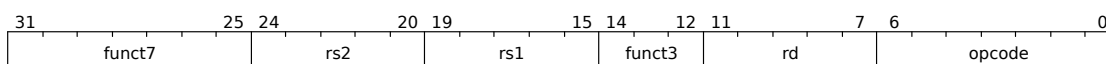


Figure 18: R-Type

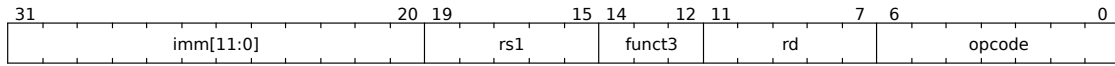


Figure 19: I-Type

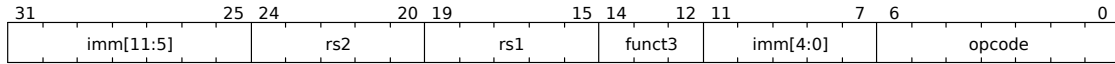


Figure 20: S-Type

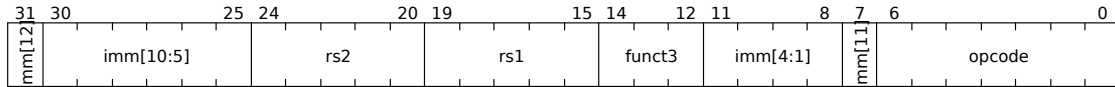


Figure 21: B-Type

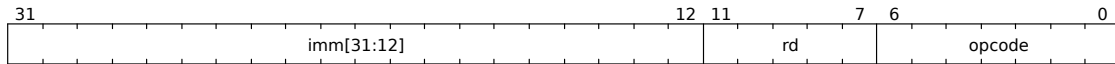


Figure 22: U-Type

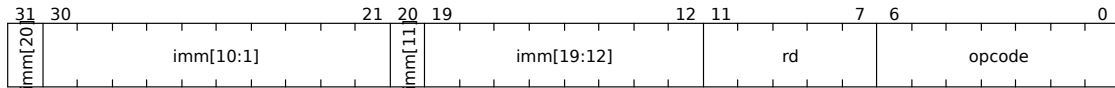


Figure 23: J-Type

The **opcode** field partially specifies an instruction, combined with **funct7** + **funct3** which describe what operation to perform. Each register field (**rs1**, **rs2**, **rd**) holds a 5-bit unsigned integer (0-31) corresponding to a register number (x0 - x31). Sign-extension is one of the most critical operations on immediates (particularly for XLEN>32), and in RISC-V the sign bit for all immediates is always held in bit 31 of the instruction to allow sign-extension to proceed in parallel with instruction decoding.

5.2 RV64I Base Integer Instruction Set

This section discusses the standard integer instructions supported by RISC-V. Integer computational instructions don't cause arithmetic exceptions.

5.2.1 R-Type (Register-Based) Integer Instructions

funct7			funct3		opcode	Instruction
00000000	rs2	rs1	000	rd	0110011	ADD
01000000	rs2	rs1	000	rd	0110011	SUB
00000000	rs2	rs1	001	rd	0110011	SLL
00000000	rs2	rs1	010	rd	0110011	SLT
00000000	rs2	rs1	011	rd	0110011	SLTU
00000000	rs2	rs1	100	rd	0110011	XOR
00000000	rs2	rs1	101	rd	0110011	SRL
01000000	rs2	rs1	101	rd	0110011	SRA
00000000	rs2	rs1	110	rd	0110011	OR
00000000	rs2	rs1	111	rd	0110011	AND

Table 18: R-Type Integer Instructions

Instruction	Description
ADD rd, rs1, rs2	Performs the addition of rs1 and rs2, result stored in rd.
SUB rd, rs1, rs2	Performs the subtraction of rs2 from rs1, result stored in rd.
SLL rd, rs1, rs2	Logical left shift (zeros are shifted into the lower bits) shift amount is encoded in the lower 5 bits of rs2.
SLT rd, x0, rs2	Signed and compare sets rd to 1 if rs2 is not equal to zero, otherwise sets rd to zero.
SLTU rd, x0, rs2	Unsigned compare sets rd to 1 if rs2 is not equal to zero, otherwise sets rd to zero.
SRL rd, rs1, rs2	Logical right shift (zeros are shifted into the lower bits) shift amount is encoded in the lower 5 bits of rs2.
SRA rd, rs1, rs2	Arithmetic right shift, shift amount is encoded in the lower 5 bits of rs2.
OR rd, rs1, rs2	Bitwise logical OR.
AND rd, rs1, rs2	Bitwise logical AND.
XOR rd, rs1, rs2	Bitwise logical XOR.

Table 19: R-Type Integer Instruction Description

Below is an example of an ADD instruction.

add x18, x19, x10

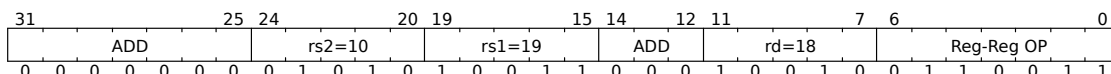


Figure 24: ADD Instruction Example

5.2.2 I-Type Integer Instructions

For I-Type integer instruction, one field is different from R-format. `rs2` and `func7` are replaced by the 12-bit signed immediate, `imm[11:0]`, which can hold values in range `[-2048, +2047]`. The immediate is always sign-extended to 64-bits before being used in an arithmetic operation. Bits `[63:12]` receive the same value as bit 11.

imm			func3		opcode	Instruction
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
00000000	shamnt	rs1	001	rd	0010011	SLLI
00000000	shamnt	rs1	101	rd	0010011	SRLI
01000000	shamnt	rs1	001	rd	0010011	SRAI

Table 20: I-Type Integer Instructions

One of the higher-order immediate bits is used to distinguish "shift right logical" (SRLI) from "shift right arithmetic" (SRAI).

Instruction	Description
ADDI	Adds the sign-extended 12-bit immediate to register <i>rs1</i> . Arithmetic overflow is ignored and the result is simply the low 64-bits of the result. ADDI <i>rd</i> , <i>rs1</i> , 0 is used to implement the MV <i>rd</i> , <i>rs1</i> assembler pseudoinstruction.
SLTI	Set less than immediate. Places the value 1 in register <i>rd</i> if register <i>rs1</i> is less than the sign extended immediate when both are treated as signed numbers, else 0 is written to <i>rd</i> .
SLTIU	Compares the values as unsigned numbers (i.e., the immediate is first sign-extended to 64-bits then treated as an unsigned number). Note: SLTIU <i>rd</i> , <i>rs1</i> , 1 sets <i>rd</i> to 1 if <i>rs1</i> equals zero, otherwise sets <i>rd</i> to 0 (assembler pseudo instruction SEQZ <i>rd</i> , <i>rs</i>).
XORI	Bitwise XOR on register <i>rs1</i> and the sign-extended 12-bit immediate and place the result in <i>rd</i> .
ORI	Bitwise OR on register <i>rs1</i> and the sign-extended 12-bit immediate and place the result in <i>rd</i> .
ANDI	Bitwise AND on register <i>rs1</i> and the sign-extended 12-bit immediate and place the result in <i>rd</i> .
SLLI	Shift Left Logical. The operand to be shifted is in <i>rs1</i> , and the shift amount is encoded in the lower 5 bits of the I-immediate field.
SRLI	Shift Right Logical. The operand to be shifted is in <i>rs1</i> , and the shift amount is encoded in the lower 5 bits of the I-immediate field.
SRAI	Shift Right Arithmetic. The operand to be shifted is in <i>rs1</i> , and the shift amount is encoded in the lower 5 bits of the I-immediate field (the original sign bit is copied into the vacated upper bits).

Table 21: I-Type Integer Instruction Description

Shift-by-immediate instructions only use lower 5 bits of the immediate value for shift amount (can only shift by 0-31 bit positions).

Below is an example of an ADDI instruction.

addi x15, x1, -50

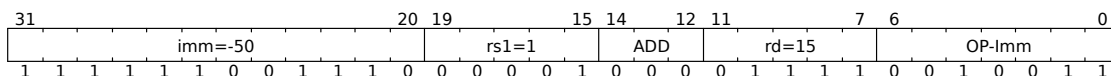


Figure 25: ADDI Instruction Example

5.2.3 I-Type Load Instructions

For I-Type load instructions, a 12-bit signed immediate is added to the base address in register *rs1* to form the memory address. In Table 22 below, **funct3** field encodes size and signedness of load data.

imm		func3		opcode	Instruction
imm[11:0]	rs1	000	rd	00000011	LB
imm[11:0]	rs1	001	rd	00000011	LH
imm[11:0]	rs1	010	rd	00000011	LW
imm[11:0]	rs1	100	rd	00000011	LBU
imm[11:0]	rs1	101	rd	00000011	LHU

Table 22: I-Type Load Instructions

Instruction	Description
LB rd, rs1, imm	Load Byte, loads 8 bits (1 byte) and sign-extends to fill destination 64-bit register.
LH rd, rs1, imm	Load Half-Word. Loads 16 bits (2 bytes) and sign-extends to fill destination 64-bit register.
LW rd, rs1, imm	Load word (32-bits) and sign-extends this to 64-bits to fill destination 64-bit register.
LBU rd, rs1, imm	Load Unsigned Byte (8-bit).
LHU rd, rs1, imm	Load Unsigned Half-Word, which zero-extends 16 bits to fill destination 64-bit register.
LWU rd, rs1, imm	Load a 32-bit value and zero-extends this to 64-bits before storing it in rd.

Table 23: I-Type Load Instruction Description

Below is an example of a LW instruction.

lw x14, 8(x2)

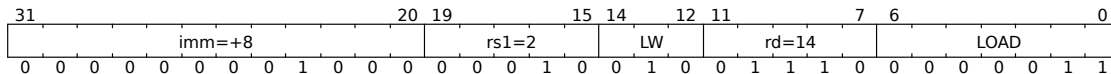


Figure 26: LW Instruction Example

5.2.4 S-Type Store Instructions

Store instructions need to read two registers: rs1 for base memory address and rs2 for data to be stored, as well as an immediate offset. The effective byte address is obtained by adding register rs1 to the sign-extended 12-bit offset. Note that stores don't write a value to the register file, as there is no rd register used by the instruction. In RISC-V, the lower 5 bits of immediate are moved to where the rd field was in other instructions, and the rs1/rs2 fields are kept in same place. The registers are kept always in the same place because a critical path for all operations includes fetching values from the registers. By always placing the read sources in the same place, the register file can read the registers without hesitation. If the data ends up being unnecessary (e.g., I-Type), it can be ignored.

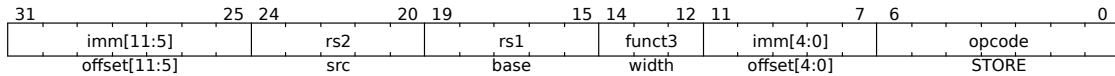


Figure 27: Store Instructions

imm			func3	imm	opcode	Instruction
imm[11:5]	rs2	rs1	000	imm[4:0]	01000011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	01000011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	01000011	SW

Table 24: S-Type Store Instructions

Instruction	Description
SB rs2, imm[11:0](rs1)	Store 8-bit value from the low bits of register rs2 to memory.
SH rs2, imm[11:0](rs1)	Store 16-bit value from the low bits of register rs2 to memory.
SW rs2, imm[11:0](rs1)	Store 32-bit value from the low bits of register rs2 to memory.

Table 25: S-Type Store Instruction Description

Below is an example SW instruction.

sw x14, 8(x2)

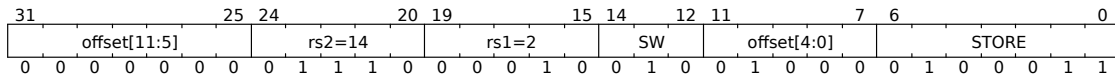


Figure 28: SW Instruction Example

5.2.5 Unconditional Jumps

The jump and link (JAL) instruction uses the J-type format, where the J-immediate encodes a signed offset in multiples of 2 bytes. The offset is sign-extended and added to the address of the jump instruction to form the jump target address. Jumps can therefore target a ± 1 MiB range. JAL stores the address of the instruction following the jump (pc+4) into register rd. The standard software calling convention uses x1 as the return address register and x5 as an alternate link register.

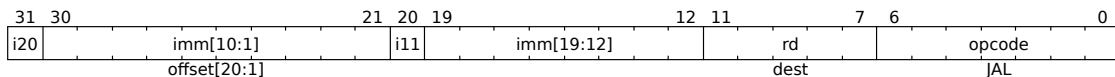


Figure 29: JAL Instruction

The indirect jump instruction JALR (jump and link register) uses the I-type encoding. The target address is obtained by adding the sign-extended 12-bit I-immediate to the register rs1, then setting the least-significant bit of the result to zero. The address of the instruction following the jump (pc+4) is written to register rd. Register x0 can be used as the destination if the result is not required.

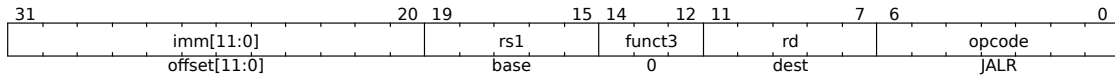


Figure 30: JALR Instruction

Both JAL and JALR instructions will generate an instruction-address-misaligned exception if the target address is not aligned to a four-byte boundary.

Instruction	Description
JAL rd, imm[20:1]	Jump and link
JALR rd, rs1, imm[11:0]	Jump and link register

Table 26: J-Type Instruction Description

5.2.6 Conditional Branches

All branch instructions use the B-Type instruction format. The 12-bit immediate represents values -4096 to +4094 in 2-byte increments. The offset is sign-extended and added to the address of the branch instruction to give the target address. The conditional branch range is ± 4 KiB.

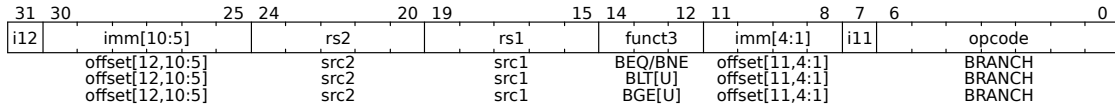


Figure 31: Branch Instructions

imm			func3	imm	opcode	Instruction
imm[12,10:5]	rs2	rs1	000	imm[4:1,11]	110011	BEQ
imm[12,10:5]	rs2	rs1	001	imm[4:1,11]	110011	BNE
imm[12,10:5]	rs2	rs1	100	imm[4:1,11]	110011	BLT
imm[12,10:5]	rs2	rs1	101	imm[4:1,11]	110011	BGE
imm[12,10:5]	rs2	rs1	110	imm[4:1,11]	110011	BLTU
imm[12,10:5]	rs2	rs1	111	imm[4:1,11]	110011	BGEU

Table 27: B-Type Instructions

Instruction	Description
BEQ rs1, rs2, imm[12:1]	Take the branch if registers rs1 and rs2 are equal.
BNE rs1, rs2, imm[12:1]	Take the branch if registers rs1 and rs2 are unequal.
BLT rs1, rs2, imm[12:1]	Take the branch if rs1 is less than rs2.
BGE rs1, rs2, imm[12:1]	Take the branch if rs1 is greater than or equal to rs2.
BLTU rs1, rs2, imm[12:1]	Take the branch if rs1 is less than rs2 (unsigned).
BGEU rs1, rs2, imm[12:1]	Take the branch if rs1 is greater than or equal to rs2 (unsigned).

Table 28: B-Type Instruction Description

ISA Base Instruction	Pseudoinstruction	Description
BEQ <i>rs</i> , <i>x0</i> , <i>offset</i>	BEQZ <i>rs</i> , <i>offset</i>	Take the branch if <i>rs</i> is equal to zero.

Table 29: RISC-V Base Instruction to Assembly Pseudoinstruction Example

Note

Software should be optimized such that the sequential code path is the most common path, with less-frequently taken code paths placed out of line. Software should also assume that backward branches will be predicted taken and forward branches as not taken, at least the first time they are encountered. Dynamic predictors should quickly learn any predictable branch behavior.

5.2.7 Upper-Immediate Instructions



Figure 32: Upper-Immediate Instructions

LUI (load upper immediate) is used to build 32-bit constants and uses the U-type format. LUI places the U-immediate value in the top 20 bits of the destination register *rd*, filling in the lowest 12 bits with zeros. Together with an ADDI to set low 12 bits, can create any 32-bit value in a register using two instructions (LUI/ADDI).

For example:

LUI x10, 0x87654 # x10 = 0x8765_4000

ADDI x10, x10, 0x321 # x10 = 0x8765_4321

AUIPC (add upper immediate to pc) is used to build pc-relative addresses and uses the U-type format. AUIPC forms a 32-bit offset from the 20-bit U-immediate, filling in the lowest 12 bits with zeros, and adds this offset to the address of the AUIPC instruction, then places the result in register *rd*.

5.2.8 Memory Ordering Operations

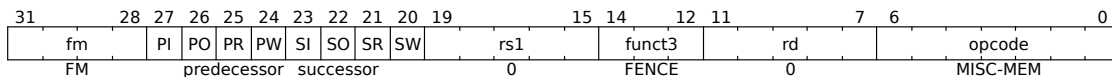


Figure 33: FENCE Instructions

The FENCE instruction is used to order device I/O and memory accesses as viewed by other RISC-V harts and external devices or coprocessors. Any combination of device input (I), device

output (O), memory reads (R), and memory writes (W) may be ordered with respect to any combination of the same. These operations are discussed further in Section 5.16.

5.2.9 Environment Call and Breakpoints

SYSTEM instructions are used to access system functionality that might require privileged access and are encoded using the I-type instruction format. These can be divided into two main classes: those that atomically read-modify-write control and status registers (CSRs), and all other potentially privileged instructions.

5.2.10 NOP Instruction

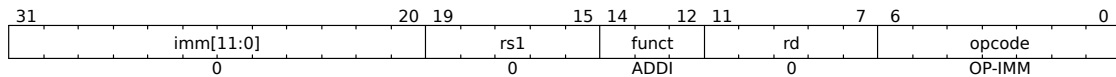


Figure 34: NOP Instructions

The NOP instruction does not change any architecturally visible state, except for advancing the pc and incrementing any applicable performance counters. NOP is encoded as **ADDI x0, x0, 0**.

5.3 M Extension: Multiplication Operations

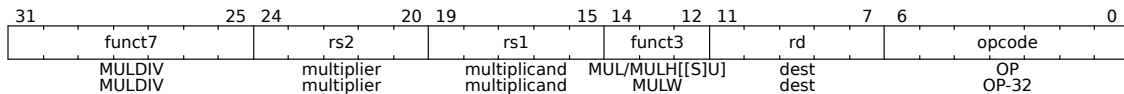


Figure 35: Multiplication Operations

Instruction	Description
MUL rd, rs1, rs2	Multiplication of rs1 by rs2 and places the lower 64-bits in the destination register.
MULH rd, rs1, rs2	Multiplication that return the upper 64-bits of the full 2×64-bit product.
MULHU rd, rs1, rs2	Unsigned multiplication that return the upper 64-bits of the full 2×64-bit product.
MULHSU rd, rs1, rs2	Signed rs1 multiple unsigned rs2 that return the upper 64-bits of the full 2×64-bit product.
MULW rd, rs1, rs2	RV64 instruction that multiplies the lower 32 bits of the source registers, placing the sign-extension of the lower 32 bits of the result into the destination register.

Table 30: Multiplication Operation Description

Combining MUL and MULH together creates one multiplication operation.

5.3.1 Division Operations

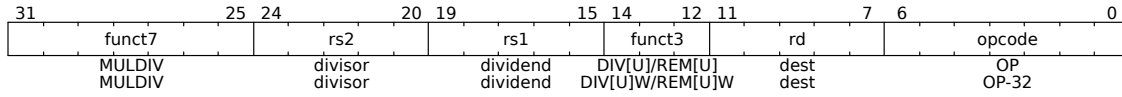


Figure 36: Division Operations

Instruction	Description
DIV rd, rs1, rs2	64-bits by 64-bits signed division of r1 by rs2 rounding towards zero.
DIVU rd, rs1, rs2	64-bits by 64-bits unsigned division of r1 by rs2 rounding towards zero.
REM rd, rs1, rs2	Remainder of the corresponding division.
REMU rd, rs1, rs2	Unsigned remainder of the corresponding division.
DIVW rd, rs1, rs2	RV64 instruction. Signed divide the lower 32 bits of rs1 by the lower 32 bits of rs2.
DIVUW rd, rs1, rs2	RV64 instruction. Unsigned divide the lower 32 bits of rs1 by the lower 32 bits of rs2.
REMW rd, rs1, rs2	Singed remainder.
REMUW rd, rs1, rs2	Unsigned remainder sign-extend the 32-bit result to 64 bits, including on a divide by zero.
MULDIV rd, rs1, rs	Multiply Divide.

Table 31: Division Operation Description

Combining DIV and REM together creates one division operation.

5.4 A Extension: Atomic Operations

Atomic operations are defined as operations that automatically read-modify-write memory to support synchronization between multiple RISC-V harts running in the same memory space.

5.4.1 Atomic Load-Reserve and Store-Conditional Instructions

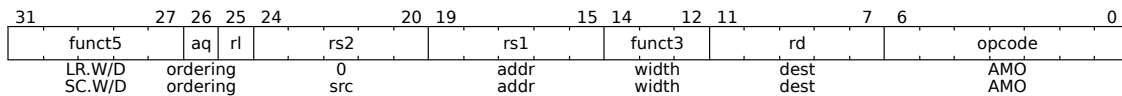


Figure 37: Atomic Operations

Instruction	Description
LR.W	Load Reserve. Loads a word from the address in rs1, places the sign-extended value in rd, and registers a reservation set—a set of bytes that subsumes the bytes in the addressed word.
SC.W	Store Conditional Conditionally writes a word in rs2 to the address in rs1: the SC.W succeeds only if the reservation is still valid and the reservation set contains the bytes being written. If the SC.W succeeds, the instruction writes the word in rs2 to memory, and it writes zero to rd. If the SC.W fails, the instruction does not write to memory, and it writes a nonzero value to rd. Executing an SC.W instruction invalidates any reservation held by this hart.
LR.D	RV64 - Loads doubleword.
SC.D	RV64 - Stores doubleword.

Table 32: Atomic Load-Reserve and Store-Conditional Instruction Description

For RV64, the sign-extended value of LR.W and SC.W is placed in rd.

Note

Only cores with data caches support the LR/SC instructions used by the A-Extension. Cores with DTIMs will *NOT*.

5.4.2 Atomic Memory Operations (AMOs)

The atomic memory operation (AMO) instructions perform read-modify-write operations for multiprocessor synchronization. These AMO instructions atomically load a data value from the address in rs1, place the value into register rd, apply a binary operator to the loaded value and the original value in rs2, then store the result back to the address in rs1.

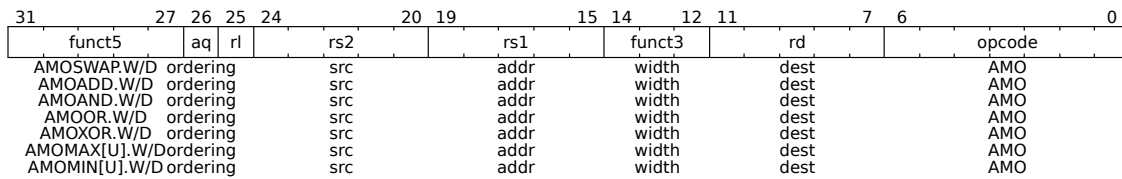


Figure 38: Atomic Memory Operations

Instruction	Description
AMOSWAP.W/D	Word / doubleword swap.
AMOADD.W/D	Word / doubleword add.
AMOAND.W/D	Word / doubleword and.
AMOOR.W/D	Word / doubleword or.
AMOXOR.W/D	Word / doubleword xor.
AMOMIN.W/D	Word / doubleword minimum.
AMOMINU.W/D	Unsigned word / doubleword minimum.
AMOMAX.W/D	Word / doubleword maximum.
AMOMAXU.W/D	Unsigned word / doubleword maximum.

Table 33: Atomic Memory Operation Description

For RV64, 32-bit AMOs always sign-extend the value placed in `rd`.

5.5 F Extension: Single-Precision Floating-Point Instructions

The F Extension implements single-precision floating-point computational instructions compliant with the IEEE 754-2008 arithmetic standard. The F Extension adds 32 floating-point registers, `f0–f31`, each 32 bits wide, and a floating-point control and status register `fcsr`. Floating-point load and store instructions transfer floating-point values between registers and memory, and instructions to transfer values to and from the integer register file are also provided.

5.5.1 Floating-Point Control and Status Registers

Floating-Point Control and Status Register, `fcsr`, is a RISC-V control and status register (CSR). The register selects the dynamic rounding mode for floating-point arithmetic operations and holds the accrued exception flags.

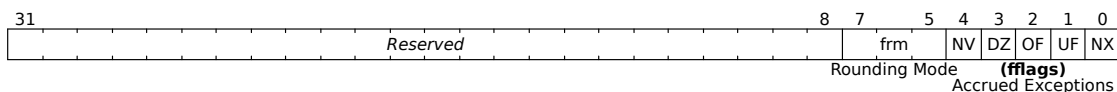


Figure 39: Floating-Point Control and Status Register

Flag Mnemonic	Flag Meaning
NV	Invalid Operation
DZ	Divide by Zero
OF	Overflow
UF	Underflow
NX	Inexact

Table 34: Accrued Exception Flags

The `fcsr` register can be read and written with the `FRCSR` and `FSCSR` instructions. The `FRRM` instruction reads the Rounding Mode field `frm`. `FSRM` swaps the value in `frm` with an integer register. `FRFLAGS` and `FSFLAGS` are defined analogously for the Accrued Exception Flags field `fflags`.

5.5.2 Rounding Modes

Floating-point operations use either a static rounding mode encoded in the instruction, or a dynamic rounding mode held in `frm`. A value of 111 in the instruction's `rm` field selects the dynamic rounding mode held in `frm`. If `frm` is set to an invalid value (101–111), any subsequent attempt to execute a floating-point operation with a dynamic rounding mode will raise an illegal instruction exception. Some instructions, including widening conversions, have the `rm` field, but are nevertheless unaffected by the rounding mode. Software should set their `rm` field to `RNE` (000).

Mode	Mnemonic	Meaning
000	RNE	Round to Nearest, ties to Even.
001	RTZ	Round towards Zero.
010	RDN	Round Down (towards $-\infty$).
011	RUP	Round Up (towards $+\infty$).
100	RMM	Round to Nearest, ties to Max Magnitude.
101		<i>Invalid. Reserved for future use.</i>
110		<i>Invalid. Reserved for future use.</i>
111	DYN	In instruction's <code>rm</code> field, selects dynamic rounding mode; In Rounding Mode register, <i>Invalid</i> .

Table 35: Floating-Point Rounding Modes

5.5.3 Single-Precision Floating-Point Load and Store Instructions

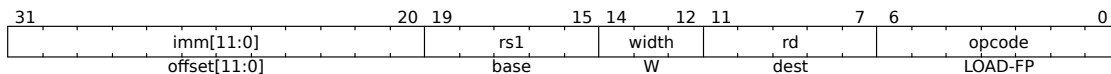


Figure 40: Single-Precision FP Load Instruction

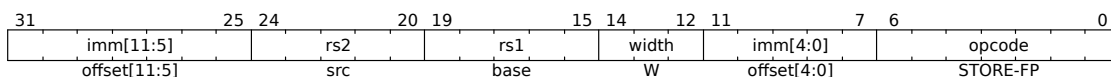


Figure 41: Single-Precision FP Store Instruction

Instruction	Operation	Description
FLW <code>rd, rs1, imm</code>	$f[rd] = M[x[rs1] + sext(offset)][31:0]$	Loads a single-precision floating-point value from memory into floating-point register <code>rd</code> .
FSW <code>imm, rs1, rs2</code>	$M[x[rs1] + sext(offset)] = f[rs2][31:0]$	Stores a single-precision value from floating-point register <code>rs2</code> to memory.

Table 36: Single-Precision FP Load and Store Instructions Description

5.5.4 Single-Precision Floating-Point Computational Instructions

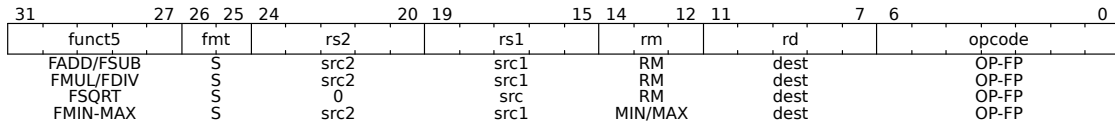


Figure 42: Single-Precision FP Computational Instructions

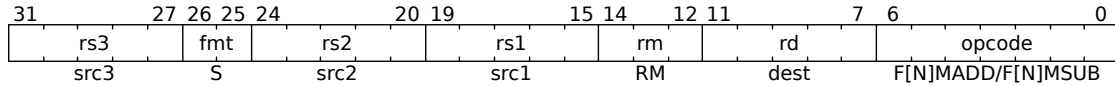


Figure 43: Single-Precision FP Fused Computational Instructions

Instruction	Operation	Description
FADD.S rd,rs1,rs2	$f[rd] = f[rs1] + f[rs2]$	Single-precision floating-point addition.
FSUB.S rd,rs1,rs2	$f[rd] = f[rs1] - f[rs2]$	Single-precision floating-point subtraction.
FMUL.S rd,rs1,rs2	$f[rd] = f[rs1] \times f[rs2]$	Single-precision floating-point multiplication.
FDIV.S rd,rs1,rs2	$f[rd] = f[rs1] \div f[rs2]$	Single-precision floating-point division.
FSQRT.S rd,rs1	$f[rd] = \sqrt{f[rs1]}$	Single-precision floating-point square root.
FMIN.S rd,rs1,rs2	$f[rd] = \min(f[rs1], f[rs2])$	Single-precision floating-point minimum-number.
FMAX.S rd,rs1,rs2	$f[rd] = \max(f[rs1], f[rs2])$	Single-precision floating-point maximum-number.
FMADD.S rd,rs1,rs2,rs3	$f[rd] = (f[rs1] \times f[rs2]) + f[rs3]$	Single-precision floating-point multiply and add.
FMSUB.S rd,rs1,rs2,rs3	$f[rd] = (f[rs1] \times f[rs2]) - f[rs3]$	Single-precision floating-point multiply and subtract.
FNMADD.S rd,rs1,rs2,rs3	$f[rd] = -(f[rs1] \times f[rs2]) + f[rs3]$	Single-precision floating-point multiply, negate, and add.
FNMSUB.S rd,rs1,rs2,rs3	$f[rd] = -(f[rs1] \times f[rs2]) - f[rs3]$	Single-precision floating-point multiply, negate, and subtract.

Table 37: Single-Precision FP Computational Instructions Description

5.5.5 Single-Precision Floating-Point Conversion and Move Instructions

5.5.5.1 Single-Precision Floating-Point Conversion Instructions

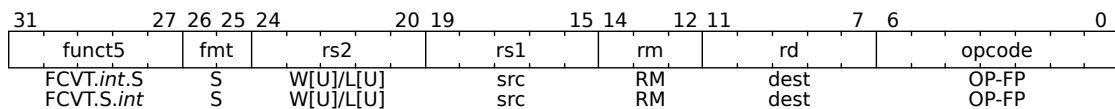


Figure 44: Single-Precision FP to Integer and Integer to FP Conversion Instructions

Instruction	Operation	Description
FCVT.W.S rd, rs1	$x[rd] = \text{sext}(s32_{f32}(f[rs1]))$	Converts a single-precision floating-point number to a signed 32-bit integer. Sign-extends the 32-bit result to the destination register width.
FCVT.S.W rd, rs1	$f[rd] = f32_{s32}(x[rs1])$	Converts a signed 32-bit integer to a single-precision floating-point number.
FCVT.WU.S rd, rs1	$x[rd] = \text{sext}(u32_{f32}(f[rs1]))$	Converts a single-precision floating-point number to an unsigned 32-bit integer. Sign-extends the 32-bit result to the destination register width.
FCVT.S.WU rd, rs1	$f[rd] = f32_{u32}(x[rs1])$	Converts an unsigned 32-bit integer to a single-precision floating-point number.
FCVT.L.S rd, rs1	$x[rd] = s64_{f32}(f[rs1])$	Converts a single-precision floating-point number to a signed 64-bit integer.
FCVT.S.L rd, rs1	$f[rd] = f32_{s64}(x[rs1])$	Converts a signed 64-bit integer to a single-precision floating-point number.
FCVT.LU.S rd, rs1	$x[rd] = u64_{f32}(f[rs1])$	Converts a single-precision floating-point number to an unsigned 64-bit integer.
FCVT.S.LU rd, rs1	$f[rd] = f32_{u64}(x[rs1])$	Converts an unsigned 64-bit integer to a single-precision floating-point number.

Table 38: Single-Precision FP Conversion Instructions Description

If the rounded result is not representable in the destination format, it is clipped to the nearest value and the invalid flag is set.

5.5.5.2 Single-Precision Floating-Point-to-Floating-Point Sign-Injection Instructions

The floating-point-to-floating-point sign-injection instructions produce a result that takes all bits except the sign bit from rs1. The sign-injection instructions provide floating-point MV, ABS and NEG.

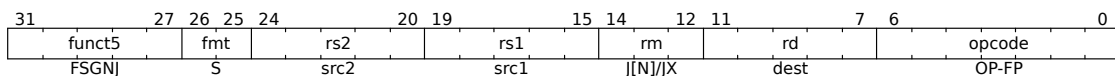


Figure 45: Single-Precision FP to FP Sign-Injection Instructions

Instruction	Operation	Description
FSGNJ.S rd, rs1, rs2	$f[rd] = \{f[rs2][31], f[rs1][30:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The result's sign bit is rs2's sign bit.
FSGNJN.S rd, rs1, rs2	$f[rd] = \{\sim f[rs2][31], f[rs1][30:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The result's sign bit is the opposite of rs2's sign bit.
FSGNJX.S rd, rs1, rs2	$f[rd] = \{f[rs1][31] \wedge f[rs2][31], f[rs1][30:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The sign bit is the XOR of the sign bits of rs1 and rs2.

Table 39: Single-Precision FP to FP Sign-Injection Instructions Description

ISA Base Instruction	Pseudoinstruction	Description
FSGNJ.S rx, ry, ry	FMV.S rx, ry	Moves ry to rx.
FSGNJN.S rx, ry, ry	FNEG.S rx, ry	Moves the negation of ry to rx.
FSGNJX.S rx, ry, ry	FABS.S rx, ry	Moves the absolute value of ry to rx.

Table 40: RISC-V Base Instruction to Assembly Pseudoinstruction Example

5.5.5.3 Single-Precision Floating-Point Move Instructions

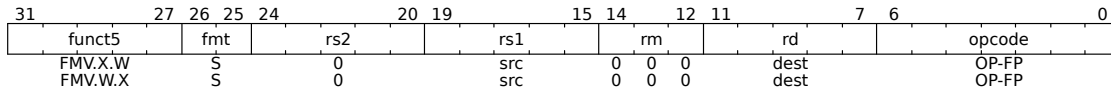


Figure 46: Single-Precision FP Move Instructions

Instruction	Operation	Description
FMV.X.W rd, rs1	$x[rd] = \text{sext}(f[rs1][31:0])$	Moves the single-precision value in floating-point register rs1 represented in IEEE 754-2008 encoding to the lower 32 bits of integer register rd. The higher 32 bits of the destination register are filled with copies of the floating-point number's sign bit.
FMV.W.X rd, rs1	$f[rd] = x[rs1][31:0]$	Moves the single-precision value encoded in IEEE 754-2008 standard encoding from the lower 32 bits of integer register rs1 to the floating-point register rd.

Table 41: Single-Precision FP Move Instructions Description

5.5.6 Single-Precision Floating-Point Compare Instructions

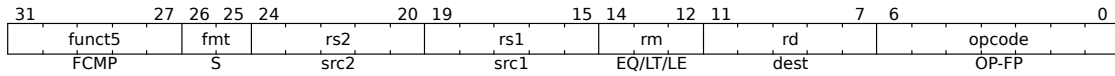


Figure 47: Single-Precision FP Compare Instructions

Instruction	Operation	Description
FEQ.S rd,rs1,rs2	$x[rd] = f[rs1] == f[rs2]$	Writes 1 to the integer register rd if rs1 is equal to rs2, 0 otherwise. Performs a quiet comparison; only sets the invalid operation exception flag if either input is a signaling NaN.
FLT.S rd,rs1,rs2	$x[rd] = f[rs1] < f[rs2]$	Writes 1 to the integer register rd if rs1 less than rs2, 0 otherwise. Performs signaling comparisons; sets the invalid operation exception flag if either input is NaN.
FLE.S rd,rs1,rs2	$x[rd] = f[rs1] \leq f[rs2]$	Writes 1 to the integer register rd if rs1 less than or equal to rs2, 0 otherwise. Performs signaling comparisons; sets the invalid operation exception flag if either input is NaN.

Table 42: Single-Precision FP Compare Instructions Description

5.5.6.1 Single-Precision Floating-Point Classify Instruction

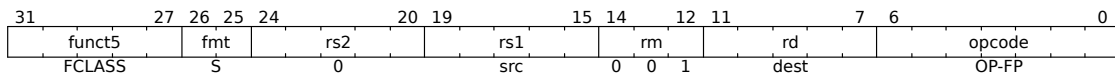


Figure 48: Single-Precision FP Classify Instruction

Instruction	Operation	Description
FCLASS.S rd,rs1	$x[rd] = \text{classify}_s(f[rs1])$	Examines the value in floating-point register rs1 and writes to integer register rd a 10-bit mask that indicates the class of the floating-point number.

Table 43: Single-Precision FP Classify Instruction Description

rd bit	Meaning
0	rs1 is $-\infty$
1	rs1 is negative normal number
2	rs1 is a negative subnormal number
3	rs1 is -0
4	rs1 is +0
5	rs1 is a positive subnormal number
6	rs1 is a positive normal number
7	rs1 is $+\infty$
8	rs1 is a signaling NaN
9	rs1 is a quiet NaN

Table 44: Floating-Point Number Classes

5.6 D Extension: Double-Precision Floating-Point Instructions

The D extension widens the 32 floating-point registers, f_0 – f_{31} , to 64 bits. The f registers can now hold either 32-bit or 64-bit floating-point values. When multiple floating-point precisions are supported, then valid values of narrower n-bit types, $n < \text{FLEN}$, are represented in the lower n bits of an FLEN-bit. Any operation that writes a narrower result to an f register must write all 1s to the uppermost FLEN-n bits to yield a legal NaN-boxed value. Floating-point n-bit transfer operations move external values held in IEEE standard formats into and out of the f registers, and comprise floating-point loads and stores and floating-point move instructions.

5.6.1 Double-Precision Floating-Point Load and Store Instructions

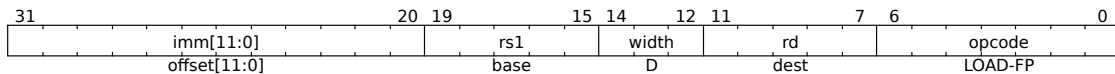


Figure 49: Double-Precision FP Load Instruction

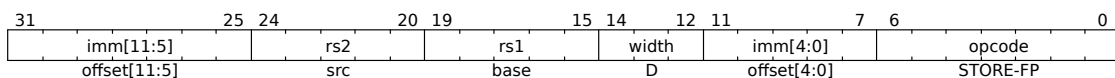


Figure 50: Double-Precision FP Store Instruction

Instruction	Operation	Description
FLD rd, rs1, imm	$f[\text{rd}] = M[x[\text{rs1}] + \text{sext}(\text{offset})][63:0]$	Loads a double-precision floating-point value from memory into floating-point register rd.
FSD imm, rs1, rs2	$M[x[\text{rs1}] + \text{sext}(\text{offset})] = f[\text{rs2}][63:0]$	Stores a double-precision value from the floating-point register rs2 to memory.

Table 45: Double-Precision FP Load and Store Instructions Description

FLD and FSD are only guaranteed to execute atomically if the effective address is naturally aligned and $XLEN \geq 64$. These instructions do not modify the bits being transferred; in particular, the payloads of non-canonical NaNs are preserved.

5.6.2 Double-Precision Floating-Point Computational Instructions

The double-precision floating-point computational instructions are defined analogously to their single-precision counterparts, but operate on double-precision operands and produce double-precision results.

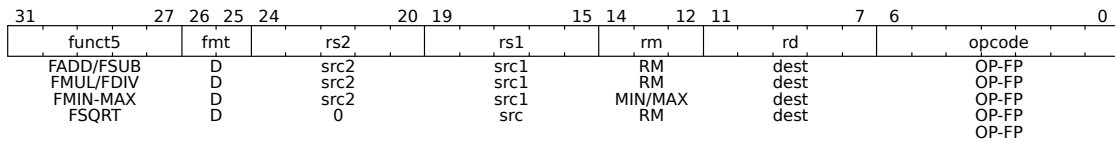


Figure 51: Double-Precision FP Computational Instructions

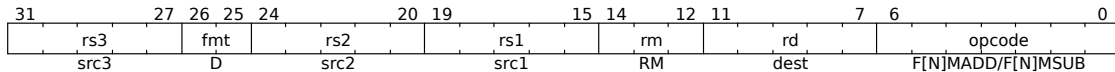


Figure 52: Double-Precision FP Fused Computational Instructions

Instruction	Operation	Description
FADD.D rd,rs1,rs2	$f[rd] = f[rs1] + f[rs2]$	Double-precision floating-point addition.
FSUB.D rd,rs1,rs2	$f[rd] = f[rs1] - f[rs2]$	Double-precision floating-point subtraction.
FMUL.D rd,rs1,rs2	$f[rd] = f[rs1] \times f[rs2]$	Double-precision floating-point multiplication.
FDIV.D rd,rs1,rs2	$f[rd] = f[rs1] \div f[rs2]$	Double-precision floating-point division.
FSQRT.D rd,rs1	$f[rd] = \sqrt{f[rs1]}$	Double-precision floating-point square root.
FMIN.D rd,rs1,rs2	$f[rd] = \min(f[rs1], f[rs2])$	Double-precision floating-point minimum-number.
FMAX.D rd,rs1,rs2	$f[rd] = \max(f[rs1], f[rs2])$	Double-precision floating-point maximum-number.
FMADD.D rd,rs1,rs2,rs3	$f[rd] = (f[rs1] \times f[rs2]) + f[rs3]$	Double-precision floating-point multiply and add.
FMSUB.D rd,rs1,rs2,rs3	$f[rd] = (f[rs1] \times f[rs2]) - f[rs3]$	Double-precision floating-point multiply and subtract.
FNMADD.D rd,rs1,rs2,rs3	$f[rd] = -(f[rs1] \times f[rs2]) + f[rs3]$	Double-precision floating-point multiply, negate, and add.
FNMSUB.D rd,rs1,rs2,rs3	$f[rd] = -(f[rs1] \times f[rs2]) - f[rs3]$	Double-precision floating-point multiply, negate, and subtract.

Table 46: Double-Precision FP Computational Instructions Description

5.6.3 Double-Precision Floating-Point Conversion and Move Instructions

5.6.3.1 Double-Precision Floating-Point Conversion Instructions

All floating-point to integer and integer to floating-point conversion instructions round according to the *rm* field.

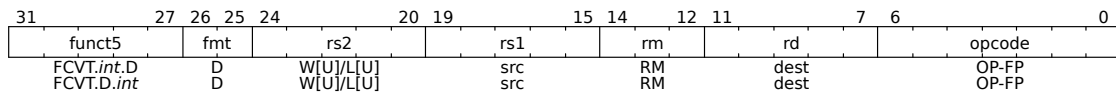


Figure 53: Double-Precision FP to Integer and Integer to FP Conversion Instructions

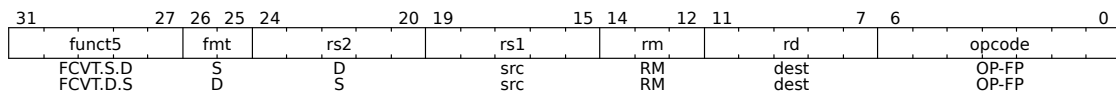


Figure 54: Double-Precision to Single-Precision and Single-Precision to Double-Precision FP Conversion Instructions

Instruction	Operation	Description
FCVT.W.D rd, rs1	$x[rd] = \text{sext}(s32_{f64}(f[rs1]))$	Converts a double-precision floating-point number to a signed 32-bit integer. Sign-extends the 32-bit result to the destination register width.
FCVT.D.W rd, rs1	$f[rd] = f64_{s32}(x[rs1])$	Converts a signed 32-bit integer to a double-precision floating-point number. Always produces an exact result and is unaffected by rounding mode.
FCVT.WU.D rd, rs1	$x[rd] = \text{sext}(u32_{f64}(f[rs1]))$	Converts a double precision floating-point number to an unsigned 32-bit integer. Sign-extends the 32-bit result to the destination register width.
FCVT.D.WU rd, rs1	$f[rd] = f64_{u32}(x[rs1])$	Converts an unsigned 32-bit integer to a double-precision floating-point number. Always produces an exact result and is unaffected by rounding mode.
FCVT.L.D rd, rs1	$x[rd] = s64_{f64}(f[rs1])$	Converts a double-precision floating-point number to a signed 64-bit integer.
FCVT.D.L rd, rs1	$f[rd] = f64_{s64}(x[rs1])$	Converts a signed 64-bit integer to a double-precision floating-point number.
FCVT.LU.D rd, rs1	$x[rd] = u64_{f64}(f[rs1])$	Converts a double-precision floating-point number to an unsigned 64-bit integer.
FCVT.D.LU rd, rs1	$f[rd] = f64_{u64}(x[rs1])$	Converts an unsigned 64-bit integer to a double-precision floating-point number.
FCVT.S.D rd, rs1	$f[rd] = f32_{f64}(f[rs1])$	Converts a double-precision floating-point number to a single-precision floating-point number.
FCVT.D.S rd, rs1	$f[rd] = f64_{f32}(f[rs1])$	Converts a single-precision floating-point number to a double-precision floating-point number.

Table 47: Double-Precision FP Conversion Instructions Description

5.6.3.2 Double-Precision Floating-Point-to-Floating-Point Sign-Injection Instructions

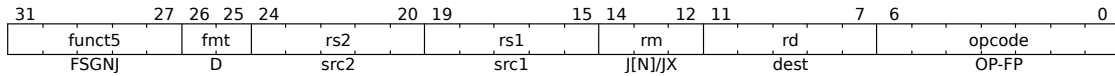


Figure 55: Double-Precision FP to FP Sign-Injection Instructions

Instruction	Operation	Description
FSGNJ.D rd, rs1, rs2	$f[rd] = \{f[rs2][63], f[rs1][62:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The result's sign bit is rs2's sign bit.
FSGNJD.D rd, rs1, rs2	$f[rd] = \{\sim f[rs2][63], f[rs1][62:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The result's sign bit is the opposite of rs2's sign bit.
FSGNJX.D rd, rs1, rs2	$f[rd] = \{f[rs1][63] \wedge f[rs2][63], f[rs1][62:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The sign bit is the XOR of the sign bits of rs1 and rs2.

Table 48: Double-Precision FP to FP Sign-Injection Instructions Description

ISA Base Instruction	Pseudoinstruction	Description
FSGNJ.D rx, ry, ry	FMV.D rx, ry	Moves ry to rx.
FSGNJD.D rx, ry, ry	FNEG.D rx, ry	Moves the negation of ry to rx.
FSGNJX.D rx, ry, ry	FABS.D rx, ry	Moves the absolute value of ry to rx.

Table 49: RISC-V Base Instruction to Assembly Pseudoinstruction Example

5.6.3.3 Double-Precision Floating-Point Move Instructions

The RV64 architecture provides instructions to move bit patterns between the floating-point and integer registers.

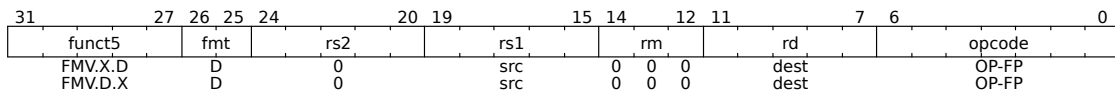


Figure 56: Double-Precision FP Move Instructions

Instruction	Operation	Description
FMV.X.D rd, rs1	$x[rd] = f[rs1][63:0]$	Moves the double-precision value in floating-point register rs1 to a representation in IEEE 754-2008 standard encoding in integer register rd.
FMV.D.X rd, rs1	$f[rd] = x[rs1][63:0]$	Moves the double-precision value encoded in IEEE 754-2008 standard encoding from the integer register rs1 to the floating-point register rd.

Table 50: Double-Precision FP Move Instructions Description

FMV.X.D and FMV.D.X do not modify the bits being transferred; in particular, the payloads of non-canonical NaNs are preserved.

5.6.4 Double-Precision Floating-Point Compare Instructions

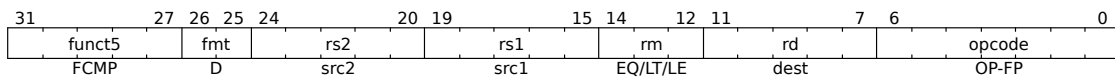


Figure 57: Double-Precision FP Compare Instructions

Instruction	Operation	Description
FEQ.D rd, rs1, rs2	$x[rd] = f[rs1] == f[rs2]$	Writes 1 to the integer register rd if rs1 is equal to rs2, 0 otherwise. Performs a quiet comparison; only sets the invalid operation exception flag if either input is a signaling NaN.
FLT.D rd, rs1, rs2	$x[rd] = f[rs1] < f[rs2]$	Writes 1 to the integer register rd if rs1 less than rs2, 0 otherwise. Performs signaling comparisons; sets the invalid operation exception flag if either input is NaN.
FLE.D rd, rs1, rs2	$x[rd] = f[rs1] \leq f[rs2]$	Writes 1 to the integer register rd if rs1 less than or equal to rs2, 0 otherwise. Performs signaling comparisons; sets the invalid operation exception flag if either input is NaN.

Table 51: Double-Precision FP Compare Instructions Description

5.6.5 Double-Precision Floating-Point Classify Instruction

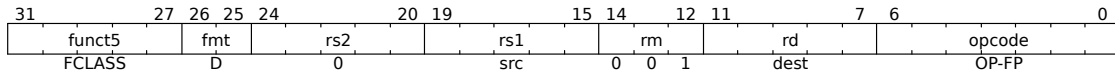


Figure 58: Double-Precision FP Classify Instruction

Instruction	Operation	Description
FCLASS.D rd, rs1	$x[rd] = \text{classify}_d(f[rs1])$	Examines the value in floating-point register rs1 and writes to integer register rd a 10-bit mask that indicates the class of the floating-point number.

Table 52: Double-Precision FP Classify Instruction Description

5.7 Zfh Extension: Half-Precision Floating-Point Instructions

The Zfh extension depends on the single-precision floating-point extension, F. This extension primarily provides extensions that consume half-precision operands and produce half-precision results. However, it is also common to compute on half-precision data using higher intermediate precision.

5.7.1 Half-Precision Floating-Point Load and Store Instructions

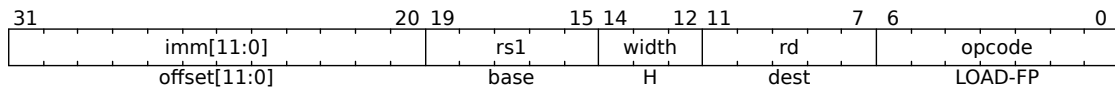


Figure 59: Half-Precision FP Load Instruction

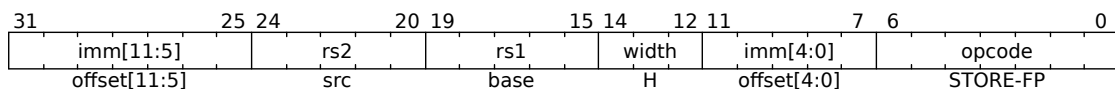


Figure 60: Half-Precision FP Store Instruction

Instruction	Operation	Description
FLH rd, rs1, imm	$f[rd] = M[x[rs1] + \text{sext}(\text{offset})][15:0]$	Loads a half-precision floating-point value from memory into floating-point register rd.
FSH imm, rs1, rs2	$M[x[rs1] + \text{sext}(\text{offset})] = f[rs2][15:0]$	Stores a half-precision value from floating-point register rs2 to memory.

Table 53: Half-Precision FP Load and Store Instructions Description

FLH and FSH are only guaranteed to execute atomically if the effective address is naturally aligned. These instructions do not modify the bits being transferred; in particular, the payloads of

non-canonical NaNs are preserved. FLH NaN-boxes the result written to rd, whereas FSH ignores all but the lower 16 bits in rs2.

5.7.2 Half-Precision Floating-Point Computational Instructions

The half-precision floating-point computational instructions are defined analogously to their single-precision counterparts, but operate on half-precision operands and produce half-precision results.

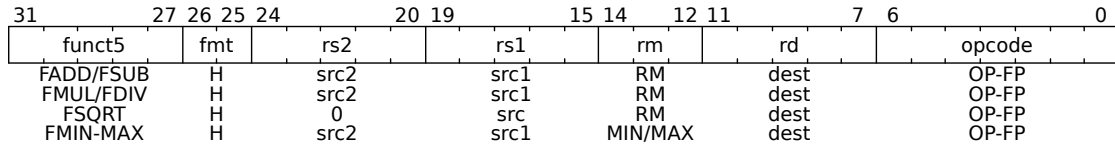


Figure 61: Half-Precision FP Computational Instructions

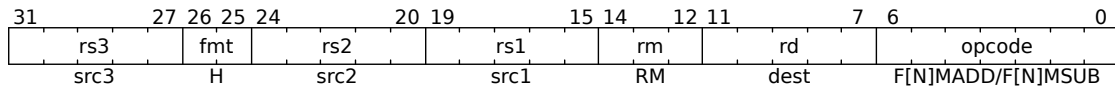


Figure 62: Half-Precision FP Fused Computational Instructions

Instruction	Operation	Description
FADD.H rd,rs1,rs2	$f[rd] = f[rs1] + f[rs2]$	Half-precision floating-point addition.
FSUB.H rd,rs1,rs2	$f[rd] = f[rs1] - f[rs2]$	Half-precision floating-point subtraction.
FMUL.H rd,rs1,rs2	$f[rd] = f[rs1] \times f[rs2]$	Half-precision floating-point multiplication.
FDIV.H rd,rs1,rs2	$f[rd] = f[rs1] \div f[rs2]$	Half-precision floating-point division.
FSQRT.H rd,rs1	$f[rd] = \sqrt{f[rs1]}$	Half-precision floating-point square root.
FMIN.H rd,rs1,rs2	$f[rd] = \min(f[rs1], f[rs2])$	Half-precision floating-point minimum-number.
FMAX.H rd,rs1,rs2	$f[rd] = \max(f[rs1], f[rs2])$	Half-precision floating-point maximum-number.
FMADD.H rd,rs1,rs2,rs3	$f[rd] = (f[rs1] \times f[rs2]) + f[rs3]$	Half-precision floating-point multiply and add.
FMSUB.H rd,rs1,rs2,rs3	$f[rd] = (f[rs1] \times f[rs2]) - f[rs3]$	Half-precision floating-point multiply and subtract.
FNMADD.H rd,rs1,rs2,rs3	$f[rd] = -(f[rs1] \times f[rs2]) + f[rs3]$	Half-precision floating-point multiply, negate, and add.
FNMSUB.H rd,rs1,rs2,rs3	$f[rd] = -(f[rs1] \times f[rs2]) - f[rs3]$	Half-precision floating-point multiply, negate, and subtract.

Table 54: Half-Precision FP Computational Instructions Description

5.7.3 Half-Precision Floating-Point Conversion and Move Instructions

5.7.3.1 Half-Precision Floating-Point Conversion Instructions

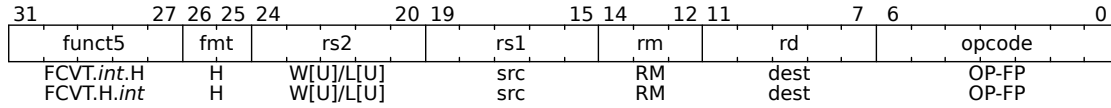


Figure 63: Half-Precision FP to Integer and Integer to FP Conversion Instructions

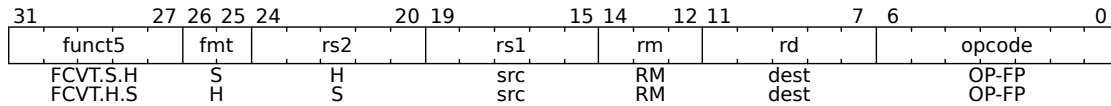


Figure 64: Half-Precision to Single-Precision and Single-Precision to Half-Precision FP Conversion Instructions

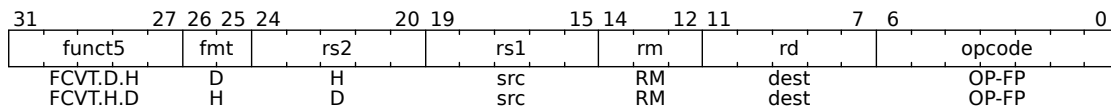


Figure 65: Half-Precision to Double-Precision and Double-Precision to Half-Precision FP Conversion Instructions

Instruction	Operation	Description
FCVT.W.H rd, rs1	$x[rd] = \text{sext}(s32_{f16}(f[rs1]))$	Converts a half-precision floating-point number to a signed 32-bit integer.
FCVT.H.W rd, rs1	$f[rd] = f16_{s32}(x[rs1])$	Converts a signed 32-bit integer to a half-precision floating-point number.
FCVT.WU.H rd, rs1	$x[rd] = \text{sext}(u32_{f16}(f[rs1]))$	Converts a half-precision floating-point number to an unsigned 32-bit integer.
FCVT.H.WU rd, rs1	$f[rd] = f16_{u32}(x[rs1])$	Converts an unsigned 32-bit integer to a half-precision floating-point number.
FCVT.L.H rd, rs1	$x[rd] = s64_{f16}(f[rs1])$	Converts a half-precision floating-point number to a signed 64-bit integer.
FCVT.H.L rd, rs1	$f[rd] = f16_{s64}(x[rs1])$	Converts a signed 64-bit integer to a half-precision floating-point number.
FCVT.LU.H rd, rs1	$x[rd] = u64_{f16}(f[rs1])$	Converts a half-precision floating-point number to an unsigned 64-bit integer.
FCVT.H.LU rd, rs1	$f[rd] = f16_{u64}(x[rs1])$	Converts an unsigned 64-bit integer to a half-precision floating-point number.
FCVT.S.H rd, rs1	$f[rd] = f32_{f16}(f[rs1])$	Converts a half-precision floating-point number to a single-precision floating-point number.
FCVT.H.S rd, rs1	$f[rd] = f16_{f32}(f[rs1])$	Converts a single-precision floating-point number to a half-precision floating-point number.
FCVT.D.H rd, rs1	$f[rd] = f64_{f16}(f[rs1])$	Converts a half-precision floating-point number to a double-precision floating-point number.
FCVT.H.D rd, rs1	$f[rd] = f16_{f64}(f[rs1])$	Converts a double-precision floating-point number to a half-precision floating-point number.

Table 55: Half-Precision FP Conversion Instructions Description

5.7.3.2 Half-Precision Floating-Point-to-Floating-Point Sign-Injection Instructions

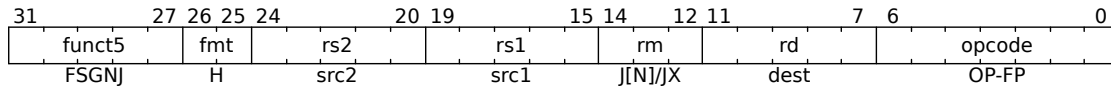


Figure 66: Half-Precision FP to FP Sign-Injection Instructions

Instruction	Operation	Description
FSGNJ.H rd, rs1, rs2	$f[rd] = \{f[rs2][15], f[rs1][14:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The result's sign bit is rs2's sign bit.
FSGNJJ.H rd, rs1, rs2	$f[rd] = \{-f[rs2][15], f[rs1][14:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The result's sign bit is the opposite of rs2's sign bit.
FSGNJJX.H rd, rs1, rs2	$f[rd] = \{f[rs1][15] \wedge f[rs2][15], f[rs1][14:0]\}$	Produces a result that takes all bits except the sign bit from rs1. The sign bit is the XOR of the sign bits of rs1 and rs2.

Table 56: Half-Precision FP to FP Sign-Injection Instructions Description

ISA Base Instruction	Pseudoinstruction	Description
FSGNJ.H rx, ry, ry	FMV.H rx, ry	Moves ry to rx.
FSGNJJ.H rx, ry, ry	FNEG.H rx, ry	Moves the negation of ry to rx.
FSGNJJX.H rx, ry, ry	FABS.H rx, ry	Moves the absolute value of ry to rx.

Table 57: RISC-V Base Instruction to Assembly Pseudoinstruction Example

5.7.3.3 Half-Precision Floating-Point Move Instructions

Instructions are provided to move bit patterns between the floating-point and integer registers.

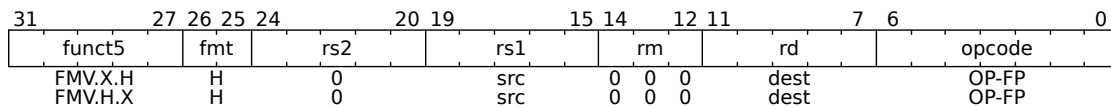


Figure 67: Half-Precision FP Move Instructions

Instruction	Operation	Description
FMV.X.H rd, rs1	$x[rd] = \text{sext}(f[rs1][15:0])$	Moves the half-precision value in floating-point register rs1 to a representation in IEEE 754-2008 standard encoding in integer register rd, filling the upper XLEN-16 bits with copies of the floating-point number's sign bit.
FMV.H.X rd, rs1	$f[rd] = x[rs1][15:0]$	Moves the half-precision value encoded in IEEE 754-2008 standard encoding from the lower 16 bits of integer register rs1 to the floating-point register rd, NaN-boxing the result.

Table 58: Half-Precision FP Move Instructions Description

FMV.X.H and FMV.H.X do not modify the bits being transferred; in particular, the payloads of non-canonical NaNs are preserved.

5.7.4 Half-Precision Floating-Point Compare Instructions

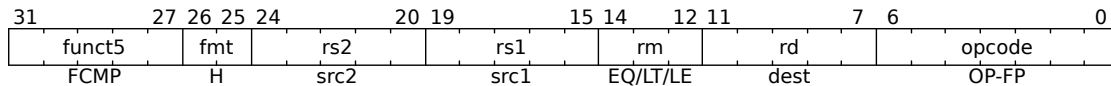


Figure 68: Half-Precision FP Compare Instructions

Instruction	Operation	Description
FEQ.H rd, rs1, rs2	$x[rd] = f[rs1] == f[rs2]$	Writes 1 to the integer register rd if rs1 is equal to rs2, 0 otherwise. Performs a quiet comparison; only sets the invalid operation exception flag if either input is a signaling NaN.
FLT.H rd, rs1, rs2	$x[rd] = f[rs1] < f[rs2]$	Writes 1 to the integer register rd if rs1 is less than rs2, 0 otherwise. Performs signaling comparisons; sets the invalid operation exception flag if either input is NaN.
FLE.H rd, rs1, rs2	$x[rd] = f[rs1] \leq f[rs2]$	Writes 1 to the integer register rd if rs1 is less than or equal to rs2, 0 otherwise. Performs signaling comparisons; sets the invalid operation exception flag if either input is NaN.

Table 59: Half-Precision FP Compare Instructions Description

5.7.5 Half-Precision Floating-Point Classify Instruction

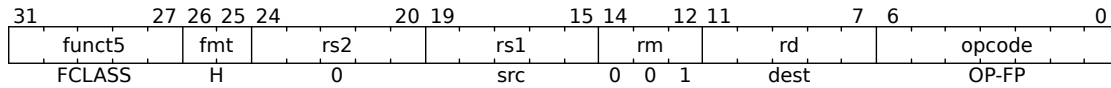


Figure 69: Half-Precision FP Classify Instruction

Instruction	Operation	Description
FCLASS.H rd, rs1	$x[rd] = \text{classify}_h(f[rs1])$	Examines the value in floating-point register rs1 and writes to integer register rd a 10-bit mask that indicates the class of the floating-point number.

Table 60: Half-Precision FP Classify Instruction Description

5.8 V Extension: Vector Instructions

This chapter describes the instructions and control and status registers (CSRs) supported by RVV and enabled on the Tenstorrent X280 MC. For more information, consult the RISC-V "V" Vector Extension specification.

For vector-enabled cores, each hart supporting the vector extension defines two parameters:

1. The maximum size of a vector element that any operation can produce or consume in bits, $ELEN \geq 8$, which must be a power of 2.
2. The number of bits in a single vector register, $VLEN$, which must be a power of 2 and must be no greater than 2^{16} .

The vector extension adds 32 vector registers and seven unprivileged CSRs to the base scalar RISC-V ISA.

Address	Privilege	Name	Description
0x008	URW	vstart	Vector start position
0x009	URW	vxstat	Fixed-Point Saturate Flag
0x00A	URW	vxrm	Fixed-Point Rounding Mode
0x00F	URW	vcsr	Vector control and status register
0xC20	URO	v1	Vector length
0xC21	URO	vtype	Vector data type register
0xC22	URO	vlenb	$VLEN/8$ (vector register length in bytes)

Table 61: Vector CSRs

5.8.1 Vector Registers

The vector extension adds 32 architectural vector registers, v0-v31 to the base scalar RISC-V ISA.

Each vector register has a fixed VLEN bits of state.

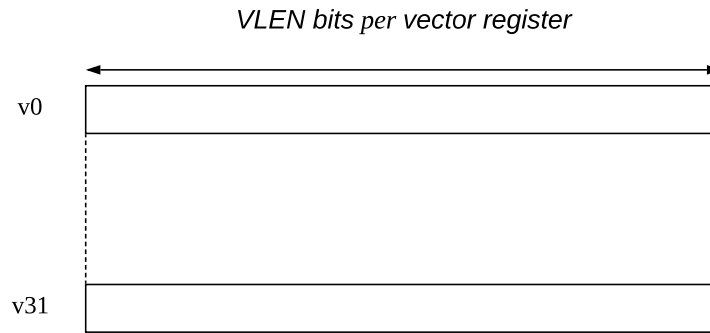


Figure 70: Vector Register VLEN

5.8.2 Vector Context Status in mstatus

A vector context status field, *VS*, is added to *mstatus*[10:9] and shadowed in *sstatus*[10:9]. It is defined analogously to the floating-point context status field, *FS*.

Attempts to execute any vector instruction, or to access the vector CSRs, raises an illegal-instruction exception when the *VS* field is set to Off.

When the *VS* field is set to Initial or Clean, executing any instruction that changes vector state, including the vector CSRs, will change *VS* to Dirty.

5.8.3 Vector Type Register (vtype)

The 64-bit-wide *vector type* CSR *vtype* provides the default type used to interpret the contents of the vector register file. The *vtype* CSR can only be updated by the *vsetvl{i}* instructions described in Section 5.8.11.1. The vector type also determines the organization of elements in each vector register, and how multiple vector registers are grouped.

In the base vector extension, the *vtype* register has five fields: *vll*, *vma*, *vta*, *vsew*[2:0], and *vlmul*[2:0].

Vector Type Register (vtype)		
CSR	0xC21	
Bits	Field Name	Description
[2:0]	<i>vlmul</i> [2:0]	Vector register group multiplier (LMUL) setting
[5:3]	<i>vsew</i> [2:0]	Selected element width (SEW) setting
7	<i>vma</i>	Vector mask agnostic
6	<i>vta</i>	Vector tail agnostic
[62:8]	Reserved	
63	<i>vll</i>	Illegal value if set

Table 62: Vector Type Register

5.8.3.1 Vector Selected Element Width (vsew[2:0])

The value in vsew sets the dynamic Selected Element Width (SEW). By default, a vector register is viewed as being divided into VLEN/SEW selected-width elements.

vsew[2:0]			SEW
0	0	0	8
0	0	1	16
0	1	0	32

Table 63: Encoding of vsew

5.8.3.2 Vector Register Grouping (vlmul[2:0])

Multiple vector registers can be grouped together, so that a single vector instruction can operate on multiple vector registers. The term *vector register group* is, therefore, defined as one or more vector registers used as a single operand to a vector instruction. Vector register groups allow double-width or larger elements to be operated on with the same vector length as selected-width elements. Vector register groups also provide greater execution efficiency for longer application vectors.

The vector length multiplier, *LMUL*, when greater than 1, represents the default number of vector registers that are combined to form a vector register group. LMUL can have the integer values 1, 2, 4, or 8.

LMUL can also be a fractional value, reducing the number of bits used in a vector register. LMUL supports fractional values of 1/2, 1/4, or 1/8. A fractional LMUL value is used to increase the number of usable architectural registers, when operating on mixed-width values, by not requiring that larger-width vectors occupy multiple vector registers. Instead, wider values can occupy a single vector register and narrower values can occupy a fraction of a vector register.

LMUL is set by the signed vlmul field in vtype ($LMUL = 2^{vlmul[2:0]}$).

The derived value $VLMAX = LMUL * VLEN / SEW$ represents the maximum number of elements that can be operated on with a single vector instruction given the current SEW and LMUL settings, as shown in the table below.

vlmul			LMUL	# Groups	VLMAX	Registers Grouped with Register N
1	0	0	—	—	—	Reserved
1	0	1	1/8	32	$VLEN/SEW/8$	vN (single register in group)
1	1	0	1/4	32	$VLEN/SEW/4$	vN (single register in group)
1	1	1	1/2	32	$VLEN/SEW/2$	vN (single register in group)
0	0	0	1	32	$VLEN/SEW$	vN (single register in group)
0	0	1	2	16	$2 * VLEN/SEW$	vN, vN+1
0	1	0	4	8	$4 * VLEN/SEW$	vN, ..., vN+3
0	1	1	8	4	$8 * VLEN/SEW$	vN, ..., vN+7

Table 64: Setting Vector Register Groups

- When LMUL=2, the vector register group contains vector register vN and vector register vN+1, providing twice the vector length in bits. Instructions specifying a vector register group with an odd-numbered vector register will raise an illegal-instruction exception.
- When LMUL=4, the vector register group contains four vector registers. Instructions specifying vector register groups using vector register numbers that are not multiples of four will raise an illegal-instruction exception.
- When LMUL=8, the vector register group contains eight vector registers. Instructions specifying vector register groups using register numbers that are not multiples of eight will raise an illegal-instruction exception.

Mask registers are always contained in a single vector register, regardless of LMUL.

5.8.3.3 Vector Tail Agnostic and Vector Mask Agnostic (vta and vma)

The vta and vma bits modify the behavior of destination tail elements and destination inactive masked-off elements, respectively, during the execution of vector instructions. The tail and inactive sets contain element positions that do not receive new results during a vector operation.

vta	vma	Tail Elements	Inactive Elements
0	0	undisturbed	undisturbed
0	1	undisturbed	agnostic
1	0	agnostic	undisturbed
1	1	agnostic	agnostic

Table 65: Encoding of vta and vma

When a set is marked *undisturbed*, the corresponding set of destination elements in any vector or mask destination operand retain the value they previously held.

When a set is marked *agnostic*, the corresponding set of destination elements in any vector or mask destination operand can either retain the value they previously held, or are overwritten with 1s. Within a single vector instruction, each destination element can be either left undisturbed or overwritten with 1s, in any combination. The pattern of being left undisturbed or overwritten with 1s is not required to be deterministic when the instruction is executed with the same inputs.

The assembly syntax adds two flags to the vsetvli instruction:

```

ta    # Tail agnostic
tu    # Tail undisturbed
ma    # Mask agnostic
mu    # Mask undisturbed

vsetvli t0, a0, e32, m4, ta, ma    # Tail agnostic, mask agnostic
vsetvli t0, a0, e32, m4, tu, ma    # Tail undisturbed, mask agnostic
vsetvli t0, a0, e32, m4, ta, mu    # Tail agnostic, mask undisturbed
vsetvli t0, a0, e32, m4, tu, mu    # Tail undisturbed, mask undisturbed

```

5.8.3.4 Vector Type Illegal (`vill`)

The `vill` bit is used to encode that a previous `vsetv1{i}` instruction attempted to write an unsupported value to `vtype`. If the `vill` bit is set, then any attempt to execute a vector instruction that depends upon `vtype` will raise an illegal-instruction exception. When the `vill` bit is set, the other 63 bits in `vtype` shall be zero.

5.8.4 Vector Length Register (`v1`)

The 64-bit-wide read-only `v1` CSR can only be updated by the `vsetv1i` and `vsetv1` instructions and the fault-only-first vector load instruction variants.

The `v1` register holds an unsigned integer specifying the number of elements to be updated by a vector instruction. Elements in any destination vector register group with indices $\geq v1$ are unmodified during execution of a vector instruction. When `vstart` $\geq v1$, no elements are updated in any destination vector register group.

5.8.5 Vector Byte Length (`vlenb`)

The 64-bit-wide read-only CSR `vlenb` holds the value `VLEN/8`, i.e., the vector register length in bytes.

5.8.6 Vector Start Index CSR (`vstart`)

The `vstart` read-write CSR specifies the index of the first element to be executed by a vector instruction.

All vector instructions are defined to begin execution with the element number given in `vstart`, leaving earlier elements in the destination vector undisturbed, and to reset `vstart` to zero at the end of execution.

Note

All vector instructions, including `vsetv1{i}`, reset `vstart` to zero.

`vstart` is not modified by vector instructions that raise illegal-instruction exceptions.

For instructions where the number of elements to be performed is set by `v1`, if the value in the `vstart` register is greater than or equal to the vector length `v1`, then no element operations are performed. The `vstart` register is then reset to zero.

5.8.7 Vector Fixed-Point Rounding Mode Register (vxrm)

The vector fixed-point rounding-mode register vxrm holds a two-bit read-write rounding-mode field. The vector fixed-point rounding-mode is given a separate CSR address to allow independent access, but is also reflected as a field in vcsr.

The fixed-point rounding algorithm is specified as follows. Suppose the pre-rounding result is v , and d bits of that result are to be rounded off. Then, the rounded result is $(v \gg d) + r$, where r depends on the rounding mode as specified in the following table.

vxrm[1:0]		Mnemonic	Rounding Mode	Rounding Increment, r
0	0	RNU	Round-to-nearest-up (add +0.5 LSB)	$v[d-1]$
0	1	RNE	Round-to-nearest-even	$v[d-1] \ \& \ (v[d-2:0] \neq 0 \mid v[d])$
1	0	RDN	Round-down (truncate)	0
1	1	ROD	Round-to-odd (OR bits into LSB, a.k.a. "jam")	$!v[d] \ \& \ v[d-1:0] \neq 0$

Table 66: Encoding of vxrm

The following rounding functions are used to represent this operation.

```
roundoff_unsigned(v, d) = (unsigned(v) >> d) + r
roundoff_signed(v, d) = (signed(v) >> d) + r
```

5.8.8 Vector Fixed-Point Saturation Flag (vxsat)

The vxsat CSR holds a single read-write bit that indicates if a fixed-point instruction has had to saturate an output value to fit into a destination format.

The vxsat bit is mirrored in vcsr.

5.8.9 Vector Control and Status Register (vcsr)

The separate vxrm and vxsat CSRs can also be accessed via fields in the vector control and status CSR, vcsr.

Vector Control and Status Register (vcsr)		
CSR	0x00F	
Bits	Field Name	Description
0	vxsat	Fixed-point accrued saturation flag
[2:1]	vxrm[1:0]	Fixed-point rounding mode

Table 67: Vector Control and Status Register

5.8.10 Vector Instruction Formats

The instructions in the vector extension fit under three existing major opcodes (LOAD-FP, STORE-FP, AMO) and one new major opcode, OP-V.

Vector instructions can have scalar or vector source operands and produce scalar or vector results, and most vector instructions can be performed either unconditionally or conditionally under a mask.

Vector loads and stores move bit patterns between vector register elements and memory. Vector arithmetic instructions operate on values held in vector register elements.

5.8.10.1 VMEM Format

The following figures illustrate the formats for vector load instructions under the LOAD-FP major opcode.

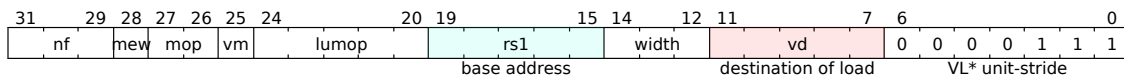


Figure 71: VL Unit Stride Instruction Format

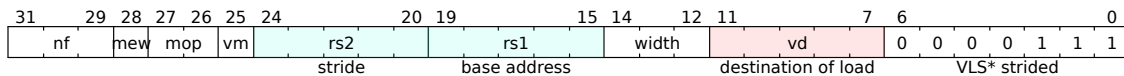


Figure 72: VLS Strided Instruction Format

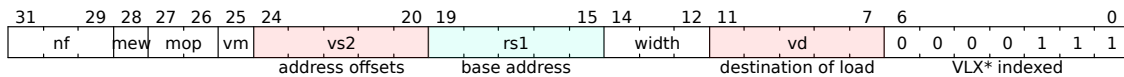


Figure 73: VLX Indexed Instruction Format

The following figures illustrate the formats for vector store instructions under the STORE-FP major opcode.

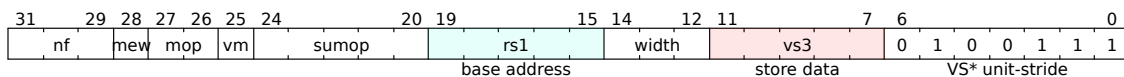


Figure 74: VS Unit-Stride Instruction Format

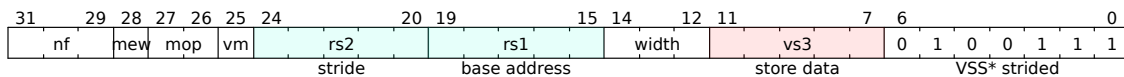


Figure 75: VSS Strided Instruction Format

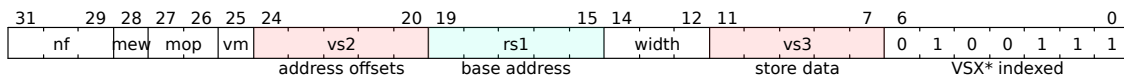


Figure 76: VSX Indexed Instruction Format

5.8.10.2 VALU Format

The following figures illustrate the formats for vector arithmetic instructions under the OP-V major opcode.

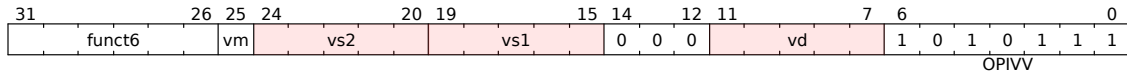


Figure 77: OPIVV Instruction Format

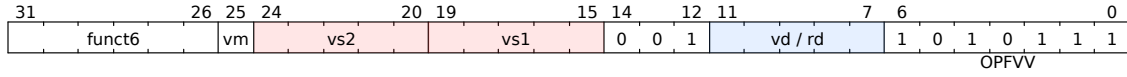


Figure 78: OPFVV Instruction Format

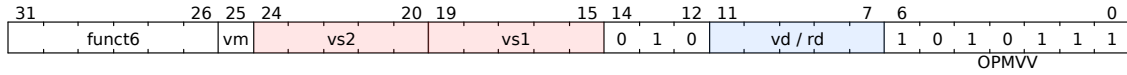


Figure 79: OPMVV Instruction Format

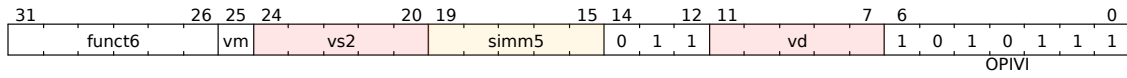


Figure 80: OPIVI Instruction Format

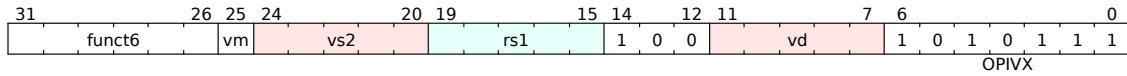


Figure 81: OPIVX Instruction Format

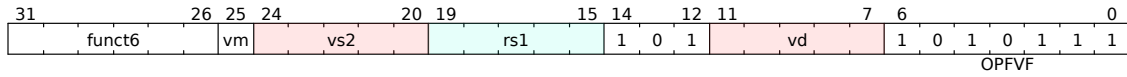


Figure 82: OPFVF Instruction Format

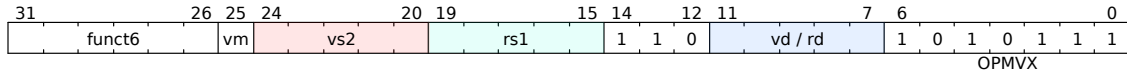


Figure 83: OPMVX Instruction Format

5.8.10.3 VCFG Format

The following figures illustrate the formats for vector configuration instructions under the OP-V major opcode.

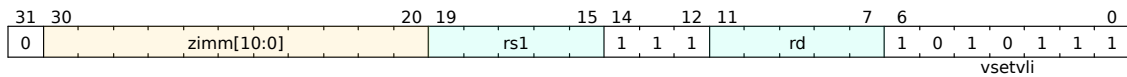


Figure 84: vsetvli Instruction Format

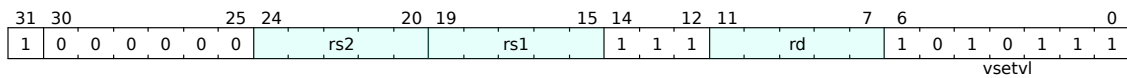


Figure 85: vsetvl Instruction Format

5.8.11 Configuration-Setting Instructions

5.8.11.1 vsetvli and vsetvl Instructions

The `vsetvli` instruction sets the `vtype` and `v1` CSRs based on its arguments, and writes the new value of `v1` into `rd`. The `vsetvl` variant operates similarly to `vsetvli` except that it takes a `vtype` value from `rs2` and can be used for context restore, and when the `vtypei` field is too small to hold the desired setting.

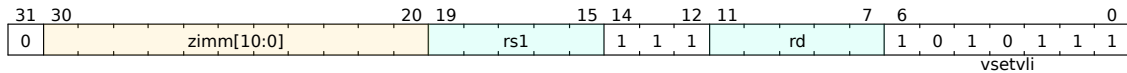


Figure 86: `vsetvli` Instruction

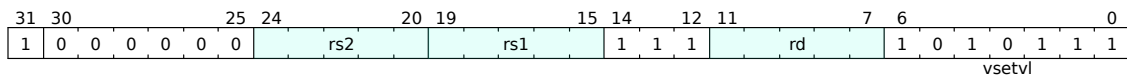


Figure 87: `vsetvl` Instruction

Instruction	Description
VSETVLI <code>rd,rs1,vtypei</code>	<code>rd</code> = new <code>v1</code> , <code>rs1</code> = AVL, <code>vtypei</code> = new <code>vtype</code> setting
VSETVL <code>rd,rs1,rs2</code>	<code>rd</code> = new <code>v1</code> , <code>rs1</code> = AVL, <code>rs2</code> = new <code>vtype</code> value

Table 68: `vsetvli` and `vsetvl` Instructions Description

The new `vtype` setting is encoded in the immediate fields of `vsetvli` and in the `rs2` register for `vsetvl`. The new vector length setting is based on the requested Application Vector Length (AVL), which is encoded in the `rs1` and `rd` fields as follows:

rd	rs1	AVL Value	Effect on <code>v1</code>
-	<code>!x0</code>	Value in <code>x[rs1]</code>	Normal stripmining
<code>!x0</code>	<code>x0</code>	<code>~0</code>	Set <code>v1</code> to VLMAX
<code>x0</code>	<code>x0</code>	Value in <code>v1</code> register	Keep existing <code>v1</code> (of course, <code>vtype</code> may change)

Table 69: Encoding of AVL in `vsetvli` and `vsetvl` Instructions

When `rs1` is not `x0`, the AVL is an unsigned integer held in the `x` register specified by `rs1`, and the new `v1` value is also written to the `x` register specified by `rd`.

When `rs1`=`x0` but `rd`≠`x0`, the maximum unsigned integer value (`~0`) is used as the AVL, and the resulting VLMAX is written to `v1` and also to the `x` register specified by `rd`.

When `rs1`=`x0` and `rd`=`x0`, the current vector length in `v1` is used as the AVL, and the resulting value is written to `v1`, but not to a destination register.

```
e8      # SEW=8b
e16     # SEW=16b
e32     # SEW=32b
e64     # SEW=64b

mf8     # LMUL=1/8
mf4     # LMUL=1/4
mf2     # LMUL=1/2
m1      # LMUL=1, assumed if m setting absent
m2      # LMUL=2
m4      # LMUL=4
m8      # LMUL=8
```

Examples:

```
vsetvli t0, a0, e8          # SEW=8,  LMUL=1
vsetvli t0, a0, e8, m2      # SEW=8,  LMUL=2
vsetvli t0, a0, e32, mf2    # SEW=32, LMUL=1/2
```

5.8.12 Vector Loads and Stores

Vector loads and stores move values between vector registers and memory. Vector loads and stores are masked and do not raise exceptions on inactive elements. Masked vector loads do not update inactive elements in the destination vector register group. Masked vector stores only update active memory elements. All vector loads and stores may generate and accept a non-zero `vstart` value.

5.8.12.1 Vector Load/Store Instruction Encoding

Vector loads and stores are encoded within the scalar floating-point load and store major opcodes (LOAD-FP/STORE-FP). The vector load and store encodings repurpose a portion of the standard scalar floating-point load/store 12-bit immediate field to provide further vector instruction encoding, with bit 25 holding the standard vector mask bit.

Field	Description
rs1[4:0]	Specifies x register holding base address
rs2[4:0]	Specifies x register holding stride
vs2[4:0]	Specifies v register holding address offsets
vs3[4:0]	Specifies v register holding store data
vd[4:0]	Specifies v register destination of load
vm	Specifies whether vector masking is enabled (0 = mask enabled, 1 = mask disabled)
width[2:0]	Specifies size of memory elements, and distinguishes from FP scalar
mew	Extended memory element width
mop[1:0]	Specifies memory addressing mode
nf[2:0]	Specifies the number of fields in each segment, for segment load/stores
lumop[4:0]/sumop[4:0]	Additional fields encoding variants of unit-stride instructions

Table 70: VMEM Format Instruction Fields

5.8.12.2 Vector Load/Store Addressing Modes

The base vector extension supports unit-stride, strided, and indexed (scatter/gather) addressing modes. Vector load/store base registers and strides are taken from the GPR x registers.

The base effective address for all vector accesses is given by the contents of the x register named in rs1.

Vector unit-stride operations access elements stored contiguously in memory starting from the base effective address.

Vector constant-strided operations access the first memory element at the base effective address, and then access subsequent elements at address increments given by the byte offset contained in the x register specified by rs2.

Vector indexed operations add the contents of each element of the vector offset operand specified by vs2 to the base effective address to give the effective address of each element. The data vector register group has EEW=SEW, EMUL=LMUL, while the offset vector register group has EEW encoding in the instruction and EMUL=(EEW/SEW)*LMUL.

The vector offset operand is treated as a vector of byte-address offsets.

The vector addressing modes are encoded using the 2-bit mop[1:0] field.

mop[1:0]		Description	Opcodes
0	0	Unit-stride	VLE<EEW>
0	1	Indexed-unordered	VLUXEI<EEW>
1	0	Strided	VLSE<EEW>
1	1	Indexed-ordered	VLOXEI<EEW>

Table 71: Encoding of mop for Loads

mop[1:0]		Description	Opcodes
0	0	Unit-stride	VSE<EEW>
0	1	Indexed-unordered	VSUXEI<EEW>
1	0	Strided	VSSE<EEW>
1	1	Indexed-ordered	VSOXEI<EEW>

Table 72: Encoding of mop for Stores

Vector unit-stride and constant-stride memory accesses do not guarantee ordering between individual element accesses. The vector indexed load and store memory operations have two forms, ordered and unordered. The indexed-ordered variants preserve element ordering on memory accesses.

For unordered instructions (mop!=11) there is no guarantee on element access order. If the accesses are to a strongly ordered IO region, the element accesses can be initiated in any order.

Additional unit-stride vector addressing modes are encoded using the 5-bit 1umop and sumop fields in the unit-stride load and store instruction encodings, respectively.

1umop[4:0]					Description
0	0	0	0	0	Unit-stride
0	1	0	0	0	Unit-stride, whole registers
1	0	0	0	0	Unit-stride fault-only-first
Note: Unspecified values are reserved.					

Table 73: Encoding of 1umop

sumop[4:0]					Description
0	0	0	0	0	Unit-stride
0	1	0	0	0	Unit-stride, whole registers
Note: Unspecified values are reserved.					

Table 74: Encoding of sumop

5.8.12.3 Vector Load/Store Width Encoding

Vector loads and stores have an EEW encoded directly in the instruction. The corresponding EMUL is calculated as $EMUL = (EEW/SEW) * LMUL$. If the EMUL would be out of range ($EMUL > 8$ or $EMUL < 1/8$), an illegal-instruction exception is raised. The vector register groups must have legal register specifiers for the selected EMUL, else an illegal-instruction is raised.

Vector unit-stride and constant-stride use the EEW/EMUL encoded in the instruction for the data values, while vector indexed loads and stores use the EEW/EMUL encoded in the instruction for the index values and the SEW/LMUL encoded in vtype for the data values.

Vector loads and stores are encoded using width values that are not claimed by the standard scalar floating-point loads and stores.

	mew	width[2:0]			Mem Bits	Data Reg Bits	Index Bits	Opcodes
Standard scalar FP	x	0	0	1	16	FLEN	—	FLH/FSH
Standard scalar FP	x	0	1	0	32	FLEN	—	FLW/FSW
Standard scalar FP	x	0	1	1	64	FLEN	—	FLD/FSD
Standard scalar FP	x	1	0	0	128	FLEN	—	FLQ/FSQ
Vector 8b element	0	0	0	0	8	8	—	VL*E8/VS*E8
Vector 16b element	0	1	0	1	16	16	—	VL*E16/VS*E16
Vector 32b element	0	1	1	0	32	32	—	VL*E32/VS*E32
Vector 64b element	0	1	1	1	64	64	—	VL*E64/VS*E64
Vector 8b index	0	0	0	0	SEW	SEW	8	VL*EI8/VS*EI8
Vector 16b index	0	1	0	1	SEW	SEW	16	VL*EI16/VS*EI16
Vector 32b index	0	1	1	0	SEW	SEW	32	VL*EI32/VS*EI32
Vector 64b index	0	1	1	1	SEW	SEW	64	VL*EI64/VS*EI64

Table 75: Encoding of Vector Loads and Stores width

- **Mem bits** is the size of each element accessed in memory.
- **Data reg bits** is the size of each data element accessed in register.
- **Index bits** is the size of each index accessed in register.

5.8.12.4 VL* Unit-Stride Instructions

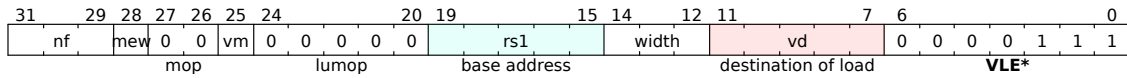


Figure 88: Vector Unit-Stride Load Instruction

vd = destination, rs1 = base address, vm is mask encoding (v0.t or <missing>)

```
vle8.v    vd, (rs1), vm # 8-bit unit-stride load
vle16.v   vd, (rs1), vm # 16-bit unit-stride load
vle32.v   vd, (rs1), vm # 32-bit unit-stride load
vle64.v   vd, (rs1), vm # 64-bit unit-stride load
```

The unit-stride fault-only-first load instructions are used to vectorize loops with data-dependent exit conditions ("while" loops). These instructions execute as a regular load except that they will only take a trap caused by a synchronous exception on element 0. If an element > 0 raises an exception, that element and all following elements in the destination vector register are not modified, and the vector length vl is reduced to the index of the element that would have raised an exception.

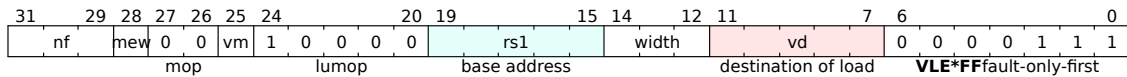


Figure 89: Vector Unit-Stride Fault-Only-First Load Instruction

Vector unit-stride fault-only-first loads

vd = destination, rs1 = base address, vm is mask encoding (v0.t or <missing>)

```
vle8ff.v   vd, (rs1), vm # 8-bit unit-stride fault-only-first load
vle16ff.v  vd, (rs1), vm # 16-bit unit-stride fault-only-first load
vle32ff.v  vd, (rs1), vm # 32-bit unit-stride fault-only-first load
vle64ff.v  vd, (rs1), vm # 64-bit unit-stride fault-only-first load
```

An additional unit-stride load and store is provided to support transferring mask values (EEW=1) to/from memory. These operate the same as unmasked byte loads or stores, except that the effective vector length is $evl = \text{ceil}(vl/8)$, and the destination register is always written with a tail-agnostic policy.

Vector unit-stride mask load

```
vle1.v vd, (rs1) # Load byte vector of length ceil(vl/8)
```

Vector unit-stride mask store

```
vsel.v vs3, (rs1) # Store byte vector of length ceil(vl/8)
```

5.8.12.5 VLS* Strided Instructions

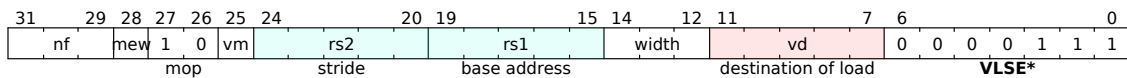


Figure 90: Vector Strided Load Instruction

vd = destination, rs1 = base address, rs2 = byte stride

```
vlse8.v    vd, (rs1), rs2, vm # 8-bit strided load
vlse16.v   vd, (rs1), rs2, vm # 16-bit strided load
vlse32.v   vd, (rs1), rs2, vm # 32-bit strided load
vlse64.v   vd, (rs1), rs2, vm # 64-bit strided load
```

5.8.12.6 VL* Indexed Instructions

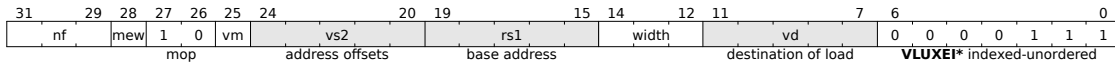


Figure 91: Vector Indexed-Unordered Load Instruction

Vector indexed-unordered load instructions

vd destination, rs1 base address, vs2 byte offsets

```
vluxei8.v   vd, (rs1), vs2, vm # unordered 8-bit indexed load of SEW data
vluxei16.v  vd, (rs1), vs2, vm # unordered 16-bit indexed load of SEW data
vluxei32.v  vd, (rs1), vs2, vm # unordered 32-bit indexed load of SEW data
vluxei64.v  vd, (rs1), vs2, vm # unordered 64-bit indexed load of SEW data
```

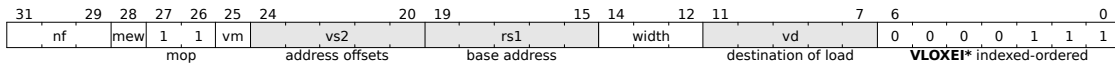


Figure 92: Vector Indexed-Ordered Load Instruction

Vector indexed-ordered load instructions

vd destination, rs1 base address, vs2 byte offsets

```
vloxei8.v   vd, (rs1), vs2, vm # ordered 8-bit indexed load of SEW data
vloxei16.v  vd, (rs1), vs2, vm # ordered 16-bit indexed load of SEW data
vloxei32.v  vd, (rs1), vs2, vm # ordered 32-bit indexed load of SEW data
vloxei64.v  vd, (rs1), vs2, vm # ordered 64-bit indexed load of SEW data
```

5.8.12.7 VS* Unit-Stride Instructions

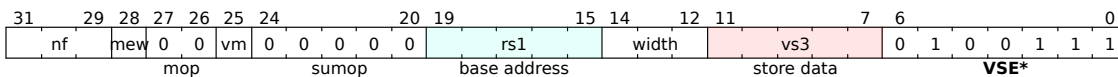


Figure 93: Vector Unit-Stride Store Instruction

vs3 = store data, rs1 = base address, vm is mask encoding (v0.t or <missing>)

```
vse8.v      vs3, (rs1), vm # 8-bit unit-stride store
vse16.v     vs3, (rs1), vm # 16-bit unit-stride store
vse32.v     vs3, (rs1), vm # 32-bit unit-stride store
vse64.v     vs3, (rs1), vm # 64-bit unit-stride store
```

5.8.12.8 VSS* Strided Instructions

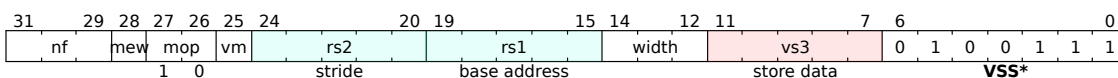


Figure 94: Vector Strided Store Instruction

vs3 = store data, rs1 = base address, rs2 = byte stride

```
vsse8.v    vs3, (rs1), rs2, vm # 8-bit strided store
vsse16.v   vs3, (rs1), rs2, vm # 16-bit strided store
vsse32.v   vs3, (rs1), rs2, vm # 32-bit strided store
vsse64.v   vs3, (rs1), rs2, vm # 64-bit strided store
```

5.8.12.9 VS* Indexed Instructions

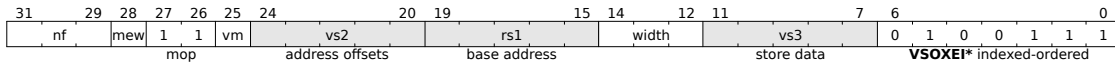


Figure 95: Vector Ordered-Indexed Store Instruction

Vector indexed-ordered store instructions

vs3 store data, rs1 base address, vs2 byte offsets

```
vsoxei8.v   vs3, (rs1), vs2, vm # ordered 8-bit indexed store of SEW data
vsoxei16.v  vs3, (rs1), vs2, vm # ordered 16-bit indexed store of SEW data
vsoxei32.v  vs3, (rs1), vs2, vm # ordered 32-bit indexed store of SEW data
vsoxei64.v  vs3, (rs1), vs2, vm # ordered 64-bit indexed store of SEW data
```

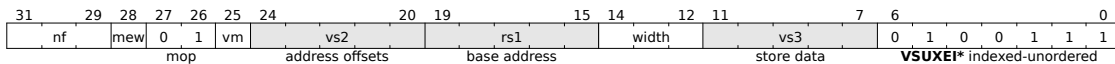


Figure 96: Vector Unordered-Indexed Store Instruction

Vector unordered-indexed store instructions

```
vsuxei8.v   vs3, (rs1), vs2, vm # unordered 8-bit indexed store of SEW data
vsuxei16.v  vs3, (rs1), vs2, vm # unordered 16-bit indexed store of SEW data
vsuxei32.v  vs3, (rs1), vs2, vm # unordered 32-bit indexed store of SEW data
vsuxei64.v  vs3, (rs1), vs2, vm # unordered 64-bit indexed store of SEW data
```

5.8.12.10 Vector Load/Store Segment Instructions

The vector load and store segment instruction subset is given the ISA string Zvlssseg.

The vector load/store segment instructions move multiple contiguous fields in memory to and from consecutively numbered vector registers.

The three-bit *nf* field in the vector instruction encoding is an unsigned integer that contains one less than the number of fields per segment, *NFIELDS*.

nf[2:0]			NFIELDS
0	0	0	1
0	0	1	2
0	1	0	3
0	1	1	4
1	0	0	5
1	0	1	6
1	1	0	7
1	1	1	8

Table 76: Encoding of nf

The EMUL setting must be such that $EMUL * NFIELDS \leq 8$, otherwise an illegal-instruction exception is raised.

Each field will be held in successively numbered vector register groups. When $EMUL > 1$, each field will occupy a vector register group held in multiple successively numbered vector registers, and the vector register group for each field must follow the usual vector register alignment constraints. For example, when $EMUL = 2$ and $NFIELDS = 4$, each field's vector register group must start at an even vector register, but does not have to start at a multiple of 8 vector register number.

If the vector register numbers accessed by the segment load or store increments past 31, then an illegal-instruction exception is raised.

The v1 register gives the number of structures to move, which is equal to the number of elements transferred to each vector register group. Masking is also applied at the level of whole structures.

For segment loads and stores, the individual memory accesses within each segment are unordered with respect to each other even for ordered indexed segment loads and stores.

If a trap is taken, vstart is in units of structures.

5.8.12.10.1 Vector Unit-Stride Segment Loads and Stores

The vector unit-stride load and store segment instructions move packed contiguous segments ("array-of-structures") into multiple destination vector register groups.

The assembler prefixes vlseg/vsseg are used for unit-stride segment loads and stores, respectively.

```
# Format
vlseg<nf>e<eew>.v vd, (rs1), vm # Unit-stride segment load template
vsseg<nf>e<eew>.v vs3, (rs1), vm # Unit-stride segment store template
```

Instruction	Description
VLSEG8E8.V vd, (rs1), vm	Load eight vector registers with eight byte fields
VSSEG3E32.V vs3, (rs1), vm	Store packed vector of 3*4-byte segments from vs3, vs3+1, vs3+2 to memory

Table 77: Unit-Stride Segmented Load/Store Instructions Description

For loads, the vd register will hold the first field loaded from the segment. For stores, the vs3 register is read to provide the first field to be stored in each segment.

```
# Example 1
# Memory structure holds packed RGB pixels (24-bit data structure, 8bpp)

vsetvli a1, t0, e8, ta, ma
vlseg3e8.v v8, (a0), vm

# v8 holds the red pixels
# v9 holds the green pixels
# v10 holds the blue pixels

# Example 2
# Memory structure holds complex values, 32b for real and 32b for imaginary

vsetvli a1, t0, e32, ta, ma
vlseg2e32.v v8, (a0), vm

# v8 holds real
# v9 holds imaginary
```

There are also fault-only-first versions of the unit-stride instructions.

```
# Template for vector fault-only-first unit-stride segment loads and stores

vlseg<nf>e<eew>ff.v vd, (rs1), vm # Unit-stride fault-only-first segment loads
```

5.8.12.10.2 Vector Strided Segment Loads and Stores

Vector strided segment loads and stores move contiguous segments where each segment is separated by the byte-stride offset given in the rs2 GPR argument. Negative and zero strides are supported.

```
# Format
vlsseg<nf>e<eew>.v vd, (rs1), rs2, vm # Strided segment loads
vssseg<nf>e<eew>.v vs3, (rs1), rs2, vm # Strided segment stores

# Examples
vsetvli a1, t0, e8, ta, ma
vlsseg3e8.v v4, (x5), x6 # Load bytes at addresses x5+i*x6 into v4[i],
                        # and bytes at addresses x5+i*x6+1 into v5[i],
                        # and bytes at addresses x5+i*x6+2 into v6[i].

# Examples
vsetvli a1, t0, e32, ta, ma
vssseg2e32.v v2, (x5), x6 # Store words from v2[i] to address x5+i*x6
                        # and words from v3[i] to address x5+i*x6+4
```

For strided segment stores where the byte stride is such that segments could overlap in memory, the segments must appear to be written in element order.

5.8.12.10.3 Vector Indexed Segment Loads and Stores

Vector indexed segment loads and stores move contiguous segments, where each segment is located at an address given by adding the scalar base address in the rs1 field to byte offsets in vector register vs2.

The data vector register group has EEW=SEW, EMUL=LMUL, while the index vector register group has EEW encoded in the instruction with EMUL=(EEW/SEW)*LMUL.

```
# Format
vluxseg<nf>ei<eew>.v vd, (rs1), vs2, vm # Unordered indexed segment loads
vloxseg<nf>ei<eew>.v vd, (rs1), vs2, vm # Ordered indexed segment loads
vsuxseg<nf>ei<eew>.v vs3, (rs1), vs2, vm # Unordered indexed segment stores
vsosseg<nf>ei<eew>.v vs3, (rs1), vs2, vm # Ordered indexed segment stores

# Examples
vsetvli a1, t0, e8, ta, ma
vluxseg3ei32.v v4, (x5), v3 # Load bytes at addresses x5+v3[i] into v4[i],
                        # and bytes at addresses x5+v3[i]+1 into v5[i],
                        # and bytes at addresses x5+v3[i]+2 into v6[i].

# Examples
vsetvli a1, t0, e32, ta, ma
vsuxseg2ei32.v v2, (x5), v5 # Store words from v2[i] to address x5+v5[i]
                        # and words from v3[i] to address x5+v5[i]+4
```

For vector indexed segment loads, the destination vector register groups cannot overlap the source vector register group (specified by vs2); otherwise, an illegal-instruction exception is raised.

Only ordered indexed segment stores are provided. The segments must appear to be written in element order.

5.8.12.11 Vector Load/Store Whole Register Instructions

These instructions load and store whole vector registers (i.e., VLEN bits), and optionally as vector register groups.

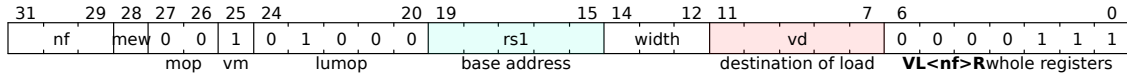


Figure 97: Vector Load Whole Register Instruction

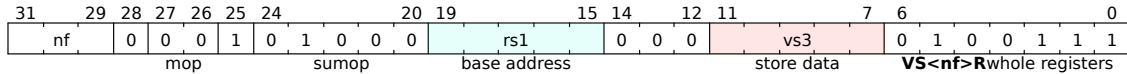


Figure 98: Vector Store Whole Register Instruction

Format of whole register load and store instructions

vl1r.v v3, (a0) # Pseudoinstruction equal to vl1re8.v v3, (a0)

vl1re8.v v3, (a0) # Load v3 with VLEN/8 bytes held at address in a0

vl1re16.v v3, (a0) # Load v3 with VLEN/16 halfwords held at address in a0

vl1re32.v v3, (a0) # Load v3 with VLEN/32 words held at address in a0

vl1re64.v v3, (a0) # Load v3 with VLEN/64 doublewords held at address in a0

vl2r.v v2, (a0) # Pseudoinstruction equal to vl2re8.v v2, (a0)

vl2re8.v v2, (a0) # Load v2-v3 with 2*VLEN/8 bytes from address in a0

vl2re16.v v2, (a0) # Load v2-v3 with 2*VLEN/16 halfwords held at address in a0

vl2re32.v v2, (a0) # Load v2-v3 with 2*VLEN/32 words held at address in a0

vl2re64.v v2, (a0) # Load v2-v3 with 2*VLEN/64 doublewords held at address in a0

vl4r.v v4, (a0) # Pseudoinstruction equal to vl4re8.v v4, (a0)

vl4re8.v v4, (a0) # Load v4-v7 with 4*VLEN/8 bytes from address in a0

vl4re16.v v4, (a0)

vl4re32.v v4, (a0)

vl4re64.v v4, (a0)

vl8r.v v8, (a0) # Pseudoinstruction equal to vl8re8.v v8, (a0)

vl8re8.v v8, (a0) # Load v8-v15 with 8*VLEN/8 bytes from address in a0

vl8re16.v v8, (a0)

vl8re32.v v8, (a0)

vl8re64.v v8, (a0)

vs1r.v v3, (a1) # Store v3 to address in a1

vs2r.v v2, (a1) # Store v2-v3 to address in a1

vs4r.v v4, (a1) # Store v4-v7 to address in a1

vs8r.v v8, (a1) # Store v8-v15 to address in a1

5.8.13 Vector Memory Alignment Constraints

If the elements accessed by a vector memory instruction are not naturally aligned to the memory element size, either an address-misaligned exception is raised on that element or the element is transferred successfully.

Vector memory accesses follow the same rules for atomicity as scalar memory accesses.

5.8.14 Vector Arithmetic Instruction Formats

The vector arithmetic instructions use a new major opcode (OP-V=1010111₂), which neighbors OP-FP. The three-bit funct3 field is used to define sub-categories of vector instructions.

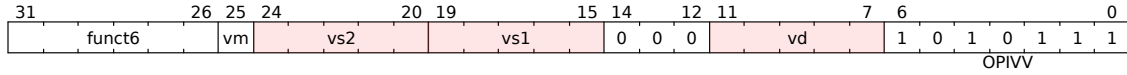


Figure 99: OPIVV Instruction

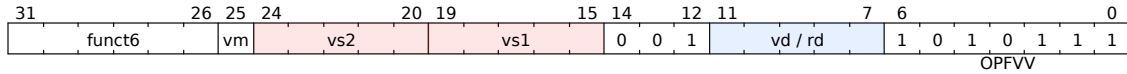


Figure 100: OPFVV Instruction

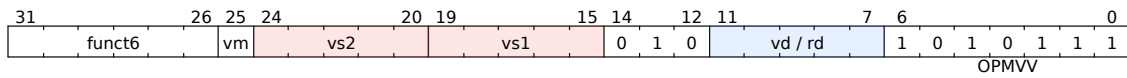


Figure 101: OPMVV Instruction

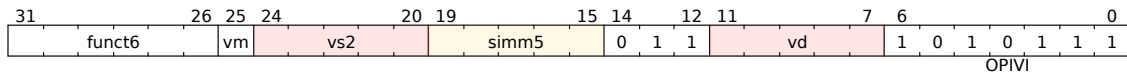


Figure 102: OPIVI Instruction

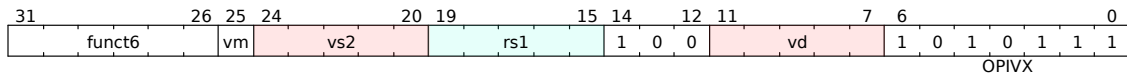


Figure 103: OPIVX Instruction

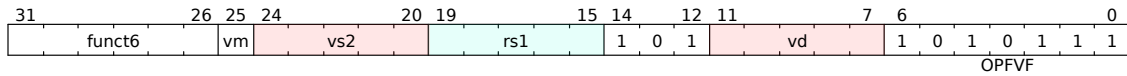


Figure 104: OPFVF Instruction

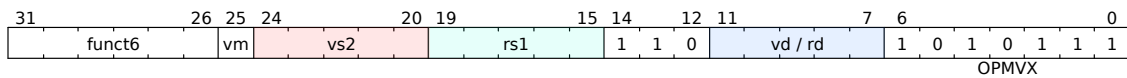


Figure 105: OPMVX Instruction

5.8.14.1 Vector Arithmetic Instruction Encoding

The funct3 field encodes the operand type and source locations.

funct3[2:0]			Operands	Source of scalar(s)	Notes
0	0	0	OPIVV	vector-vector	
0	0	1	OPFVV	vector-vector	
0	1	0	OPMVV	vector-vector	
0	1	1	OPIVI	vector-immediate	imm[4:0]
1	0	0	OPIVX	vector-scalar	GPR x register rs1
1	0	1	OPFVF	vector-scalar	FP f register rs1
1	1	0	OPMVX	vector-scalar	GPR x register rs1
1	1	1	OPCFG	scalar-immediate	GPR x register rs1 & rs2/imm

Table 78: funct3 Fields for Arithmetic Instructions

Integer operations are performed using unsigned or two's-complement signed integer arithmetic depending on the opcode.

All standard vector floating-point arithmetic operations follow the IEEE 754-2008 standard. All vector floating-point operations use the dynamic rounding mode in the `frm` register. If the `frm` field contains an invalid rounding mode, attempting to execute any vector floating-point instruction, even those that do not depend on the rounding mode, or when `v1=0`, or when `vstart ≥ v1`, will raise an illegal-instruction exception.

Vector-vector operations take two vectors of operands from vector register groups specified by `vs2` and `vs1`, respectively.

Vector-scalar operations can have three possible forms, but in all cases take one vector of operands from a vector register group specified by `vs2` and a second scalar source operand from one of three alternative sources.

1. For integer operations, the scalar can be a 5-bit immediate encoded in the `rs1` field. The value is sign- or zero-extended to SEW bits.
2. For integer operations, the scalar can be taken from the scalar `x` register specified by `rs1`. If $64 > \text{SEW}$, the least-significant SEW bits of the `x` register are used. If $64 < \text{SEW}$, the value from the `x` register is sign-extended to SEW bits.
3. For floating-point operations, the scalar can be taken from a scalar `f` register. If $\text{FLEN} > \text{SEW}$, the value in the `f` registers is checked for a valid NaN-boxed value, in which case the least-significant SEW bits of the `f` register are used. Otherwise, the canonical NaN value is used. If execution is attempted of a vector instruction where any floating-point vector operand's EEW is not a supported floating-point type width (which includes when $\text{FLEN} < \text{SEW}$), an illegal-instruction exception is raised.

Vector arithmetic instructions are masked under control of the `vm` field.

Assembly syntax pattern for vector binary arithmetic instructions

```
# Operations returning vector results, masked by vm (v0.t, <nothing>)
vop.vv vd, vs2, vs1, vm # integer vector-vector      vd[i] = vs2[i] op vs1[i]
vop.vx vd, vs2, rs1, vm # integer vector-scalar      vd[i] = vs2[i] op x[rs1]
vop.vi vd, vs2, imm, vm # integer vector-immediate   vd[i] = vs2[i] op imm

vfop.vv vd, vs2, vs1, vm # FP vector-vector operation vd[i] = vs2[i] fop vs1[i]
vfop.vf vd, vs2, rs1, vm # FP vector-scalar operation vd[i] = vs2[i] fop f[rs1]
```

Note

In the encoding, `vs2` is the first operand, while `rs1/simm5` is the second operand. This is the opposite of the standard scalar ordering. This arrangement retains the existing encoding conventions such that instructions that read only one scalar register, read it from `rs1`, and that 5-bit immediates are sourced from the `rs1` field.

```
# Assembly syntax pattern for vector ternary arithmetic instructions (multiply-add)

# Integer operations overwriting sum input
vop.vv vd, vs1, vs2, vm # vd[i] = vs1[i] * vs2[i] + vd[i]
vop.vx vd, rs1, vs2, vm # vd[i] = x[rs1] * vs2[i] + vd[i]

# Integer operations overwriting product input
vop.vv vd, vs1, vs2, vm # vd[i] = vs1[i] * vd[i] + vs2[i]
vop.vx vd, rs1, vs2, vm # vd[i] = x[rs1] * vd[i] + vs2[i]

# Floating-point operations overwriting sum input
vfop.vv vd, vs1, vs2, vm # vd[i] = vs1[i] * vs2[i] + vd[i]
vfop.vf vd, rs1, vs2, vm # vd[i] = f[rs1] * vs2[i] + vd[i]

# Floating-point operations overwriting product input
vfop.vv vd, vs1, vs2, vm # vd[i] = vs1[i] * vd[i] + vs2[i]
vfop.vf vd, rs1, vs2, vm # vd[i] = f[rs1] * vd[i] + vs2[i]
```

5.8.14.2 Widening Vector Arithmetic Instructions

A few vector arithmetic instructions are defined to be *widening* operations where the destination elements have $EEW=2*SEW$ and $EMUL=2*LMUL$.

The first operand can be either single- or double-width. These are generally written with a *vw** prefix on the opcode or *vfw** for vector floating-point operations.

```
# Assembly syntax pattern for vector widening arithmetic instructions

# Double-width result, two single-width sources: 2*SEW = SEW op SEW
vwop.vv vd, vs2, vs1, vm # integer vector-vector vd[i] = vs2[i] op vs1[i]
vwop.vx vd, vs2, rs1, vm # integer vector-scalar vd[i] = vs2[i] op x[rs1]

# Double-width result, double-width and single-width sources: 2*SEW = 2*SEW op SEW
vwop.wv vd, vs2, vs1, vm # integer vector-vector vd[i] = vs2[i] op vs1[i]
vwop.wx vd, vs2, rs1, vm # integer vector-scalar vd[i] = vs2[i] op x[rs1]
```

5.8.14.3 Narrowing Vector Arithmetic Instructions

A few instructions are provided to convert double-width source vectors into single-width destination vectors. These instructions convert a vector register group with $EEW/EMUL = 2*SEW/2*LMUL$ to a vector register group with the current $LMUL/SEW$ vectors/elements.

If $EEW > ELEN$ or $EMUL > 8$, an illegal-instruction exception is raised.

5.8.15 Vector Integer Arithmetic Instructions

A set of vector integer arithmetic instructions is provided.

5.8.15.1 Vector Single-Width Integer Add and Subtract

Vector integer add and subtract are provided. Reverse-subtract instructions are also provided for the vector-scalar forms.

```
# Integer add
vadd.vv vd, vs2, vs1, vm # vector-vector
vadd.vx vd, vs2, rs1, vm # vector-scalar
vadd.vi vd, vs2, imm, vm # vector-immediate

# Integer subtract
vsub.vv vd, vs2, vs1, vm # vector-vector
vsub.vx vd, vs2, rs1, vm # vector-scalar

# Integer reverse subtract
vrsub.vx vd, vs2, rs1, vm # vd[i] = x[rs1] - vs2[i]
vrsub.vi vd, vs2, imm, vm # vd[i] = imm - vs2[i]
```

5.8.15.2 Vector Widening Integer Add/Subtract

The widening add/subtract instructions are provided in both signed and unsigned variants, depending on whether the narrower source operands are first sign- or zero-extended before forming the double-width sum.

```
# Widening unsigned integer add/subtract, 2*SEW = SEW +/- SEW
vwaddu.vv vd, vs2, vs1, vm # vector-vector
vwaddu.vx vd, vs2, rs1, vm # vector-scalar
vwsubu.vv vd, vs2, vs1, vm # vector-vector
vwsubu.vx vd, vs2, rs1, vm # vector-scalar

# Widening signed integer add/subtract, 2*SEW = SEW +/- SEW
vwadd.vv vd, vs2, vs1, vm # vector-vector
vwadd.vx vd, vs2, rs1, vm # vector-scalar
vwsub.vv vd, vs2, vs1, vm # vector-vector
vwsub.vx vd, vs2, rs1, vm # vector-scalar

# Widening unsigned integer add/subtract, 2*SEW = 2*SEW +/- SEW
vwaddu.wv vd, vs2, vs1, vm # vector-vector
vwaddu.wx vd, vs2, rs1, vm # vector-scalar
vwsubu.wv vd, vs2, vs1, vm # vector-vector
vwsubu.wx vd, vs2, rs1, vm # vector-scalar

# Widening signed integer add/subtract, 2*SEW = 2*SEW +/- SEW
vwadd.wv vd, vs2, vs1, vm # vector-vector
vwadd.wx vd, vs2, rs1, vm # vector-scalar
vwsub.wv vd, vs2, vs1, vm # vector-vector
vwsub.wx vd, vs2, rs1, vm # vector-scalar
```

Note

An integer value can be doubled in width using the widening add instructions with a scalar operand of x0. Can define assembly pseudoinstructions `vwcvt.x.x.v vd, vs, vm = vwadd.vx vd, vs, x0, vm` and `vwcvtu.x.x.v vd, vs, vm = vwaddu.vx vd, vs, x0, vm`.

5.8.15.3 Vector Integer Extension

The vector integer extension instructions zero- or sign-extend a source vector integer operand with EEW less than SEW to fill SEW-sized elements in the destination. The EEW of the source is 1/2, 1/4, or 1/8 of SEW, while the EMUL of the source is (EEW/SEW)*LMUL. The destination has EEW equal to SEW and EMUL equal to LMUL.

```
vzext.vf2 vd, vs2, vm # Zero-extend SEW/2 source to SEW destination
vsext.vf2 vd, vs2, vm # Sign-extend SEW/2 source to SEW destination
vzext.vf4 vd, vs2, vm # Zero-extend SEW/4 source to SEW destination
vsext.vf4 vd, vs2, vm # Sign-extend SEW/4 source to SEW destination
vzext.vf8 vd, vs2, vm # Zero-extend SEW/8 source to SEW destination
vsext.vf8 vd, vs2, vm # Sign-extend SEW/8 source to SEW destination
```

If the source EEW is not a supported width, or source EMUL would be below the minimum legal LMUL, an illegal-instruction exception is raised.

5.8.15.4 Vector Integer Add-with-Carry / Subtract-with-Borrow Instructions

To support multi-word integer arithmetic, instructions that operate on a carry bit are provided. For each operation (add or subtract), two instructions are provided: one to provide the result (SEW width), and the second to generate the carry output (single bit encoded as a mask boolean).

vadc and vsbc add or subtract the source operands and the carry-in or borrow-in, and write the result to vector register vd. These instructions are encoded as masked instructions (vm=0), but they operate on and write-back all body elements. Encodings corresponding to the unmasked versions (vm=1) are reserved.

vadc and vsbc add or subtract the source operands, and optionally add the carry-in or subtract the borrow-in if masked (vm=0), and write the result back to mask register vd. If unmasked (vm=1), there is no carry-in or borrow-in. These instructions operate on and write back all body elements, even if masked.

```
# Produce sum with carry

# vd[i] = vs2[i] + vs1[i] + v0[i].LSB
vadc.vvm vd, vs2, vs1, v0 # vector-vector

# vd[i] = vs2[i] + x[rs1] + v0[i].LSB
vadc.vxm vd, vs2, rs1, v0 # vector-scalar

# vd[i] = vs2[i] + imm + v0[i].LSB
vadc.vim vd, vs2, imm, v0 # vector-immediate

# Produce carry out in mask register format

# vd[i] = carry_out(vs2[i] + vs1[i] + v0[i].LSB)
vmadc.vvm vd, vs2, vs1, v0 # vector-vector

# vd[i] = carry_out(vs2[i] + x[rs1] + v0[i].LSB)
vmadc.vxm vd, vs2, rs1, v0 # vector-scalar

# vd[i] = carry_out(vs2[i] + imm + v0[i].LSB)
vmadc.vim vd, vs2, imm, v0 # vector-immediate

# vd[i] = carry_out(vs2[i] + vs1[i])
vmadc.vv vd, vs2, vs1 # vector-vector, no carry-in

# vd[i] = carry_out(vs2[i] + x[rs1])
vmadc.vx vd, vs2, rs1 # vector-scalar, no carry-in

# vd[i] = carry_out(vs2[i] + imm)
vmadc.vi vd, vs2, imm # vector-immediate, no carry-in
```

Implementing a carry propagation requires executing two instructions with unchanged inputs; as such, destructive accumulations will require an additional move to obtain correct results.

```
# Example multi-word arithmetic sequence, accumulating into v4
vmadc.vvm v1, v4, v8, v0 # Get carry into temp register v1
vadc.vvm v4, v4, v8, v0 # Calc new sum
vmmv.m v0, v1 # Move temp carry into v0 for next word
```

The subtract-with-borrow instruction vsbc performs the equivalent function to support long word arithmetic for subtraction. There are no subtract-with-immediate instructions.

```
# Produce difference with borrow

# vd[i] = vs2[i] - vs1[i] - v0[i].LSB
vsbc.vvm vd, vs2, vs1, v0 # vector-vector

# vd[i] = vs2[i] - x[rs1] - v0[i].LSB
vsbc.vxm vd, vs2, rs1, v0 # vector-scalar

# Produce borrow out in mask register format

# vd[i] = borrow_out(vs2[i] - vs1[i] - v0[i].LSB)
vmsbc.vvm vd, vs2, vs1, v0 # vector-vector

# vd[i] = borrow_out(vs2[i] - x[rs1] - v0[i].LSB)
vmsbc.vxm vd, vs2, rs1, v0 # vector-scalar

# vd[i] = borrow_out(vs2[i] - vs1[i])
vmsbc.vv vd, vs2, vs1 # vector-vector, no borrow-in

# vd[i] = borrow_out(vs2[i] - x[rs1])
vmsbc.vx vd, vs2, rs1 # vector-scalar, no borrow-in
```

For vmsbc, the borrow is defined to be 1 if the difference, prior to truncation, is negative.

For vadc and vsbc, an illegal-instruction exception is raised if the destination vector register is v0.

5.8.15.5 Vector Bitwise Logical Instructions

```
# Bitwise logical operations
vand.vv vd, vs2, vs1, vm # vector-vector
vand.vx vd, vs2, rs1, vm # vector-scalar
vand.vi vd, vs2, imm, vm # vector-immediate

vor.vv vd, vs2, vs1, vm # vector-vector
vor.vx vd, vs2, rs1, vm # vector-scalar
vor.vi vd, vs2, imm, vm # vector-immediate

vxor.vv vd, vs2, vs1, vm # vector-vector
vxor.vx vd, vs2, rs1, vm # vector-scalar
vxor.vi vd, vs2, imm, vm # vector-immediate
```

5.8.15.6 Vector Single-Width Bit Shift Instructions

The full complement of vector shift instructions are provided, including logical shift left, and logical (zero-extending) and arithmetic (sign-extending) shift right.

```
# Bit shift operations
vsll.vv vd, vs2, vs1, vm # vector-vector
vsll.vx vd, vs2, rs1, vm # vector-scalar
vsll.vi vd, vs2, uimm, vm # vector-immediate

vsrl.vv vd, vs2, vs1, vm # vector-vector
vsrl.vx vd, vs2, rs1, vm # vector-scalar
vsrl.vi vd, vs2, uimm, vm # vector-immediate

vsra.vv vd, vs2, vs1, vm # vector-vector
vsra.vx vd, vs2, rs1, vm # vector-scalar
vsra.vi vd, vs2, uimm, vm # vector-immediate
```

Only the low $\log_2(\text{SEW})$ bits are read to obtain the shift amount from a register value.

The immediate is treated as an unsigned shift amount, with a maximum shift amount of 31.

5.8.15.7 Vector Narrowing Integer Right Shift Instructions

The narrowing right shifts extract a smaller field from a wider operand and have both zero-extending (srl) and sign-extending (sra) forms. The shift amount can come from a vector or a scalar x register or a 5-bit immediate. The low $\log_2(2*\text{SEW})$ bits of the vector or scalar shift amount value are used (e.g., the low 6 bits for a SEW=64-bit to SEW=32-bit narrowing operation). The unsigned immediate form supports shift amounts up to 31 only.

```
# Narrowing shift right logical, SEW = (2*SEW) >> SEW
vnsrl.vv vd, vs2, vs1, vm # vector-vector
vnsrl.vx vd, vs2, rs1, vm # vector-scalar
vnsrl.vi vd, vs2, uimm, vm # vector-immediate

# Narrowing shift right arithmetic, SEW = (2*SEW) >> SEW
vnsra.vv vd, vs2, vs1, vm # vector-vector
vnsra.vx vd, vs2, rs1, vm # vector-scalar
vnsra.vi vd, vs2, uimm, vm # vector-immediate
```

5.8.15.8 Vector Integer Comparison Instructions

The following integer compare instructions write 1 to the destination mask register element if the comparison evaluates to true, and 0 otherwise.

```
# Set if equal
vmseq.vv vd, vs2, vs1, vm # vector-vector
vmseq.vx vd, vs2, rs1, vm # vector-scalar
vmseq.vi vd, vs2, imm, vm # vector-immediate

# Set if not equal
vmsne.vv vd, vs2, vs1, vm # vector-vector
vmsne.vx vd, vs2, rs1, vm # vector-scalar
vmsne.vi vd, vs2, imm, vm # vector-immediate

# Set if less than, unsigned
vmsltu.vv vd, vs2, vs1, vm # vector-vector
vmsltu.vx vd, vs2, rs1, vm # vector-scalar

# Set if less than, signed
vmslt.vv vd, vs2, vs1, vm # vector-vector
vmslt.vx vd, vs2, rs1, vm # vector-scalar

# Set if less than or equal, unsigned
vmsleu.vv vd, vs2, vs1, vm # vector-vector
vmsleu.vx vd, vs2, rs1, vm # vector-scalar
vmsleu.vi vd, vs2, imm, vm # vector-immediate

# Set if less than or equal, signed
vmsle.vv vd, vs2, vs1, vm # vector-vector
vmsle.vx vd, vs2, rs1, vm # vector-scalar
vmsle.vi vd, vs2, imm, vm # vector-immediate

# Set if greater than, unsigned
vmgtu.vx vd, vs2, rs1, vm # vector-scalar
vmgtu.vi vd, vs2, imm, vm # vector-immediate

# Set if greater than, signed
vmgt.vx vd, vs2, rs1, vm # vector-scalar
vmgt.vi vd, vs2, imm, vm # vector-immediate
```

5.8.15.9 Vector Integer Min/Max Instructions

Signed and unsigned integer minimum and maximum instructions are supported.

```
# Unsigned minimum
vminu.vv vd, vs2, vs1, vm # vector-vector
vminu.vx vd, vs2, rs1, vm # vector-scalar

# Signed minimum
vmin.vv vd, vs2, vs1, vm # vector-vector
vmin.vx vd, vs2, rs1, vm # vector-scalar

# Unsigned maximum
vmaxu.vv vd, vs2, vs1, vm # vector-vector
vmaxu.vx vd, vs2, rs1, vm # vector-scalar

# Signed maximum
vmax.vv vd, vs2, vs1, vm # vector-vector
vmax.vx vd, vs2, rs1, vm # vector-scalar
```

5.8.15.10 Vector Single-Width Integer Multiply Instructions

The single-width multiply instructions perform a SEW-bit*SEW-bit multiply and return an SEW-bit-wide result. The `vmulh*` versions write the high word of the product to the destination register.

```
# Signed multiply, returning low bits of product
vmul.vv vd, vs2, vs1, vm    # vector-vector
vmul.vx vd, vs2, rs1, vm    # vector-scalar

# Signed multiply, returning high bits of product
vmulh.vv vd, vs2, vs1, vm   # vector-vector
vmulh.vx vd, vs2, rs1, vm   # vector-scalar

# Unsigned multiply, returning high bits of product
vmulhu.vv vd, vs2, vs1, vm  # vector-vector
vmulhu.vx vd, vs2, rs1, vm  # vector-scalar

# Signed(vs2)-Unsigned multiply, returning high bits of product
vmulhsu.vv vd, vs2, vs1, vm # vector-vector
vmulhsu.vx vd, vs2, rs1, vm # vector-scalar
```

5.8.15.11 Vector Integer Divide Instructions

The divide and remainder instructions are equivalent to the RISC-V standard scalar integer multiply/divides, with the same results for extreme inputs.

```
# Unsigned divide
vdivu.vv vd, vs2, vs1, vm # vector-vector
vdivu.vx vd, vs2, rs1, vm # vector-scalar

# Signed divide
vdiv.vv vd, vs2, vs1, vm  # vector-vector
vdiv.vx vd, vs2, rs1, vm  # vector-scalar

# Unsigned remainder
vremu.vv vd, vs2, vs1, vm # vector-vector
vremu.vx vd, vs2, rs1, vm # vector-scalar

# Signed remainder
vrem.vv vd, vs2, vs1, vm  # vector-vector
vrem.vx vd, vs2, rs1, vm  # vector-scalar
```

Note

There is no instruction to perform a "scalar divide by vector" instruction.

5.8.15.12 Vector Widening Integer Multiply Instructions

The widening integer multiply instructions return the full 2*SEW-bit product from an SEW-bit*SEW-bit multiply.

```
# Widening signed-integer multiply
vwmul.vv vd, vs2, vs1, vm # vector-vector
vwmul.vx vd, vs2, rs1, vm # vector-scalar

# Widening unsigned-integer multiply
vwmulu.vv vd, vs2, vs1, vm # vector-vector
vwmulu.vx vd, vs2, rs1, vm # vector-scalar

# Widening signed-unsigned integer multiply
vwmulsu.vv vd, vs2, vs1, vm # vector-vector
vwmulsu.vx vd, vs2, rs1, vm # vector-scalar
```

5.8.15.13 Vector Single-Width Integer Multiply-Add Instructions

The integer multiply-add instructions are destructive and are provided in two forms, one that overwrites the addend or minuend (vmacc, vnmsac) and one that overwrites the first multiplicand (vmadd, vnmsub).

The low half of the product is added or subtracted from the third operand.

```
# Integer multiply-add, overwrite addend
vmacc.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) + vd[i]
vmacc.vx vd, rs1, vs2, vm # vd[i] = +(x[rs1] * vs2[i]) + vd[i]

# Integer multiply-sub, overwrite minuend
vnmsac.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vs2[i]) + vd[i]
vnmsac.vx vd, rs1, vs2, vm # vd[i] = -(x[rs1] * vs2[i]) + vd[i]

# Integer multiply-add, overwrite multiplicand
vmadd.vv vd, vs1, vs2, vm # vd[i] = (vs1[i] * vd[i]) + vs2[i]
vmadd.vx vd, rs1, vs2, vm # vd[i] = (x[rs1] * vd[i]) + vs2[i]

# Integer multiply-sub, overwrite multiplicand
vnmsub.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vd[i]) + vs2[i]
vnmsub.vx vd, rs1, vs2, vm # vd[i] = -(x[rs1] * vd[i]) + vs2[i]
```

5.8.15.14 Vector Widening Integer Multiply-Add Instructions

The widening integer multiply-add instructions add a SEW-bit*SEW-bit multiply result to/from a 2*SEW-bit value and produce a 2*SEW-bit result. All combinations of signed and unsigned multiply operands are supported.


```
# Widening unsigned-integer multiply-add, overwrite addend
vwmaccu.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) + vd[i]
vwmaccu.vx vd, rs1, vs2, vm # vd[i] = +(x[rs1] * vs2[i]) + vd[i]

# Widening signed-integer multiply-add, overwrite addend
vwmac.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) + vd[i]
vwmac.vx vd, rs1, vs2, vm # vd[i] = +(x[rs1] * vs2[i]) + vd[i]

# Widening signed-unsigned-integer multiply-add, overwrite addend
vwmaccsu.vv vd, vs1, vs2, vm # vd[i] = +(signed(vs1[i]) * unsigned(vs2[i])) + vd[i]
vwmaccsu.vx vd, rs1, vs2, vm # vd[i] = +(signed(x[rs1]) * unsigned(vs2[i])) + vd[i]

# Widening unsigned-signed-integer multiply-add, overwrite addend
vwmaccus.vx vd, rs1, vs2, vm # vd[i] = +(unsigned(x[rs1]) * signed(vs2[i])) + vd[i]
```

5.8.15.15 Vector Integer Merge Instructions

The vector integer merge instructions combine two source operands based on a mask. Unlike regular arithmetic instructions, the merge operates on all body elements (i.e., the set of elements from `vstart` up to the current vector length in `v1`).

The `vmerge` instructions are always masked (`vm=0`). The instructions combine two sources as follows. At elements where the mask value is zero, the first operand is copied to the destination element; otherwise, the second operand is copied to the destination element. The first operand is always a vector register group specified by `vs2`. The second operand is a vector register group specified by `vs1` or a scalar `x` register specified by `rs1` or a 5-bit sign-extended immediate.

```
vmerge.vvm vd, vs2, vs1, v0 # vd[i] = v0.mask[i] ? vs1[i] : vs2[i]
vmerge.vxm vd, vs2, rs1, v0 # vd[i] = v0.mask[i] ? x[rs1] : vs2[i]
vmerge.vim vd, vs2, imm, v0 # vd[i] = v0.mask[i] ? imm : vs2[i]
```

5.8.15.16 Vector Integer Move Instructions

The vector integer move instructions copy a source operand to a vector register group. The `vmv.v.v` variant copies a vector register group, whereas the `vmv.v.x` and `vmv.v.i` variants *splat* a scalar register or immediate to all active elements of the destination vector register group. These instructions are always unmasked (`vm=1`). The first operand specifier (`vs2`) must contain `v0`, and any other vector register number in `vs2` is *reserved*.

```
vmv.v.v vd, vs1 # vd[i] = vs1[i]
vmv.v.x vd, rs1 # vd[i] = x[rs1]
vmv.v.i vd, imm # vd[i] = imm
```

5.8.16 Vector Fixed-Point Arithmetic Instructions

A set of vector arithmetic instructions is provided to support fixed-point arithmetic.

An N-bit element can hold two's-complement signed integers in the range $-2^{N-1} \dots +2^{N-1}-1$, and unsigned integers in the range $0 \dots +2^N-1$. The fixed-point instructions help preserve precision in narrow operands by supporting scaling and rounding, and can handle overflow by saturating results into the destination format range.

Note

The widening integer operations described in Section 5.8.15.12 and Section 5.8.15.2 can also be used to remove the possibility of overflow.

5.8.16.1 Vector Single-Width Saturating Add and Subtract

Saturating forms of integer add and subtract are provided, for both signed and unsigned integers. If the result would overflow the destination, the result is replaced with the closest representable value, and the `vxsat` bit is set.

```
# Saturating adds of unsigned integers
vsaddu.vv vd, vs2, vs1, vm # vector-vector
vsaddu.vx vd, vs2, rs1, vm # vector-scalar
vsaddu.vi vd, vs2, imm, vm # vector-immediate

# Saturating adds of signed integers
vsadd.vv vd, vs2, vs1, vm # vector-vector
vsadd.vx vd, vs2, rs1, vm # vector-scalar
vsadd.vi vd, vs2, imm, vm # vector-immediate

# Saturating subtract of unsigned integers
vssubu.vv vd, vs2, vs1, vm # vector-vector
vssubu.vx vd, vs2, rs1, vm # vector-scalar

# Saturating subtract of signed integers
vssub.vv vd, vs2, vs1, vm # vector-vector
vssub.vx vd, vs2, rs1, vm # vector-scalar
```

5.8.16.2 Vector Single-Width Averaging Add and Subtract

The averaging add and subtract instructions right shift the result by one bit and round off the result according to the setting in `vxrm`. Both unsigned and signed versions are provided. For `vaaddu` and `vaadd`, there can be no overflow in the result. For `vasub` and `vasubu`, overflow is ignored and the result wraps around.

Note

For `vasub`, overflow occurs only when subtracting the smallest number from the largest number under RNU or RNE rounding.

```
# Averaging add

# Averaging adds of unsigned integers
vaaddu.vv vd, vs2, vs1, vm # roundoff_unsigned(vs2[i] + vs1[i], 1)
vaaddu.vx vd, vs2, rs1, vm # roundoff_unsigned(vs2[i] + x[rs1], 1)

# Averaging adds of signed integers
vaadd.vv vd, vs2, vs1, vm # roundoff_signed(vs2[i] + vs1[i], 1)
vaadd.vx vd, vs2, rs1, vm # roundoff_signed(vs2[i] + x[rs1], 1)

# Averaging subtract

# Averaging subtract of unsigned integers
vasubu.vv vd, vs2, vs1, vm # roundoff_unsigned(vs2[i] - vs1[i], 1)
vasubu.vx vd, vs2, rs1, vm # roundoff_unsigned(vs2[i] - x[rs1], 1)

# Averaging subtract of signed integers
vasub.vv vd, vs2, vs1, vm # roundoff_signed(vs2[i] - vs1[i], 1)
vasub.vx vd, vs2, rs1, vm # roundoff_signed(vs2[i] - x[rs1], 1)
```

5.8.16.3 Vector Single-Width Fractional Multiply with Rounding and Saturation

The signed fractional multiply instruction produces a 2*SEW product of the two SEW inputs, then shifts the result right by SEW-1 bits, rounding these bits according to vxrm, and then saturates the result to fit into SEW bits. If the result causes saturation, the vxsat bit is set.

```
# Signed saturating and rounding fractional multiply
# See vxrm description for rounding calculation
vsmul.vv vd, vs2, vs1, vm # vd[i] = clip(roundoff_signed(vs2[i]*vs1[i], SEW-1))
vsmul.vx vd, vs2, rs1, vm # vd[i] = clip(roundoff_signed(vs2[i]*x[rs1], SEW-1))
```

5.8.16.4 Vector Single-Width Scaling Shift Instructions

These instructions shift the input value right, and rounds off the shifted out bits according to vxrm. The scaling right shifts have both zero-extending (vssrl) and sign-extending (vssra) forms. The low log₂(SEW) bits of the vector or scalar shift amount value are used. The immediate form supports shift amounts up to 31 only.

```
# Scaling shift right logical
vssrl.vv vd, vs2, vs1, vm # vd[i] = roundoff_unsigned(vs2[i], vs1[i])
vssrl.vx vd, vs2, rs1, vm # vd[i] = roundoff_unsigned(vs2[i], x[rs1])
vssrl.vi vd, vs2, uimm, vm # vd[i] = roundoff_unsigned(vs2[i], uimm)

# Scaling shift right arithmetic
vssra.vv vd, vs2, vs1, vm # vd[i] = roundoff_signed(vs2[i], vs1[i])
vssra.vx vd, vs2, rs1, vm # vd[i] = roundoff_signed(vs2[i], x[rs1])
vssra.vi vd, vs2, uimm, vm # vd[i] = roundoff_signed(vs2[i], uimm)
```

5.8.16.5 Vector Narrowing Fixed-Point Clip Instructions

The vnclip instructions are used to pack a fixed-point value into a narrower destination. The instructions support rounding, scaling, and saturation into the final destination format.

The second argument (vector element, scalar value, immediate value) gives the amount to right shift the source as in the narrowing shift instructions, which provides the scaling. The low $\log_2(2 \cdot \text{SEW})$ bits of the vector or scalar shift amount value are used (e.g., the low 6 bits for a SEW=64-bit to SEW=32-bit narrowing operation). The immediate form supports shift amounts up to 31 only.

```
# Narrowing unsigned clip
                                # SEW          2*SEW  SEW
vnclipu.wv vd, vs2, vs1, vm    # vd[i] = clip(roundoff_unsigned(vs2[i], vs1[i]))
vnclipu.wx vd, vs2, rs1, vm    # vd[i] = clip(roundoff_unsigned(vs2[i], x[rs1]))
vnclipu.wi vd, vs2, uimm, vm   # vd[i] = clip(roundoff_unsigned(vs2[i], uimm5))

# Narrowing signed clip
vnclip.wv vd, vs2, vs1, vm     # vd[i] = clip(roundoff_signed(vs2[i], vs1[i]))
vnclip.wx vd, vs2, rs1, vm     # vd[i] = clip(roundoff_signed(vs2[i], x[rs1]))
vnclip.wi vd, vs2, uimm, vm    # vd[i] = clip(roundoff_signed(vs2[i], uimm5))
```

- For vnclipu/vnclip, the rounding mode is specified in the vxrm CSR. Rounding occurs around the least-significant bit of the destination and before saturation.
- For vnclipu, the shifted rounded source value is treated as an unsigned integer and saturates if the result would overflow the destination viewed as an unsigned integer.
- For vnclip, the shifted rounded source value is treated as a signed integer and saturates if the result would overflow the destination viewed as a signed integer.

If any destination element is saturated, the vxsat bit is set in the vxsat register.

5.8.17 Vector Floating-Point Instructions

The standard vector floating-point instructions treat 16-bit, 32-bit, and 64-bit elements as IEEE 754-2008-compatible values. If the EEW of a vector floating-point operand does not correspond to a supported IEEE floating-point type, an illegal-instruction exception is raised.

If the floating-point unit status field `mstatus.FS` is Off then any attempt to execute a vector floating-point instruction will raise an illegal-instruction exception. Any vector floating-point instruction that modifies any floating-point extension state (i.e., floating-point CSRs or f registers) must set `mstatus.FS` to Dirty.

The vector floating-point instructions have the same behavior as the scalar floating-point instructions with regard to NaNs.

Scalar values for vector-scalar operations can be sourced from the standard scalar f registers, as described in Section 5.8.14.1.

5.8.17.1 Vector Floating-Point Exception Flags

A vector floating-point exception at any active floating-point element sets the standard floating-point exception flags in the `fflags` register. Inactive elements do not set floating-point exception flags.

5.8.17.2 Vector Single-Width Floating-Point Add/Subtract Instructions

```
# Floating-point add
vfadd.vv vd, vs2, vs1, vm # vector-vector
vfadd.vf vd, vs2, rs1, vm # vector-scalar

# Floating-point subtract
vfsub.vv vd, vs2, vs1, vm # vector-vector
vfsub.vf vd, vs2, rs1, vm # vector-scalar vd[i] = vs2[i] - f[rs1]
vfrsub.vf vd, vs2, rs1, vm # scalar-vector vd[i] = f[rs1] - vs2[i]
```

5.8.17.3 Vector Widening Floating-Point Add/Subtract Instructions

```
# Widening FP add/subtract, 2*SEW = SEW +/- SEW
vfwadd.vv vd, vs2, vs1, vm # vector-vector
vfwadd.vf vd, vs2, rs1, vm # vector-scalar
vfwsb.vv vd, vs2, vs1, vm # vector-vector
vfwsb.vf vd, vs2, rs1, vm # vector-scalar

# Widening FP add/subtract, 2*SEW = 2*SEW +/- SEW
vfwadd.wv vd, vs2, vs1, vm # vector-vector
vfwadd.wf vd, vs2, rs1, vm # vector-scalar
vfwsb.wv vd, vs2, vs1, vm # vector-vector
vfwsb.wf vd, vs2, rs1, vm # vector-scalar
```

5.8.17.4 Vector Single-Width Floating-Point Multiply/Divide Instructions

```
# Floating-point multiply
vfmul.vv vd, vs2, vs1, vm # vector-vector
vfmul.vf vd, vs2, rs1, vm # vector-scalar

# Floating-point divide
vfdiv.vv vd, vs2, vs1, vm # vector-vector
vfdiv.vf vd, vs2, rs1, vm # vector-scalar

# Reverse floating-point divide vector = scalar / vector
vfrdiv.vf vd, vs2, rs1, vm # scalar-vector, vd[i] = f[rs1]/vs2[i]
```

5.8.17.5 Vector Single-Width Floating-Point Fused Multiply-Add Instructions

All four varieties of fused multiply-add are provided, as well as in two destructive forms that overwrite one of the operands, either the addend or the first multiplicand.

```
# FP multiply-accumulate, overwrites addend
vfmac.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) + vd[i]
vfmac.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vs2[i]) + vd[i]

# FP negate-(multiply-accumulate), overwrites subtrahend
vfnm.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vs2[i]) - vd[i]
vfnm.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vs2[i]) - vd[i]

# FP multiply-subtract-accumulator, overwrites subtrahend
vfms.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) - vd[i]
vfms.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vs2[i]) - vd[i]

# FP negate-(multiply-subtract-accumulator), overwrites minuend
vfms.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vs2[i]) + vd[i]
vfms.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vs2[i]) + vd[i]

# FP multiply-add, overwrites multiplicand
vfma.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vd[i]) + vs2[i]
vfma.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vd[i]) + vs2[i]

# FP negate-(multiply-add), overwrites multiplicand
vfma.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vd[i]) - vs2[i]
vfma.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vd[i]) - vs2[i]

# FP multiply-sub, overwrites multiplicand
vfms.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vd[i]) - vs2[i]
vfms.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vd[i]) - vs2[i]

# FP negate-(multiply-sub), overwrites multiplicand
vfms.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vd[i]) + vs2[i]
vfms.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vd[i]) + vs2[i]
```

5.8.17.6 Vector Widening Floating-Point Fused Multiply-Add Instructions

The widening floating-point fused multiply-add instructions all overwrite the wide addend with the result. The multiplier inputs are all SEW wide, while the addend and destination is 2*SEW bits wide.

```
# FP widening multiply-accumulate, overwrites addend
vfwma.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) + vd[i]
vfwma.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vs2[i]) + vd[i]

# FP widening negate-(multiply-accumulate), overwrites addend
vfwma.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vs2[i]) - vd[i]
vfwma.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vs2[i]) - vd[i]

# FP widening multiply-subtract-accumulator, overwrites addend
vfwms.vv vd, vs1, vs2, vm # vd[i] = +(vs1[i] * vs2[i]) - vd[i]
vfwms.vf vd, rs1, vs2, vm # vd[i] = +(f[rs1] * vs2[i]) - vd[i]

# FP widening negate-(multiply-subtract-accumulator), overwrites addend
vfwms.vv vd, vs1, vs2, vm # vd[i] = -(vs1[i] * vs2[i]) + vd[i]
vfwms.vf vd, rs1, vs2, vm # vd[i] = -(f[rs1] * vs2[i]) + vd[i]
```

5.8.17.7 Vector Floating-Point Square-Root Instruction

This is a unary vector-vector instruction.

```
# Floating-point square root
vfsqrt.v vd, vs2, vm # vector-vector square root
```

5.8.17.8 Vector Floating-Point Reciprocal Estimate Instruction

This is a unary vector-vector instruction that returns an estimate of $1/x$, accurate to 7 bits. The instruction behaves identically to a full IEEE $+1.0/x$ operation for values outside its domain.

```
# Floating-point reciprocal estimate to 7 bits.
vfrece7.v vd, vs2, vm
```

5.8.17.9 Vector Floating-Point Reciprocal Square-Root Estimate Instruction

This is a unary vector-vector instruction that returns an estimate of $1/\sqrt{x}$, accurate to 7 bits.

```
# Floating-point reciprocal square-root estimate to 7 bits.
vfrsqste7.v vd, vs2, vm
```

5.8.17.10 Vector Floating-Point MIN/MAX Instructions

The vector floating-point `vfmin` and `vfmax` instructions have the same behavior as the corresponding scalar floating-point instructions in version 2.2 of the RISC-V F/D/Q extensions.

```
# Floating-point minimum
vfmin.vv vd, vs2, vs1, vm # vector-vector
vfmin.vf vd, vs2, rs1, vm # vector-scalar

# Floating-point maximum
vfmax.vv vd, vs2, vs1, vm # vector-vector
vfmax.vf vd, vs2, rs1, vm # vector-scalar
```

5.8.17.11 Vector Floating-Point Sign-Injection Instructions

Vector versions of the scalar sign-injection instructions. The result takes all bits except the sign bit from the vector `vs2` operands.

```
vfsgnj.vv vd, vs2, vs1, vm # vector-vector
vfsgnj.vf vd, vs2, rs1, vm # vector-scalar

vfsgnjn.vv vd, vs2, vs1, vm # vector-vector
vfsgnjn.vf vd, vs2, rs1, vm # vector-scalar

vfsgnjx.vv vd, vs2, vs1, vm # vector-vector
vfsgnjx.vf vd, vs2, rs1, vm # vector-scalar
```

5.8.17.12 Vector Floating-Point Compare Instructions

These vector floating-point compare instructions compare two source operands and write the comparison result to a mask register. The destination mask vector register may be the same as the source vector mask register (*vm*).

The compare instructions follow the semantics of the scalar floating-point compare instructions. *vmfeq* and *vmfne* raise the invalid operation exception only on signaling NaN inputs. *vmflt*, *vmfle*, *vmfgt*, and *vmfge* raise the invalid operation exception on both signaling and quiet NaN inputs. *vmfne* writes 1 to the destination element when either operand is NaN, whereas the other comparisons write 0 when either operand is NaN.

```
# Compare equal
vmfeq.vv vd, vs2, vs1, vm # vector-vector
vmfeq.vf vd, vs2, rs1, vm # vector-scalar

# Compare not equal
vmfne.vv vd, vs2, vs1, vm # vector-vector
vmfne.vf vd, vs2, rs1, vm # vector-scalar

# Compare less than
vmflt.vv vd, vs2, vs1, vm # vector-vector
vmflt.vf vd, vs2, rs1, vm # vector-scalar

# Compare less than or equal
vmfle.vv vd, vs2, vs1, vm # vector-vector
vmfle.vf vd, vs2, rs1, vm # vector-scalar

# Compare greater than
vmfgt.vf vd, vs2, rs1, vm # vector-scalar

# Compare greater than or equal
vmfge.vf vd, vs2, rs1, vm # vector-scalar
```

In the instructions below:

- **va**, **vb** are vector register groups.
- **f** is the scalar floating-point register.

Comparison	Assembler Mapping	Assembler Pseudoinstruction
Vector to Vector		
$va < vb$	<code>vmflt.vv vd, va, vb, vm</code>	
$va \leq vb$	<code>vmfle.vv vd, va, vb, vm</code>	
$va > vb$	<code>vmflt.vv vd, vb, va, vm</code>	<code>vmfgt.vv vd, va, vb, vm</code>
$va \geq vb$	<code>vmfle.vv vd, vb, va, vm</code>	<code>vmfge.vv vd, va, vb, vm</code>
Vector to Scalar		
$va < f$	<code>vmflt.vf vd, va, f, vm</code>	
$va \leq f$	<code>vmfle.vf vd, va, f, vm</code>	
$va > f$	<code>vmfgt.vf vd, va, f, vm</code>	
$va \geq f$	<code>vmfge.vf vd, va, f, vm</code>	

Table 79: Compare Instructions in Assembly Format

5.8.17.13 Vector Floating-Point Classify Instruction

This is a unary vector-vector instruction that operates in the same way as the scalar classify instruction.

```
vfclass.v vd, vs2, vm # vector-vector
```

The 10-bit mask produced by this instruction is placed in the least-significant bits of the result elements. The upper (SEW-10) bits of the result are filled with zeros. The instruction is only defined for SEW=16-bit and above, so the result will always fit in the destination elements.

5.8.17.14 Vector Floating-Point Merge Instruction

A vector-scalar floating-point merge instruction is provided, which operates on all body elements, from `vstart` up to the current vector length in `v1` regardless of mask value.

The `vfmerge.vfm` instruction is always masked (`vm=0`). At elements where the mask value is zero, the first vector operand is copied to the destination element; otherwise, a scalar floating-point register value is copied to the destination element.

```
vfmerge.vfm vd, vs2, rs1, v0 # vd[i] = v0.mask[i] ? f[rs1] : vs2[i]
```

5.8.17.15 Vector Floating-Point Move Instruction

The vector floating-point move instruction *sp/ats* a floating-point scalar operand to a vector register group. The instruction copies a scalar `f` register value to all active elements of a vector register group. This instruction is always unmasked (`vm=1`). The instruction must have the `vs2` field set to `v0`, with all other values for `vs2` reserved.

```
vfmv.v.f vd, rs1 # vd[i] = f[rs1]
```

5.8.17.16 Single-Width Floating-Point/Integer Type-Convert Instructions

Conversion operations are provided to convert to and from floating-point values and unsigned and signed integers, where both source and destination are SEW wide.

Instruction	Description
VFCVT.XU.F.V vd, vs2, vm	Convert float to unsigned integer
VFCVT.X.F.V vd, vs2, vm	Convert float to signed integer
VFCVT.RTZ.XU.F.V vd, vs2, vm	Convert float to unsigned integer, truncating
VFCVT.RTZ.X.F.V vd, vs2, vm	Convert float to signed integer, truncating
VFCVT.F.XU.V vd, vs2, vm	Convert unsigned integer to float
VFCVT.F.X.V vd, vs2, vm	Convert signed integer to float

Table 80: Single-Width Floating-Point/Integer Type-Convert Instructions Description

5.8.17.17 Widening Floating-Point/Integer Type-Convert Instructions

A set of conversion instructions is provided to convert between narrower integer and floating-point datatypes to a type of twice the width.

Instruction	Description
VFWCVT.XU.F.V vd, vs2, vm	Convert float to double-width unsigned integer
VFWCVT.X.F.V vd, vs2, vm	Convert float to double-width signed integer
VFWCVT.RTZ.XU.F.V vd, vs2, vm	Convert float to double-width unsigned integer, truncating
VFWCVT.RTZ.X.F.V vd, vs2, vm	Convert float to double-width signed integer, truncating
VFWCVT.F.XU.V vd, vs2, vm	Convert unsigned integer to double-width float
VFWCVT.F.X.V vd, vs2, vm	Convert signed integer to double-width float
VFWCVT.F.F.V vd, vs2, vm	Convert single-width float to double-width float

Table 81: Widening Floating-Point/Integer Type-Convert Instructions Description

5.8.17.18 Narrowing Floating-Point/Integer Type-Convert Instructions

A set of conversion instructions is provided to convert wider integer and floating-point datatypes to a type of half the width.

Instruction	Description
VFNCVT.XU.F.W <i>vd, vs2, vm</i>	Convert double-width float to unsigned integer
VFNCVT.X.F.W <i>vd, vs2, vm</i>	Convert double-width float to signed integer
VFNCVT.RTZ.XU.F.W <i>vd, vs2, vm</i>	Convert double-width float to unsigned integer, truncating
VFNCVT.RTZ.X.F.W <i>vd, vs2, vm</i>	Convert double-width float to signed integer, truncating
VFNCVT.F.XU.W <i>vd, vs2, vm</i>	Convert double-width unsigned integer to float
VFNCVT.F.X.W <i>vd, vs2, vm</i>	Convert double-width signed integer to float
VFNCVT.F.F.W <i>vd, vs2, vm</i>	Convert double-width float to single-width float
VFNCVT.ROD.F.F.W <i>vd, vs2, vm</i>	Convert double-width float to single-width float, rounding towards odd

Table 82: Narrowing Floating-Point/Integer Type-Convert Instructions

5.8.18 Vector Reduction Operations

Vector reduction operations take a vector register group of elements and a scalar held in element 0 of a vector register, and performs a reduction using some binary operator, producing a scalar result in element 0 of a vector register. The scalar input and output operands are held in element 0 of a single vector register, not a vector register group, so any vector register can be the scalar source or destination of a vector reduction regardless of LMUL setting.

The destination vector register can overlap the source operands, including the mask register.

Inactive elements from the source vector register group are excluded from the reduction, but the scalar operand is always included regardless of the mask values.

The other elements in the destination vector register ($0 < \text{index} < \text{VLEN}/\text{SEW}$) are left unchanged.

If $v1=0$, no operation is performed and the destination register is not updated.

Traps on vector reduction instructions are always reported with a *vstart* of 0. Vector reduction operations raise an illegal-instruction exception if *vstart* is non-zero.

The assembler syntax for a reduction operation is *vredop .vs*, where the *.vs* suffix denotes the first operand is a vector register group and the second operand is a scalar stored in element 0 of a vector register.

5.8.18.1 Vector Single-Width Integer Reduction Instructions

All operands and results of single-width reduction instructions have the same SEW width. Overflows wrap around on arithmetic sums.

```
# Simple reductions, where [*] denotes all active elements
vredsum.vs vd, vs2, vs1, vm # vd[0] = sum(vs1[0], vs2[*])
vredmaxu.vs vd, vs2, vs1, vm # vd[0] = maxu(vs1[0], vs2[*])
vredmax.vs vd, vs2, vs1, vm # vd[0] = max(vs1[0], vs2[*])
vredminu.vs vd, vs2, vs1, vm # vd[0] = minu(vs1[0], vs2[*])
vredmin.vs vd, vs2, vs1, vm # vd[0] = min(vs1[0], vs2[*])
vredand.vs vd, vs2, vs1, vm # vd[0] = and(vs1[0], vs2[*])
vredor.vs vd, vs2, vs1, vm # vd[0] = or(vs1[0], vs2[*])
vredxor.vs vd, vs2, vs1, vm # vd[0] = xor(vs1[0], vs2[*])
```

5.8.18.2 Vector Widening Integer Reduction Instructions

The unsigned `vwredsumu.vs` instruction zero-extends the SEW-wide vector elements before summing them, then adds the 2*SEW-width scalar element, and stores the result in a 2*SEW-width scalar element.

The `vwredsum.vs` instruction sign-extends the SEW-wide vector elements before summing them.

```
# Unsigned sum reduction into double-width accumulator
vwredsumu.vs vd, vs2, vs1, vm # 2*SEW = 2*SEW + sum(zero-extend(SEW))

# Signed sum reduction into double-width accumulator
vwredsum.vs vd, vs2, vs1, vm # 2*SEW = 2*SEW + sum(sign-extend(SEW))
```

5.8.18.3 Vector Single-Width Floating-Point Reduction Instructions

```
# Simple reductions
vfredosum.vs vd, vs2, vs1, vm # Ordered sum
vfredusum.vs vd, vs2, vs1, vm # Unordered sum
vfredmax.vs vd, vs2, vs1, vm # Maximum value
vfredmin.vs vd, vs2, vs1, vm # Minimum value
```

5.8.18.4 Vector Widening Floating-Point Reduction Instructions

Widening forms of the sum reductions are provided that read and write a double-width reduction result.

```
# Simple reductions.
vfwredosum.vs vd, vs2, vs1, vm # Ordered sum
vfwredusum.vs vd, vs2, vs1, vm # Unordered sum
```

The reduction of the SEW-width elements is performed, as in the single-width reduction case, with the elements in `vs2` promoted to 2*SEW bits before adding to the 2*SEW-bit accumulator.

5.8.19 Vector Mask Instructions

Several instructions are provided to help operate on mask values held in a vector register.

5.8.19.1 Vector Mask-Register Logical Instructions

Vector mask-register logical operations operate on mask registers. Each element in a mask register is a single bit, so these instructions all operate on single vector registers regardless of the setting of the `v1mul` field in `vtype`. They do not change the value of `v1mul`. The destination vector register may be the same as either source vector register.

As with other vector instructions, the elements with indices less than `vstart` are unchanged, and `vstart` is reset to zero after execution. Vector mask logical instructions are always unmasked, so there are no inactive elements, and the encodings with `vm=0` are reserved. Mask elements past `v1`, the tail elements, are always updated with a tail-agnostic policy.

```
vmmand.mm vd, vs2, vs1    # vd.mask[i] = vs2.mask[i] && vs1.mask[i]
vmnand.mm vd, vs2, vs1    # vd.mask[i] = !(vs2.mask[i] && vs1.mask[i])
vmmandnot.mm vd, vs2, vs1 # vd.mask[i] = vs2.mask[i] && !vs1.mask[i]
vmxor.mm  vd, vs2, vs1    # vd.mask[i] = vs2.mask[i] ^^ vs1.mask[i]
vmor.mm   vd, vs2, vs1    # vd.mask[i] = vs2.mask[i] || vs1.mask[i]
vmnor.mm  vd, vs2, vs1    # vd.mask[i] = !(vs2.mask[i] || vs1.mask[i])
vmornot.mm vd, vs2, vs1   # vd.mask[i] = vs2.mask[i] || !vs1.mask[i]
vmxnor.mm vd, vs2, vs1    # vd.mask[i] = !(vs2.mask[i] ^^ vs1.mask[i])
```

Several assembler pseudoinstructions are defined as shorthand for common uses of mask logical operations:

```
vmmv.m  vd, vs -> vmmand.mm  vd, vs, vs # Copy mask register
vmclr.m vd      -> vmxor.mm   vd, vd, vd # Clear mask register
vmset.m vd      -> vmxnor.mm  vd, vd, vd # Set mask register
vmnot.m vd, vs  -> vmnand.mm  vd, vs, vs # Invert bits
```

The set of eight mask logical instructions can generate any of the 16 possible binary logical functions of the two input masks: `src1=0011` and `src2=0101`.

Output				Instruction	Pseudoinstruction
0	0	0	0	vmxor.mm vd, vd, vd	vmclr.m vd
0	0	0	1	vmand.mm vd, src1, src2	
0	0	1	0	vmandnot.mm vd, src1, src2	
0	0	1	1	vmand.mm vd, src1, src1	vmmv.m vd, src1
0	1	0	0	vmandnot.mm vd, src2, src1	
0	1	0	1	vmand.mm vd, src2, src2	vmmv.m vd, src2
0	1	1	0	vmxor.mm vd, src1, src2	
1	0	0	0	vmnor.mm vd, src1, src2	
1	0	0	1	vmxnor.mm vd, src1, src2	
1	0	1	0	vmnand.mm vd, src2, src2	vmnot.m vd, src2
1	0	1	1	vmornot.mm vd, src1, src2	
1	1	0	0	vmnand.mm vd, src1, src1	vmnot.m vd, src1
1	1	0	1	vmornot.mm vd, src2, src1	
1	1	1	0	vmnand.mm vd, src1, src2	
1	1	1	1	vmxnor.mm vd, vd, vd	vmset.m vd

Table 83: Vector Mask Logical Instructions

Note

The vector mask logical instructions are designed to be easily fused with a following masked vector operation to effectively expand the number of predicate registers by moving values into v0 before use.

5.8.19.2 Vector Count Population in Mask (vcpop.m)

vcpop.m rd, vs2, vm

The vcpop.m instruction counts the number of mask elements of the active elements of the vector source mask register that have the value 1 and writes the result to a scalar x register.

The operation can be performed under a mask, in which case only the masked elements are counted.

vcpop.m rd, vs2, v0.t # x[rd] = sum_i(vs2.mask[i] && v0.mask[i])

Traps on vcpop.m are always reported with a vstart of 0. The vcpop instruction will raise an illegal-instruction exception if vstart is non-zero.

5.8.19.3 Find-First-Set Mask Bit (vfirst)

vfirst.m rd, vs2, vm

The `vfirst` instruction finds the lowest-numbered active element of the source mask vector that has the value 1 and writes that element's index to a GPR. If no active element has the value 1, -1 is written to the GPR.

Traps on `vfirst` are always reported with a `vstart` of 0. The `vfirst` instruction will raise an illegal-instruction exception if `vstart` is non-zero.

5.8.19.4 Set-Before-First Mask Bit (`vmsbf.m`)

`vmsbf.m vd, vs2, vm`

Example

7 6 5 4 3 2 1 0	Element number
1 0 0 1 0 1 0 0	v3 contents
0 0 0 0 0 0 1 1	vmsbf.m v2, v3
	v2 contents
1 0 0 1 0 1 0 1	v3 contents
0 0 0 0 0 0 0 0	vmsbf.m v2, v3
	v2
0 0 0 0 0 0 0 0	v3 contents
1 1 1 1 1 1 1 1	vmsbf.m v2, v3
	v2
1 1 0 0 0 0 1 1	v0 vcontents
1 0 0 1 0 1 0 0	v3 contents
	vmsbf.m v2, v3, v0.t
0 1 x x x x 1 1	v2 contents

The `vmsbf.m` instruction takes a mask register as input and writes results to a mask register. The instruction writes a 1 to all active mask elements before the first source element that is a 1, then writes a 0 to that element and all following active elements. If there is no set bit in the source vector, then all active elements in the destination are written with a 1.

The tail elements in the destination mask register are handled according to the setting of the `vta` bit in `vtype` (see Section 5.8.3.3).

Traps on `vmsbf.m` are always reported with a `vstart` of 0. The `vmsbf` instruction will raise an illegal-instruction exception if `vstart` is non-zero.

The destination register cannot overlap the source register and, if masked, cannot overlap the mask register ('v0').

5.8.19.5 Set-Including-First Mask Bit (`vmsif.m`)

The vector mask set-including-first instruction is similar to set-before-first, except it also includes the element with a set bit.

`vmsif.m vd, vs2, vm`

Example

7 6 5 4 3 2 1 0	Element number
1 0 0 1 0 1 0 0	v3 contents
	<code>vmsif.m v2, v3</code>
0 0 0 0 0 1 1 1	v2 contents
1 0 0 1 0 1 0 1	v3 contents
	<code>vmsif.m v2, v3</code>
0 0 0 0 0 0 0 1	v2
1 1 0 0 0 0 1 1	v0 vcontents
1 0 0 1 0 1 0 0	v3 contents
	<code>vmsif.m v2, v3, v0.t</code>
1 1 x x x x 1 1	v2 contents

The tail elements in the destination mask register are handled according to the setting of the `vta` bit in `vtype` (see Section 5.8.3.3).

Traps on `vmsif.m` are always reported with a `vstart` of 0. The `vmsif` instruction will raise an illegal-instruction exception if `vstart` is non-zero.

The destination register cannot overlap the source register and, if masked, cannot overlap the mask register (`'v0'`).

5.8.19.6 Set-Only-First Mask Bit (`vmsof.m`)

The vector mask set-only-first instruction is similar to set-before-first, except it only sets the first element with a bit set, if any.

`vmsof.m vd, vs2, vm`

Example

7 6 5 4 3 2 1 0	Element number
1 0 0 1 0 1 0 0	v3 contents
	<code>vmsof.m v2, v3</code>
0 0 0 0 0 1 0 0	v2 contents
1 0 0 1 0 1 0 1	v3 contents
	<code>vmsof.m v2, v3</code>
0 0 0 0 0 0 0 1	v2
1 1 0 0 0 0 1 1	v0 vcontents
1 1 0 1 0 1 0 0	v3 contents
	<code>vmsof.m v2, v3, v0.t</code>
0 1 x x x x 0 0	v2 contents

The tail elements in the destination mask register are handled according to the setting of the `vta` bit in `vtype` (see Section 5.8.3.3).

Traps on `vmsof.m` are always reported with a `vstart` of 0. The `vmsof` instruction will raise an illegal-instruction exception if `vstart` is non-zero.

The destination register cannot overlap the source register and, if masked, cannot overlap the mask register ('v0').

5.8.19.7 Using Vector Mask Instructions

The following is an example of vectorizing a data-dependent exit loop.

```
.text
.balign 4
.global strcpy
# char* strcpy(char *dst, const char* src)
strcpy:
    mv a2, a0                # Copy dst
    li t0, -1                # Infinite AVL
loop:
    vsetvli x0, t0, e8, m8, ta, ma # Max length vectors of bytes
    vle8ff.v v8, (a1)          # Get src bytes
    csrr t1, vl               # Get number of bytes fetched
    vmseq.vi v1, v8, 0        # Flag zero bytes
    vfirst.m a3, v1           # Zero found?
    add a1, a1, t1            # Bump pointer
    vmsif.m v0, v1            # Set mask up to and including zero byte
    vse8.v v8, (a2), v0.t     # Write out bytes
    add a2, a2, t1            # Bump pointer
    bltz a3, loop             # Zero byte not found, so loop

    ret
```

```
.text
.balign 4
.global strncpy
# char* strncpy(char *dst, const char* src, size_t n)
strncpy:
    mv a3, a0                # Copy dst
loop:
    vsetvli x0, a2, e8, m8, ta, ma    # Vectors of bytes
    vle8ff.v v8, (a1)                # Get src bytes
    vmseq.vi v1, v8, 0                # Flag zero bytes
    csrr t1, vl                      # Get number of bytes fetched
    vfirst.m a4, v1                  # Zero found?
    vmsif.m v0, v1                   # Set mask up to and including zero byte
    vse8.v v8, (a3), v0.t            # Write out bytes
    sub a2, a2, t1                   # Decrement count
    bgez a4, zero_tail              # Zero remaining bytes
    add a1, a1, t1                   # Bump pointer
    add a3, a3, t1                   # Bump pointer
    bnez a2, loop                   # Anymore?

ret

zero_tail:
    vsetvli x0, a2, e8, m8, ta, ma    # Vectors of bytes
    vmv.v.i v0, 0                    # Splat zero

zero_loop:
    vsetvli t1, a2, e8, m8, ta, ma    # Vectors of bytes
    vse8.v v0, (a3)                  # Store zero
    sub a2, a2, t1                   # Decrement count
    add a3, a3, t1                   # Bump pointer
    bnez a2, zero_loop              # Anymore?

ret
```

5.8.19.8 Vector Iota Instruction

The `viota.m` instruction reads a source vector mask register and writes to each element of the destination vector register group the sum of all the bits of elements in the mask register whose index is less than the element, e.g., a parallel prefix sum of the mask values.

This instruction can be masked, in which case only the enabled elements contribute to the sum and only the enabled elements are written.

`viota.m vd, vs2, vm`

Example

7 6 5 4 3 2 1 0	Element number
1 0 0 1 0 0 0 1	v2 contents
	<code>viota.m v4, v2</code> # Unmasked
2 2 2 1 1 1 1 0	v4 result
1 1 1 0 1 0 1 1	v0 contents
1 0 0 1 0 0 0 1	v2 contents
2 3 4 5 6 7 8 9	v4 contents
	<code>viota.m v4, v2, v0.t</code> # Masked, <code>vtype.vma=0</code>
1 1 1 5 1 7 1 0	v4 results

The result value is zero-extended to fill the destination element if SEW is wider than the result. If the result value would overflow the destination SEW, the least-significant SEW bits are retained.

Traps on `viota.m` are always reported with a `vstart` of 0, and execution is always restarted from the beginning when resuming after a trap handler. An illegal-instruction exception is raised if `vstart` is non-zero.

The destination register group cannot overlap the source register and, if masked, cannot overlap the mask register ('v0').

The `viota.m` instruction can be combined with memory scatter instructions (indexed stores) to perform vector compress functions.

```
# Compact non-zero elements from input memory array to output memory array
#
# size_t compact_non_zero(size_t n, const int* in, int* out)
# {
#     size_t i;
#     size_t count = 0;
#     int *p = out;
#
#     for (i=0; i<n; i++)
#     {
#         const int v = *in++;
#         if (v != 0)
#             *p++ = v;
#     }
#
#     return (size_t) (p - out);
# }
#
# a0 = n
# a1 = &in
# a2 = &out

compact_non_zero:
    li a6, 0                                # Clear count of non-zero elements
loop:
    vsetvli a5, a0, e32, m8, ta, ma        # 32-bit integers
    vle32.v v8, (a1)                       # Load input vector
    sub a0, a0, a5                          # Decrement number done
    slli a5, a5, 2                          # Multiply by four bytes
    vmsne.vi v0, v8, 0                     # Locate non-zero values
    add a1, a1, a5                          # Bump input pointer
    vcpop.m a5, v0                          # Count number of elements set in v0
    viota.m v16, v0                         # Get destination offsets of active elements
    add a6, a6, a5                          # Accumulate number of elements
    vsll.vi v16, v16, 2, v0.t               # Multiply offsets by four bytes
    slli a5, a5, 2                          # Multiply number of non-zero elements by 4 bytes
    vsuxei32.v v8, (a2), v16, v0.t          # Scatter using scaled viota results under mask
    add a2, a2, a5                          # Bump output pointer
    bnez a0, loop                          # Any more?

    mv a0, a6                              # Return count
    ret
```

5.8.19.9 Vector Element Index Instruction

The `vid.v` instruction writes each element's index to the destination vector register group, from 0 to `v1-1`.

`vid.v vd, vm` # Write element ID to destination

The instruction can be masked.

The `vs2` field of the instruction must be set to `v0`; otherwise, the encoding is *reserved*.

The result value is zero-extended to fill the destination element if SEW is wider than the result. If the result value would overflow the destination SEW, the least-significant SEW bits are retained.

5.8.20 Vector Permutation Instructions

A range of permutation instructions are provided to move elements around within the vector registers.

5.8.20.1 Integer Scalar Move Instructions

The integer scalar read/write instructions transfer a single value between a scalar x register and element 0 of a vector register. The instructions ignore LMUL and vector register groups.

```
vmv.x.s rd, vs2 # x[rd] = vs2[0] (rs1=0)
vmv.s.x vd, rs1 # vd[0] = x[rs1] (vs2=0)
```

The `vmv.x.s` instruction copies a single SEW-wide element from index 0 of the source vector register to a destination integer register. If $SEW > XLEN$, the least-significant XLEN bits are transferred and the upper SEW-XLEN bits are ignored. If $SEW < XLEN$, the value is sign-extended to XLEN bits.

The `vmv.s.x` instruction copies the scalar integer register to element 0 of the destination vector register. If $SEW < XLEN$, the least-significant bits are copied and the upper XLEN-SEW bits are ignored. If $SEW > XLEN$, the value is sign-extended to SEW bits. The other elements in the destination vector register ($0 < \text{index} < VLEN/SEW$) are unchanged. If $vstart \geq v1$, no operation is performed and the destination register is not updated.

5.8.20.2 Floating-Point Scalar Move Instructions

The floating-point scalar read/write instructions transfer a single value between a scalar f register and element 0 of a vector register. The instructions ignore LMUL and vector register groups.

```
vfmv.f.s rd, vs2 # f[rd] = vs2[0] (rs1=0)
vfmv.s.f vd, rs1 # vd[0] = f[rs1] (vs2=0)
```

The `vfmv.f.s` instruction copies a single SEW-wide element from index 0 of the source vector register to a destination scalar floating-point register.

The `vfmv.s.f` instruction copies the scalar floating-point register to element 0 of the destination vector register. The other elements in the destination vector register ($0 < \text{index} < VLEN/SEW$) are unchanged. If $vstart \geq v1$, no operation is performed and the destination register is not updated.

5.8.20.3 Vector Slide Instructions

The slide instructions move elements up and down a vector register group.

For all of the `vslideup`, `vslidedown`, `v[f]slide1up`, and `v[f]slide1down` instructions, if $vstart \geq vl$, the instruction performs no operation and leaves the destination vector register unchanged.

The slide instructions may be masked, with mask element i controlling whether *destination* element i is written.

5.8.20.3.1 Vector Slideup Instructions

```
vslideup.vx vd, vs2, rs1, vm      # vd[i+rs1] = vs2[i]
vslideup.vi vd, vs2, uimm[4:0], vm # vd[i+uimm] = vs2[i]
```

For `vslideup`, the value in vl specifies the maximum number of destination elements that are written. The start index (*OFFSET*) for the destination can be either specified using an unsigned integer in the x register specified by $rs1$, or a 5-bit immediate treated as an unsigned 5-bit quantity. If $XLEN > SEW$, *OFFSET* is *not* truncated to SEW bits. Destination elements *OFFSET* through $vl-1$ are written if unmasked and if $OFFSET < vl$.

`vslideup` behavior for destination elements

OFFSET is amount to slideup, either from x register or a 5-bit immediate

$0 < i < \max(vstart, OFFSET)$	# Unchanged
$\max(vstart, OFFSET) \leq i < vl$	# $vd[i] = vs2[i-OFFSET]$ if mask[i] # enabled, else unchanged
$vl \leq i < VLMAX$	# Tail elements, unchanged

The destination vector register group for `vslideup` cannot overlap the source vector register group; otherwise, an illegal-instruction exception is raised.

Note

The non-overlap constraint avoids WAR hazards on the input vectors during execution, and enables restart with non-zero $vstart$.

5.8.20.3.2 Vector Slidedown Instructions

```
vslidedown.vx vd, vs2, rs1, vm      # vd[i] = vs2[i+rs1]
vslidedown.vi vd, vs2, uimm[4:0], vm # vd[i] = vs2[i+uimm]
```

For `vslidedown`, the value in vl specifies the number of destination elements that are written.

The start index (*OFFSET*) for the source can be either specified using an unsigned integer in the x register specified by $rs1$, or a 5-bit immediate treated as an unsigned 5-bit quantity. If $XLEN > SEW$, *OFFSET* is *not* truncated to SEW bits.

vslidedown behavior for source elements for element i in slide

```

    0 ≤ i+OFFSET < VLMAX # Read vs2[i+OFFSET]
VLMAX ≤ i+OFFSET        # Read as 0

```

vslidedown behavior for destination element i in slide

```

    0 < i < vstart # Unchanged
vstart ≤ i < vl    # Updated if mask[i] enabled, else unchanged
vl ≤ i < VLMAX    # Unchanged

```

5.8.20.3.3 Vector Slide1up

Variants of slide are provided that only move by one element but which also allow a scalar integer value to be inserted at the vacated element position.

```

vslide1up.vx vd, vs2, rs1, vm # vd[0]=x[rs1], vd[i+1] = vs2[i]
vslide1up.vf vd, vs2, rs1, vm # vd[0]=f[rs1], vd[i+1] = vs2[i]

```

The `vslide1up` instruction places the x register argument at location 0 of the destination vector register group, provided that element 0 is active, otherwise the destination element is unchanged. If $XLEN < SEW$, the value is sign-extended to SEW bits. If $XLEN > SEW$, the least-significant bits are copied over and the high $SEW-XLEN$ bits are ignored.

The remaining active $vl-1$ elements are copied over from index i in the source vector register group to index $i+1$ in the destination vector register group.

The vl register specifies how many of the destination vector register elements are written with source values, and all tail elements are unchanged.

vslide1up behavior

```

    i < vstart # Unchanged
    0 = i = vstart # vd[i] = x[rs1] if mask[i] enabled, else unchanged
max(vstart, 1) ≤ i < vl    # vd[i] = vs2[i-1] if mask[i] enabled, else unchanged
vl ≤ i < VLMAX    # Unchanged

```

The `vslide1up` instruction requires that the destination vector register group does not overlap the source vector register group. Otherwise, an illegal instruction exception is raised.

The `vslide1up` instruction is defined analogously, but sources its scalar argument from an f register.

5.8.20.3.4 Vector Slide1down Instruction

The `vslide1down` instruction copies the first $vl-1$ active elements values from index $i+1$ in the source vector register group to index i in the destination vector register group.

The vl register specifies how many of the destination vector register elements are written with source values, and all tail elements are unchanged.

```
vslide1down.vx vd, vs2, rs1, vm # vd[i] = vs2[i+1], vd[vl-1]=x[rs1]
vslide1down.vf vd, vs2, rs1, vm # vd[i] = vs2[i+1], vd[vl-1]=f[rs1]
```

The `vslide1down` instruction places the `x` register argument at location `v1-1` in the destination vector register, provided that element `v1-1` is active, otherwise the destination element is unchanged. If `XLEN < SEW`, the value is sign-extended to `SEW` bits. If `XLEN > SEW`, the least-significant bits are copied over and the high `SEW-XLEN` bits are ignored.

`vslide1down` behavior

```
        i < vstart # Unchanged
vstart ≤ i < vl-1 # vd[i] = vs2[i+1] if mask[i] enabled, else unchanged
vstart ≤ i = vl-1 # vd[vl-1] = x[rs1] if mask[i] enabled, else unchanged
        vl ≤ i < VLMAX # Unchanged
```

The `vslide1down` instruction is defined analogously, but sources its scalar argument from an `f` register.

Note

The `vslide1down` instruction can be used to load values into a vector register without using memory and without disturbing other vector registers. This provides a path for debuggers to modify the contents of a vector register, albeit slowly, with multiple repeated `vslide1down` invocations.

5.8.20.4 Vector Register Gather Instructions

The vector register gather instructions read elements from a first source vector register group at locations given by a second source vector register group. The index values in the second vector are treated as unsigned integers. The source vector can be read at any index `< VLMAX` regardless of `v1`. The number of elements to write to the destination register is given by `v1`, and the remaining elements past `v1` are handled according to the current tail policy (see Section 5.8.3.3). The operation can be masked.

```
vrgather.vv vd, vs2, vs1, vm # vd[i] = (vs1[i] >= VLMAX) ? 0 : vs2[vs1[i]];
vrgatherei16.vv vd, vs2, vs1, vm # vd[i] = (vs1[i] >= VLMAX) ? 0 : vs2[vs1[i]];
```

The `vrgather.vv` form uses `SEW/LMUL` for both the data and indices. The `vrgatherei16.vv` form uses `SEW/LMUL` for the data in `vs2` but `EEW=16` and `EMUL = (16/SEW)*LMUL` for the indices in `vs1`.

Note

When `SEW=8`, `vrgather.vv` can only reference vector elements 0-255. The `vrgatherei16` form can index 64K elements, and can also be used to reduce the register capacity needed to hold indices for wider `SEW` data.

If the element indices are out of range ($vs1[i] \geq VLMAX$), then zero is returned for the element value.

Vector-scalar and vector-immediate forms of the register gather are also provided. These read one element from the source vector at the given index, and write this value to the $v1$ elements at the start of the destination vector register. The index value in the scalar register and the immediate are treated as unsigned integers. If $XLEN > SEW$, the index value is *not* truncated to SEW bits.

Note

These forms allow any vector element to be "splatted" to an entire vector.

```
vrgather.vx vd, vs2, rs1, vm # vd[i] = (x[rs1] >= VLMAX) ? 0 : vs2[x[rs1]]
vrgather.vi vd, vs2, uimm, vm # vd[i] = (uimm >= VLMAX) ? 0 : vs2[uimm]
```

For any vrgather instruction, the destination vector register group cannot overlap with the source vector register groups; otherwise, an illegal-instruction exception is raised.

5.8.20.5 Vector Compress Instruction

The vector compress instruction allows elements selected by a vector mask register from a source vector register group to be packed into contiguous elements at the start of the destination vector register group.

```
vcompress.vm vd, vs2, vs1 # Compress into vd elements of vs2 where vs1 is enabled
```

The vector mask register specified by $vs1$ indicates which of the first $v1$ elements of vector register group $vs2$ should be extracted and packed into contiguous elements at the beginning of vector register vd . The remaining elements of vd are treated as tail elements according to the current tail policy (see Section 5.8.3.3).

Example use of vcompress instruction

```
1 1 0 1 0 0 1 0 1 v0
8 7 6 5 4 3 2 1 0 v1
1 2 3 4 5 6 7 8 9 v2
vcompress.vm v2, v1, v0
1 2 3 4 8 7 5 2 0 v2
```

vcompress is encoded as an unmasked instruction ($vm=1$). The equivalent masked instruction ($vm=0$) is reserved.

The destination vector register group cannot overlap the source vector register group or the source mask register; otherwise, an illegal-instruction exception is raised.

A trap on a `vcompress` instruction is always reported with a `vstart` of 0. Executing a `vcompress` instruction with a non-zero `vstart` raises an illegal-instruction exception.

Note

Although possible, `vcompress` is one of the more difficult instructions to restart with a non-zero `vstart`. The assumption is implementations will choose not to do that, but will instead restart from element 0. This does mean elements in the destination register after `vstart` will already have been updated.

5.8.20.6 Whole Vector Register Move

The `vmv<nr>r.v` instructions copy whole vector registers (i.e., all `VLEN` bits), and can copy whole vector register groups. The instructions operate as if `EEW=8`, `EMUL = nr`, and effective length `evl=VLEN/8 * EMUL`, regardless of current settings in `vtype` and `v1`.

The instruction is encoded as an `OPIVI` instruction. The number of vector registers to copy is encoded in the low three bits of the `simm` field using the same encoding as the `nf` field for memory instructions, i.e., `simm = nr - 1`. `nr` must be 1, 2, 4, or 8.

Note

The instruction uses the same `funct6` encoding as the `vsmul` instruction, but with an immediate operand, and only the unmasked version (`vm=1`). This encoding is chosen as it is close to the related `vmerge` encoding, and it is unlikely the `vsmul` instruction would benefit from an immediate form.

`vmv<nr>r.v vd, vs2 # General form`

```
vmv1r.v v1, v2    # Copy v1=v2
vmv2r.v v10, v12  # Copy v10=v12; v11=v13
vmv4r.v v4, v8    # Copy v4=v8; v5=v9; v6=v10; v7=v11
vmv8r.v v0, v8    # Copy v0=v8; v1=v9; ...; v7=v15
```

The source and destination vector register numbers must be aligned appropriately for the vector register group size.

Note

If `vd` is equal to `vs2` the instruction is an architectural NOP, but is treated as a HINT to implementations that rearrange data internally such that the register group will next be accessed with an `EEW` equal to `SEW`.

5.8.21 Xsfvqmacccqoq Extension: Int8 Matrix Multiplication Instructions

The Tenstorrent X280 MC supports int8 Matrix-Multiply Instructions.

These instructions are signed or unsigned integer matrix-multiply-add instructions that widen 8-bit integer input values to 32-bits. To understand the functionality of these instructions, consider the arithmetic below:

```
# Dest (C)   Input (A) * Input (B)
int32 +=     uint8 *   uint8
int32 +=     uint8 *   int8
int32 +=     int8  *   uint8
int32 +=     int8  *   int8
```

Here, the (u)int8 A- and B-matrices are 4-by-8 and 8-by-4, respectively, while the int32 C-matrix is 4-by-4. This represents a quad-widening of the arithmetic values, but only a double-widening of the data width due to the destination matrix being smaller than the input matrices.

The products of the 8-bit signed or unsigned integer inputs are calculated exactly, and are added to the 32-bit integer accumulators, with wraparound if the 32-bit destination overflows.

These instructions are encoded similarly to other vector instructions, but within the *custom-2* major opcode, with funct3=2 (OPMVV), and with funct6 as specified below.

```
# Quad-widening unsigned-integer matrix-multiply-add, overwrite addend
sf.vqmaccu.4x8x4 vd, vs1, vs2 # funct6=111100
```

```
# Quad-widening signed-integer matrix-multiply-add, overwrite addend
sf.vqmacc.4x8x4 vd, vs1, vs2 # funct6=111101
```

```
# Quad-widening unsigned-signed-integer matrix-multiply-add, overwrite addend
sf.vqmaccus.4x8x4 vd, vs1, vs2 # funct6=111110
```

```
# Quad-widening signed-unsigned-integer matrix-multiply-add, overwrite addend
sf.vqmaccsu.4x8x4 vd, vs1, vs2 # funct6=111111
```

The above instructions perform $v1/32$ matrix operations,

$C[j] += A * B[j]$, for j in $[0, v1/32)$

where

- The 4-by-8 (u)int8 matrix A resides in row-major layout in bytes $[0, 32)$ of vs1.
- The 8-by-4 (u)int8 matrix B[j] resides in row-major layout in bytes $[j*32, (j+1)*32)$ of vs2.
- The 4-by-4 int32 matrix C[j] resides in row-major layout in bytes $[j*64, (j+1)*64)$ of vd.

vs1 has EMUL=1, vs2 has EMUL=LMUL, and vd has EMUL=2*LMUL.

The instructions are reserved if v1 is not divisible by 32, or if SEW≠8, or if masked. vd must not overlap vs1.

5.8.22 Xsfvnrclipxfqf Extension: FP32-to-int8 Ranged Clip Instructions

The Tenstorrent X280 MC supports the SiFive custom Xsfvnrclipxfqf extension, which provides support for FP32-to-int8 Ranged Clip Instructions. These instructions convert an FP32 number to a signed or unsigned 8-bit integer, clipping the result to a range specified by a scalar register.

```
sf.vfnrclip.xu.f.qf vd, vs2, rs1 # unsigned; funct6=100010
sf.vfnrclip.x.f.qf  vd, vs2, rs1 # signed;   funct6=100011
```

These instructions are reserved if SEW8. vs2 holds the input FP32 vector and has EMUL=LMUL*4 and EEW=SEW*4. vd holds the int8 result and has EMUL=LMUL and EEW=SEW. Standard overlap constraints for narrowing instructions apply.

f[rs1] holds the bounds within which to clip the results, with the lower bound specified as an 8-bit integer in bits 15:8 of f[rs1], and the upper bound specified as an 8-bit integer in bits 7:0.

These bounds are of the same signedness as the result. Result element i is given by the expression $\max(f[rs1][15:8], \min(\text{round}(vs2[i]), f[rs1][7:0]))$, rounding according to the dynamic floating-point rounding mode, frm.

Note

By implication, if the lower bound is greater than the upper bound, the result always equals the lower bound.

Note

The lower bound is sometimes statically known to be zero. Placing the upper bound in the lower bits of rs1, while less intuitive, reduces instruction count in this case.

These instructions are encoded similarly to other vector instructions, but within the custom-2 major opcode, with funct3=5 (OPFVF), and with funct6 as specified above.

No floating-point exception flags are generated by this operation and vxsat is never asserted.

5.8.23 Xsfvfhbfmin Extension: Bfloat16 Conversion Instructions

The Tenstorrent X280 MC supports vector bfloat16 conversion instructions. The following instructions enable the user to convert from single-precision floating-point (FP32) to bfloat16 (BF16) and vice-versa. These instructions obey the full set of static or dynamic rounding modes, set the floating-point exception flags, and generate special values as appropriate. These instructions are reserved if SEW $\neq$ 16.

```
# Convert BF16 to FP32
sf.vfwcvt.f.bf16.v # source EEW=16, dest EEW=32
```

```
# Convert FP32 to BF16
sf.vfncvt.bf16.f.v # source EEW=32, dest EEW=16
```

`sf.vfwcvt.f.bf16.v` is encoded identically to `vfwcvt.f.f.v` but with `vs1=01101`.

`sf.vfncvt.bf16.f.v` is encoded identically to `vfncvt.f.f.v` but with `vs1=11101`.

5.9 C Extension: Compressed Instructions

The C Extension reduces static and dynamic code size by adding short 16-bit instruction encodings for common operations. The C extension can be added to any of the base ISAs (RV32, RV64, RV128), and we use the generic term "RVC" to cover any of these. Typically, 50%–60% of the RISC-V instructions in a program can be replaced with RVC instructions, resulting in a 25%–30% code-size reduction. The C extension is compatible with all other standard instruction extensions. The C extension allows 16-bit instructions to be freely intermixed with 32-bit instructions, with the latter now able to start on any 16-bit boundary, i.e., `IALIGN=16`. With the addition of the C extension, no instructions can raise instruction-address-misaligned exceptions. It is important to note that the C extension is not designed to be a stand-alone ISA, and is meant to be used alongside a base ISA. The compressed 16-bit instruction format is designed around the assumption that `x1` is the return address register and `x2` is the stack pointer.

5.9.1 Compressed 16-bit Instruction Formats

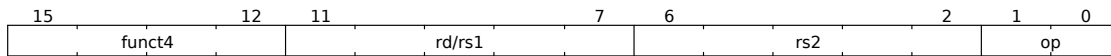


Figure 106: CR Format - Register

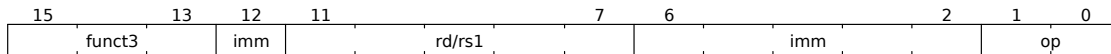


Figure 107: CI Format - Immediate

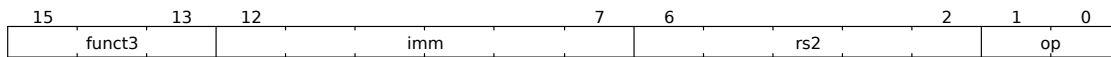


Figure 108: CSS Format - Stack-relative Store

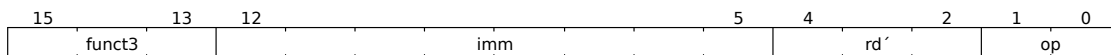


Figure 109: CIW Format - Wide Immediate

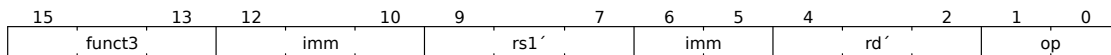


Figure 110: CL Format - Load

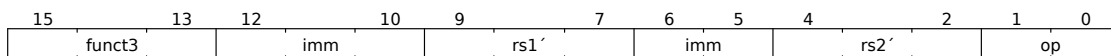


Figure 111: CS Format - Store

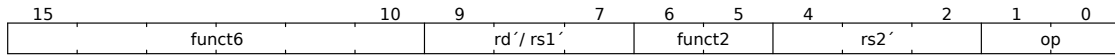


Figure 112: CA Format - Arithmetic

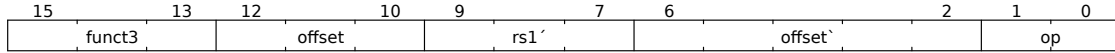


Figure 113: CJ Format - Jump

5.9.2 Stack-Pointed-Based Loads and Stores

The compressed load instructions are expressed in CI format.

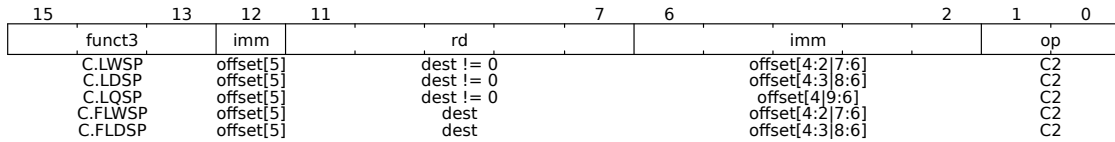


Figure 114: Stack-Pointed-Based Loads

Instruction	Description
C.LWSP	Loads a 32-bit value from memory into register rd.
C.LDSP	RV64C Instruction which loads a 64-bit value from memory into register rd.
C.LQSP	RV128C loads a 128-bit value from memory into register rd.
C.FLWSP	RV32FC Instruction that loads a single-precision floating-point value from memory into floating-point register rd.
C.FLDSP	RV32DC/RV64DC Instruction that loads a double-precision floating-point value from memory into floating-point register rd.

Table 84: Stack-Pointed-Based Load Instruction Description

The compressed store instructions are expressed in CSS format.

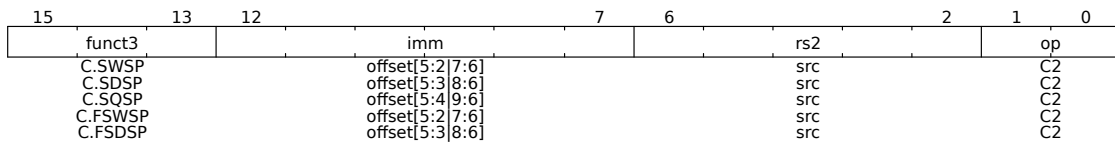


Figure 115: Stack-Pointed-Based Stores

Instruction	Description
C.LWSP	Loads a 32-bit value from memory into register rd.
C.SWSP	Stores a 32-bit value in register rs2 to memory.
C.SDSP	RV64C/RV128C instruction that stores a 64-bit value in register rs2 to memory.
C.SQSP	RV128C instruction that stores a 128-bit value in register rs2 to memory.
C.FSWSP	RV32FC instruction that stores a single-precision floating-point value in floating-point register rs2 to memory.
C.FSDSP	RV32DC/RV64DC instruction that stores a double-precision floating-point value in floating-point register rs2 to memory.

Table 85: Stack-Pointed-Based Store Instruction Description

5.9.3 Register-Based Loads and Stores

The compressed register-based load instructions are expressed in CL format.

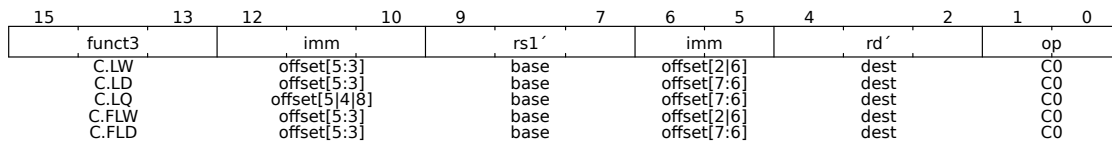


Figure 116: Register-Based Loads

Instruction	Description
C.LW	Loads a 32-bit value from memory into register rd.
C.LD	RV64C/RV128C-only instruction that loads a 64-bit value from memory into register rd.
C.LQ	RV128C-only instruction that loads a 128-bit value from memory into register rd.
C.FLW	RV32FC-only instruction that loads a single-precision floating-point value from memory into floating-point register rd.
C.FLD	RV32DC/RV64DC-only instruction that loads a double-precision floating-point value from memory into floating-point register rd.

Table 86: Register-Based Load Instruction Description

The compressed register-based store instructions are expressed in CS format.

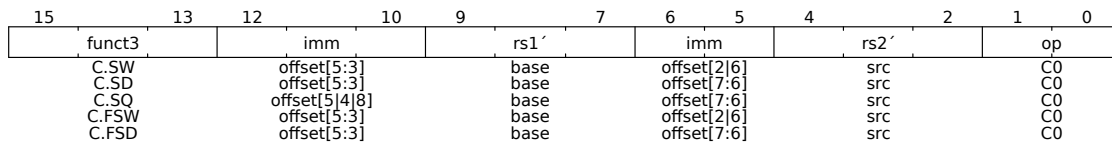


Figure 117: Register-Based Stores

Instruction	Description
C.SW	Stores a 32-bit value in register rs2 to memory.
C.SD	RV64C/RV128C instruction that stores a 64-bit value in register rs2 to memory.
C.SQ	RV128C instruction that stores a 128-bit value in register rs2 to memory.
C.FSW	RV32FC instruction that stores a single-precision floating-point value in floating-point register rs2 to memory.
C.FSD	RV32DC/RV64DC instruction that stores a double-precision floating-point value in floating-point register rs2 to memory.

Table 87: Register-Based Store Instruction Description

5.9.4 Control Transfer Instructions

RVC provides unconditional jump instructions and conditional branch instructions.

The unconditional jump instructions are expressed in CJ format.

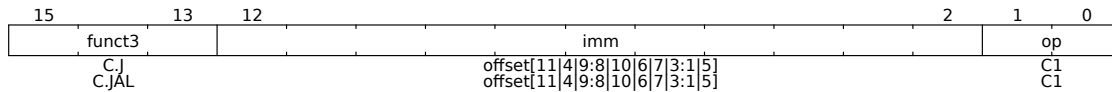


Figure 118: Unconditional Jump Instructions

Instruction	Description
C.J	Unconditional control transfer.
C.JAL	RV32C instruction that performs the same operation as C.J, but additionally writes the address of the instruction following the jump (pc+2) to the link register, x1.

Table 88: Unconditional Jump Instruction Description

The unconditional control transfer instructions are expressed in CR format.

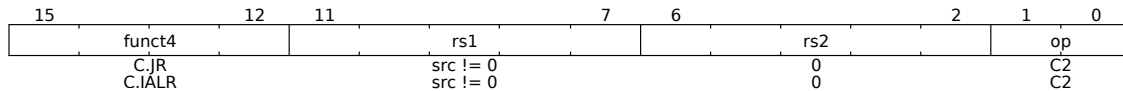


Figure 119: Unconditional Control Transfer Instructions

Instruction	Description
C.JR	Performs an unconditional control transfer to the address in register rs1.
C.JALR	Performs the same operation as C.JR, but additionally writes the address of the instruction following the jump (pc+2) to the link register, x1.

Table 89: Unconditional Control Transfer Instruction Description

The conditional control transfer instructions are expressed in CB format.

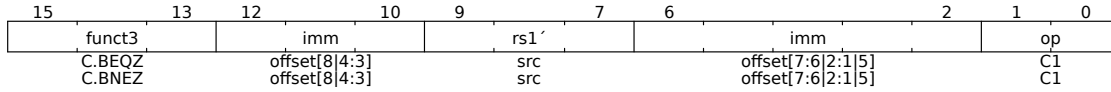


Figure 120: Conditional Control Transfer Instructions

Instruction	Description
C.BEQZ	Conditional control transfers. Takes the branch if the value in register <code>rs1'</code> is zero.
C.BNEZ	Conditional control transfers. Takes the branch if <code>rs1'</code> contains a nonzero value.

Table 90: Conditional Control Transfer Instruction Description

5.9.5 Integer Computational Instructions

5.9.5.1 Integer Constant-Generation Instructions

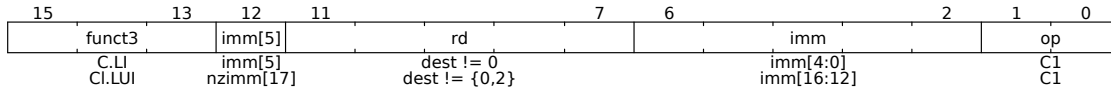


Figure 121: Integer Constant-Generation Instructions

Instruction	Description
C.LI	Loads the sign-extended 6-bit immediate, <code>imm</code> , into register <code>rd</code> .
C.LUI	Loads the non-zero 6-bit immediate field into bits 17–12 of the destination register, clears the bottom 12 bits, and sign-extends bit 17 into all higher bits of the destination

Table 91: Integer Constant-Generation Instruction Description

5.9.5.2 Integer Register-Immediate Operations

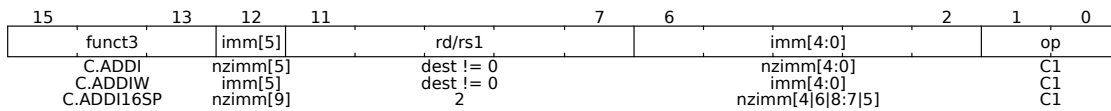


Figure 122: Integer Register-Immediate Operations

Instruction	Description
C.ADDI	Adds the non-zero sign-extended 6-bit immediate to the value in register rd then writes the result to rd.
C.ADDIW	RV64C/RV128C instruction that performs the same computation but produces a 32-bit result, then sign-extends result to 64 bits.
C.ADDI16SP	Adds the non-zero sign-extended 6-bit immediate to the value in the stack pointer (sp=x2), where the immediate is scaled to represent multiples of 16 in the range (-512,496). C.ADDI16SP is used to adjust the stack pointer in procedure prologues and epilogues.

Table 92: Integer Register-Immediate Operation Description

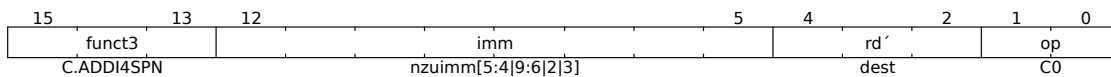


Figure 123: Integer Register-Immediate Operations (cont.)

Instruction	Description
C.ADDI4SPN	Adds a zero-extended non-zero immediate, scaled by 4, to the stack pointer, x2, and writes the result to rd'.

Table 93: Integer Register-Immediate Operation Description (cont.)

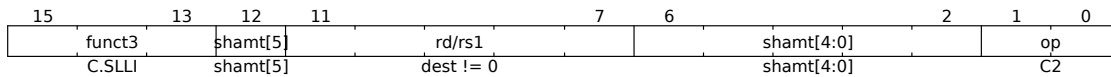


Figure 124: Integer Register-Immediate Operations (cont.)

Instruction	Description
C.SLLI	Performs a logical left shift of the value in register rd then writes the result to rd. The shift amount is encoded in the shamt field.

Table 94: Integer Register-Immediate Operation Description (cont.)

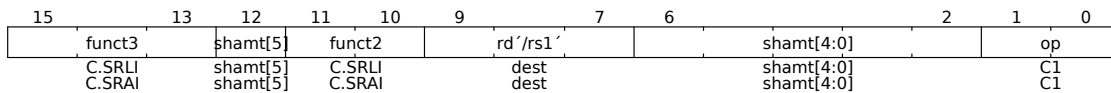


Figure 125: Integer Register-Immediate Operations (cont.)

Instruction	Description
C.SRLI	Logical right shift of the value in register rd' then writes the result to rd'. The shift amount is encoded in the shamt field.
C.SRAI	Arithmetic right shift of the value in register rd' then writes the result to rd'. The shift amount is encoded in the shamt field.

Table 95: Integer Register-Immediate Operation Description (cont.)

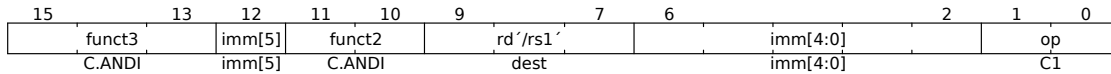


Figure 126: Integer Register-Immediate Operations (cont.)

Instruction	Description
C.ANDI	Computes the bitwise AND of the value in register rd' and the sign-extended 6-bit immediate, then writes the result to rd'.

Table 96: Integer Register-Immediate Operation Description (cont.)

5.9.5.3 Integer Register-Register Operations

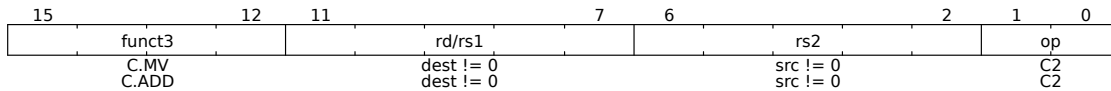


Figure 127: Integer Register-Register Operations

Instruction	Description
C.MV	Copies the value in register rs2 into register rd.
C.ADD	Adds the values in registers rd and rs2 and writes the result to register rd.

Table 97: Integer Register-Register Operation Description

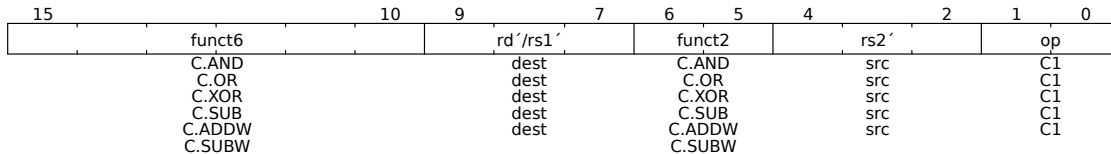


Figure 128: Integer Register-Register Operations (cont.)

Instruction	Description
C.AND	Computes the bitwise AND of the values in registers rd' and rs2'.
C.OR	Computes the bitwise OR of the values in registers rd' and rs2'.
C.XOR	Computes the bitwise XOR of the values in registers rd' and rs2'.
C.SUB	Subtracts the value in register rs2' from the value in register rd'.
C.ADDW	RV64C/RV128C-only instruction that adds the values in registers rd' and rs2', then sign-extends the lower 32 bits of the sum before writing the result to register rd.
C.SUBW	RV64C/RV128C-only instruction that subtracts the value in register rs2' from the value in register rd', then sign-extends the lower 32 bits of the difference before writing the result to register rd.

Table 98: Integer Register-Register Operation Description (cont.)

5.9.5.4 Defined Illegal Instruction

A 16-bit instruction with all bits zero is permanently reserved as an illegal instruction.

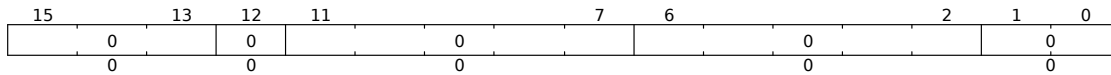


Figure 129: Defined Illegal Instruction

5.10 Zba Extension: Address Calculation Instructions

The Zba instructions are used to accelerate the generation of addresses that index into arrays of basic types (halfword, word, doubleword) using both unsigned word-sized and 64-sized indices; that is, a shifted index is added to a base address.

5.10.1 Address Calculation Instructions

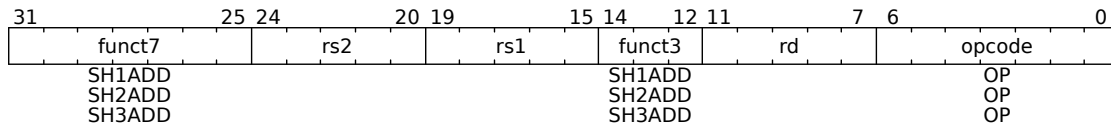


Figure 130: Address Calculation Instructions

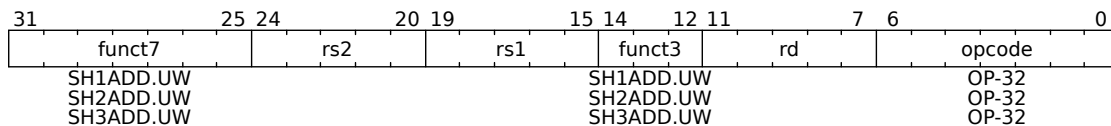


Figure 131: Address Calculation Instructions (cont.)

Instruction	Description
SH1ADD rd, rs1, rs2	Shifts rs1 by 1 bit, then adds the result to rs2
SH2ADD rd, rs1, rs2	Shifts rs1 by 2 bits, then adds the result to rs2
SH3ADD rd, rs1, rs2	Shifts rs1 by 3 bits, then adds the result to rs2
SH1ADD.UW rd, rs1, rs2	Performs an 64-wide addition of rs2, and the unsigned value formed by extracting the least-significant word of rs1 and shifting it left by 1 bit
SH2ADD.UW rd, rs1, rs2	Performs an 64-wide addition of rs2, and the unsigned value formed by extracting the least-significant word of rs1 and shifting it left by 2 bits
SH3ADD.UW rd, rs1, rs2	Performs an 64-wide addition of rs2, and the unsigned value formed by extracting the least-significant word of rs1 and shifting it left by 3 bits

Table 99: Address Calculation Instructions Description

5.10.2 Add/Shift with Prefix Zero-Extend Instructions

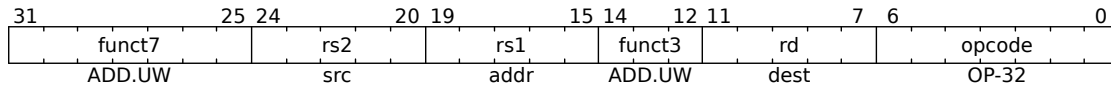


Figure 132: Add with Prefix Zero-Extend Instruction

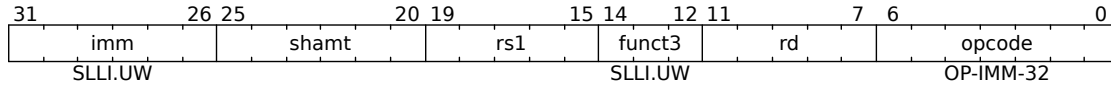


Figure 133: Shift with Prefix Zero-Extend Instruction

Instruction	Description
ADD.UW rd, rs1, rs2	Performs a 64-wide addition between rs2 and the zero-extended least-significant word of rs1
SLLI.UW rd, rs1, shamt	Takes the least-significant word of rs1, zero-extends it, and shifts it left by the immediate

Table 100: Add/Shift with Prefix Zero-Extend Instructions Description

5.11 Zbb Extension: Basic Bit Manipulation Instructions

The Zbb instructions are used for basic bit manipulation.

5.11.1 Count Leading/Trailing Zeroes Instructions

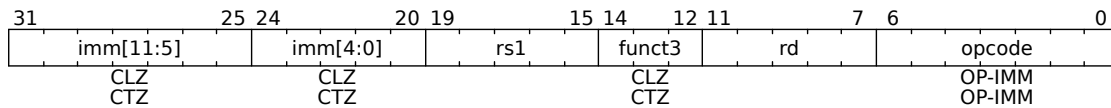


Figure 134: Count Leading/Trailing Zeroes Instructions

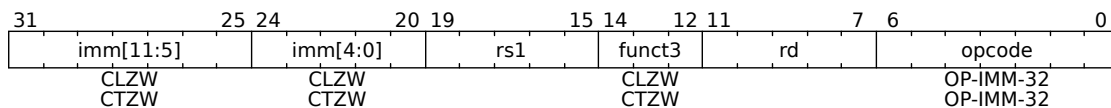


Figure 135: Count Leading/Trailing Zeroes Instructions (cont.)

Instruction	Description
CLZ rd, rs	Counts the number of 0 bits before the first 1 bit, starting at the most-significant bit and progressing to bit 0. If the input is 0, the output is 64. If the most-significant bit of the input is 1, the output is 0.
CTZ rd, rs	Counts the number of 0 bits before the first 1 bit, starting at the least-significant bit and progressing to the most-significant bit. If the input is 0, the output is 64. If the least-significant bit of the input is 1, the output is 0.
CLZW rd, rs	Counts the number of 0 bits before the first 1 bit, starting at bit 31 and progressing to bit 0. If the least-significant word is 0, the output is 32. If the most-significant bit of the word is 1, the output is 0.
CTZW rd, rs	Counts the number of 0 bits before the first 1 bit, starting at the least-significant bit and progressing to the most-significant word. If the least-significant word is 0, the output is 32. If the least-significant bit of the input is 1, the output is 0.

Table 101: Count Leading/Trailing Zeroes Instructions Description

5.11.2 Count Population Instructions

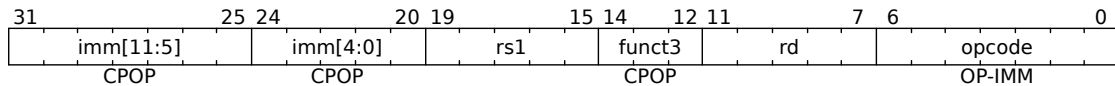


Figure 136: Count Population Instruction

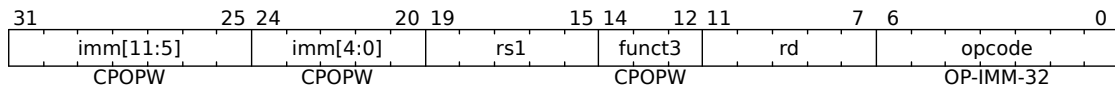


Figure 137: Count Population Instruction (cont.)

Instruction	Description
CPOP rd, rs	Counts the number of 1 bits in the source register
CPOPW rd, rs	Counts the number of 1 bits in the least-significant word of the source register

Table 102: Count Population Instructions Description

5.11.3 Logic-With-Negate Instructions

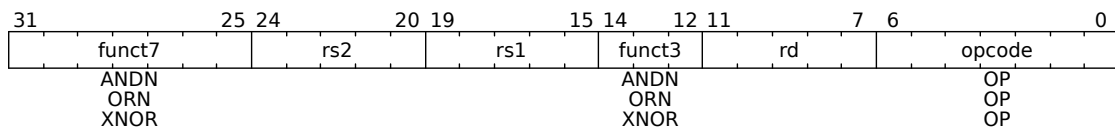


Figure 138: Logic-With-Negate Instructions

Instruction	Description
ANDN rd,rs1,rs2	Performs bitwise logical AND between rs1 and the bitwise inversion of rs2
ORN rd,rs1,rs2	Performs bitwise logical OR between rs1 and the bitwise inversion of rs2
XNOR rd,rs1,rs2	Performs bitwise exclusive-NOR on rs1 and rs2

Table 103: Logic-With-Negate Instructions Description

5.11.4 Comparison Instructions

These instructions are arithmetic R-type instructions that return the smaller or larger value of two operands.

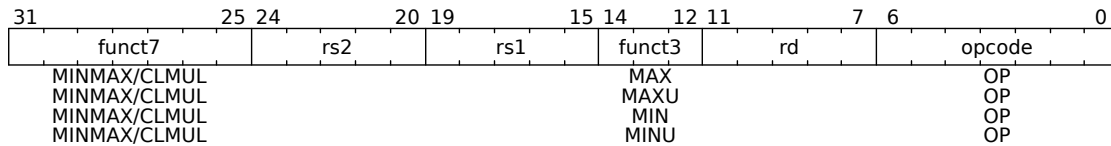


Figure 139: Comparison Instructions

Instruction	Description
MIN rd,rs1,rs2	Returns the smaller of two signed integers
MINU rd,rs1,rs2	Returns the smaller of two unsigned integers
MAX rd,rs1,rs2	Returns the larger of two signed integers
MAXU rd,rs1,rs2	Returns the larger of two unsigned integers

Table 104: Comparison Instructions Description

5.11.5 Sign-Extend and Zero-Extend Instructions

These instructions perform the sign-extension or zero-extension of the least-significant 8 or 16 bits of the source register.

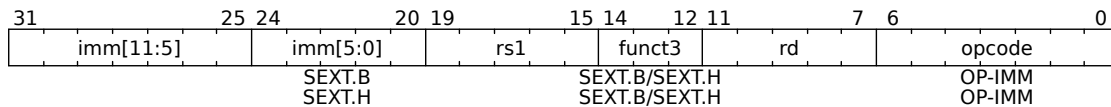


Figure 140: Sign-Extend Instructions

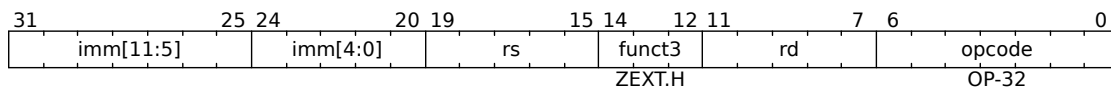


Figure 141: Zero-Extend Instruction

Instruction	Description
SEXT.B rd, rs	Sign-extends the least-significant byte in the source to 64 by copying the most-significant bit in the byte (i.e., bit 7) to all of the more-significant bits
SEXT.H rd, rs	Sign-extends the least-significant halfword in <i>rs</i> to 64 by copying the most-significant bit in the halfword (i.e., bit 15) to all of the more-significant bits
ZEXT.H rd, rs	Zero-extends the least-significant halfword of the source to 64 by inserting 0's into all of the bits more significant than 15

Table 105: Sign- and Zero-Extend Instructions

5.11.6 Bitwise Rotation Instructions

Bitwise rotation instructions are similar to the shift-logical operations from the base ISA specification. However, where the shift-logical instructions shift in zeros, the rotate instructions shift in the bits that were shifted out of the other side of the values.

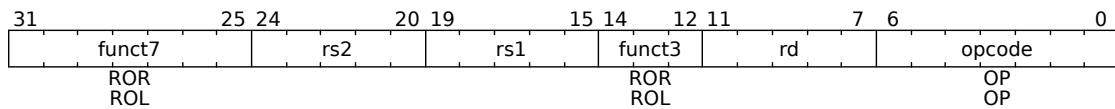


Figure 142: Bitwise Rotation Instructions

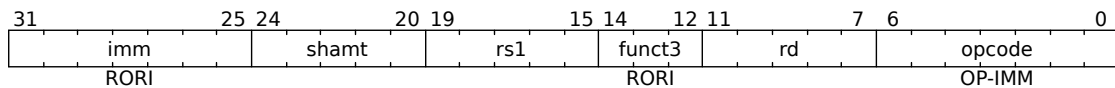


Figure 143: Bitwise Rotation Instructions (cont.)

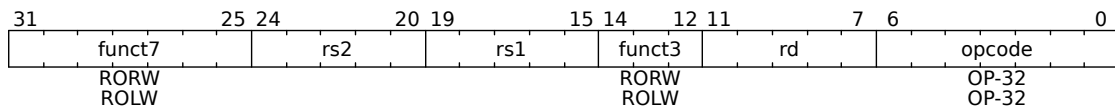


Figure 144: Bitwise Rotation Instructions (cont.)

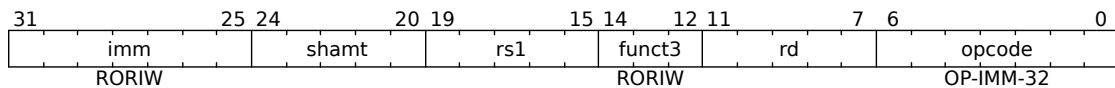


Figure 145: Bitwise Rotation Instructions (cont.)

Instruction	Description
ROR rd, rs1, rs2	Performs a rotate right shift of rs1 by the amount in the least-significant 6 bits of rs2
ROL rd, rs1, rs2	Performs a rotate left shift of rs1 by the amount in the least-significant 6 bits of rs2
RORI rd, rs1, shamt	Performs a rotate right shift of rs1 by the amount in the least significant 6 bits of shamt
RORW rd, rs1, rs2	Performs a rotate right shift of the least-significant word of rs1 by the amount in the least-significant 5 bits of rs2. The resulting word value is sign-extended by copying bit 31 to all of the more-significant bits.
ROLW rd, rs1, rs2	Performs a rotate left shift on the least-significant word of rs1 by the amount in the least-significant 5 bits of rs2. The resulting word value is sign-extended by copying bit 31 to all of the more-significant bits.
RORIW rd, rs1, imm	Performs a rotate right shift on the least-significant word of rs1 by the amount in the least-significant 6 bits of shamt. The resulting word value is sign-extended by copying bit 31 to all of the more significant bits.

Table 106: Bitwise Rotation Instructions Description

5.11.7 OR Combine Instruction

Instruction	Description
ORC.B rd, rs	Combines the bits within every byte through a reciprocal bitwise logical OR. This sets the bits of each byte in the result rd to all zeros if no bit within the respective byte of rs is set, otherwise it sets the bits to all ones if any bit within the respective byte of rs is set.

Table 107: OR Combine Instruction Description

5.11.8 Byte-Reverse Instruction

Instruction	Description
REV8 rd, rs	Reverses the order of the bytes in a register

Table 108: Byte-Reverse Instruction Description

5.12 Zicsr Extension: Control and Status Register Instructions

RISC-V defines a separate address space of 4,096 Control and Status registers associated with each hart. The defined instructions access counter, timers and floating-point status registers.

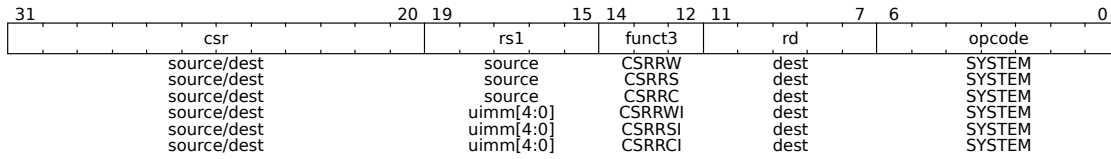


Figure 146: Zicsr Instructions

Instruction	Description
CSRRW rd, rs1 csr	Instruction atomically swaps values in the CSRs and integer registers.
CSRRS rd, rs1 csr	Instruction reads the value of the CSR, zero-extends the value to 64-bits, and writes it to integer register rd. The initial value in integer register rs1 is treated as a bit mask that specifies bit positions to be set in the CSR.
CSRRC rd, rs1 csr	Instruction reads the value of the CSR, zero-extends the value to 64-bits, and writes it to integer register rd. The initial value in integer register rs1 is treated as a bit mask that specifies bit positions to be cleared in the CSR.
CSRRWI rd, rs1 csr	Update the CSR using an 64-bit value obtained by zero-extending a 5-bit unsigned immediate (uimm[4:0]) field encoded in the rs1 field instead of a value from an integer register.
CSRRSI rd, rs1 csr	Update the CSR using an 64-bit value obtained by zero-extending a 5-bit unsigned immediate (uimm[4:0]) field encoded in the rs1 field instead of a value from an integer register.
CSRRCI rd, rs1 csr	If the uimm[4:0] field is zero, then these instructions will not write to the CSR.

Table 109: Control and Status Register Instruction Description

The CSRRWI, CSRRSI, and CSRRCI instructions are similar in kind to CSRRW, CSRRS, and CSRRC respectively, except in that they update the CSR using an 64-bit value obtained by zero-extending a 5-bit unsigned immediate (uimm[4:0]) field encoded in the rs1 field instead of a value from an integer register. For CSRRSI and CSRRCI, these instructions will not write to the CSR if the uimm[4:0] field is zero, and they shall not cause any of the size effects that might otherwise occur on a CSR write. For CSRRWI, if rd = x0, then the instruction shall not read the CSR and shall not cause any of the side effects that might occur on a CSR read. Both CSRRSI and CSRRCI will always read the CSR and cause any read side effects regardless of the rd and rs1 fields.

Table 110 shows if a CSR reads or writes given a particular CSR.

Register Operand				
Instruction	rd	rs1	Read CSR?	Write CSR?
CSRRW	x0	—	No	Yes
CSRRW	!x0	—	Yes	Yes
CSRRS/C	—	x0	Yes	No
CSRRS/C	—	!x0	Yes	Yes
Immediate Operand				
Instruction	rd	uimm	Read CSR?	Write CSR?
CSRRWI	x0	—	No	Yes
CSRRWI	!x0	—	Yes	Yes
CSRRS/CI	—	0	Yes	No
CSRRS/CI	—	!0	Yes	Yes

Table 110: CSR Reads and Writes

5.12.1 Control and Status Registers

The control and status registers (CSRs) are only accessible using variations of the CSRR (Read) and CSRRW (Write) instructions. Only the CPU executing the `csr` instruction can read or write these registers, and they are not visible by software outside of the core they reside on. The standard RISC-V ISA sets aside a 12-bit encoding space (`csr[11:0]`) for up to 4,096 CSRs. Attempts to access a non-existent CSR raise an illegal instruction exception. Attempts to access a CSR without appropriate privilege level or to write a read-only register also raise an illegal instruction exception. A read/write register may also contain some bits that are read-only, in which case, writes to the read-only bits are ignored. Each core functionality has its own control and status registers which are described in the following section.

5.12.2 Defined CSRs

The following tables describe the currently defined CSRs, categorized by privilege level. The usage of the CSRs below is implementation specific. CSRs are only accessible when operating within a specific access mode (user mode, debug mode, supervisor mode, or machine mode). Therefore, attempts to access a non-existent CSR raise an illegal instruction exception, and attempts to access a CSR without appropriate privilege level or to write a read-only register also raise illegal instruction exceptions.

Number	Privilege	Name	Description
User Trap Setup			
0x000	RW	ustatus	User status register.
0x004	RW	uie	User interrupt-enable register.
0x005	RW	utvec	User trap handler base address.
User Trap Handling			
0x040	RW	uscratch	Scratch register for use trap handlers.
0x041	RW	uepc	User exception program counter.
0x042	RW	ucause	User trap cause.
0x043	RW	ubadaddr	User bad address.
0x044	RW	uip	User interrupt pending.
User Floating-Point CSRs			
0x001	RW	fflags	Floating-Point Accrued Exceptions.
0x002	RW	frm	Floating-Point Dynamic Rounding Mode.
0x003	RW	fcsr	Floating-Point Control and Status Register (frm + fflags).
User Counter/Timers			
0xC00	RO	cycle	Cycle counter for RDCYCLE instruction.
0xC01	RO	time	Timer for RDTIME instruction.
0xC02	RO	instret	Instructions-retired counter for RDINSTRET instruction.
0xC03	RO	hpmcounter3	Performance-monitoring counter.
0xC04	RO	hpmcounter4	Performance-monitoring counter.
		...	
0xC1F	RO	hpmcounter31	Performance-monitoring counter.

Table 111: User Mode CSRs

Number	Privilege	Name	Description
Supervisor Trap Setup			
0x100	RW	sstatus	Supervisor status register.
0x102	RW	sedeleg	Supervisor exception delegation register.
0x103	RW	sideleg	Supervisor interrupt delegation register.
0x104	RW	sie	Supervisor interrupt-enable register.
0x105	RW	stvec	Supervisor trap handler base address.
0x106	RW	scounteren	Supervisor counter enable.
Supervisor Trap Handling			
0x140	RW	sscratch	Scratch register for supervisor trap handlers.
0x141	RW	sepc	Supervisor exception program counter.
0x142	RW	scause	Supervisor trap cause.
0x143	RW	stval	Supervisor bad address or instruction.
0x144	RW	sip	Supervisor interrupt pending.
Supervisor Protection and Translation			
0x180	RW	satp	Supervisor address translation and protection.
Supervisor Interrupts and Exceptions			
0xDA0	RO	scountovf	32-bit read-only register that contains shadow copies of the 0F bits in the 29 mhpmevent CSRs.

Table 112: Supervisor Mode CSRs

Number	Privilege	Name	Description
Machine Information Registers			
0xF11	RO	mvendorid	Vendor ID.
0xF12	RO	marchid	Architecture ID.
0xF13	RO	mimpid	Implementation ID.
0xF14	RO	mhartid	Hardware thread ID.
Machine Trap Setup			
0x300	RW	mstatus	Machine status register.
0x301	RW	misa	ISA and extensions.
0x302	RW	medeleg	Machine exception delegation register.
0x303	RW	mideleg	Machine interrupt delegation register.
0x304	RW	mie	Machine interrupt-enable register.
0x305	RW	mtvec	Machine trap-handler base address.
0x306	RW	mcounteren	Machine counter enable.
Machine Trap Handling			
0x340	RW	mscratch	Scratch register for machine trap handlers.
0x341	RW	mepc	Machine exception program counter.
0x342	RW	mcause	Machine trap cause.
0x343	RW	mtval	Machine bad address or instruction.
0x344	RW	mip	Machine interrupt pending.
Machine Memory Protection			
0x3A0	RW	pmpcfg0	Physical memory protection configuration.
0x3A1	RW	pmpcfg1	Physical memory protection configuration, RV32 only.
0x3A2	RW	pmpcfg2	Physical memory protection configuration.
0x3A3	RW	pmpcfg3	Physical memory protection configuration, RV32 only.
0x3B0	RW	pmpaddr0	Physical memory protection address register.
0x3B1	RW	pmpaddr1	Physical memory protection address register.
		...	
0x3BF	RW	pmpaddr15	Physical memory protection address register.
Machine Counter/Timers			
0xB00	RW	mcycle	Machine cycle counter.
0xB02	RW	minstret	Machine instruction-retired counter.
Machine Counter Setup			
0x320	RW	mcountinhibit	Machine counter-inhibit register.
0x323	RW	mhpmevent3	Machine performance-monitoring event selector.
0x324	RW	mhpmevent4	Machine performance-monitoring event selector.
		...	
0x33F	RW	mhpmevent31	Machine performance-monitoring event selector.
Debug/Trace Register (shared with Debug Mode)			
0x7A0	RW	tselect	Debug/Trace trigger register select.

Table 113: Machine Mode CSRs

Number	Privilege	Name	Description
0x7A1	RW	tdata1	First Debug/Trace trigger data register.
0x7A2	RW	tdata2	Second Debug/Trace trigger data register.
0x7A3	RW	tdata3	Third Debug/Trace trigger data register.

Table 113: Machine Mode CSRs

Number	Privilege	Name	Description
0x7B0	RW	dcsr	Debug control and status register.
0x7B1	RW	dpc	Debug PC.
0x7B2	RW	dscratch	Debug scratch register.

Table 114: Debug Mode Registers

5.12.3 CSR Access Ordering

On a given hart, explicit and implicit CSR accesses are performed in program order with respect to those instructions whose execution behavior is affected by the state of the accessed CSR. In particular, a CSR access is performed after the execution of any prior instructions in program order whose behavior modifies or is modified by the CSR state and before the execution of any subsequent instructions in program order whose behavior modifies or is modified by the CSR state.

Furthermore, a CSR read access instruction returns the accessed CSR state before the execution of the instruction, while a CSR write access instruction updates the accessed CSR state after the execution of the instruction. Where the above program order does not hold, CSR accesses are weakly ordered, and the local hart or other harts may observe the CSR accesses in an order different from program order. In addition, CSR accesses are not ordered with respect to explicit memory accesses, unless a CSR access modifies the execution behavior of the instruction that performs the explicit memory access or unless a CSR access and an explicit memory access are ordered by either the syntactic dependencies defined by the memory model or the ordering requirements defined by the Memory-Ordering PMAs. To enforce ordering in all other cases, software should execute a FENCE instruction between the relevant accesses. For the purposes of the FENCE instruction, CSR read accesses are classified as device input (I), and CSR write accesses are classified as device output (O). For more about the FENCE instructions, see Section 5.16. For CSR accesses that cause side effects, the above ordering constraints apply to the order of the initiation of those side effects but does not necessarily apply to the order of the completion of those side effects.

5.12.4 SiFive RISC-V Implementation Version Registers

The following sections further describe SiFive's implementation of the RISC-V implementation version registers.

5.12.4.1 Machine Vendor ID Register (`mvendorid`)

The value in `mvendorid` is 0x489, corresponding to SiFive's JEDEC number.

5.12.4.2 Machine Architecture ID Register (`marchid`)

The value in `marchid` indicates the overall microarchitecture of the core and at SiFive we use this to distinguish between core generators. The RISC-V standard convention separates `marchid` into open-source and proprietary namespaces using the most-significant bit (MSB) of the `marchid` register; where if the MSB is clear, the `marchid` is for an open-source core, and if the MSB is set, then `marchid` is a proprietary microarchitecture. The open-source namespace is managed by the RISC-V Foundation and the proprietary namespace is managed by SiFive.

SiFive's E3 and S5 cores are based on the open-source 3/5-Series microarchitecture, which has a Foundation-allocated `marchid` of 1. Our other generators are numbered according to the core series.

Value	Core Generator
0x8000_0007	6/7/P200/X200-Series Processor

Table 115: Core Generator Encoding of `marchid`

5.12.4.3 Machine Implementation ID Register (`mimpid`)

The value in `mimpid` holds an encoded value that uniquely identifies the version of the generator used to build this implementation. If your release version is not included in Table 116, contact your SiFive account manager for more information.

Value	Generator Release Version
0x0000_0000	Pre-19.02
0x2019_0228	19.02
0x2019_0531	19.05
0x2019_0919	19.08p0p0 / 19.08.00
0x2019_1105	19.08p1p0 / 19.08.01.00
0x2019_1204	19.08p2p0 / 19.08.02.00
0x2020_0423	19.08p3p0 / 19.08.03.00
0x0120_0626	19.08p4p0 / 19.08.04.00
0x0220_0515	koala.00.00-preview and koala.01.00-preview
0x0220_0603	koala.02.00-preview
0x0220_0630	20G1.03.00 / koala.03.00-general
0x0220_0710	20G1.04.00 / koala.04.00-general
0x0220_0826	20G1.05.00 / koala.05.00-general
0x0320_0908	kiwi.00.00-preview
0x0220_1013	20G1.06.00 / koala.06.00-general
0x0220_1120	20G1.07.00 / koala.07.00-general
0x0421_0205	llama.00.00-preview
0x0421_0324	21G1.01.00 / llama.01.00-general
0x0421_0427	21G1.02.00 / llama.02.00-general
0x0521_0528	mongoose.00.00-preview
0x0521_0714	21G2.01.00 / mongoose.01.00-general
0x0521_1008	21G2.02.00 / mongoose.02.00-general
0x0621_1027	narwhal.00.00-preview
0x0621_1203	narwhal.01.00-preview
0x0621_1222	21G3.02.00 / narwhal.02.00-general
0x0622_0316	21G3.03.00 / narwhal.03.00-general
0x0622_0425	21G3.04.00 / narwhal.04.00-general

Table 116: Generator Release Encoding of mimpid

5.12.4.4 Reading Implementation Version Registers

To read the mvendorid, marchid, and mimpid registers, simply replace mimpid with mvendorid or marchid as needed.

- In C:

```
uintptr_t mimpid;
__asm__ volatile("csrr %0, mimpid" : "=r"(mimpid));
```

- In Assembly:

```
csrr a5, mimpid
```

5.12.5 RISC-V Status Registers

The following sections further describe the RISC-V status registers.

5.12.5.1 Machine Status Register (mstatus)

The mstatus register keeps track of and controls the hart's current operating state, including whether or not interrupts are enabled. A summary of the mstatus fields related to hart state is provided in Table 117. Note that this is not a complete description of mstatus as it contains fields related to interrupts, which are described in Section 7.7.1. For the full description of mstatus, consult the *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*.

Machine Status Register (mstatus)			
CSR	0x300		
Bits	Field Name	Attr.	Description
[10:9]	VS[1:0]	WARL	Vector context status
[12:11]	Reserved	WPRI	
[14:13]	FS[1:0]	WARL	Floating-point context status
[16:15]	XS[1:0]	RO	Additional user-mode extensions and associated state status
17	MPRV	RW	Modify privilege level
18	SUM	RW	Permit supervisor user memory access
19	MXR	RW	Make executable readable
20	TVM	WARL	Trap virtual memory
21	TW	WARL	Timeout wait
22	TSR	WARL	Trap SRET
[31:23]	Reserved	WPRI	
[33:32]	UXL[1:0]	WARL	User mode XLEN
[35:34]	SXL[1:0]	WARL	Supervisor mode XLEN
[62:36]	Reserved	WPRI	
63	SD	RO	FS dirty state status

Table 117: Machine Status Register (partial)

5.12.5.2 Supervisor Status Register (sstatus)

Similar to machine mode, supervisor mode has a register dedicated to keeping track of the hart's current state called sstatus. sstatus is effectively a restricted view of mstatus, in that changes made to sstatus are reflected in mstatus and vice-versa, with the exception of the machine mode fields, which are not visible in sstatus.

A summary of the sstatus fields related to hart state is provided in Table 118. Note that this is not a complete description of sstatus as it also contains fields related to interrupts, which are described in Section 7.8.2. For the full description of sstatus, consult the *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*.

Supervisor Status Register (sstatus)			
CSR	0x100		
Bits	Field Name	Attr.	Description
[10:9]	VS[1:0]	WARL	Vector context status
[12:11]	Reserved	WPRI	
[14:13]	FS[1:0]	WARL	Floating-point unit state status
[16:15]	XS[1:0]	RO	Additional user-mode extensions and associated state status
17	Reserved	WPRI	
18	SUM	RW	Permit supervisor user memory access
19	MXR	RW	Make executable readable
[31:20]	Reserved	WPRI	
[33:32]	UXL[1:0]	WARL	User mode XLEN
[62:34]	Reserved	WPRI	
63	SD	RO	FS dirty state status

Table 118: Supervisor Status Register (partial)

5.12.6 Custom CSRs

SiFive implements some custom CSRs that are specific to the implementation. For these CSRs, including the Feature Disable CSR, consider Chapter 6.

5.13 Base Counters and Timers

RISC-V ISAs provide a set of up to 32×64-bit performance counters and timers that are accessible via unprivileged 64-bit read-only CSR registers 0xc00–0xc1f. The first three of these (CYCLE, TIME, and INSTRET) have dedicated functions; while the remaining counters, if implemented, provide programmable event counting.

The Tenstorrent X280 MC implements `mcycle`, `mtime`, and `minstret` counters, which have dedicated functions: cycle count, real-time clock, and instructions-retired, respectively. The timer functionality is based on the `mtime` register. Additionally, the Tenstorrent X280 MC implements event counters in the form of `mhpmpcounter`, which is used to monitor user requested events.

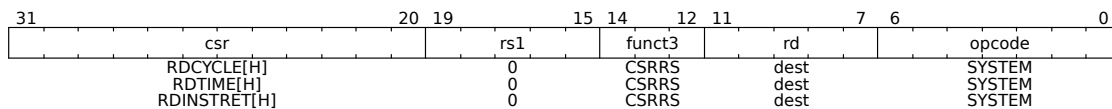


Figure 147: Timer and Counter Pseudoinstructions

Instruction	Description
RDCYCLE rd	Reads the 64-bits of the cycle CSR which holds a count of the number of clock cycles executed by the processor core on which the hart is running from an arbitrary start time in the past.
RDTIME rd	Generates an illegal instruction exception. The <code>mtime</code> register is memory mapped to the CLINT register space and can be read using a regular load instruction.
RDINSTRET rd	Reads the 64-bits of the instret CSR, which counts the number of instructions retired by this hart from some arbitrary start point in the past.

Table 119: Timer and Counter Pseudoinstruction Description

RDCYCLE, RDTIME, and RDINSTRET pseudoinstructions read the full 64 bits of the cycle, time, and instret counters. The RDCYCLE pseudoinstruction reads the low 64-bits of the cycle CSR (`mcycle`), which holds a count of the number of clock cycles executed by the processor core on which the hart is running from an arbitrary start time in the past. The RDTIME pseudoinstruction reads the low 64-bits of the time CSR (`mtime`), which counts wall-clock real time that has passed from an arbitrary start time in the past. The RDINSTRET pseudoinstruction reads the low 64-bits of the instret CSR (`minstret`), which counts the number of instructions retired by this hart from some arbitrary start point in the past. The rate at which the cycle counter advances is `rtc_clock`. To determine the current rate (cycles per second) of instruction execution, call the `metal_timer_get_timebase_frequency` API. The `metal_timer_get_timebase_frequency` and additional APIs are described in Section 5.13.2 below.

Number	Privilege	Name	Description
0xC00	RO	cycle	Cycle counter for RDCYCLE instruction
0xC01	RO	time	Timer for RDTIME instruction
0xC02	RO	instret	Instruction-retired counter for RDINSTRET instruction

Table 120: Timer and Counter CSRs

5.13.1 Timer Register

`mtime` is a 64-bit read-write register that contains the number of cycles counted from the `rtc_toggle` signal described in the Tenstorrent X280 MC User Guide. On reset, `mtime` is cleared to zero.

5.13.2 Timer API

The APIs below are used for reading and manipulating the machine timer. Other APIs are described in more detail within the Freedom Metal documentation. <https://sifive.github.io/freedom-metal-docs/>

5.13.2.1 Functions

int metal_timer_get_cyclecount(int hartid, unsigned long long *cyclecount)

Read the machine cycle count.

Return

0 upon success

Parameters

- **hartid**: The hart ID to read the cycle count of
- **cyclecount**: The variable to hold the value

int metal_timer_get_timebase_frequency(int hartid, unsigned long long *timebase)

Get the machine timebase frequency.

Return

0 upon success

Parameters

- **hartid**: The hart ID to read the cycle count of
- **timebase**: The variable to hold the value

int metal_timer_set_tick(int hartid, int second)

Set the machine timer tick interval in seconds.

Return

0 upon success

Parameters

- **hartid**: The hart ID to read the cycle count of
- **second**: The number of seconds to set the tick interval to

5.14 Privileged Instructions

The RISC-V architecture implements privileged instructions that can only be executed when the Tenstorrent X280 MC is operating in a privileged mode. The SYSTEM major opcode is used to encode all of the privileged instructions.

5.14.1 Machine-Mode Privileged Instructions

5.14.1.1 Environment Call and Breakpoint

These ECALL and EBREAK instructions cause a precise requested trap to the supporting execution environment. The ECALL instruction is used to make a service request to the execution environment. The EBREAK instruction is used to return control to a debugging environment.

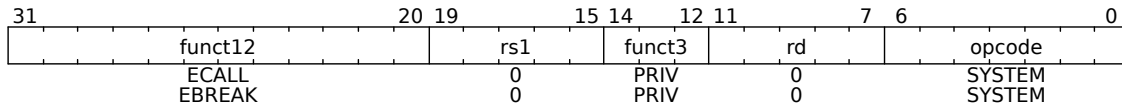


Figure 148: ECALL and EBREAK Instructions

5.14.1.2 Trap-Return Instructions

To return after handling a trap, there are separate trap return instructions per privilege level: MRET and SRET. MRET is always provided, while SRET is provided if the respective privilege mode is supported. An xRET instruction can be executed in privilege mode x or higher, where executing a lower-privilege xRET instruction will pop the relevant lower-privilege interrupt enable and privilege mode stack.

In addition to manipulating the privilege stack, xRET instructions sets the pc to the value stored in the corresponding xepc register.

5.14.1.3 Wait for Interrupt

The Wait for Interrupt (WFI) instruction provides a hint to the Tenstorrent X280 MC that the current hart can be stalled until an interrupt might need servicing. Execution of the WFI instruction can also be used to inform the hardware platform that suitable interrupts should preferentially be routed to this hart.

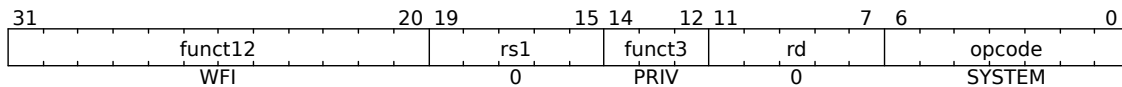


Figure 149: Wait for Interrupt Instruction

If an enabled interrupt is present or later becomes present while the hart is stalled, the interrupt exception will be taken on the following instruction, i.e., execution resumes in the trap handler and mepc = pc + 4. The WFI instruction can also be executed when interrupts are disabled. The operation of WFI must be unaffected by the global interrupt bits in mstatus (MIE/SIE) (i.e., the hart must resume if a locally enabled interrupt becomes pending), but should honor the individual interrupt enables (e.g., MTIE). WFI is also required to resume execution for locally enabled interrupts pending at any privilege level, regardless of the global interrupt enable at each privilege level. If the event that causes the hart to resume execution does not cause an interrupt to be taken, execution will resume at pc + 4, and software must determine what action to take, including looping back to repeat the WFI if there was no actionable event.

The suggested way to call WFI is inside an infinite loop as described below.

```
while (1) {
    __asm__ volatile ("wfi");
}
```

In SiFive's implementation of WFI, the WFI instruction is issued and the core goes into an internal clock gating state.

5.14.2 Supervisor-Mode Privileged Instructions

For S-mode enabled cores, one new supervisor-level instruction is provided in addition to SRET.

5.14.2.1 Supervisor Memory-Management Fence Instruction

The supervisor memory-management fence instruction SFENCE.VMA is used to synchronize updates to in-memory memory-management data structures with current execution.

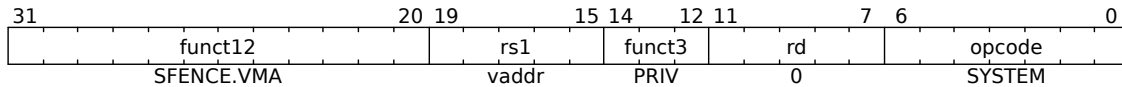


Figure 150: Supervisor Memory-Management Fence Instruction

Instruction execution causes implicit reads and writes to these data structures; however, these implicit references are ordinarily not ordered with respect to loads and stores in the instruction stream. Executing an SFENCE.VMA instruction guarantees that any stores in the instruction stream prior to the execution of SFENCE.VMA are ordered before all implicit references subsequent to the SFENCE.VMA. Furthermore, executing an SFENCE.VMA guarantees that any implicit writes caused by instructions prior to the SFENCE.VMA are ordered before all loads and stores subsequent to the SFENCE.VMA.

5.15 ABI - Register File Usage and Calling Conventions

RV64GCV_Zfh_Zba_Zbb_Sscopfpmf has 32 x registers that are each 64 bits wide.

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	-
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	-
x4	tp	Thread pointer	-
x5	t0	Temporary / alternate link register	Caller
x6-7	t1-2	Temporaries	Caller
x8	s0/fp	Saved-register / frame-pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments / return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller
Floating-Point Registers			
f0-7	ft0-7	FP temporaries	Caller
f8-9	fs0-1	FP saved registers	Callee
f10-11	fa0-1	FP arguments / return values	Caller
f12-17	fa2-7	FP arguments	Caller
f18-27	fa2-11	FP saved registers	Callee
f28-31	ft8-11	FP temporaries	Caller

Table 121: RISC-V Registers

The programmer counter PC hold the address of the current instruction.

- x1 / ra - holds the return address for a call.
- x2 / sp - stack pointer, points to the current routine stack.
- x8 / fp / s0 - frame pointer, points to the bottom of the top stack frame.
- x3 / gp - global pointer, points into the middle of the global data section.
The common definition is: .data + 0x800. RISC-V immediate values are 12-bit signed values, which is +/- 2048 in decimal or +/- 0x800 in hex. So that global pointer relative accesses can reach their full extent, the global pointer point + 0x800 into the data section. The linker can then relax LUI+LW, LUI+SW into gp-relative LW or SW, i.e., shorter instruction sequences and access most global data using LW at gp +/- offset

```
LW t0 , 0x800(gp)
LW t1 , 0x7FF(gp)
```

- x4 / tp - thread pointer, point to thread-local storage (TLS-mostly used in Linux and RTOS).
If you create a variable in TLS, every thread has its own copy of the variable, i.e., changes to the variable are local to the thread. This is a static area of memory that gets copied for each thread in a program. It is also used to create libraries that have thread-safe functions,

because of the fact that each call to a function has its copy of the same global data, so it's safe.

5.15.1 RISC-V Assembly

RISC-V instructions have opcodes and operands.

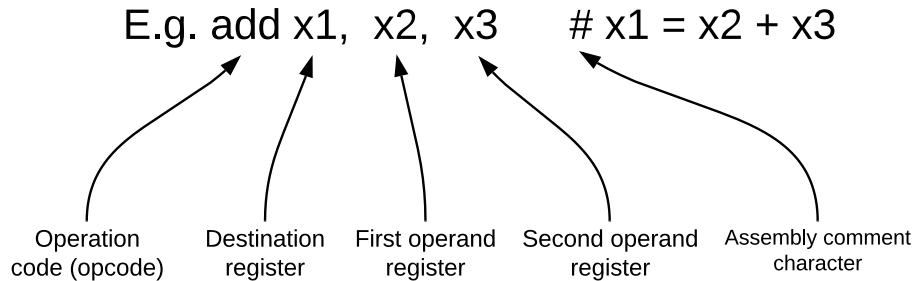


Figure 151: RISC-V Assembly Example

Assembly	C	Description
<code>add x1, x2, x3</code>	<code>a = b + c</code>	<code>a=x1, b=x2, c=x3</code>
<code>sub x3, x4, x5</code>	<code>d = e - f</code>	<code>d=x3, e=x4, f=x5</code>
<code>add x0, x0, x0</code>	NOP	Writes to x0 are always ignored
<code>add x3, x4, x0</code>	<code>f = g</code>	<code>f=x3, g=x4</code>
<code>addi x3, x4, -10</code>	<code>f = g - 10</code>	<code>f=x3, g=x4</code>
<code>lw x10, 12(x13) # 12 = 3x4</code> <code>add x11, x12, x10</code>	<code>int A[100];</code> <code>g = h + A[3];</code>	Reg x10 gets A[3] <code>g=x11, h=x12</code>
<code>lw x10, 12(x13) # 12 = 3x4</code> <code>add x10, x12, x10</code> <code>sw x10, 40(x13) # 40 = 10x4</code>	<code>int A[100];</code> <code>A[10] = h + A[3];</code>	Reg x10 gets A[3] <code>h=x12</code> Reg x10 gets <code>h + A[3]</code>
<code>bne x13, x14, done</code> <code>add x10, x11, x12</code> <code>done:</code>	<code>if (i == j)</code> <code> f = g + h;</code>	<code>f=x10, g=x11, h=x12, i=x13, j=x14</code>
<code>bne x10, x14, else</code> <code>add x10, x11, x12</code> <code>j done</code> <code>else: sub x10, x11, x12</code> <code>done:</code>	<code>if (i == j)</code> <code> f = g + h;</code> <code>else</code> <code> f = g - h;</code>	<code>f=x10, g=x11, h=x12, i=x13, j=x14</code>

Table 122: RISC-V Assembly and C Examples

5.15.2 Assembler to Machine Code

The following flowchart describes how the assembler converts the RISC-V assembly code to machine code.

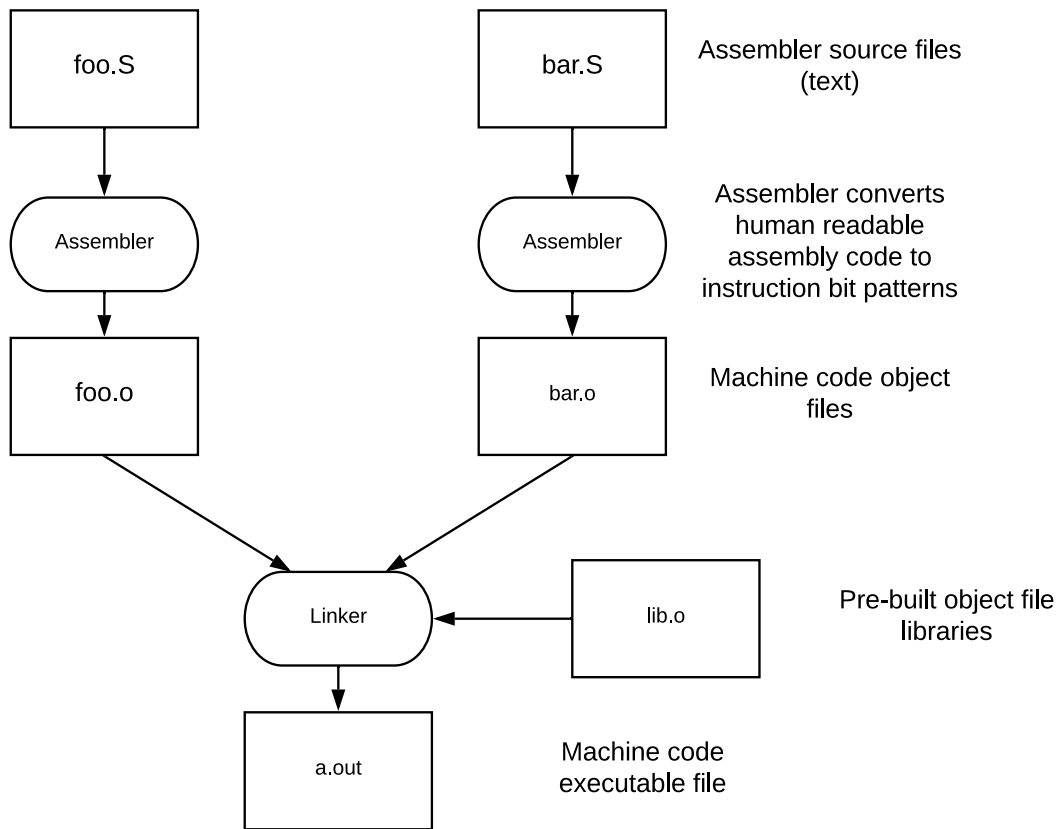


Figure 152: RISC-V Assembly to Machine Code

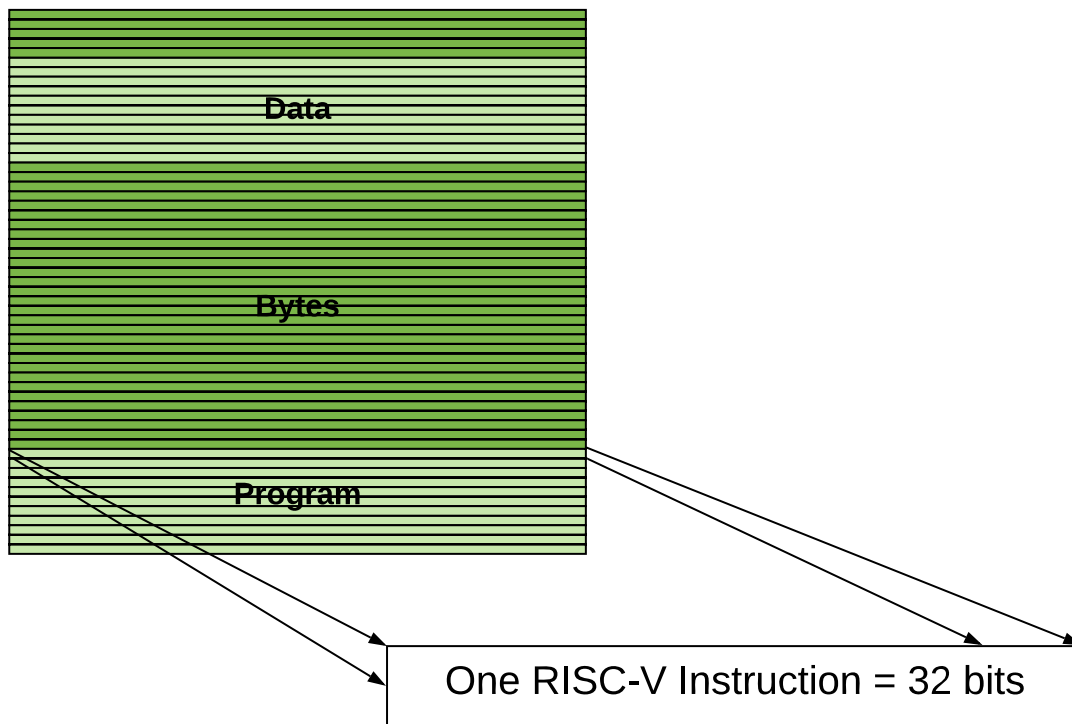


Figure 153: One RISC-V Instruction

5.15.3 Calling a Function (Calling Convention)

1. Put parameters in place where function can access them.
2. Transfer control to function.
3. Acquire local resources needed for function.
4. Perform function task.
5. Place result values where calling code can access and restore any registers might have used.
6. Return control to original caller.

Caller-saved The function invoked can do whatever it likes with the registers. Callee-saved If a function wants to use registers it needs to store and restore them.

Take, for example, the following function:

```
int leaf(int g, int h, int i, int j) {  
    int f;  
    f = (g+h) - (i+j);  
    return f;  
}
```

In this function above, arguments are passed in a0, a1, a2 and a3. The return value is returned in a0.

```
addi sp, sp, -8    # adjust stack for 2 items
sw s1, 4(sp)       # save s1 for use afterwards
sw s0, 0(sp)       # save s0 for use afterwards

add s0,a0,a1       # s0 = g + h
add s1,a2,a3       # s1 = i + j
sub a0,s0,s1       # return value (g + h) - (i + j)

lw s0, 0(sp)       # restore register s0 for caller
lw s1, 4(sp)       # restore register s1 for caller
addi sp, 4(sp)     # adjust stack to delete 2 items
jr ra              # jump back to calling routine
```

In the assembly above, notice that the stack pointer was decremented by 8 to make room to save the registers. Also, s1 and s0 are saved and will be stored at the end.

5.15.3.1 Nested Functions

In the case of nested function calls, values held in a0-7 and ra will be clobbered.

Take, for example, the following function:

```
int sumSquare(int x, int y) {
    return mult(x,x) + y;
}
```

In the function above, a function called sumSquare is calling mult. To execute the function, there's a value in ra that sumSquare wants to jump back to, but this value will be overwritten by the call to mult.

To avoid this, the sumSquare return address must be saved before the call to mult. To save the return address of sumSquare, the function can utilize stack memory. The user can use stack memory to preserve automatic (local) variables that don't fit within the registers.

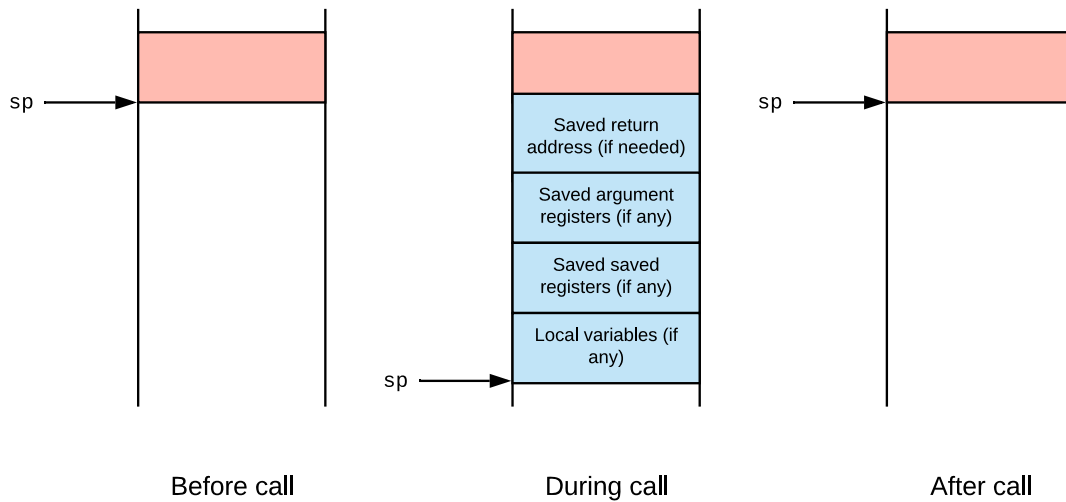


Figure 154: Stack Memory during Function Calls

Consider the assembly for sumSquare below:

```
sumSquare:
addi sp,sp,-8      # reserve space on stack
sw ra, 4(sp)       # save return address
sw a1, 0(sp)       # save y
mv a1,a0           # mult(x,x)
jal mult           # call mult
lw a1, 0(sp)       # restore y
add a0,a0,a1       # mult()+y
lw ra, 4(sp)       # get return address
addi sp,sp,8       # restore stack
mult:...
```

5.16 Memory Ordering - FENCE Instructions

In the RISC-V ISA, each thread, referred to as a hart, observes its own memory operations as if they executed sequentially in program order. RISC-V also has a relaxed memory model, which requires explicit FENCE instructions to guarantee the ordering of memory operations.

The FENCE instructions include FENCE and FENCE.I. The FENCE instruction simply ensures that the memory access instructions before the FENCE instruction get committed before the FENCE instruction is committed. It does not guarantee that those memory access instructions have actually completed. For example, a load instruction before a FENCE instruction can commit without waiting for its value to come back from the memory system. FENCE.I functions the same as FENCE, as well as flushes the instruction cache.

For example, without FENCE instructions:

Hart 1 executes:

Load X
Store Y
Store Z

Because of relaxed memory model, Hart 2 could see stores/loads arranged in any order:

Store Z
Load X
Store Y

With FENCE instructions:

Hart 1 executes:

Load X
Store Y
FENCE
Store Z

Hart 2 sees:

Store Y
Load X
Store Z

With FENCE instructions, Hart 2 is forced to see the Load X and the Store Y prior to the Store Z, but could arbitrarily see Store Y before Load X or Load X before Store Y. Functionally, FENCE instructions order the completion of older memory accesses prior to newer accesses. However, unnecessary FENCE instructions slow processes and can hide bugs, so it is essential to identify where and when FENCE should be used.

5.17 Boot Flow

This process is managed as part of the Freedom Metal source code. The freedom-metal boot code supports single core boot or multi-core boot, and contains all the necessary initialization code to enable every core in the system.

1. ENTRY POINT: File: freedom-metal/src/entry.S, label: _enter.
2. Write mtvec register with early_trap_vector as default exception handler.
3. Initialize global pointer gp register using the generated symbol __global_pointer\$.
4. Clear feature disable CSR 0x7c1.
5. Read mhartid into register a0 and call _start, which exists in crt0.S.
6. We now transition to File: freedom-metal/gloss/crt0.S, label: _start.
7. Initialize stack pointer, sp, with _sp generated symbol. Harts with mhartid of one or larger are offset by ($_sp + __stack_size \times mhartid$). The `__stack_size` field is generated in the linker file.

8. Check if `mhartid == __metal_boot_hart` and run the init code if they are equal. All other harts skip init and go to the Post-Init Flow, step #15.
9. Boot Hart Init Flow begins here.
10. Init data section to destination in defined RAM space.
11. Copy ITIM section, if ITIM code exists, to destination.
12. Zero out bss section.
13. Call `atexit` library function that registers the `libc` and `freedom-metal` destructors to run after main returns.
14. Call the `__libc_init_array` library function, which runs all functions marked with `__attribute__((constructor))`.
 - a. For example, PLL, UART, hardware prefetcher, L2, L2 prefetcher, and/or L3, if they exist in the design. This method provides full early initialization prior to entering the main application. Note that prefetchers are initialized to a known value at startup, but not tuned for any specific workload; refer to the relevant driver software in `freedom-metal/src/drivers/` for a demonstration of this tuning.
15. Post-Init Flow Begins Here.
16. Call the C routine `__metal_synchronize_harts`, where hart 0 will release all harts once their individual `msip` bits are set. The `msip` bit is typically used to assert a software interrupt on individual harts, however interrupts are not yet enabled, so `msip` in this case is used as a gatekeeping mechanism.
17. Check `misa` register to see if floating-point hardware is part of the design, and set up `mstatus` accordingly.
18. Single or multi-hart design redirection step.
 - a. If design is a single hart only, or a multi-hart design without a C-implemented function `secondary_main`, ONLY the boot hart will continue to `main()`.
 - b. For multi-hart designs, all other CPUs will enter sleep via WFI instruction via the weak `secondary_main` label in `crt0.S`, while boot hart runs the application program.
 - c. In a multi-hart design which includes a C-defined `secondary_main` function, all harts will enter `secondary_main` as the primary C function.

5.18 Linker File

The linker file generates important symbols that are used in the boot code. The linker file options are found in the `freedom-e-sdk/bsp` path.

There are usually three different linker file options:

- `metal.default.lds` — Use flash and RAM sections

- `metal.ramrodata.lds` — Place read only data in RAM for better performance
- `metal.scratchpad.lds` — Places all code + data sections into available RAM location

Each linker option can be selected by specifying `LINK_TARGET` on the command line.

For example:

```
make PROGRAM=hello TARGET=design-rtl CONFIGURATION=release LINK_TARGET=scratchpad
software
```

The `metal.default.lds` linker file is selected by default when `LINK_TARGET` is not specified. If there is a scenario where a custom linker is required, one of the supplied linker files can be copied and renamed and used for the build. For example, if a new linker file named `metal.newmap.lds` was generated, this can be used at build time by specifying `LINK_TARGET=newmap` on the command line.

5.18.1 Linker File Symbols

The linker file generates symbols that are used by the startup code, so that software can use these symbols to assign the stack pointer, initialize or copy certain RAM sections, and provide the boot hart information. These symbols are made visible to software using the `PROVIDE` keyword.

For example:

```
__stack_size = DEFINED(__stack_size) ? __stack_size : 0x400;
PROVIDE(__stack_size = __stack_size);
```

5.18.1.1 Generated Linker Symbols

A description list of the generated linker symbols is shown below.

`__metal_boot_hart`

This is an integer number to describe which hart runs the main init flow. The `mhartid` CSR contains the integer value for each hart. For example, hart 0 has `mhartid==0`, hart 1 has `mhartid==1`, and so on. An assembly example is shown below, where `a0` already contains the `mhartid` value.

```
/* If we're not hart 0, skip the initialization work */
la t0, __metal_boot_hart
bne a0, t0, _skip_init
```

An example on how to use this symbol in C code is shown below.

```
extern int __metal_boot_hart;
int boot_hart = (int)&__metal_boot_hart;
```

Additional linker file generated symbols, along with descriptions are shown below.

__metal_chicken_bit

Status bit to tell startup code to zero out the Feature Disable CSR. Details of this register are internal use only.

__global_pointer\$

Static value used to write the gp register at startup.

_sp

Address of the end of stack for hart 0, used to initialize the beginning of the stack since the stack grows lower in memory. On a multi-hart system, the start address of the stack for each hart is calculated using $(_sp + _stack_size \times mhartid)$

metal_segment_bss_target_start

metal_segment_bss_target_end

Used to zero out global data mapped to .bss section.

- Only __metal_boot_hart runs this code.

metal_segment_data_source_start

metal_segment_data_target_start

metal_segment_data_target_end

Used to copy data from image to its destination in RAM.

- Only __metal_boot_hart runs this code.

metal_segment_itim_source_start

metal_segment_itim_target_start

metal_segment_itim_target_end

Code or data can be placed in itim sections using the `__attribute__((section(".itim")))`.

- When this attribute is applied to code or data, the `metal_segment_itim_source_start`, `metal_segment_itim_target_start`, and `metal_segment_itim_target_end` symbols get updated accordingly, and these symbols allow the startup code to copy code and data into the ITIM area.
 - Only __metal_boot_hart runs this code.

Note

At the time of this writing, the boot flow does not support C++ projects

5.19 RISC-V Compiler Flags

5.19.1 arch, abi, and mtune

RISC-V targets are described using three arguments:

1. `-march=ISA`: selects the architecture to target.
2. `-mabi=ABI`: selects the ABI to target.
3. `-mtune=CODENAME`: selects the microarchitecture to target.

5.19.1.1 `-march`

This argument controls which instructions and registers are available for the compiler, as defined by the RISC-V user-level ISA specification.

The RISC-V ISA with 32, 32-bit integer registers and the instructions for multiplication would be denoted as RV32IM. Users can control the set of instructions that GCC uses when generating assembly code by passing the lower-case ISA string to the `-march` GCC argument; for example, `-march=rv32im`. On RISC-V systems that don't support particular operations, emulation routines may be used to provide the missing functionality.

Example:

```
double dmul(double a, double b) {  
    return a * b;  
}
```

will compile directly to a FP multiplication instruction when compiled with the D extension:

```
$ riscv64-unknown-elf-gcc test.c -march=rv64imafdc -mabi=lp64d -o- -S -O3  
dmul:  
    fmul.d    fa0,fa0,fa1  
    ret
```

but will compile to an emulation routine without the D extension:

```
$ riscv64-unknown-elf-gcc test.c -march=rv64i -mabi=lp64 -o- -S -O3  
dmul:  
    add      sp,sp,-16  
    sd      ra,8(sp)  
    call     __muldf3  
    ld      ra,8(sp)  
    add      sp,sp,16  
    jr      ra
```

Similar emulation routines exist for the C intrinsics that are trivially implemented by the M and F extensions.

5.19.1.2 -mabi

-mabi selects the ABI to target. This controls the calling convention (which arguments are passed in which registers) and the layout of data in memory. The -mabi argument to GCC specifies both the integer and floating-point ABIs to which the generated code complies. Much like how the -march argument specifies which hardware generated code can run on, the -mabi argument specifies which software-generated code can link against. We use the standard naming scheme for integer ABIs (ilp32 or lp64), with an argumental single letter appended to select the floating-point registers used by the ABI (ilp32 vs. ilp32f vs. ilp32d). In order for objects to be linked together, they must follow the same ABI.

RISC-V defines two integer ABIs and three floating-point ABIs.

- ilp32: int, long, and pointers are all 32-bits long. long long is a 64-bit type, char is 8-bit, and short is 16-bit.
- lp64: long and pointers are 64-bits long, while int is a 32-bit type. The other types remain the same as ilp32.

The floating-point ABIs are a RISC-V specific addition:

- "" (the empty string): No floating-point arguments are passed in registers.
- f: 32-bit and smaller floating-point arguments are passed in registers. This ABI requires the F extension, as without F there are no floating-point registers.
- d: 64-bit and smaller floating-point arguments are passed in registers. This ABI requires the D extension.

5.19.1.3 arch/abi Combinations

- march=rv32imafdc -mabi=ilp32d: Hardware floating-point instructions can be generated and floating-point arguments are passed in registers. This is like the -mfloat-abi=hard argument for the Arm® architecture GCC.
- march=rv32imac -mabi=ilp32: No floating-point instructions can be generated and no floating-point arguments are passed in registers. This is like the -mfloat-abi=soft argument for the Arm architecture GCC.
- march=rv32imafdc -mabi=ilp32: Hardware floating-point instructions can be generated, but no floating-point arguments will be passed in registers. This is like the -mfloat-abi=softfp argument for the Arm architecture GCC, and is usually used when interfacing with soft-float binaries on a hard-float system.
- march=rv32imac -mabi=ilp32d: Illegal, as the ABI requires floating-point arguments are passed in registers but the ISA defines no floating-point registers to pass them in.

Example:

```
double dmul(double a, double b) {
    return b * a;
}
```

If neither the ABI nor ISA contains the concept of floating-point hardware then the C compiler cannot emit any floating-point-specific instructions. In this case, emulation routines are used to perform the computation and the arguments are passed in integer registers:

```
$ riscv64-unknown-elf-gcc test.c -march=rv32imac -mabi=ilp32 -o- -S -O3
dmul:
    mv     a4,a2
    mv     a5,a3
    add    sp,sp,-16
    mv     a2,a0
    mv     a3,a1
    mv     a0,a4
    mv     a1,a5
    sw     ra,12(sp)
    call   __muldf3
    lw     ra,12(sp)
    add    sp,sp,16
    jr     ra
```

The second case is the exact opposite of this one: everything is supported in hardware. In this case we can emit a single `fmul.d` instruction to perform the computation.

```
$ riscv64-unknown-elf-gcc test.c -march=rv32imafdc -mabi=ilp32d -o- -S -O3
dmul:
    fmul.d fa0,fa1,fa0
    ret
```

The third combination is for users who may want to generate code that can be linked with code designed for systems that don't subsume a particular extension while still taking advantage of the extra instructions present in a particular extension. This is a common problem when dealing with legacy libraries that need to be integrated into newer systems. For this purpose, the compiler arguments and multilib paths designed to cleanly integrate with this workflow. The generated code is essentially a mix between the two above outputs: the arguments are passed in the registers specified by the `ilp32` ABI (as opposed to the `ilp32d` ABI, which could pass these arguments in registers) but then once inside the function the compiler is free to use the full power of the RV32IMAFDC ISA to actually compute the result. While this is less efficient than the code the compiler could generate if it was allowed to take full advantage of the D-extension registers, it's a lot more efficient than computing the floating-point multiplication without the D-extension instructions

```
$ riscv64-unknown-elf-gcc test.c -march=rv32imafdc -mabi=ilp32 -o- -S -O3
dmul:
    add    sp,sp,-16
    sw     a0,8(sp)
    sw     a1,12(sp)
    fld    fa5,8(sp)
    sw     a2,8(sp)
    sw     a3,12(sp)
    fld    fa4,8(sp)
    fmul.d  fa5,fa5,fa4
    fsd    fa5,8(sp)
    lw     a0,8(sp)
    lw     a1,12(sp)
    add    sp,sp,16
    jr     ra
```

5.20 Compilation Process

GCC driver script is actually running the preprocessor, then the compiler, then the assembler and finally the linker. If the user runs GCC with the `--save-temps` argument, several intermediate files will be generated.

```
$ riscv64-unknown-linux-gnu-gcc relocation.c -o relocation -O3 --save-temps
```

- `relocation.i`: The preprocessed source, which expands any preprocessor directives (things like `#include` or `#ifdef`).
- `relocation.s`: The output of the actual compiler, which is an assembly file (a text file in the RISC-V assembly format).
- `relocation.o`: The output of the assembler, which is an un-linked object file (an ELF file, but not an executable ELF).
- `relocation`: The output of the linker, which is a linked executable (an executable ELF file).

5.21 Large Code Model Workarounds

RISC-V software currently requires that linked symbols reside within a 32-bit range. There are two types of code models defined for RISC-V, **medlow** and **medany**. The medany code model generates `auipc/ld` pairs to refer to global symbols, which allows the code to be linked at any address, while medlow generates `lui/ld` pairs to refer to global symbols, which restricts the code to be linked around address zero. They both generate 32-bit signed offsets for referring to symbols, so they both restrict the generated code to being linked within a 2 GiB window. When building software, the code model parameter is passed into the RISC-V tool suite and it defines a method to generate the necessary instruction combinations to access global symbols within the software program. This is done using `-mcode1=medany/medlow`. For 32-bit architectures, we use the medlow code model, while medany is used for 64-bit architectures. This is controlled within the 'setting.mk' file in the `freedom-e-sdk/bsp` folder.

The real problem occurs when:

1. Total program size exceeds 2 GiB, which is rare
2. When global symbols within a single compiled image are required to reside in a region outside of the 32-bit space

Example for symbols within 32-bit address space:

```
MEMORY
{
  ram (wxa!ri) : ORIGIN = 0x80000000, LENGTH = 0x4000
  flash (rxai!w) : ORIGIN = 0x20400000, LENGTH = 0x1fc00000
}
```

Example for symbols outside 32-bit address space:

```
MEMORY
{
  ram (wxa!ri) : ORIGIN = 0x100000000, LENGTH = 0x4000 /* Updated ORIGIN from
0x80000000 */
  flash (rxai!w) : ORIGIN = 0x20400000, LENGTH = 0x1fc00000
}
```

If a software example uses the above memory map, and uses either medlow or medany code models, it will not link successfully. Generated errors will generally contain the following phrase:

relocation truncated to fit:

A workaround for the linker error “relocation truncated to fit:” is to use `LINK_TARGET=scratchpad` since both the code and data sections get placed into the ram section, as defined by the linker script. Note that this doesn’t always solve the problem, since some designs do not have enough memory allocated to the ram section to fit the compiled software example. To solve these cases, SiFive provides support for the compact code model.

5.21.1 RISC-V Code Model Summary

	Medlow	Medany	Compact
Code	Small	Small	Small
Data	Small	Small	Small
Distance	< 4GB	< 2GB (PC to GOT)	No limitation
Address	4GB (absolute)	No limitation	No limitation

Table 123: RISC-V Code Model Table

As shown in the above table, the compact code model option has no limits on the base address, or the distance between, the code and data sections.

5.21.2 Enabling the Compact Code Model

To enable the large code model, follow the steps below:

1. Enable compact code model in settings.mk file to use `RISCV_CMODEL = compact` instead of `RISCV_CMODEL = medany`.
2. Update assembly files to use new instructions if `__riscv_cmodel_compact` is defined by the toolsuite.
3. Use the latest GCC+LLVM toolsuite from SiFive, starting with 2021.06.x.

5.21.2.1 Makefile Update

To enable the toolsuite to generate the proper code sequences for the compact code model, first update settings.mk file, which can be found in the board support package (BSP) path, similar to freedom-e-sdk/bsp/design-rtl.

Change: `RISCV_CMODEL = medany` $\Rightarrow$ `RISCV_CMODEL = compact`

The `RISCV_CMODEL` definition gets passed into the toolsuite using the `-mcmodel` switch.

Note

For 32-bit designs, which do not require the use of the compact code model, you will find `RISCV_CMODEL = medlow` in settings.mk.

This `-mcmodel=compact` option will enable the symbol `__riscv_cmodel_compact` to be visible within the code, and can be used to determine the correct code sequences to use within assembly files.

5.21.2.2 Assembly File Updates

Assembly files may need to be hand-edited to support the compact code model if they reference a global symbol. The following freedom-metal source files now support the compact code model option:

1. freedom-metal/src/entry.S
2. freedom-metal/src/scrub.S
3. freedom-metal/gloss/crt0.S

5.22 Pipeline Hazards

The pipeline only interlocks on read-after-write and write-after-write hazards, so instructions may be scheduled to avoid stalls.

5.22.1 Read-After-Write Hazards

Read-after-Write (RAW) hazards occur when an instruction tries to read a register before a preceding instruction tries to write to it. This hazard describes a situation where an instruction refers to a result that has not been calculated or retrieved. This situation is possible because even though an instruction was executed after a prior instruction, the prior instruction may only have processed partly through the core pipeline.

Example:

- Instruction 1: $x1 + x3$ is saved in $x2$
- Instruction 2: $x2 + x3$ is saved in $x4$

The first instruction is calculating a value ($x1 + x3$) to be saved in $x2$. The second instruction is going to use the value of $x2$ to compute a result to be saved in $x4$. However, in the core pipeline, when operations are fetched for the second operation, the results from the first operation have not yet been saved.

5.22.2 Write-After-Write Hazards

Write-after-write (WAW) hazards occur when an instruction tries to write an operand before it is written by a preceding instruction.

Example:

- Instruction 1: $x4 + x7$ is saved in $x2$
- Instruction 2: $x1 + x3$ is saved in $x2$

Write-back of instruction 2 must be delayed until instruction 1 finishes executing.

In general, MMIO accesses stall when there is a hazard on the result caused by either RAW or WAW. So, instructions may be scheduled to avoid stalls.

5.23 Reading CSRs

There are several methods for reading the CSRs that are implemented in the Tenstorrent X280 MC. A full list of the defined RISC-V CSRs are described in Section 5.12.2.

1. Inline assembly using `csrr` instruction and the register name. For example, reading the `misa` CSR:

```
int misa;
__asm__ volatile("csrr %0, misa" : "=r" (misa));
```

2. Using the Freedom Metal API `METAL_CPU_GET_CSR`. Again, reading the `misa` CSR:


```
int misa_value;  
METAL_CPU_GET_CSR(misa,misa_value);
```

In the second method, the first argument is the register name and the second is the variable to store the result in.

Both inline assembly and Freedom Metal API methods can receive the CSR number instead of its name. For example:

```
int mscratch;  
METAL_CPU_GET_CSR(0x340, mscratch_value); // reading mscratch csr
```

Note

Accessing CSRs has to be according to the privilege level you are in. Attempting to access a CSR in a privilege level higher than the current level of operation will result in an exception.

To access a privileged CSR, the user must switch to the appropriate privilege level. This can be done using the following Freedom Metal API:

```
metal_privilege_drop_to_mode(METAL_PRIVILEGE_USER,  
                             my_regfile,  
                             user_mode_entry_point);
```

The Freedom Metal API routines and more examples located in freedom-e-sdk/software directory.

Chapter 6

Custom Instructions and CSRs

This chapter describes some of the custom instructions and CSRs configured in the Tenstorrent X280 MC.

6.1 SiFive Custom Instructions

These custom instructions use the SYSTEM instruction encoding space, which is the same as the custom CSR encoding space, but with `funct3=0`.

6.1.1 CFLUSH.D.L1

- Implemented as state machine in L1 data cache, for cores with data caches.
- Only available in M-mode. Execution from modes other than M-mode raises an illegal-instruction exception.
- Opcode `0xFC000073`, with optional `rs1` field in bits [19:15].
- When `rs1 = x0`, `CFLUSH.D.L1` writes back and invalidates all lines in the L1 data cache.
- When `rs1 ≠ x0`, an illegal-instruction exception is raised.
- If the effective privilege mode does not have write permissions to the address in `rs1`, then a store access or store page-fault exception is raised.
- If the address in `rs1` is in an uncacheable region with write permissions, the instruction has no effect but raises no exceptions.
- Note that if the PMP scheme write-protects only part of a cache line, then using a value for `rs1` in the write-protected region will cause an exception, whereas using a value for `rs1` in the write-permitted region will write back the entire cache line.

6.1.2 CDISCARD.D.L1

- Implemented as state machine in L1 data cache, for cores with data caches.
- Only available in M-mode. Execution from modes other than M-mode raises an illegal-instruction exception.
- Opcode `0xFC200073`, with optional `rs1` field in bits [19:15].

- When `rs1 = x0`, `CDISCARD.D.L1` invalidates, but does not write back, all lines in the L1 data cache. Dirty data within the cache is lost.
- When `rs1 ≠ x0`, an illegal-instruction exception is raised.
- If the effective privilege mode does not have write permissions to the address in `rs1`, then a store access or store page-fault exception is raised.
- If the address in `rs1` is in an uncacheable region with write permissions, the instruction has no effect but raises no exceptions.
- Note that if the PMP scheme write-protects only part of a cache line, then using a value for `rs1` in the write-protected region will cause an exception, whereas using a value for `rs1` in the write-permitted region will invalidate and discard the entire cache line.

6.1.3 CEASE

- Privileged instruction only available in M-mode.
- Opcode `0x30500073`.
- After retiring `CEASE`, hart will not retire another instruction until reset.
- Instigates power-down sequence, which will eventually raise the `cease_from_tile_N` signal to the outside of the Core Complex, indicating that it is safe to power down.
- `CEASE` has no effect on System Bus Access.
- Debug `haltreq` will not work after a `CEASE` instruction has retired.

6.1.4 PAUSE

- Opcode `0x0100000F`, which is a `FENCE` instruction with predecessor set `W` and null successor set. Therefore, `PAUSE` is a `HINT` instruction that executes as a no-op on all RISC-V implementations.
- This instruction may be used for more efficient idling in spin-wait loops.
- This instruction causes a stall of up to 32 cycles or until a cache eviction occurs, whichever comes first.

6.2 SiFive Custom CSRs

These custom CSRs use the custom CSR encoding space. Refer to Appendix C for an exhaustive list of the custom CSRs available on the Tenstorrent X280 MC.

6.2.1 Branch Prediction Mode CSR

This SiFive custom extension adds an M-mode CSR to control the current branch prediction mode, `mbpm` at CSR `0x7C0`.

The Tenstorrent X280 MC's branch prediction system includes a Return Address Stack (RAS), a Branch Target Buffer (BTB), and a Branch History Table (BHT). While branch predictors are essential to achieve high performance in pipelined processors, they can also cause undesirable timing variability for hard real-time systems. The `mbpm` register provides a means to customize the branch predictor behavior to trade average performance for a more predictable execution time.

Branch Prediction Mode CSR				
CSR	0x7C0			
Bits	Field Name	Attr.	Rst.	Description
0	static	RW	0x0	Branch-Direction Prediction
[7:1]	Reserved	RO	0x0	

Table 124: Branch Prediction Mode CSR

The `static` bit determines the value returned by the BHT component of the branch prediction system. A zero value indicates dynamic direction prediction, and a non-zero value indicates static-taken direction prediction. The BTB is cleared on any write to `static`, and the RAS is unaffected by writes to `static`.

6.2.2 SiFive Feature Disable CSR

The SiFive custom M-mode Feature Disable CSR is provided to enable or disable certain microarchitectural features. In the Tenstorrent X280 MC, CSR 0x7C1 has been allocated for this purpose. These features are described in Table 125.

Warning

The features that can be controlled by this CSR are subject to change or removal in future releases. It is not advised to depend on this CSR for development.

A feature is fully enabled when the associated bit is zero. If a particular core does not support the disabling of a feature, the corresponding bit is hardwired to zero.

On reset, all implemented bits are set to 1, disabling all features. The bootloader is responsible for turning on all required features and can simply write zero to turn on the maximal set of features. SiFive's Freedom Metal bootloader handles turning on these features; when using a custom bootloader, clearing the Feature Disable CSR must be implemented.

Note that arbitrary toggling of the Feature Disable CSR bits is neither recommended nor supported; they are only intended to be set from 1 to 0. A particular Feature Disable CSR bit is only to be used in a very limited number of situations, as detailed in the **Example Usage** entry in Table 126.

SiFive Feature Disable CSR				
CSR	0x7C1			
Bits	Field Name	Attr.	Rst.	Description
0	disableDCacheClockGate	RW	0x1	Disable data cache clock gating
1	disableICacheClockGate	RW	0x1	Disable instruction cache clock gating
2	disableCoreClockGate	RW	0x1	Disable core clock gating
3	disableSpeculativeICacheRefill	RW	0x1	Disable speculative instruction cache refill
[6:4]	Reserved	RO	0x0	
7	disableTileClockGate	RW	0x1	Disable tile clock gating
8	Reserved	RO	0x0	
9	suppressCorruptOnGrantData	RW	0x1	Suppress corrupt signal on GrantData messages
[15:10]	Reserved	RO	0x0	
16	disableBranchPredication	RW	0x1	Disable short forward branch optimization
17	disableICachePrefetch	RW	0x1	Disable instruction cache next-line prefetcher
[31:18]	Reserved	RO	0x0	

Table 125: SiFive Feature Disable CSR

Feature Disable CSR Usage	
Bit	Description / Usage
3	Disable speculative instruction cache refill Example Usage A particular integration might require that execution from the System Port range be disallowed. Startup code would first configure PMP to prevent execution from the System Port range, followed by clearing bit 3 of the Feature Disable CSR. This would enable speculative instruction cache refill accesses, without allowing those to access the System Port range because PMP would prohibit such accesses.
9	Suppress corrupt signal on GrantData messages Example Usage 1 When running in debug mode on configurations having both ECC and a BEU, setting bit 9 of the Feature Disable CSR will suppress debug mode errors. Example Usage 2 Startup code could scrub errors present in RAMs at power-on, followed by clearing bit 9 of the Feature Disable CSR to allow normal operation.

Table 126: SiFive Feature Disable CSR Usage

6.2.3 Power Dial CSR

The Power Dial CSR, at 0x7C8, provides a method of scaling down dynamic power in a core in order to limit maximum power without frequency changes. This CSR is further described in Section 15.2.1.

6.3 Other Custom Instructions and CSRs

Other custom instructions and CSRs may be implemented, but their functionality is not documented further here, and they should not be used in this version of the Tenstorrent X280 MC.

Chapter 7

Interrupts and Exceptions

This chapter describes how interrupt and exception concepts in the RISC-V architecture apply to the Tenstorrent X280 MC.

7.1 Interrupt Concepts

Interrupts are *asynchronous* events that cause program execution to change to a specific location in the software application to handle the interrupting event. When processing of the interrupt is complete, program execution resumes back to the original program execution location. For example, a timer that triggers every 10 milliseconds will cause the CPU to branch to the interrupt handler, acknowledge the interrupt, and set the next 10 millisecond interval.

The Tenstorrent X280 MC supports machine mode and supervisor mode interrupts. By default, all interrupts are handled in machine mode. For harts that support supervisor mode, it is possible to selectively delegate interrupts to supervisor mode.

The Core Complex also has support for the following types of RISC-V interrupts: local and global. Local interrupts are signaled directly to an individual hart with a dedicated interrupt exception code and fixed priority. This allows for reduced interrupt latency as no arbitration is required to determine which hart will service a given request and no additional memory accesses are required to determine the cause of the interrupt. Software and timer interrupts are local interrupts generated by the Core-Local Interruptor (CLINT). The Tenstorrent X280 MC contains no other local interrupt sources.

Global interrupts are routed through a Platform-Level Interrupt Controller (PLIC), which can direct interrupts to any hart in the system via the external interrupt. Decoupling global interrupts from the harts allow the design of the PLIC to be tailored to the platform, permitting a broad range of attributes like the number of interrupts and the prioritization and routing schemes.

Chapter 8 describes the CLINT. Chapter 9 describes the global interrupt architecture and the PLIC design.

7.2 Exception Concepts

Exceptions are different from interrupts in that they typically occur *synchronously* to the instruction execution flow, and most often are the result of an unexpected event that results in the pro-

gram to enter an exception handler. For example, if a hart is operating in supervisor mode and attempts to access a machine mode only Control and Status Register (CSR), it will immediately enter the exception handler and determine the next course of action. The exception code in the `mstatus` register will hold a value of 0x2, showing that an illegal instruction exception occurred. Based on the requirements of the system, the supervisor mode application may report an error and/or terminate the program entirely.

There are no specific enable bits to allow exceptions to occur since they are always enabled by default. However, early in the boot flow, software should set up `mtvec.BASE` to a defined value, which contains the base address of the default exception handler. All exceptions will trap to `mtvec.BASE`. Software must read the `mcause` CSR to determine the source of the exception, and take appropriate action.

Synchronous exceptions that occur from within an interrupt handler will immediately cause program execution to abort the interrupt handler and enter the exception handler. Exceptions within an interrupt handler are usually the result of a software bug and should generally be avoided since `mepc` and `mcause` CSRs will be overwritten from the values captured in the original interrupt context.

The RISC-V defined synchronous exceptions have a priority order which may need to be considered when multiple exceptions occur simultaneously from a single instruction. Table 127 describes the synchronous exception priority order.

Priority	Exception Code	Description
<i>Highest</i>	3	Instruction address breakpoint
	12	Instruction page fault
	1	Instruction access fault
	2	Illegal instruction
	0	Instruction address misaligned
	8, 9, 11	Environment call
	3	Environment break
	3	Load/Store/AMO address breakpoint
	6	Store/AMO address misaligned
	4	Load address misaligned
	15	Store/AMO page fault
	13	Load page fault
<i>Lowest</i>	7	Store/AMO access fault
	5	Load access fault

Table 127: Exception Priority

Refer to Table 136 for the full list of exception codes.

Data address breakpoints (watchpoints), Instruction address breakpoints, and environment break exceptions (EBREAK) all have the same exception code (3), but different priority, as shown in the table above.

Instruction address misaligned exceptions (0) have lower priority than other instruction address exceptions because they are the result of control-flow instructions with misaligned targets, rather than from instruction fetch.

Some of the helpful CSRs for debugging exceptions and interrupts are described below:

CSR	Description
exception	SiFive Scope signal. Indicates the moment that an exception occurs in the write-back (commit) stage.
mcause	Contains the cause value of the exception/interrupt. See Section 7.7.5 for more description.
mepc	Contains the pc where the exception occurs.
mtval	If the cause is a load/store fault, this register has the value of the problematic address. If it is an invalid instruction, it provides the instruction that the core tried to execute.
mstatus	Contains the interrupt enables, privilege modes, and general status of execution. See Section 7.7.1 for more description.
mtvec	Contains the vector that the core will jump to when an exception occurs. If this is not a valid executable value, you may get a double exception when jumping to the exception handler, so it is important to look at all these registers when the exception FIRST occurs. See Section 7.7.2 for more description.

Table 128: Summary of Exception and Interrupt CSRs

7.3 Trap Concepts

The term trap describes the transfer of control in a software application, where trap handling typically executes in a more privileged environment. For example, a particular hart contains three privilege modes: machine, supervisor, and user. Each privilege mode has its own software execution environment including a dedicated stack area. Additionally, each privilege mode contains separate control and status registers (CSRs) for trap handling. While operating in user mode, a context switch is required to handle an event in supervisor mode. The software sets up the system for a context switch and then an ECALL instruction is executed, which synchronously switches control to the environment-call-from-user-mode exception handler.

The default mode out of reset is machine mode. Software begins execution at the highest privilege level, which allows all CSRs and system resources to be initialized before any privilege level changes. The steps below describe the required steps necessary to change privilege mode from machine to user mode, on a particular design that also includes supervisor mode.

1. Interrupts should first be disabled globally by writing `mstatus.MIE` to 0, which is the default reset value.
2. Write `mtvec` CSR with the base address of the machine mode exception handler. This is a required step in any boot flow.
3. Write `mstatus.MPP` to 0 to set the previous mode to user, which allows us to *return* to that mode.

4. Setup the Physical Memory Protection (PMP) regions to grant the required regions to user and supervisor mode, and optionally, revoke permissions from machine mode.
5. Write `stvec` CSR with the base address of the supervisor mode exception handler.
6. Write `medeleg` register to delegate exceptions to supervisor mode. Consider ECALL and page fault exceptions.
7. Write `mstatus.FS` to enable floating-point (if supported).
8. Store machine mode user registers to stack or to an application-specific frame pointer.
9. Write `mepc` with the entry point of user mode software
10. Execute `mret` instruction to enter user mode.

Note

There is only one set of user registers (x1-x31) that are used across all privilege levels, so application software is responsible for saving and restoring state when entering and exiting different levels.

7.4 Interrupt Block Diagram

The Tenstorrent X280 MC interrupt architecture is depicted in Figure 155.

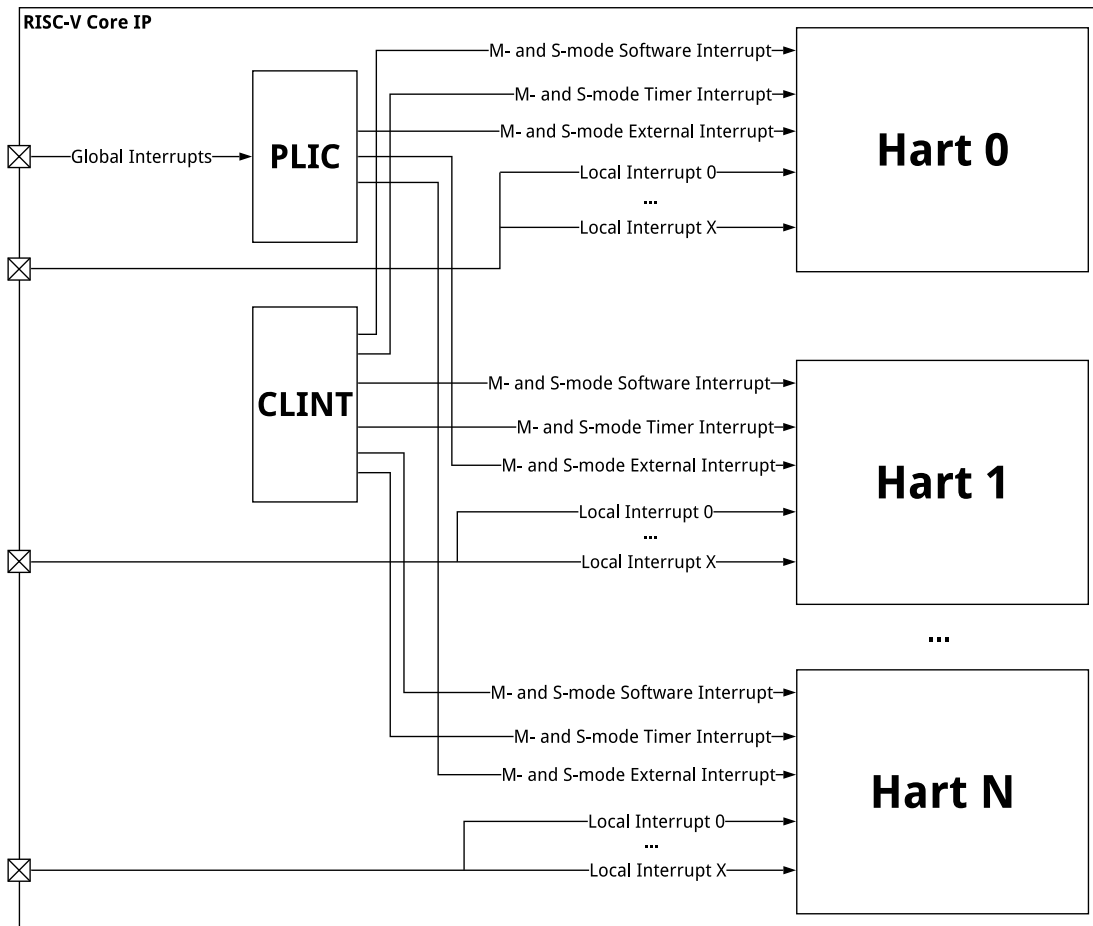


Figure 155: Tenstorrent X280 MC Interrupt Architecture Block Diagram

7.5 Local Interrupts

Software interrupts (Interrupt ID #3) are triggered by writing the memory-mapped interrupt pending register `msip` for a particular hart. Other harts are able to write `msip` to trigger a software interrupt on any other hart in the Tenstorrent X280 MC. This allows for efficient interprocessor communication. The `msip` register is described in Table 134.

Timer interrupts (Interrupt ID #7) are triggered when the memory-mapped register `mtime` is greater than or equal to the global timebase register `mtimecmp`, and both registers are part of the CLINT memory map. `mtimecmp` can be written by other harts to set up timer interrupts. The `mtime` and `mtimecmp` registers are generally only available in machine mode, unless the PMP grants user or supervisor mode access to the memory-mapped region in which they reside.

Global interrupts are usually first routed to the PLIC, then into the hart using external interrupts (Interrupt ID #11).

7.6 Interrupt Operation

Within a privilege mode M , if the associated global interrupt-enable $\{ie\}$ is clear, then no interrupts will be taken in that privilege mode, but a pending-enabled interrupt in a higher privilege mode will preempt current execution. If $\{ie\}$ is set, then pending-enabled interrupts at a higher interrupt level in the same privilege mode will preempt current execution and run the interrupt handler for the higher interrupt level.

When an interrupt or synchronous exception is taken, the privilege mode is modified to reflect the new privilege mode. The global interrupt-enable bit of the handler's privilege mode is cleared.

7.6.1 Interrupt Entry and Exit

When an interrupt occurs:

- The value of `mstatus.MIE` is copied into `mstatus.MPIE`, and then `mstatus.MIE` is cleared, effectively disabling interrupts.
- The privilege mode prior to the interrupt is encoded in `mstatus.MPP`.
- The current `pc` is copied into the `mepc` register, and then `pc` is set to the value specified by `mtvec` as defined by the `mtvec.MODE` described in Table 132.

At this point, control is handed over to software in the interrupt handler with interrupts disabled. When an `mret` instruction is executed, the following occurs:

- The privilege mode is set to the value encoded in `mstatus.MPP`.
- The global interrupt enable, `mstatus.MIE`, is set to the value of `mstatus.MPIE`.
- The `pc` is set to the value of `mepc`.

At this point, control is handed over to software.

At the software level, interrupt attributes can be applied to interrupt processing functions, as described in Section 8.4.

The Control and Status Registers (CSRs) involved in handling RISC-V interrupts are described in Section 7.7.

7.7 Interrupt Control and Status Registers

The Tenstorrent X280 MC specific implementation of interrupt CSRs is described below. For a complete description of RISC-V interrupt behavior and how to access CSRs, please consult *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*.

7.7.1 Machine Status Register (mstatus)

The mstatus register keeps track of and controls the hart's current operating state, including whether or not interrupts are enabled. A summary of the mstatus fields related to interrupts in the Tenstorrent X280 MC is provided in Table 129. Note that this is not a complete description of mstatus as it contains fields unrelated to interrupts, which are described in Section 5.12.5.1. For the full description of mstatus, please consult *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*.

Machine Status Register (mstatus)			
CSR	0x300		
Bits	Field Name	Attr.	Description
0	Reserved	WPRI	
1	SIE	RW	Supervisor Interrupt Enable
2	Reserved	WPRI	
3	MIE	RW	Machine Interrupt Enable
4	Reserved	WPRI	
5	SPIE	RW	Supervisor Previous Interrupt Enable
6	Reserved	WPRI	
7	MPIE	RW	Machine Previous Interrupt Enable
8	SPP	RW	Supervisor Previous Privilege Mode
[10:9]	Reserved	WPRI	
[12:11]	MPP[1:0]	RW	Machine Previous Privilege Mode

Table 129: Machine Status Register (partial)

Interrupts are enabled by setting the MIE bit in mstatus. Prior to writing mstatus.MIE=1, it is recommended to first enable interrupts in mie.

The privilege mode encoding for mstatus.MPP is described in Table 130.

Level	Encoding	Name
0	00	User mode (U)
1	01	Supervisor mode (S)
2	10	Reserved
3	11	Machine mode (M)

Table 130: Encoding of RISC-V Privilege Levels

7.7.2 Machine Trap Vector (mtvec)

The mtvec register has two main functions: defining the base address of the trap vector, and setting the mode by which the Tenstorrent X280 MC will process interrupts. For Direct and Vectored modes, the interrupt processing mode is defined in the MODE field of the mtvec register. The mtvec register is described in Table 131, and the mtvec.MODE field is described in Table 132.

Machine Trap Vector Register (mtvec)			
CSR	0x305		
Bits	Field Name	Attr.	Description
[1:0]	MODE	WARL	MODE Sets the interrupt processing mode. The encoding for the Tenstorrent X280 MC supported modes is described in Table 132.
[63:2]	BASE[63:2]	WARL	Interrupt Vector Base Address. Operating in Direct Mode requires 4-byte alignment. Operating in Vectored Mode requires 256-byte alignment.

Table 131: Machine Trap Vector Register

MODE Field Encoding mtvec.MODE		
Value	Mode	Description
0x0	Direct	All asynchronous interrupts and synchronous exceptions set pc to BASE.
0x1	Vectored	Exceptions set pc to BASE, interrupts set pc to BASE + 4 × mcause.EXCCODE.
≥0x2	Reserved	

Table 132: Encoding of mtvec.MODE

7.7.2.1 Mode Direct

When operating in direct mode, all interrupts and exceptions trap to the mtvec.BASE address. Inside the trap handler, software must read the mcause register to determine what triggered the trap. The mcause register is described in Table 135.

When operating in Direct Mode, BASE must be 4-byte aligned.

7.7.2.2 Mode Vectored

While operating in vectored mode, interrupts set the pc to mtvec.BASE + 4 × exception code (mcause.EXCCODE). For example, if a machine timer interrupt is taken, the pc is set to mtvec.BASE + 0x1C. Typically, the trap vector table is populated with jump instructions to transfer control to interrupt-specific trap handlers.

In vectored interrupt mode, BASE must be 256-byte aligned.

All machine external interrupts (global interrupts) are mapped to exception code 11. Thus, when interrupt vectoring is enabled, the pc is set to address mtvec.BASE + 0x2C for any global interrupt.

7.7.3 Machine Interrupt Enable (mie)

Individual interrupts are enabled by setting the appropriate bit in the mie register. The mie register is described in Table 133.

Machine Interrupt Enable Register (mie)			
CSR	0x304		
Bits	Field Name	Attr.	Description
0	Reserved	WPRI	
1	SSIE	RW	Supervisor Software Interrupt Enable
2	Reserved	WPRI	
3	MSIE	RW	Machine Software Interrupt Enable
4	Reserved	WPRI	
5	STIE	RW	Supervisor Timer Interrupt Enable
6	Reserved	WPRI	
7	MTIE	RW	Machine Timer Interrupt Enable
8	Reserved	WPRI	
9	SEIE	RW	Supervisor External Interrupt Enable
10	Reserved	WPRI	
11	MEIE	RW	Machine External Interrupt Enable
12	Reserved	WPRI	
13	LCOFIE	RW	Local Count Overflow Interrupt Enable
[63:12]	Reserved	WPRI	

Table 133: Machine Interrupt Enable Register

7.7.4 Machine Interrupt Pending (mip)

The machine interrupt pending (mip) register indicates which interrupts are currently pending. The mip register is described in Table 134.

Machine Interrupt Pending Register (mip)			
CSR	0x344		
Bits	Field Name	Attr.	Description
0	Reserved	WIRI	
1	SSIP	RW	Supervisor Software Interrupt Pending
2	Reserved	WIRI	
3	MSIP	RO	Machine Software Interrupt Pending
4	Reserved	WIRI	
5	STIP	RW	Supervisor Timer Interrupt Pending
6	Reserved	WIRI	
7	MTIP	RO	Machine Timer Interrupt Pending
8	Reserved	WIRI	
9	SEIP	RW	Supervisor External Interrupt Pending
10	Reserved	WIRI	
11	MEIP	RO	Machine External Interrupt Pending
12	Reserved	WPRI	
13	LCOFIP	RW	Local Count Overflow Interrupt Pending
[63:12]	Reserved	WIRI	

Table 134: Machine Interrupt Pending Register

7.7.5 Machine Cause (mcause)

When a trap is taken in machine mode, `mcause` is written with a code indicating the event that caused the trap. When the event that caused the trap is an interrupt, the most-significant bit of `mcause` is set to 1, and the least-significant bits indicate the interrupt number, using the same encoding as the bit positions in `mip`. For example, a Machine Timer Interrupt causes `mcause` to be set to `0x8000_0000_0000_0007`. `mcause` is also used to indicate the cause of synchronous exceptions, in which case the most-significant bit of `mcause` is set to 0.

See Table 135 for more details about the `mcause` register. Refer to Table 136 for a list of synchronous exception codes.

Machine Cause Register (mcause)			
CSR	0x342		
Bits	Field Name	Attr.	Description
[9:0]	EXCCODE	WLRL	A code identifying the last exception.
[62:10]	Reserved	WLRL	
63	Interrupt	WARL	1, if the trap was caused by an interrupt; 0 otherwise.

Table 135: Machine Cause Register

Interrupt	Exception Code	Description
1	0	Reserved
1	1	Supervisor software interrupt
1	2	Reserved
1	3	Machine software interrupt
1	4	Reserved
1	5	Supervisor timer interrupt
1	6	Reserved
1	7	Machine timer interrupt
1	8	Reserved
1	9	Supervisor external interrupt
1	10	Reserved
1	11	Machine external interrupt
1	12	Reserved
1	13	HPM Counter Overflow
1	14	Debug interrupt
1	≥15	Reserved
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10	Reserved
0	11	Environment call from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	14	Debug exception
0	15	Store/AMO page fault
0	≥16	Reserved

Table 136: mcause Exception Codes

Note that there are scenarios where a misaligned load or store will generate an access exception instead of an address-misaligned exception. The access exception is raised when the misaligned access should not be emulated in a trap handler, e.g., emulating an access in an I/O region, as such emulation could cause undesirable side-effects.

7.7.6 Minimum Interrupt Configuration

The minimum configuration needed to configure an interrupt is shown below.

- Write `mtvec` to configure the interrupt mode and the base address for the interrupt vector table.
- Enable interrupts in memory-mapped PLIC register space. The CLINT does not contain interrupt enable bits.
- Write `mie` CSR to enable the software, timer, and external interrupt enables for each privilege mode.
- Write `mstatus` to enable interrupts globally for each supported privilege mode.

7.8 Supervisor Mode Interrupts

The Tenstorrent X280 MC supports the ability to selectively direct interrupts and exceptions to supervisor mode, resulting in improved performance by eliminating the need for additional mode changes.

This capability is enabled by the interrupt and exception delegation CSRs; `mideleg` and `medeleg`, respectively. Supervisor interrupts and exceptions can be managed via supervisor versions of the interrupt CSRs, specifically: `stvec`, `sip`, `sie`, and `scause`.

Machine mode software can also directly write to the `sip` register, which effectively sends an interrupt to supervisor mode. This is especially useful for timer and software interrupts as it may be desired to handle these interrupts in both machine mode and supervisor mode.

The delegation and supervisor CSRs are described in the sections below. The definitive resource for information about RISC-V supervisor interrupts is *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*.

7.8.1 Delegation Registers (`mideleg` and `medeleg`)

By default, all traps are handled in machine mode. Machine mode software can selectively delegate interrupts and exceptions to supervisor mode by setting the corresponding bits in `mideleg` and `medeleg` CSRs. The exact mapping is provided in Table 137 and Table 138 and matches the `mcause` interrupt and exception codes defined in Table 136.

Note that local interrupts may be delegated to supervisor mode.

Machine Interrupt Delegation Register (mideleg)		
CSR	0x303	
Bits	Attr.	Description
0	WARL	Reserved
1	RW	Delegate Supervisor Software Interrupt
[4:2]	WARL	Reserved
5	RW	Delegate Supervisor Timer Interrupt
[8:6]	WARL	Reserved
9	RW	Delegate Supervisor External Interrupt
13	RW	Delegate HPM Counter Overflow Interrupt
[63:14]	WARL	Reserved

Table 137: Machine Interrupt Delegation Register

Machine Exception Delegation Register (medeleg)		
CSR	0x302	
Bits	Attr.	Description
0	RW	Delegate Instruction Access Misaligned Exception
1	RW	Delegate Instruction Access Fault Exception
2	RW	Delegate Illegal Instruction Exception
3	RW	Delegate Breakpoint Exception
4	RW	Delegate Load Access Misaligned Exception
5	RW	Delegate Load Access Fault Exception
6	RW	Delegate Store/AMO Address Misaligned Exception
7	RW	Delegate Store/AMO Access Fault Exception
8	RW	Delegate Environment Call from U-Mode
9	RW	Delegate Environment Call from S-Mode
[11:10]	WARL	Reserved
12	RW	Delegate Instruction Page Fault
13	RW	Delegate Load Page Fault
14	WARL	Reserved
15	RW	Delegate Store/AMO Page Fault Exception
[63:16]	WARL	Reserved

Table 138: Machine Exception Delegation Register

7.8.2 Supervisor Status Register (sstatus)

Similar to machine mode, supervisor mode has a register dedicated to keeping track of the hart's current state called `sstatus`. `sstatus` is effectively a restricted view of `mstatus`, described in Section 7.7.1, in that changes made to `sstatus` are reflected in `mstatus` and vice-versa, with the exception of the machine mode fields, which are not visible in `sstatus`.

A summary of the `sstatus` fields related to interrupts in the Tenstorrent X280 MC is provided in Table 139. Note that this is not a complete description of `sstatus` as it also contains fields unre-

lated to interrupts, which are described in Section 5.12.5.2. For the full description of `sstatus`, consult the *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*.

Supervisor Status Register (<code>sstatus</code>)			
CSR	0x100		
Bits	Field Name	Attr.	Description
0	Reserved	WPRI	
1	SIE	RW	Supervisor Interrupt Enable
[4:2]	Reserved	WPRI	
5	SPIE	RW	Supervisor Previous Interrupt Enable
[7:6]	Reserved	WPRI	
8	SPP	RW	Supervisor Previous Privilege Mode
[12:9]	Reserved	WPRI	

Table 139: Supervisor Status Register (partial)

Interrupts are enabled by setting the SIE bit in `sstatus` and by enabling the desired individual interrupt in the `sie` register, described in Section 7.8.3.

7.8.3 Supervisor Interrupt Enable Register (`sie`)

Supervisor interrupts are enabled by setting the appropriate bit in the `sie` register. The Tenstorrent X280 MC `sie` register is described in Table 140.

Supervisor Interrupt Enable Register (<code>sie</code>)			
CSR	0x104		
Bits	Field Name	Attr.	Description
0	Reserved	WPRI	
1	SSIE	RW	Supervisor Software Interrupt Enable
[4:2]	Reserved	WPRI	
5	STIE	RW	Supervisor Timer Interrupt Enable
[8:6]	Reserved	WPRI	
9	SEIE	RW	Supervisor External Interrupt Enable
[12:10]	Reserved	WPRI	
13	LCOFIE	RW	Local Count Overflow Interrupt Enable
[63:14]	Reserved	WPRI	

Table 140: Supervisor Interrupt Enable Register

7.8.4 Supervisor Interrupt Pending (`sip`)

The supervisor interrupt pending (`sip`) register indicates which interrupts are currently pending. The Tenstorrent X280 MC `sip` register is described in Table 141.

Supervisor Interrupt Pending Register (sip)			
CSR	0x144		
Bits	Field Name	Attr.	Description
0	Reserved	WIRI	
1	SSIP	RW	Supervisor Software Interrupt Pending
[4:2]	Reserved	WIRI	
5	STIP	RW	Supervisor Timer Interrupt Pending
[8:6]	Reserved	WIRI	
9	SEIP	RW	Supervisor External Interrupt Pending
[12:10]	Reserved	WPRI	
13	LCOFIP	RW	Local Count Overflow Interrupt Pending
[63:14]	Reserved	WRPI	

Table 141: Supervisor Interrupt Pending Register

7.8.5 Supervisor Cause Register (scause)

When a trap is taken in supervisor mode, scause is written with a code indicating the event that caused the trap. When the event that caused the trap is an interrupt, the most-significant bit of scause is set to 1, and the least-significant bits indicate the interrupt number, using the same encoding as the bit positions in sip. For example, a Supervisor Timer Interrupt causes scause to be set to 0x8000_0000_0000_0005.

scause is also used to indicate the cause of synchronous exceptions, in which case the most-significant bit of scause is set to 0. Refer to Table 143 for a list of synchronous exception codes.

Supervisor Cause Register (scause)			
CSR	0x142		
Bits	Field Name	Attr.	Description
[62:0]	EXCCODE	WLRL	A code identifying the last exception.
63	Interrupt	WARL	1 if the trap was caused by an interrupt; 0 otherwise.

Table 142: Supervisor Cause Register

Interrupt	Exception Code	Description
1	0	Reserved
1	1	Supervisor software interrupt
1	2–4	Reserved
1	5	Supervisor timer interrupt
1	6–8	Reserved
1	9	Supervisor external interrupt
1	10–12	Reserved
1	13	HPM Counter Overflow
1	≥14	Reserved
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Reserved
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9–11	Reserved
0	12	Instruction page fault
0	13	Load page fault
0	14	Reserved
0	15	Store/AMO page fault
0	≥16	Reserved

Table 143: scause Exception Codes

7.8.6 Supervisor Trap Vector (stvec)

By default, all interrupts trap to a single address defined in the `stvec` register. It is up to the interrupt handler to read `scause` and react accordingly. RISC-V and the Tenstorrent X280 MC also support the ability to optionally enable interrupt vectors. When vectoring is enabled, each interrupt defined in `sie` will trap to its own specific interrupt handler.

Vectored interrupts are enabled when the `MODE` field of the `stvec` register is set to 1.

Supervisor Trap Vector Register (stvec)			
CSR	0x105		
Bits	Field Name	Attr.	Description
[1:0]	MODE	WARL	MODE determines whether or not interrupt vectoring is enabled. The encoding for the MODE field is described in Table 145.
[63:2]	BASE[63:2]	WARL	Interrupt Vector Base Address. Must be aligned on a 128-byte boundary when MODE=1. Note, BASE[1:0] is not present in this register and is implicitly 0.

Table 144: Supervisor Trap Vector Register

MODE Field Encoding stvec.MODE		
Value	Name	Description
0x0	Direct	All exceptions and interrupts set pc to BASE
0x1	Vectored	Exceptions set pc to BASE, interrupts set pc to BASE + 4 × scause.EXCCODE
≥0x2	Reserved	

Table 145: Encoding of stvec.MODE

If vectored interrupts are disabled (stvec.MODE=0), all interrupts trap to the stvec.BASE address. If vectored interrupts are enabled (stvec.MODE=1), interrupts set the pc to stvec.BASE + 4 × exception code (scause.EXCCODE). For example, if a supervisor timer interrupt is taken, the pc is set to stvec.BASE + 0x14. Typically, the trap vector table is populated with jump instructions to transfer control to interrupt-specific trap handlers.

In vectored interrupt mode, BASE must be 128-byte aligned.

All supervisor external interrupts (global interrupts) are mapped to exception code of 9. Thus, when interrupt vectoring is enabled, the pc is set to address stvec.BASE + 0x24 for any global interrupt.

See Table 144 for a description of the stvec register. See Table 145 for a description of the stvec.MODE field. See Table 143 for the Tenstorrent X280 MC supervisor mode interrupt exception code values.

7.8.7 Delegated Interrupt Handling

Upon taking a delegated trap, the following occurs:

- The value of sstatus.SIE is copied into sstatus.SPIE, then sstatus.SIE is cleared, effectively disabling interrupts.

- The current pc is copied into the sepc register, and then pc is set to the value of stvec. In the case where vectored interrupts are enabled, pc is set to stvec.BASE + 4 × exception code (scause.EXCCODE).
- The privilege mode prior to the interrupt is encoded in sstatus.SPP.

At this point, control is handed over to software in the interrupt handler with interrupts disabled. Interrupts can be re-enabled by explicitly setting sstatus.SIE or by executing an SRET instruction to exit the handler. When an SRET instruction is executed, the following occurs:

- The privilege mode is set to the value encoded in sstatus.SPP.
- The value of sstatus.SPIE is copied into sstatus.SIE.
- The pc is set to the value of sepc.

At this point, control is handed over to software.

7.9 Interrupt Priorities

Individual priorities of global interrupts are determined by the PLIC, as discussed in Chapter 9.

Tenstorrent X280 MC interrupts are prioritized as follows, in decreasing order of priority:

- Machine external interrupts
- Machine software interrupts
- Machine timer interrupts
- Supervisor external interrupts
- Supervisor software interrupts
- Supervisor timer interrupts

7.10 Interrupt Latency

Interrupt latency for the Tenstorrent X280 MC is four `external_source_for_core_N_clock` cycles, as counted by the number of cycles it takes from signaling of the interrupt to the hart to the first instruction fetch of the handler.

Global interrupts routed through the PLIC incur additional latency of three clock cycles, where the PLIC is clocked by `clock`. This means that the total latency, in cycles, for a global interrupt is: $4 + 3 \times (\text{external_source_for_core_N_clock Hz} \div \text{clock Hz})$. This is a best-case cycle count and assumes the handler is cached. It does not take into account additional latency from a peripheral source.

7.11 Non-Maskable Interrupt

The `rnmi` (resumable non-maskable interrupt) interrupt signal is a level-sensitive input to the hart. Non-maskable interrupts have higher priority than any other interrupt or exception on the hart and cannot be disabled by software. Specifically, they are not disabled by clearing the `mstatus.mie` register.

7.11.1 Handler Addresses

The NMI has an associated exception trap handler address. This address is set by external input signals, described in the Tenstorrent X280 MC User Guide.

7.11.2 RNMI CSRs

These M-mode CSRs enable a resumable non-maskable interrupt (RNMI).

Number	Name	Description
0x350	<code>mnscratch</code>	Resumable Non-maskable scratch register
0x351	<code>mnepc</code>	Resumable Non-maskable EPC value
0x352	<code>mncause</code>	Resumable Non-maskable cause value
0x353	<code>mnstatus</code>	Resumable Non-maskable status

Table 146: RNMI CSRs

- The `mnscratch` CSR holds a 64-bit read-write register, which enables the NMI trap handler to save and restore the context that was interrupted.
- The `mnepc` CSR is a 64-bit read-write register, which, on entry to the NMI trap handler, holds the PC of the instruction that took the interrupt. The lowest bit of `mnepc` is hardwired to zero.
- The `mncause` CSR holds the reason for the NMI, with bit 63 set to 1, and the NMI cause encoded in the least-significant bits, or zero if NMI causes are not supported. The lower bits of `mncause`, defined as the `exception_code`, are as follows:

<code>mncause</code>	NMI Cause	Function
1	Reserved	Reserved
2	RNMI input pin	External <code>rnmi_N</code> input
3	Bus error	RNMI caused by BEU

Table 147: `mncause.exception_code` Fields

- The `mnstatus` CSR holds a two-bit field `mnpp` encoded in the same manner as `mstatus.mpp`, which, on entry to the trap handler, holds the privilege mode of the interrupted context.

- `mnstatus` also hold the one-bit field `mnie` indicating whether NMIs are currently enabled. This bit can only be cleared by hardware, but can be set by software to indicate a further NMI can be taken.

Bits	Field Name	Description
3	<code>mnie</code>	RMNI interrupt enable
7	<code>mnpv</code>	Hardwired to zero
12:11	<code>mnpp</code>	RMNI previous priority level

Table 148: `mnstatus` CSR Fields

7.11.3 MNRET Instruction

This M-mode only instruction uses the values in `mnepc` and `mnstatus` to return to the program counter and privileged mode of the interrupted context. This instruction also sets the internal `rnmie` state bits.

Encoding is same as MRET except with bit 30 set (i.e., `funct7=0111000`). For example:

```
.word 0x70200073      // opcode for MNRET (return from RNMI)
```

7.11.4 RNMI Operation

When an RNMI interrupt is detected, the interrupted PC is written to the `mnepc` CSR, the type of RNMI to the `mncause` CSR, and the privilege mode of the interrupted context to the `mnstatus` CSR. An internal microarchitectural state bit, `rnmie`, is cleared to indicate that the processor is in an RNMI handler and cannot take a new RNMI interrupt. When clear, the internal `rnmie` bit also disables all other interrupts.

Note

These interrupts are called non-maskable because software cannot mask the interrupts. However, for correct operation, other instances of the same interrupt must be held off until the handler is completed, hence the internal state bit.

The RNMI handler can resume original execution using the MNRET instruction (described in Section 7.11.3), which restores the PC from `mnepc`, the privilege mode from `mnstatus`, and also sets the internal `rnmie` state bit, which re-enables other interrupts.

If the hart encounters an exception while the `rn mie` bit is clear, the exception state is written to `mepc` and `mcause`, `mstatus.mpp` is set to M-mode, and the hart jumps to the RNMI exception handler address.

Note

Traps in the RNMI handler can only be resumed if they occur while the handler was servicing an interrupt that occurred outside of machine mode.

Chapter 8

Core-Local Interruptor (CLINT)

This chapter describes the operation of the Core-Local Interruptor (CLINT). The Tenstorrent X280 MC CLINT complies with *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11*.

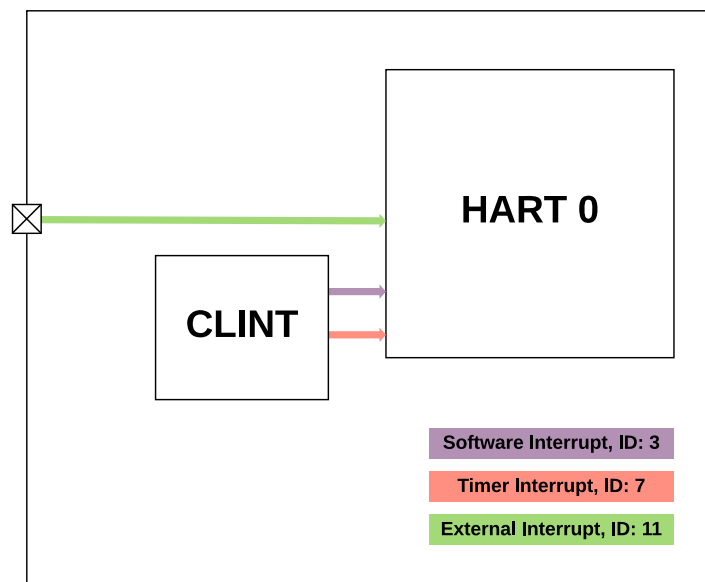


Figure 156: CLINT Block Diagram

The CLINT has a small footprint and provides software, timer, and external interrupts directly to the hart. The CLINT block also holds memory-mapped control and status registers associated with software and timer interrupts.

8.1 CLINT Priorities and Preemption

The CLINT has a fixed priority scheme, described in Section 7.9, and nested interrupts (pre-emption) within a given privilege level is not supported. Higher privilege levels may preempt lower privilege levels, however. The CLINT offers two modes of operation, Direct mode and Vectored mode.

In Direct mode, all interrupts and exceptions trap to `mtvec.BASE`. In Vectored mode, exceptions trap to `mtvec.BASE`, but interrupts will jump directly to their vector table index. See Section 7.7.2 for more information about `mtvec.BASE`.

8.2 CLINT Vector Table

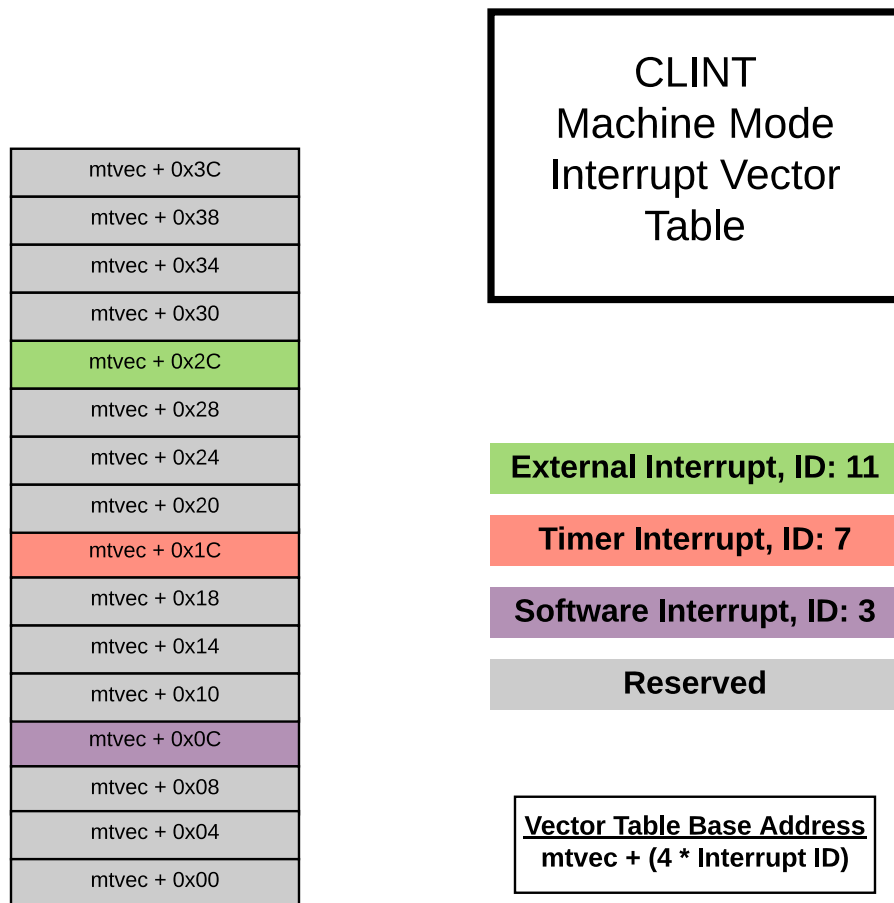


Figure 157: CLINT Interrupts and Vector Table

The CLINT vector table is populated with jump instructions, since hardware jumps to the index in the vector table first, then subsequently jumps to the handler. All exception types trap to the first entry in the table, which is `mtvec.BASE`.

An example CLINT vector table is shown below.

```

.weak default_exception_handler
.balign 4, 0
.global default_exception_handler

.weak software_handler
.balign 4, 0
.global software_handler

.weak timer_handler
.balign 4, 0
.global timer_handler

.weak external_handler
.balign 4, 0
.global external_handler

.option norvc
.weak __mtvec_clint_vector_table
#if __riscv_xlen == 32
.balign 128, 0
#else
.balign 256, 0
#endif
.global __mtvec_clint_vector_table
__mtvec_clint_vector_table:

IRQ_0:
    j default_exception_handler
IRQ_1:
    j default_vector_handler
IRQ_2:
    j default_vector_handler
IRQ_3:
    j software_handler
IRQ_4:
    j default_vector_handler
IRQ_5:
    j default_vector_handler
IRQ_6:
    j default_vector_handler
IRQ_7:
    j timer_handler
IRQ_8:
    j default_vector_handler
IRQ_9:
    j default_vector_handler
IRQ_10:
    j default_vector_handler
IRQ_11:
    j external_handler
IRQ_12:
    j default_vector_handler
IRQ_13:
    j default_vector_handler
IRQ_14:
    j default_vector_handler
IRQ_15:
    j default_vector_handler

```

Figure 158: CLINT Vector Table Example

8.3 CLINT Interrupt Sources

The Tenstorrent X280 MC supports the standard RISC-V software, timer, and external interrupts.

CLINT Interrupt IDs are provided in Table 149.

Tenstorrent X280 MC Interrupt IDs		
ID	Interrupt	Notes
0	Reserved	
1	ssip	Supervisor Software Interrupt
2	Reserved	
3	msip	Machine Software Interrupt
4	Reserved	
5	stip	Supervisor Timer Interrupt
6	Reserved	
7	mtip	Machine Timer Interrupt
8	Reserved	
9	seip	Supervisor External Interrupt
10	Reserved	
11	meip	Machine External Interrupt

Table 149: Tenstorrent X280 MC Interrupt IDs

8.4 CLINT Interrupt Attribute

To help with efficiency of save and restore context, interrupt attributes can be applied to functions used for interrupt handling.

```
void __attribute__((interrupt))
software_handler (void) {
    // handler code
}
```

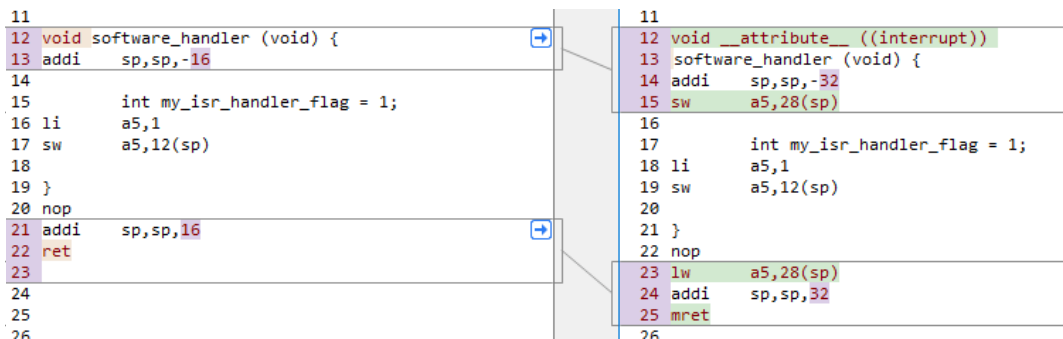


Figure 159: CLINT Interrupt Attribute Example

This attribute will save and restore registers that are used within the handler, and insert an `mret` instruction at the end of the handler.

8.5 CLINT Memory Map

Table 150 shows the memory map for CLINT on the Tenstorrent X280 MC. Note that there are no enable bits for specific interrupts within the CLINT memory map, as the enables for these interrupts reside in the `mie` CSR for each interrupt, and the `mstatus.mie` CSR bit, which enables all machine interrupts globally. See Section 7.7.3 for a description of the interrupt enable bits in the `mie` CSR, and Section 7.7.4 for a description of the interrupt pending bits in the `mip` CSR.

Address	Width	Attr.	Description	Notes
0x0200_0000	4B	RW	msip for hart 0	MSIP Registers (1-bit wide)
0x0200_0004	4B	RW	msip for hart 1	
0x0200_0008	4B	RW	msip for hart 2	
0x0200_000C	4B	RW	msip for hart 3	
0x0200_0010			Reserved	
...				
0x0200_3FFF				
0x0200_4000	8B	RW	mtimecmp for hart 0	MTIMECMP Registers
0x0200_4008	8B	RW	mtimecmp for hart 1	
0x0200_4010	8B	RW	mtimecmp for hart 2	
0x0200_4018	8B	RW	mtimecmp for hart 3	
0x0200_4020			Reserved	
...				
0x0200_BFF7				
0x0200_BFF8	8B	RW	mtime	Timer Register
0x0200_C000			Reserved	

Table 150: CLINT Memory Map

8.6 Register Descriptions

This section describes the functionality of the memory-mapped registers in the CLINT.

8.6.1 MSIP Register

Machine mode software interrupts are generated by writing to the memory-mapped control register `msip`. Each `msip` register is a 32-bit wide **WARL** register, where the upper 31 bits are tied to 0. The least-significant bit is reflected in the MSIP bit of the `mip` CSR. Other bits in the `msip` register are hardwired to zero. On reset, each `msip` register is cleared to zero.

Software interrupts are most useful for interprocessor communication in multi-hart systems, as harts may write each other's `msip` bits to effect interprocessor interrupts.

8.6.2 Timer Registers

`mtime` is a 64-bit read-write register that contains the number of cycles counted from the `rtc_toggle` signal, which is described in the Tenstorrent X280 MC User Guide. A timer interrupt is pending whenever `mtime` is greater than or equal to the value in the `mtimecmp` register. The timer interrupt is reflected in the `mtip` bit of the `mip` register, described in Chapter 7.

On reset, `mtime` is cleared to zero. The `mtimecmp` registers are not reset.

Note that `mtime` is volatile and may be masked in IP-XACT.

8.7 Supervisor Mode Delegation

By default, all interrupts trap to machine mode, including timer and software interrupts. In order for supervisor timer and software interrupts to trap directly to supervisor mode, supervisor timer and software interrupts must first be delegated to supervisor mode.

Please see Section 7.8 for more details on supervisor mode interrupts.

Chapter 9

Platform-Level Interrupt Controller (PLIC)

This chapter describes the operation of the Platform-Level Interrupt Controller (PLIC) on the Tenstorrent X280 MC. The PLIC complies with *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11* and can support a maximum of 135 external interrupt sources with 7 priority levels.

The Tenstorrent X280 MC PLIC resides in the `clock` timing domain, allowing for relaxed timing requirements. The latency of global interrupts, as perceived by a hart, increases with the ratio of the `external_source_for_core_N_clock` frequency and the `clock` frequency.

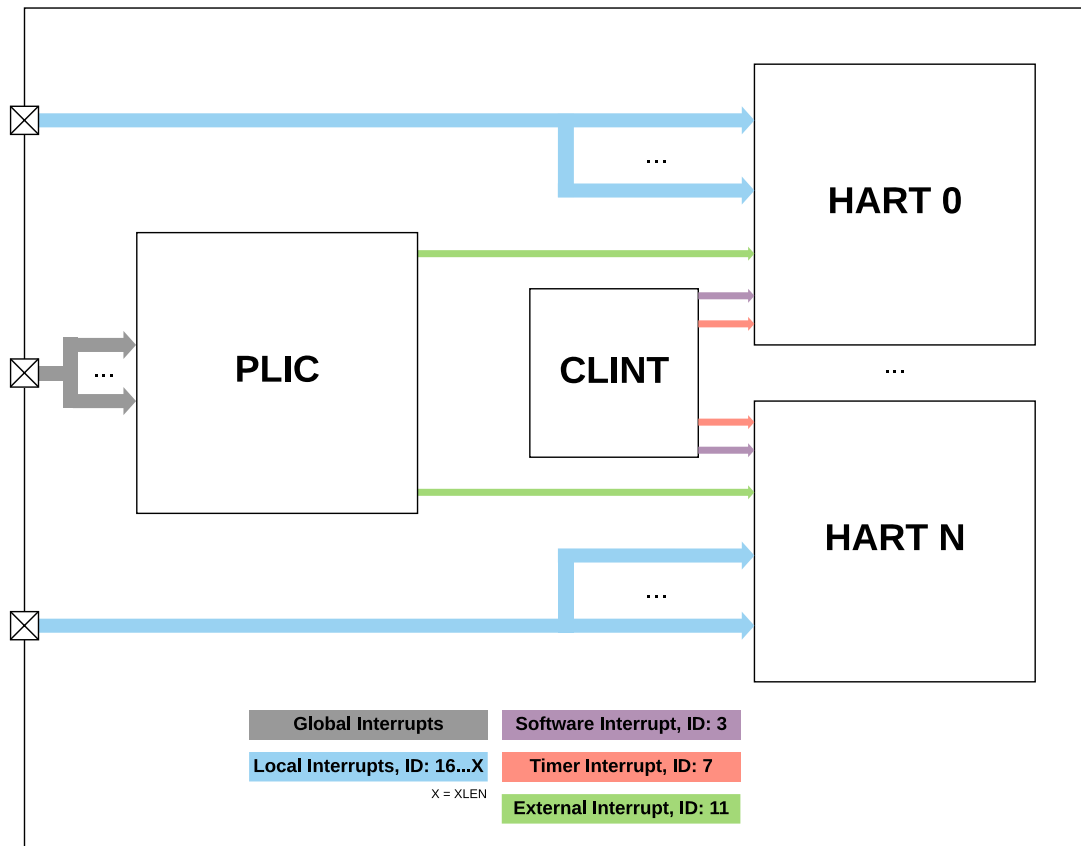


Figure 160: PLIC Multi-Core Block Diagram

9.1 Memory Map

The memory map for the Tenstorrent X280 MC PLIC control registers is shown in Table 151. The PLIC memory map only supports aligned 32-bit memory accesses.

Address	Width	Attr.	Description	Notes
0x0C00_0000			Reserved	
0x0C00_0004	4B	RW	Source 1 priority	See Section 9.3 for more information
...				
0x0C00_021C	4B	RW	Source 135 priority	
0x0C00_0220			Reserved	
...				
0x0C00_1000	4B	RO	Start of pending array	See Section 9.4 for more information
...				
0x0C00_1010	4B	RO	Last word of pending array	
0x0C00_1014			Reserved	
...				
0x0C00_2000	4B	RW	Start Hart 0 M-Mode interrupt enables	See Section 9.5 for more information
...				
0x0C00_2010	4B	RW	End Hart 0 M-Mode interrupt enables	
0x0C00_2014			Reserved	
...				
0x0C00_2080	4B	RW	Start Hart 0 S-Mode interrupt enables	See Section 9.5 for more information
...				
0x0C00_2090	4B	RW	End Hart 0 S-Mode interrupt enables	
0x0C00_2094			Reserved	
...				
0x0C00_2100	4B	RW	Start Hart 1 M-Mode interrupt enables	See Section 9.5 for more information
...				
0x0C00_2110	4B	RW	End Hart 1 M-Mode interrupt enables	
0x0C00_2114			Reserved	
...				
0x0C00_2180	4B	RW	Start Hart 1 S-Mode interrupt enables	See Section 9.5 for more information
...				
0x0C00_2190	4B	RW	End Hart 1 S-Mode interrupt enables	
0x0C00_2194			Reserved	
...				
0x0C00_2200	4B	RW	Start Hart 2 M-Mode interrupt enables	See Section 9.5 for more information
...				

Table 151: PLIC Memory Map

Address	Width	Attr.	Description	Notes
0x0C00_2210	4B	RW	End Hart 2 M-Mode interrupt enables	
0x0C00_2214			Reserved	
...				
0x0C00_2280	4B	RW	Start Hart 2 S-Mode interrupt enables	See Section 9.5 for more information
...				
0x0C00_2290	4B	RW	End Hart 2 S-Mode interrupt enables	
0x0C00_2294			Reserved	
...				
0x0C00_2300	4B	RW	Start Hart 3 M-Mode interrupt enables	See Section 9.5 for more information
...				
0x0C00_2310	4B	RW	End Hart 3 M-Mode interrupt enables	
0x0C00_2314			Reserved	
...				
0x0C00_2380	4B	RW	Start Hart 3 S-Mode interrupt enables	See Section 9.5 for more information
...				
0x0C00_2390	4B	RW	End Hart 3 S-Mode interrupt enables	
0x0C00_2394			Reserved	
...				
0x0C1F_F000	1B	RW	PLIC global clock gating disable feature	See Section 9.6 for more information
0x0C1F_F001			Reserved	
...				
0x0C20_0000	4B	RW	Hart 0 M-Mode priority threshold	See Section 9.7 for more information
0x0C20_0004	4B	RW	Hart 0 M-Mode claim/complete	See Section 9.8 for more information
0x0C20_0008			Reserved	
...				
0x0C20_1000	4B	RW	Hart 0 S-Mode priority threshold	See Section 9.7 for more information
0x0C20_1004	4B	RW	Hart 0 S-Mode claim/complete	See Section 9.8 for more information
0x0C20_1008			Reserved	
...				
0x0C20_2000	4B	RW	Hart 1 M-Mode priority threshold	See Section 9.7 for more information

Table 151: PLIC Memory Map

Address	Width	Attr.	Description	Notes
0x0C20_2004	4B	RW	Hart 1 M-Mode claim/ complete	See Section 9.8 for more information
0x0C20_2008			Reserved	
...				
0x0C20_3000	4B	RW	Hart 1 S-Mode priority threshold	See Section 9.7 for more information
0x0C20_3004	4B	RW	Hart 1 S-Mode claim/complete	See Section 9.8 for more information
0x0C20_3008			Reserved	
...				
0x0C20_4000	4B	RW	Hart 2 M-Mode priority threshold	See Section 9.7 for more information
0x0C20_4004	4B	RW	Hart 2 M-Mode claim/ complete	See Section 9.8 for more information
0x0C20_4008			Reserved	
...				
0x0C20_5000	4B	RW	Hart 2 S-Mode priority threshold	See Section 9.7 for more information
0x0C20_5004	4B	RW	Hart 2 S-Mode claim/complete	See Section 9.8 for more information
0x0C20_5008			Reserved	
...				
0x0C20_6000	4B	RW	Hart 3 M-Mode priority threshold	See Section 9.7 for more information
0x0C20_6004	4B	RW	Hart 3 M-Mode claim/ complete	See Section 9.8 for more information
0x0C20_6008			Reserved	
...				
0x0C20_7000	4B	RW	Hart 3 S-Mode priority threshold	See Section 9.7 for more information
0x0C20_7004	4B	RW	Hart 3 S-Mode claim/complete	See Section 9.8 for more information
0x0C20_7008			Reserved	
...				
0x1000_0000			End of PLIC Memory Map	

Table 151: PLIC Memory Map

9.2 Interrupt Sources

The Tenstorrent X280 MC has a total of 135 global interrupt sources, in addition to the local interrupts described in Table 149. 127 of these are external global interrupts, and the remainder are driven by various on-chip devices as listed in Table 152.

Note

In the *RISC-V Platform-Level Interrupt Controller Specification*, interrupt source 0 (ID 0) is unused, so the first usable PLIC Interrupt ID has a value of 1.

Table 152 describes the global interrupt sources on the Tenstorrent X280 MC.

PLIC Interrupt ID	Source
0	<i>Unused</i>
1	L3 Cache DirError
2	L3 Cache DirFail
3	L3 Cache DataError
4	L3 Cache DataFail
5–131	External Global Interrupts
132	Hart 0 Bus-Error Unit
133	Hart 1 Bus-Error Unit
134	Hart 2 Bus-Error Unit
135	Hart 3 Bus-Error Unit

Table 152: PLIC Interrupt Source Mapping

Table 153 describes the mapping of external global interrupts to its corresponding top-level `global_interrupts` signal bit. This signal is positive-level triggered and not configurable. See the Tenstorrent X280 MC User Guide for further description of `global_interrupts`.

<code>global_interrupts</code> Signal	PLIC Interrupt ID	PLIC Pending / Enable Register
<code>global_interrupts[0]</code>	5	<code>pending1[5]</code> / <code>enable1[5]</code>
<code>global_interrupts[1]</code>	6	<code>pending1[6]</code> / <code>enable1[6]</code>
<code>global_interrupts[2]</code>	7	<code>pending1[7]</code> / <code>enable1[7]</code>
...		
<code>global_interrupts[126]</code>	131	<code>pending5[3]</code> / <code>enable5[3]</code>

Table 153: Mapping of `global_interrupts` Signal Bits to PLIC Interrupt ID

9.3 Interrupt Priorities

Each PLIC interrupt source can be assigned a priority by writing to its 32-bit memory-mapped priority register. The Tenstorrent X280 MC supports 7 levels of priority. A priority value of 0 is

reserved to mean "never interrupt" and effectively disables the interrupt. Priority 1 is the lowest active priority, and priority 7 is the highest. Ties between global interrupts of the same priority are broken by the Interrupt ID; interrupts with the lowest ID have the highest effective priority. See Table 154 for the detailed register description.

PLIC Interrupt Priority Register				
Base Address		0x0C00_0000 + 4 × Interrupt ID		
Bits	Field Name	Attr.	Rst.	Description
[2:0]	Priority	RW	X	Global interrupt priority
[31:3]	Reserved	RO	0x0	

Table 154: PLIC Interrupt Priority Register

9.4 Interrupt Pending Bits

The current status of the interrupt source pending bits in the PLIC core can be read from the pending array, organized as 5 words of 32 bits. The pending bit for interrupt ID N is stored in bit $(N \bmod 32)$ of word $(N / 32)$. As such, the Tenstorrent X280 MC has five interrupt pending registers. Bit 0 of word 0, which represents the non-existent interrupt source 0, is hardwired to zero.

A pending bit in the PLIC core can be cleared by setting the associated enable bit then performing a claim as described in Section 9.8.

PLIC Interrupt Pending Register 1				
Base Address		0x0C00_1000		
Bits	Field Name	Attr.	Rst.	Description
0	Interrupt 0 Pending	RO	X	Non-existent global interrupt 0 is hardwired to zero
1	Interrupt 1 Pending	RO	X	Pending bit for global interrupt 1
2	Interrupt 2 Pending	RO	X	Pending bit for global interrupt 2
...				
31	Interrupt 31 Pending	RO	X	Pending bit for global interrupt 31

Table 155: PLIC Interrupt Pending Register 1

PLIC Interrupt Pending Register 5				
Base Address		0x0C00_1010		
Bits	Field Name	Attr.	Rst.	Description
0	Interrupt 128 Pending	RO	X	Pending bit for global interrupt 128
...				
7	Interrupt 135 Pending	RO	X	Pending bit for global interrupt 135
[31:8]	Reserved	WIRI	X	

Table 156: PLIC Interrupt Pending Register 5

9.5 Interrupt Enables

Each global interrupt can be enabled by setting the corresponding bit in the enable registers. The enable registers are accessed as a contiguous array of 5×32 -bit words, packed the same way as the pending bits. Bit 0 of enable word 0 represents the non-existent interrupt ID 0 and is hardwired to 0.

64-bit and 32-bit word accesses are supported by the enables array in SiFive RV64 systems.

PLIC Interrupt Enable Register 1 for Hart 0 M-Mode				
Base Address		0x0C00_2000		
Bits	Field Name	Attr.	Rst.	Description
0	Interrupt 0 Enable	RO	0x0	Non-existent global interrupt 0 is hardwired to zero
1	Interrupt 1 Enable	RW	X	Enable bit for global interrupt 1
2	Interrupt 2 Enable	RW	X	Enable bit for global interrupt 2
...				
31	Interrupt 31 Enable	RW	X	Enable bit for global interrupt 31

Table 157: PLIC Interrupt Enable Register 1 for Hart 0 M-Mode

PLIC Interrupt Enable Register 5 for Hart 0 M-Mode				
Base Address		0x0C00_2010		
Bits	Field Name	Attr.	Rst.	Description
0	Interrupt 128 Enable	RW	X	Enable bit for global interrupt 128
...				
7	Interrupt 135 Enable	RW	X	Enable bit for global interrupt 135
[31:8]	Reserved	RO	0x0	

Table 158: PLIC Interrupt Enable Register 5 for Hart 0 M-Mode

PLIC Interrupt Enable Register 1 for Hart 0 S-Mode				
Base Address		0x0C00_2080		
Bits	Field Name	Attr.	Rst.	Description
0	Interrupt 0 Enable	RO	0x0	Non-existent global interrupt 0 is hardwired to zero
1	Interrupt 1 Enable	RW	X	Enable bit for global interrupt 1
2	Interrupt 2 Enable	RW	X	Enable bit for global interrupt 2
...				
31	Interrupt 31 Enable	RW	X	Enable bit for global interrupt 31

Table 159: PLIC Interrupt Enable Register 1 for Hart 0 S-Mode

PLIC Interrupt Enable Register 5 for Hart 0 S-Mode				
Base Address		0x0C00_2090		
Bits	Field Name	Attr.	Rst.	Description
0	Interrupt 128 Enable	RW	X	Enable bit for global interrupt 128
...				
7	Interrupt 135 Enable	RW	X	Enable bit for global interrupt 135
[31:8]	Reserved	RO	0x0	

Table 160: PLIC Interrupt Enable Register 5 for Hart 0 S-Mode

9.6 PLIC Clock Gate Disable

The PLIC implements a clock gating feature to gate the module clock node when not active. PLIC clock gating is disabled out of reset and should be enabled in startup code, unless otherwise specified by SiFive erratum. Once enabled, clock is only available when there is activity on the PLIC control bus or on any interrupt line when the corresponding interrupt is not in flight. Clock gating is further described in the Tenstorrent X280 MC User Guide.

PLIC Clock Gate Disable Register				
Base Address		0x0C1F_F000		
Bits	Field Name	Attr.	Rst.	Description
0	disablePlicClockGateFeature	RW	0x1	Used to enable/disable PLIC clock gating feature. Clear to enable.
[7:1]	Reserved	RO	0x0	

Table 161: PLIC Clock Gate Disable Register

9.7 Priority Thresholds

The Tenstorrent X280 MC supports setting of an interrupt priority threshold via the `threshold` register. The `threshold` is a **WARL** field, where the Tenstorrent X280 MC supports a maximum threshold of 7.

The Tenstorrent X280 MC masks all PLIC interrupts of a priority less than or equal to `threshold`. For example, a `threshold` value of zero permits all interrupts with non-zero priority, whereas a value of 7 masks all interrupts. If the `threshold` register contains a value of 5, all PLIC interrupt configured with priorities from 1 through 5 will not be allowed to propagate to the CPU.

PLIC Interrupt Priority Threshold Register for Hart 0 M-Mode				
Base Address		0x0C20_0000		
Bits	Field Name	Attr.	Rst.	Description
[2:0]	Threshold	WARL	X	Sets the priority threshold
[31:3]	Reserved	RO	0x0	

Table 162: PLIC Interrupt Priority Threshold Register for Hart 0 M-Mode

PLIC Interrupt Priority Threshold Register for Hart 0 S-Mode				
Base Address		0x0C20_1000		
Bits	Field Name	Attr.	Rst.	Description
[2:0]	Threshold	WARL	X	Sets the priority threshold
[31:3]	Reserved	RO	0x0	

Table 163: PLIC Interrupt Priority Threshold Register for Hart 0 S-Mode

9.8 Interrupt Claim Process

A Tenstorrent X280 MC hart can perform an interrupt claim by reading the `claim_complete` register (Table 164), which returns the ID of the highest-priority pending interrupt or zero if there is no pending interrupt. A successful claim also atomically clears the corresponding pending bit on the interrupt source.

A Tenstorrent X280 MC hart can perform a claim at any time, even if the `MEIP` bit in its `mip` (Table 134) register is not set.

The claim operation is not affected by the setting of the priority threshold register.

9.9 Interrupt Completion

A Tenstorrent X280 MC hart signals it has completed executing an interrupt handler by writing the interrupt ID it received from the claim to the `claim_complete` register (Table 164). The PLIC does not check whether the completion ID is the same as the last claim ID for that target. If the

completion ID does not match an interrupt source that is currently enabled for the target, the completion is silently ignored.

PLIC Claim/Complete Register for Hart 0 M-Mode				
Base Address		0x0C20_0004		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	Interrupt Claim/Complete for Hart 0 M-Mode	RW	X	A read of zero indicates that no interrupts are pending. A non-zero read contains the ID of the highest pending interrupt. A write to this register signals completion of the interrupt ID written. Consecutive reads will not return the same result. Note that this field is volatile and may be masked in IP-XACT.

Table 164: PLIC Claim/Complete Register for Hart 0 M-Mode

PLIC Claim/Complete Register for Hart 0 S-Mode				
Base Address		0x0C20_1004		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	Interrupt Claim/Complete for Hart 0 S-Mode	RW	X	A read of zero indicates that no interrupts are pending. A non-zero read contains the ID of the highest pending interrupt. A write to this register signals completion of the interrupt ID written. Consecutive reads will not return the same result. Note that this field is volatile and may be masked in IP-XACT.

Table 165: PLIC Claim/Complete Register for Hart 0 S-Mode

The PLIC cannot forward a new interrupt to a hart that has claimed an interrupt, but has not yet finished the complete step of the interrupt handler. Thus, the PLIC does not support preemption of global interrupts to an individual hart.

Interrupt IDs for global interrupts routed through the PLIC are independent of the interrupt IDs for local interrupts. The PLIC handler may check for additional pending global interrupts once the initial claim/complete process has finished, prior to exiting the handler. This method could save additional PLIC save/restore context for global interrupts.

9.10 Example PLIC Interrupt Handler

Since the PLIC interfaces with the CPU through external interrupt #11, the external handler must contain an additional claim/complete step that is used to handshake with the PLIC logic.

```
void external_handler() {  
    //get the highest priority pending PLIC interrupt  
    uint32_t int_num = plic.claim_complete;  
  
    //branch to handler  
    plic_handler[int_num]();  
  
    //complete interrupt by writing interrupt number back to PLIC  
    plic.claim_complete = int_num;  
  
    // Add additional checks for PLIC pending here, if desired  
}
```

If a CPU reads `claim_complete` and it returns 0, the interrupt does not require processing, and thus write-back of the claim/complete is not necessary.

The `plic_handler[]()` routine shown above demonstrates one method to implement a software table where the offset of the function that resides within the table is determined by the PLIC interrupt ID. The PLIC interrupt ID is unique to the PLIC, in that it is completely independent of the interrupt IDs of local interrupts.

Chapter 10

TileLink Error Device

The Error Device is a TileLink slave that responds to all requests with a TileLink denied error and all reads with a corrupt error. It has no registers. The entire memory range discards writes and returns zeros on read. Both operation acknowledgements carry an error indication.

The Error Device serves a dual role. Internally, it is used as a landing pad for illegal off-chip requests. However, it is also useful for testing software handling of bus errors.

Chapter 11

Bus-Error Unit

This chapter describes the operation of the SiFive Bus-Error Unit (BEU).

11.1 Bus-Error Unit Overview

The Bus-Error Unit is a per-processor device that records erroneous events and reports them using platform-level and hart-local interrupts. Figure 161 shows the connections from the core to the BEU.

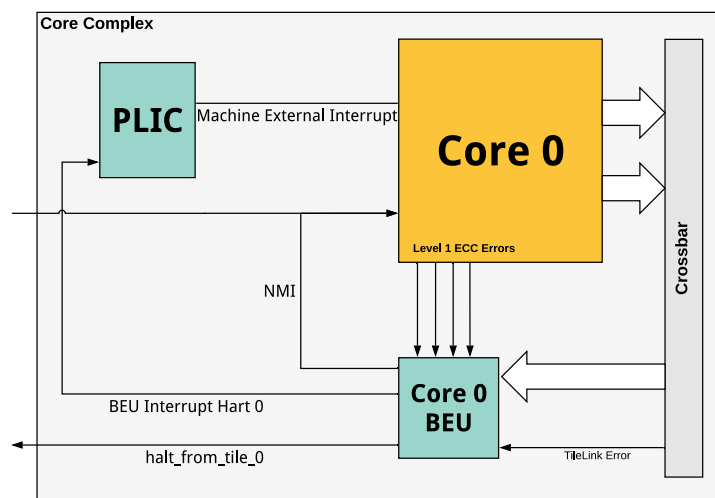


Figure 161: Bus-Error Unit Block Diagram

The BEU can be configured to generate interrupts on correctable memory errors, uncorrectable memory errors, and/or TileLink bus errors. When an error occurs, the BEU will hold the address of the error and a code to describe the error.

11.2 Memory Map

The Bus-Error Unit memory map is shown in Table 166.

Offset	Name	Attr.	Description
0x00	cause	RW	Cause of error event
0x08	value	RW	Physical address of error event
0x10	enable	RW	Event enable mask
0x18	plic_interrupt	RW	Platform-level interrupt enable mask
0x20	accrued	RW	Accrued event mask
0x28	local_interrupt	RW	Hart-local interrupt-enable mask

Table 166: Bus-Error Unit Memory Map

Each bit in `enable`, `plic_interrupt`, `accrued`, and `local_interrupt` is associated with an error event, described in Table 167. The `cause` register represents which event occurred, and the `value` register contains the address of that error event. The `value` register will be updated when errors occur on memory that supports data cache. For example, the Memory Port has data cache support, where the System and Peripheral Ports do not.

11.3 Reportable Errors

Table 167 lists the events that the Bus-Error Unit may report.

Value	Description
0	<i>No error</i>
1	Instruction cache bus error
2	Instruction cache or ITIM correctable ECC error
3	Reserved
4	Reserved
5	Load/Store/PTW bus error
6	Data cache or DTIM or DTIM correctable ECC error
7	Data cache or DTIM or DTIM uncorrectable ECC error
8	Reserved
9	Private Private L2 bus error
10	Private Private L2 cache correctable ECC error
11	Private Private L2 cache uncorrectable ECC error

Table 167: Bus-Error Unit Error Events

11.4 Functional Description

When one of the events listed in Table 167 occurs, the Bus-Error Unit can record information about that event and can generate an interrupt to a platform-level interrupt controller (i.e., the PLIC) or locally to the hart. The `enable` register contains a mask of which events the BEU can record. Each bit in `enable` corresponds to an event in Table 167. For example, if `enable[5]` is set, the BEU will record load, store, or PTW TileLink bus errors. If `plic_interrupt[5]` is also

set, then a global interrupt will be asserted to the platform-level interrupt controller. Alternatively, if `local_interrupt[5]` is set, then a BEU error will result in a local interrupt to the hart. The cause register indicates the event the BEU has most recently recorded, e.g., a value of 5 indicates a load, store, or PTW TileLink bus error was recorded.

The cause value 0 is reserved to indicate "no error". cause is only written for events enabled in the `enable` register. Furthermore, cause is only written when its current value is 0; that is, if multiple events occur, only the first one is latched, until software clears the cause register.

The `value` register supplies the physical address that caused the event, or 0 if the address is unknown. The BEU writes the `value` register whenever it writes the cause register, such as when an event enabled in the `enable` register occurs or when cause contains 0.

The bit position in the accrued register indicates which events have occurred since the last time it was cleared by software. Its format is the same as the `enable` register. The BEU sets bits in the accrued register whether or not they are enabled in the `enable` register.

11.4.1 BEU Global Interrupt

The bit position in the `plic_interrupt` register indicates which accrued events should generate an interrupt into the PLIC. An interrupt is generated when the same bit is set in both accrued and `plic_interrupt`. The PLIC drives the machine external interrupt to the core, which has an interrupt ID of 11 (0xB). The exception code value, located in `mcause` (machine trap cause) CSR, will be 11 (0xB) when BEU interrupts are routed through the PLIC. This exception code value is independent of the PLIC interrupt number used to connect the BEU to the PLIC.

The external interrupt handler must clear the BEU accrued bit before exiting.

11.4.2 BEU Local Interrupt

The BEU output signal connects to the non-maskable interrupt (NMI) logic, described in Section 7.11. The `local_interrupt` register indicates which accrued events will cause a trap to the RNMI interrupt vector address. Within the NMI handler, the `nmcause` register can be read to determine if it was a BEU error, or an error triggered directly via the NMI `rnmi_n` input signal.

11.4.3 Global Interrupt Configuration

In addition to writing the BEU registers to enable interrupts, the `mstatus.MIE` CSR bit should be written to enable Machine level interrupts globally. The `mie.MEIE` CSR bit should also be configured to enable external interrupts when `plic_interrupt` is enabled to route the PLIC interrupt from the BEU interrupts to the core. Likewise, the corresponding PLIC Hart N M-Mode interrupt enable register should be enabled, and the PLIC Hart N interrupt priority threshold must be set. Additionally, the PLIC interrupt priority register must be set for the BEU's corresponding PLIC interrupt ID. Refer to the local interrupt chapter for more details regarding interrupt configurations.

Chapter 12

Level 2 Prefetcher

The SiFive L2 Prefetcher is a per-hart device that allows the L2 to perform accesses to memory based on the patterns of data accesses made by the harts in the Core Complex. For instance, if a hart is reading every 100th byte of a large array and accesses are missing the L1 data cache, the prefetcher will detect this. It will then allocate the appropriate memory addresses to the L2 Cache so subsequent accesses to the array will hit in the L2 Cache. This reduces overall access time to the array and improves performance of the application.

The Tenstorrent X280 MC contains four L2 Prefetcher instances. Their addresses are shown in Table 168.

Instance	Base Address
Hart 0 L2 Prefetcher	0x0203_0000
Hart 1 L2 Prefetcher	0x0203_2000
Hart 2 L2 Prefetcher	0x0203_4000
Hart 3 L2 Prefetcher	0x0203_6000

Table 168: L2 Prefetcher Instances

12.1 Operation

The L2 Prefetcher can monitor eight different data streams per hart, where a stream consists of a base address and a stride between memory addresses. Prefetches are automatically issued to the memory system when possible and a prefetch queue of 16 entries holds the accesses before they are issued. The range of a prefetch (or maximum stride length) in terms of cache lines is set by the `additionalCtrl.window` register. For instance, if this is set to 0x4, then streams with a stride of 256 bytes or greater will be ignored (assuming each cache line is 64 bytes).

The initial number of prefetches (or prefetch distance) made per stream is set by `basicCtrl.initialDist`. The prefetching distance will adapt based on the success of the prefetching and the overall range of prefetching. If a hart continues to make direct accesses which match the stride of the stream (both in magnitude and sign), then additional prefetches will be made, and the prefetch distance can increase. It's possible that the direct accesses made by a hart will be too fast for the prefetcher, and the hart will have to wait for the L2 to populate. When this occurs, the L2 Prefetcher will increase the prefetch distance (i.e., emit more

prefetches) to minimize the chance of this occurring. The threshold to increase the prefetching distance is set by `additionalCtrl.hitMSHRThrd`.

The maximum number of prefetches emitted for a stream is limited by `basicCtrl.maxAllowedDist`. The prefetch distance will increase gradually, but at times this may not be fast enough. The speed of the ramp-up from the initial distance to the maximum is managed automatically by the prefetcher. The `basicCtrl.linToExpThrd` setting can be used to fine-tune this adaptive ramp-up. The smaller the value of this field, the faster the prefetcher will reach the maximum prefetch distance.

Prefetches are issued only if there are idle L2 Miss Status Holding Registers (MSHRs). The `additionalCtrl.qFullnessThrd` value is used to control when the prefetcher will stop issuing hints to the memory system. This 4-bit field allows the total number of MSHRs in the L2 Cache to be represented in fractions of 1/16th. For instance, a threshold value of `0xc` means that if 75% of the MSHRs are allocated, prefetches will be stalled until the number of MSHRs in use drops below 75% available. This setting allows the prefetcher to be tuned such that the direct accesses made by the Core Complex (from harts and Front Port masters) aren't stalled by the prefetcher.

The prefetcher monitors both reads and writes to memory and the strides can either be incrementing or decrementing addresses. Prefetching is disabled at reset; scalar support can be enabled with the `basicCtrl.scalarLoadSupportEn` and `additionalCtrl.scalarStoreSupportEn` bits, and vector support can be enabled with the `additionalCtrl.vectorLoadSupportEn` and `additionalCtrl.vectorStoreSupportEn` bits.

12.2 Retiring Streams

It's unlikely that a prefetch stream will continue indefinitely for the life of an application, so the L2 Prefetcher accommodates a method to retire old streams. This allows new streams to be tracked.

A stream can be retired by successfully hitting in the L2 Cache without the need for prefetching. If enough consecutive accesses hit in the L2 Cache, which were not prefetched, then there is no need for the prefetcher to continue monitoring the stream. The `additionalCtrl.hitCacheThrd` field sets this threshold for retiring the stream.

12.3 Page Boundaries

The L2 Prefetcher can be programmed to cross 4 KiB page boundaries of memory. This is useful in an application environment in which there is no OS page protection, or where OS page protection can be ignored. When `basicCtrl.crossPageEn` is set, prefetching can cross 4 KiB boundaries. Prefetching will resume if the hart continues to make accesses that match the stride in the new 4 KiB page of memory.

12.4 Memory Map

The memory map for the L2 Prefetcher control registers is shown in Table 169.

Offset	Name	Description
0x0	basicCtrl	L2 Prefetcher basic control register
0x4	additionalCtrl	L2 Prefetcher additional control register

Table 169: L2 Prefetcher Memory Map

12.5 Control Registers

The L2 Prefetcher control registers basicCtrl and additionalCtrl are described in the below tables.

Basic Control Register (basicCtrl)				
Register Address		L2 Prefetcher Base Address		
Bits	Field Name	Attr.	Rst.	Description
0	scalarLoadSupportEn	RW	0x0	Enable hardware prefetcher support for scalar loads
1	Reserved	RW	0x0	
[7:2]	initialDist	RW	0x3	Initial prefetch distance
[13:8]	maxAllowedDist	RW	0xA	Maximum allowed prefetch distance
[19:14]	linToExpThrd	RW	0x5	Linear-to-exponential prefetch distance threshold
[27:20]	Reserved	RW	0x0	
28	crossPageEn	RW	0x0	Enable prefetches to cross-pages. When crossPageEn == 1, the prefetches will cross 4K boundary, if needed.
[30:29]	forgiveThrd	RW	0x0	Threshold for forgiving loads with mismatching strides when L2 Prefetcher is in trained state. Set to 0 to disable this feature (default).
31	Reserved	RO	0x0	

Table 170: Basic Control Register

Additional Control Register (additionalCtrl)				
Register Address		L2 Prefetcher Base Address + 0x4		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	qFullnessThrd	RW	0xE	Threshold fraction/16 of MSHRs to stop sending hits
[8:4]	hitCacheThrd	RW	0x5	Threshold number of cache tag hits for evicting prefetch entry
[12:9]	hitMSHRThrd	RW	0x2	Threshold number of demand hits on hint MSHRs for increasing prefetch distance
[18:13]	window	RW	0x6	Size of the comparison window for address matching
19	scalarStoreSupportEn	RW	0x0	Enable hardware prefetcher support for scalar stores
20	vectorLoadSupportEn	RW	0x0	Enable hardware prefetcher support for vector loads
21	vectorStoreSupportEn	RW	0x0	Enable hardware prefetcher support for vector stores
[31:22]	Reserved			

Table 171: Additional Control Register

12.6 L2 Prefetcher Initialization

The L2 Prefetcher needs to be initialized in the boot software prior to use. SiFive initialization software writes 0x15811 to basicCtrl and 0x38c84e to additionalCtrl and recommends these values as a starting point. Once initialized, the L2 Prefetcher can be tuned on a per-configuration basis as needed.

Chapter 13

Private Level 2 Cache

In addition to the L1 memory system, the Tenstorrent X280 MC also includes a per hart Private L2 (pL2) Cache. The pL2 Cache communicates with both the upstream L1 caches and downstream L3 cache or memory, enabling a high-performance cache subsystem. The pL2 Cache is responsible for managing requests from the L1 instruction and data caches of the core. The overall cache structure is depicted in Figure 162. The features of the pL2 Cache are summarized in Table 173.

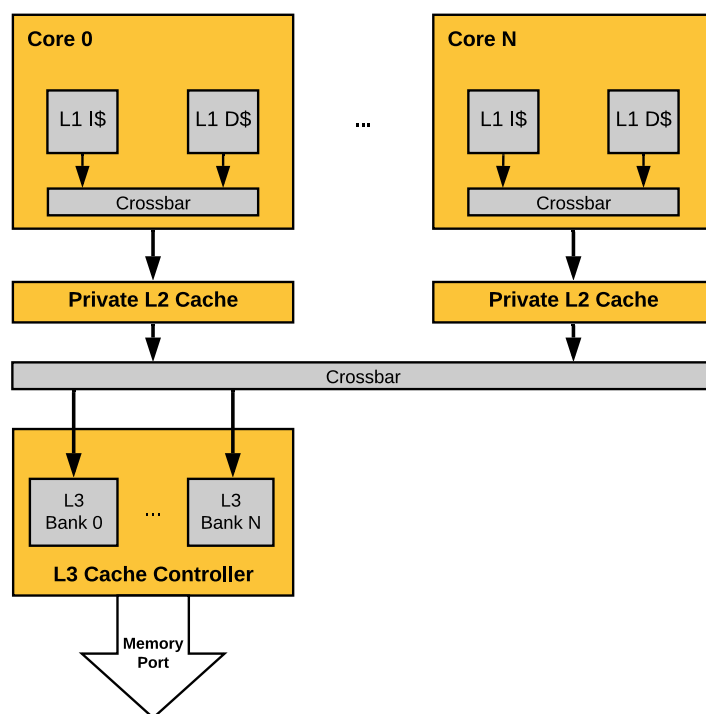


Figure 162: Cache Structure in the Tenstorrent X280 MC

The Tenstorrent X280 MC contains four pL2 Cache instances. Their addresses are shown in Table 172.

Instance	Base Address
Hart 0 Private L2 Cache	0x1010_4000
Hart 1 Private L2 Cache	0x1010_8000
Hart 2 Private L2 Cache	0x1010_C000
Hart 3 Private L2 Cache	0x1011_0000

Table 172: Private L2 Cache Instances

13.1 Functional Description

The pL2 Cache is a 128 KiB 8-way cache. It has a line size of 64 bytes and is read/write-allocate with a random replacement policy. The pL2 Cache implements the MESI coherence protocol, and operates in write-back mode.

The pL2 cache line data are stored in the data RAM. The data RAM is composed of 2 banks. This subdivision into banks helps to increase available bandwidth between the CPU and its pL2 Cache. As such, multiple requests to different banks may proceed in parallel. Each bank consists of one or more sub-banks and each sub-bank consists of a group of physical SRAM blocks. SRAM memories are protected by ECC.

The pL2 Cache also has a 11-16 cycle load-to-use latency on L2 hits depending on cache capacity, as well as a high L1/L2 and L2/L3 bandwidth.

Features	Description
Size	128 KiB
Associativity	8 ways
Number of banks	2 banks
Line size	64 bytes
Coherence protocol	MESI Coherence Protocol
ECC protection	SECCDED (8 bits ECC)
Latency	11-16 cycle load-to-use latency
Number of MSHRs	40 MSHRs

Table 173: Private L2 Cache Feature Set

13.1.1 Coherence and Ordering

The pL2 Cache supports the classic MESI, invalidate-based cache coherence protocol.

It is inclusive of the L1 data cache and partially inclusive of the L1 instruction cache. When a block of data is allocated to the L1 cache, it is also allocated to the pL2. When a block is evicted from the L1 data cache, the corresponding block in the pL2 is then updated.

The pL2 Cache also supports cacheable atomic memory operations.

The pL2 Cache guarantees ordering of MMIO requests. An MMIO read will follow the cache read pipeline and allocate into a Miss Status Handling Register (MSHR). There are 40 MSHRs in the pL2 Cache.

13.1.2 Cache Maintenance

The pL2 Cache supports cache maintenance messages through an MMIO CSR interface. A cache maintenance message must specify its target and type.

The target can be a specific pair of set/ways of the target cache, a specific cache line address of the target cache, or all cache lines of the target cache.

The type can be one of the following:

- Refresh: a read-modify-write performed on the target. This is mainly used for in-line ECC correction.
- CleanInvalid: evict the line out of cache. This is used to enable software coherence.

All legal combinations of targets and types are listed in Section 13.3.4.

The pL2 Cache maintains coherence for any cache maintenance operation (CMO), so the above operations may back probe to the L1 caches, if necessary. The invalid operations do not impact the L3 Cache and clean operations terminate in L3. As the L3 is inclusive, clean operations terminate in L3.

13.1.2.1 Cache Maintenance Operation Completion

The two CMO completion types are as follows:

- Early completion: when the order of requests is established in the pL2
- Late completion: when the cache line is evicted out of the targeted cache for the set/way target CMO, and the whole cache subsystem for the physical address target CMO

13.1.3 Error Correction Codes (ECC)

The pL2 Cache supports ECC. ECC is applied to the data RAMs and tag/state RAMs. The data RAMs use Single-Error Correcting, Double-Error Detecting (SECCDED). The tag/state RAMs use Single-Error Correcting, Double-Error Detecting (SECCDED).

The pL2 Cache can observe errors from three sources:

- The core, when the core passes down data with corrupt
- The L3 cache and memory, when the pL2 receives data with corrupt
- PL2 tag/state/data SRAMs, when the pL2 observes ECC errors from these sources

All correctable and uncorrectable ECC errors are reported to the Bus-Error Unit, described in Chapter 11. When any ECC error is detected after a tag/state lookup and/or a data RAM read, the pL2 manages this request and initiates a read-modify-write flow for correctable ECC errors, or a clean/invalid flow for uncorrectable ECC errors.

13.1.4 Private L2 Performance Monitor

Similar to the hardware performance monitor (HPM) for the core, the pL2 Cache also has a pL2 performance monitoring (pL2PM) facility. It consists of a set of event-programmable counters and their event selector registers. The registers are available to control the behavior of the counters. The performance event selector and other control registers are configured in machine mode, and the event-programmable counters can be read in user mode.

13.1.4.1 PL2PM Event Selector Registers (pmEventSelectX)

The pL2PM event selector CSRs (pmEventSelect0-5) follow the definition of mhpmeventX as defined in the RISC-V ISA. They are 64-bit bit **WARL** registers. To control the event type to count, these CSRs are used to program the corresponding event counters.

Offset	Bits	Access	Description
0x2000	64	RW	pmEventSelect0
0x2008	64	RW	pmEventSelect1
0x2010	64	RW	pmEventSelect2
0x2018	64	RW	pmEventSelect3
0x2020	64	RW	pmEventSelect4
0x2028	64	RW	pmEventSelect5

Table 174: PL2 Performance Monitor Event Selectors

The event selectors are partitioned into two fields: the lower 8 bits select an event class, and the upper bits form a mask of events in that class.

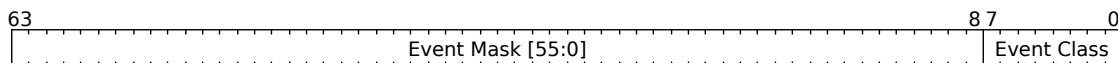


Figure 163: Event Selector Fields

The counter increments if the event corresponding to any set mask bit occurs.

13.1.4.2 PL2PM Counter Client Filter CSR (pmClientFilter)

The pL2PM counter client filter register pmClientFilter is a 64-bit **WARL** register that controls which client's performance events are excluded from incrementing the performance monitor counters. However, some events might not be attributed to a particular client in a specific implementation (for example, the pL2 release event in SiFive's pL2 Cache design).

Offset	Bits	Access	Description
0x2800	64	RW	pmClientFilter

Table 175: PL2 Performance Monitor Counter Client Filter

The mapping between masters and their pL2 master IDs is shown in Table 176.

Master ID	Description
0	Fetch Unit
1	D-Cache
2	D-Cache
3	D-Cache MMIO
4	L3 Prefetcher

Table 176: Master IDs in the Private L2 Cache

Note that the default value of pmClientFilter is 0, meaning all clients' events are counted. To disable event counting for a particular client, set its corresponding bit in pmClientFilter.

13.1.4.3 PL2 Performance Monitor Counters (pmEventCounterX)

The pL2 Performance Monitor counters (pmEventCounter0-5), follow the definition of mhpmonitorX as defined in the RISC-V ISA. They are 64-bit **WARL** registers.

Offset	Bits	Access	Description
0x3000	64	RW	pmEventCounter0
0x3008	64	RW	pmEventCounter1
0x3010	64	RW	pmEventCounter2
0x3018	64	RW	pmEventCounter3
0x3020	64	RW	pmEventCounter4
0x3028	64	RW	pmEventCounter5

Table 177: PL2 Performance Monitor Counters

Note

If a counter is not implemented, such as when there is no pL2PM, both the counter and its corresponding event selector are hard-wired to 0, meaning that the counter always returns 0.

13.1.4.4 Event Selector Encodings

Table 178 describes the event selector encodings available. Events are categorized into two classes based on the Event Class field encoded in pmEventSelectX[7:0]. One or more events

can be programmed by setting the respective Event Mask bit for a given event class. An event selector encoding of 0 means "count nothing". Multiple events will cause the counter to increment any time any of the selected events occur.

pL2 Performance Monitor Event Register		
Inner Transaction Events, pmEventSelectX[7:0]=0x1		
Bits	Description	Can use ClientFilter?
0	PutFullData request from inner cache	Yes
1	PutPartialData request from inner cache	Yes
2	ArithmeticData / LogicalData request from inner cache	Yes
3	Get request from inner cache	Yes
4	PrefetchRead request from inner cache	Yes
5	PrefetchWrite request from inner cache	Yes
6	AcquireBlock.NtoB request from inner cache	Yes
7	AcquireBlock.NtoT request from inner cache	Yes
8	AcquireBlock.BtoT request from inner cache	Yes
9	AcquirePerm.NtoT request from inner cache	Yes
10	AcquirePerm.BtoT request from inner cache	Yes
11	Release.TtoB request from inner cache	Yes
12	Release.TtoN request from inner cache	Yes
13	Release.BtoN request from inner cache	Yes
14	ReleaseData.TtoB request from inner cache	Yes
15	ReleaseData.TtoN request from inner cache	Yes
16	ReleaseData.BtoN request from inner cache	Yes
17	ReleaseData.TtoT request from inner cache	Yes
18	ProbeBlock.toT request from inner cache	Yes
19	ProbeBlock.toB request from inner cache	Yes
20	ProbeBlock.toN request from inner cache	Yes
21	ProbePerm.toN request from inner cache	Yes
22	ProbeAck.TtoB request from inner cache	Yes
23	ProbeAck.TtoN request from inner cache	Yes
24	ProbeAck.BtoN request from inner cache	Yes
25	ProbeAck.TtoT request from inner cache	Yes
26	ProbeAck.BtoB request from inner cache	Yes
27	ProbeAck.NtoN request from inner cache	Yes
28	ProbeAckData.TtoB request from inner cache	Yes
29	ProbeAckData.TtoN request from inner cache	Yes
30	ProbeAckData.TtoT request from inner cache	Yes
pL2 Query Result Events, pmEventSelectX[7:0]=0x2		
Bits	Description	Can use ClientFilter?
0	PutFullData.Hit request from inner cache hits a valid line in pL2	Yes
1	PutPartialData.Hit request from inner cache hits a valid line in pL2	Yes

Table 178: PL2PM pmEventSelect Register

2	ArithmeticData / LogicalData request from inner cache hits a valid line in pL2	Yes
3	Get request from inner cache hits a valid line in pL2	Yes
4	PrefetchRead request from inner cache hits a valid line in pL2	Yes
5	AcquireBlock.NtoB request from inner cache hits a valid line in pL2	Yes
6	AcquirePerm.NtoT request from inner cache hits a valid line in pL2	Yes
7	Release.TtoB request from inner cache hits a valid line in pL2	Yes
8	ReleaseData.TtoB request from inner cache hits a valid line in pL2	Yes
9	ProbeBlock.toT request from outer cache hits a valid line in pL2	No
10	PutFullData request from inner cache hits a shared line in pL2	Yes
11	PutPartialData request from inner cache hits a shared line in pL2	Yes
12	ArithmeticData / LogicalData request from inner cache hits a shared line in pL2	Yes
13	Get request from inner cache hits a shared line in pL2	Yes
14	PrefetchRead request from inner cache hits a shared line in pL2	Yes
15	AcquireBlock.NtoB request from inner cache hits a shared line in pL2	Yes
16	AcquirePerm.NtoT request from inner cache hits a shared line in pL2	Yes
17	ProbeBlock.toT request from outer cache hits a shared line in pL2	No
18	ProbeBlock.toT request from outer cache hits a dirty line in pL2	No
Outer Transaction Events, pmEventSelectX[7:0]=0x3		
Bits	Description	Can use ClientFilter?
0	PutFullData request to outer cache	No
1	PutPartialData request to outer cache	No
2	ArithmeticData / LogicalData request to outer cache	No
3	Get request to outer cache	No
4	PrefetchRead request to outer cache	No
5	PrefetchWrite request to outer cache	No
6	AcquireBlock.NtoB request to outer cache	No
7	AcquireBlock.NtoT request to outer cache	No
8	AcquireBlock.BtoT request to outer cache	No
9	AcquirePerm.NtoT request to outer cache	No
10	AcquirePerm.BtoT request to outer cache	No
11	Release.TtoB request to outer cache	No
12	Release.TtoN request to outer cache	No
13	Release.BtoN request to outer cache	No

Table 178: PL2PM pmEventSelect Register

14	ReleaseData.TtoB request to outer cache	No
15	ReleaseData.TtoN request to outer cache	No
16	ReleaseData.BtoN request to outer cache	No
17	ReleaseData.TtoT request to outer cache	No
18	ProbeBlock.toT request from outer cache	No
19	ProbeBlock.toB request from outer cache	No
20	ProbeBlock.toN request from outer cache	No
21	ProbePerm.toN request from outer cache	No
22	ProbeAck.TtoB request to outer cache	No
23	ProbeAck.TtoN request to outer cache	No
24	ProbeAck.BtoN request to outer cache	No
25	ProbeAck.TtoT request to outer cache	No
26	ProbeAck.BtoB request to outer cache	No
27	ProbeAck.NtoN request to outer cache	No
28	ProbeAckData.TtoB request to outer cache	No
29	ProbeAckData.TtoN request to outer cache	No
30	ProbeAckData.TtoT request to outer cache	No
Other pL2 Microarchitectural Events, pmEventSelectX[7:0]=0x4		
Bits	Description	Can use ClientFilter?
0	Hint request from inner cache hits an inflight miss request	Yes
1	Acquire / Get request from inner cache hits an inflight miss request	Yes
2	Put / ArithmeticData / LogicalData request from inner cache hits an inflight miss request	Yes
3	Hint / Acquire / Get request from inner cache replays	Yes
4	Put / ArithmeticData / LogicData request from inner cache replays	Yes
5	Probe request from outer cache replays	No
Note: "Inner cache" refers to the inner side of the pL2 cache, i.e., transactions from the core. "Outer cache" refers to the other side of the pL2 Cache, i.e., read/write transactions from the pL2 and probe requests from outside.		

Table 178: PL2PM pmEventSelect Register

13.2 Memory Map

The pL2 Cache memory map is shown in Table 179.

Offset	Name	Description
0x0000	Config	Information about the cache configuration
0x0040	ECCInjectError	Inject an ECC error
0x0060	EccInjectErrorDone	ECC inject is done
0x0200	CFlush64	MMIO cache flush command register
0x0208	FlushCount	The number of outstanding CFLUSH requests
0x1008	pL2Config0	Config bits of private L2 cache
0x2000	pmEventSelect0	Performance monitor event select 0
0x2008	pmEventSelect1	Performance monitor event select 1
0x2010	pmEventSelect2	Performance monitor event select 2
0x2018	pmEventSelect3	Performance monitor event select 3
0x2020	pmEventSelect4	Performance monitor event select 4
0x2028	pmEventSelect5	Performance monitor event select 5
0x2800	pmClientFilter	Performance counter client disable mask
0x3000	pmEventCounter0	Performance monitor event counter 0
0x3008	pmEventCounter1	Performance monitor event counter 1
0x3010	pmEventCounter2	Performance monitor event counter 2
0x3018	pmEventCounter3	Performance monitor event counter 3
0x3020	pmEventCounter4	Performance monitor event counter 4
0x3028	pmEventCounter5	Performance monitor event counter 5

Table 179: Private L2 Cache Memory Map

13.3 Register Descriptions

This section describes the functionality of the memory-mapped registers in the Private L2 Cache.

13.3.1 PL2 Cache Configuration Register (Config)

The Config Register can be used to programmatically determine information regarding the cache size and organization.

PL2 Cache Configuration Register (Config)				
Register Offset		0x0		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	Banks	RO	0x2	Number of banks in the cache
[15:8]	Ways	RO	0x8	Number of ways per bank
[23:16]	lgSets	RO	0x8	Base-2 logarithm of the total number of sets
[31:24]	lgBlockBytes	RO	0x6	Base-2 logarithm of the bytes per cache block

Table 180: PL2 Cache Configuration Register

13.3.2 ECC Error Injection Register (ECCInjectError)

The ECCInjectError register can be used to inject single-bit or double-bit ECC errors into either the tag/state or data SRAM. This function can be used to test error correction logic, measurement, and recovery. Note that when an uncorrectable ECC error is injected into either tag/state or data SRAM, the program may lose data coherence and consistency, which may impact normal behavior.

ECC Error Injection Register (ECCInjectError)				
Register Offset		0x40		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	ECCToggleBit	RW	0x0	Toggle (corrupt) this bit index on the next cache operation
[15:8]	Reserved			
[17:16]	InjectionTarget	RW	0x0	Inject error in 0=data or 1=tag/state or 2=lru
18	InjectOnWrite	RW	0x0	Inject error on (0) read data or (1) write data
19	InjectionType	RW	0x0	Inject correctable (0) or uncorrectable (1) error
[31:20]	Reserved			

Table 181: ECC Error Injection Register

ECC injection only occurs on the next cacheable read or write. If software wants to know the status of the injection, it needs to poll the EccInjectErrorDone register.

13.3.3 ECC Error Injection Done Register (EccInjectErrorDone)

ECCInjectErrorDone indicates if the ECC error injection is complete.

ECC Error Injection Done Register (EccInjectErrorDone)				
Register Offset		0x60		
Bits	Field Name	Attr.	Rst.	Description
0	ECCToggleDone	RO	0x1	Toggle (corrupt) is done. Note that this field is volatile and may be masked in IP-XACT.
[7:1]	Reserved			

Table 182: ECC Error Injection Done Register

13.3.4 PL2 Cache Flush Register (Cflush64)

Writes to the Flush64_Command field generates a cache flush request; the response indicates that the pL2 cache has sunk the request, but may or may not perform it. This behavior is to

enable multiple outstanding cache flush requests. If software wants to know the status of previously issued requests, it needs to poll the FlushCount register.

PL2 Cache Flush Register (Cflush64)				
Register Offset		0x200		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	Flush64_Address_Low	WO	0x0	For CFlush_PA_* commands, defines the lower 32-bit physical address. For CFlush_SetWay_* commands, defines the set number.
[55:32]	Flush64_Address_High	WO	0x0	For CFlush_PA_* commands, defines the upper 24-bit physical address. For CFlush_SetWay_* commands, defines the set number.
[63:56]	Flush64_Command	WO	0x0	The command of cache flush

Table 183: PL2 Cache Flush Register

Table 184 describes the defined cache flush commands. For further descriptions of the Cache Flush Commands supported by the pL2, see Section 13.1.2.

Message	Bits		Description
	[7:3]	[2:0]	
Reserved	0	0	
Reserved	0	1	
Reserved	0	2	
CFlush_PA_CleanInv	0	3	Evict the cache line of the target physical address
CFlush_SetWay_Refresh	1	0	Read the cache line of the specified set/way from cache, and write it back into cache
Reserved	1	1	
Reserved	1	2	
CFlush_SetWay_CleanInv	1	3	Evict the cache line of the target set/way
CFlush_Cache_Refresh	2	0	Read the cache line of each set/way from cache, and write it back into cache
Reserved	2	1	
Reserved	2	2	
CFlush_Cache_CleanInv	2	3	Evict all valid cache lines

Table 184: Cache Flush Commands

When an illegal command is received, or an illegal set or way is given for a set/way target request, it is treated as a nop.

13.3.5 Flush Count Register (FlushCount)

FlushCount is a read-only register that contains the number of outstanding cache flush requests. 0 indicates that all previously issued cache flush requests have been completed.

PL2 Flush Count Register (FlushCount)				
Register Offset		0x208		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	FlushCount	RO	0x0	The number of outstanding cflush requests. Note that this field is volatile and may be masked in IP-XACT.

Table 185: PL2 Flush Count Register

13.3.6 PL2 Configuration Register (pL2Config)

This register contains bits to configure various features of the pL2 cache.

PL2 Configuration 0 Register (pL2Config0)				
Register Address		0x1008		
Bits	Field Name	Attr.	Rst.	Description
[2:0]	Reserved	RO	0x0	
3	cleanEvictEnable	RW	0x1	When set, enable clean eviction from pL2 to an outer cache/ directory. This bit will affect the L3 cache miss rate.
[8:4]	Reserved	RO	0x0	
9	l2AvoidL1LineDisable	RW	0x1	When set, L2 victim can pick L1 cache lines as well. This bit will affect pL2 cache miss rate.
[15:10]	Reserved	RO	0x0	
16	softwareEccInjectEnable	RW	0x0	When set, enable software ECC injection
17	errInjectOnWriteEnable	RW	0x1	When set, enable error injecton on tag/data RAM write (latent error). When clear, inject only on the next read. This is useful to test ECC correction logic.
18	errInjectOnReadEnable	RW	0x0	When set, enable error injection on tag/data RAM read.
19	dataUceInjectEnable	RW	0x0	When set, enable uncorrectable data error injection. When clear, inject only correctable data errors.
20	dirUceInjectEnable	RW	0x0	When set, enable uncorrectable metadata error injection. When clear, inject only correctable metadata errors.
[31:21]	Reserved	RO	0x0	

Table 186: PL2 Configuration 0 Register

Chapter 14

Level 3 Cache Controller

This chapter describes the functionality of the Level 3 Cache Controller used in the Tenstorrent X280 MC.

14.1 Level 3 Cache Controller Overview

The SiFive Level 3 Cache Controller is used to provide access to fast copies of memory for masters in a Core Complex. The Level 3 Cache Controller also acts as a directory-based coherency manager.

The SiFive Level 3 Cache Controller offers extensive flexibility, as it allows for several features in addition to the Level 3 Cache functionality. These include memory-mapped access to L3 Cache RAM for disabled cache ways, scratchpad functionality, way masking and locking, ECC support with error tracking statistics, error injection, and interrupt signaling capabilities.

These features are described in Section 14.2.

14.2 Functional Description

The Tenstorrent X280 MC L3 Cache is a 2 MiB 16-way set-associative cache. It has a line size of 64 bytes and is read/write-allocate with a random replacement policy. The cache operates in write-back mode only. The L3 Cache is composed of 4 banks. This subdivision into banks helps facilitate increased available bandwidth between CPU masters and the L3 Cache, as each bank has its own dedicated 512-bit TL-C inner port. As such, multiple requests to different banks may proceed in parallel.

The outer port of the L3 Cache Controller is a 512-bit TL-C port shared among all banks and typically connected to a DDR controller. The outer Memory Port of the L3 Cache Controller is shared among all banks and typically connected to cacheable memory. The overall organization of the L3 Cache Controller is depicted in Figure 164.

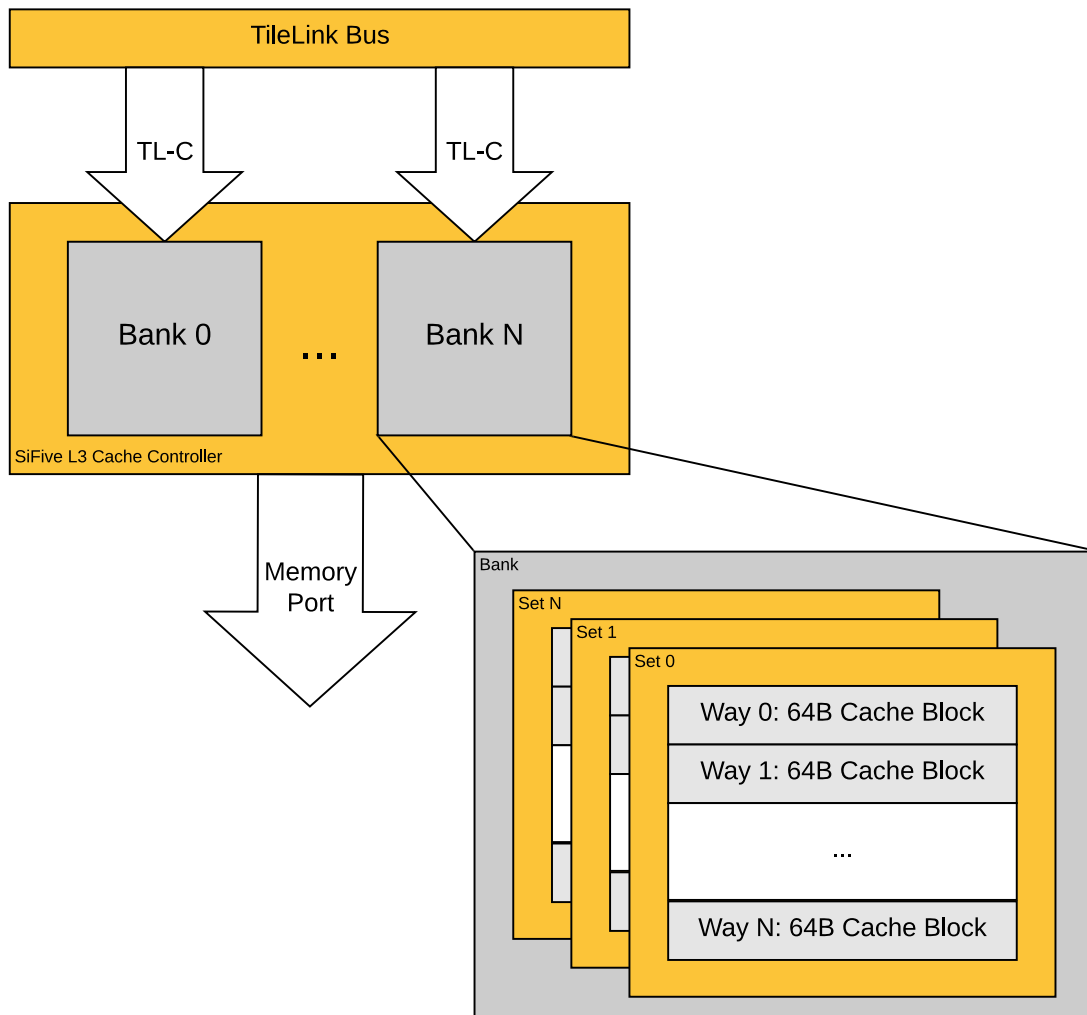


Figure 164: Organization of the SiFive L3 Cache Controller

14.2.1 Way Enable and the L3 Loosely-Integrated Memory (L3 LIM)

The SiFive Level 3 Cache Controller allows for its SRAMs to act either as direct-addressed memory in the Core Complex address space or as a cache that is controlled by the L3 Cache Controller, which can contain a copy of any cacheable address.

When cache ways are disabled, they are addressable in the L3 Loosely-Integrated Memory (L3 LIM) address space as described in the Tenstorrent X280 MC memory map in Section 4.2. The L3 LIM is an uncacheable port into unused L3 SRAM and provides deterministic access time. It is neither cached by the L1 data cache nor memory backed, as it is just a dedicated software-addressable, low latency, uncached memory. Fetching instructions or data from the L3 LIM provides deterministic behavior equivalent to an L3 Cache hit, with no possibility of a cache miss. Accesses to the L3 LIM are always given priority over cache way accesses, which target the same L3 Cache bank.

Out of reset, all ways, except for way 0, are disabled. Cache ways can be enabled by writing to the wayEnable register described in Section 14.4.2. Once a cache way is enabled, it cannot be disabled unless the Tenstorrent X280 MC is reset. The highest numbered L3 cache way is mapped to the lowest L3 LIM address space, and way 1 occupies the highest L3 LIM address range. As L3 cache ways are enabled, the size of the L3 LIM address space shrinks. The mapping of L3 cache ways to L3 LIM address space is shown in Figure 165, where N is the number of L3 cache ways, each of size 128 KiB (0x0002_0000).

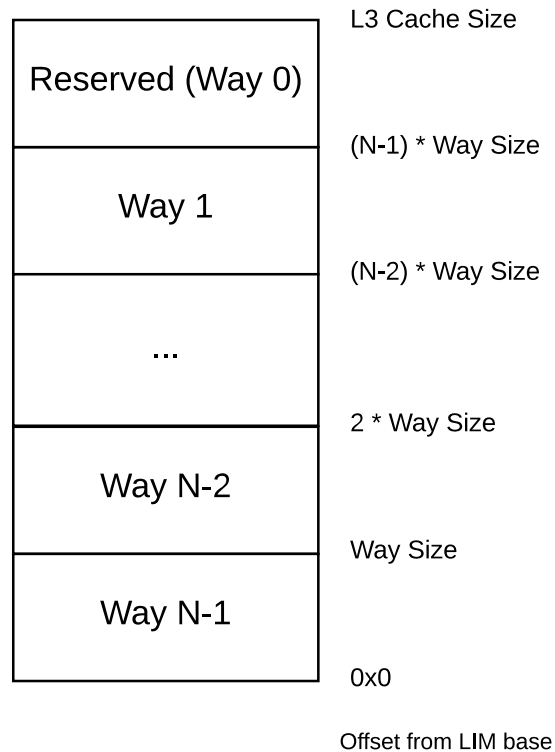


Figure 165: Mapping of L3 Cache Ways to L3 LIM Addresses

14.2.2 Way Masking and Locking

The SiFive L3 Cache Controller can control the amount of cache memory a CPU master is able to allocate into by using the wayMaskN register described in Section 14.4.10. Note that wayMaskN registers only affect allocations, and reads can still occur to ways that are masked. As such, it becomes possible to lock down specific cache ways by masking them in all wayMaskN registers. In this scenario, all masters can still read data in the locked cache ways but cannot evict data.

The following example shows how to lock the L3 cache ways:

1. For the following example, assume the data to be locked is not present in any of the L3 ways.
2. Select an L3 way where the data is to be locked to.

3. From Table 197, select the Master ID(s) to be used to load the data that is to be locked into the L3.
4. Clear bit M of the wayMaskN register for the other masters, besides the master(s) selected in the step above, to enable allocation by the selected master(s) into locked way M. The wayMask0_M - wayMaskN_M registers are described in Table 192.
 - a. Use Table 192 to locate the wayMaskN register(s).
5. Enable the WayMask registers for the selected master(s).
 - a. Set bit M of the wayMaskN register to 1, and clear other bits in that register, to allow master N to be able to allocate data only into way M of the L3.
6. Load the data to be locked into the L3 way M. Note that because the L3 is an inclusive cache, any load access by the master N will allocated the cache line for that access into the L3 cache.
7. Set the wayMaskN_M bit for this way to zero. At this point, no other master can evict data from this way.
8. If unlocking the data from way M is desired at some point in the future, set bit M in some or all of the wayMaskN registers.

14.2.3 L3 Zero Device

The SiFive L3 Cache Controller has a dedicated scratchpad address region that allows for allocation into the cache using an address range that is not memory backed. This address region is denoted as the L3 Zero Device in the Section 4.2 memory map. Writes to the scratchpad region allocate into cache ways that are enabled and not masked.

A Zero Device ignores write data and always returns zero on reads. The Tenstorrent X280 MC provides a Zero Device behind the L3 Cache, similar to the Memory Port. When combined with locked L3 cache ways, which prevent eviction, locations within a Zero Device's address range appear to retain their value. This provides a mechanism to create L1 and L2 cacheable memory that are essentially backed by L3 SRAM until the way is released (and the value resets to zero). The L3 Zero Device is cacheable like the Memory Port. However, if dirty data is evicted and a write-back to the L3 Zero Device occurs, the Zero Device will discard the write data. Therefore, care must be taken with the scratchpad, as there is no memory backing this address space. Cache evictions from addresses in the scratchpad result in data loss.

The main advantage of the L3 Zero Device over the L3 LIM is that it is a cacheable region allowing for data stored to the scratchpad to also be cached in a master's L1 data cache and pL2 cache, which results in faster access.

To understand the difference between the L3 LIM and the L3 Zero Device, consider Figure 166. Notice that the L3 LIM accesses the same blocks of memory as the main path into the L3 Cache, whereas the L3 Zero Device sits behind L3 Cache much like the Memory Port:

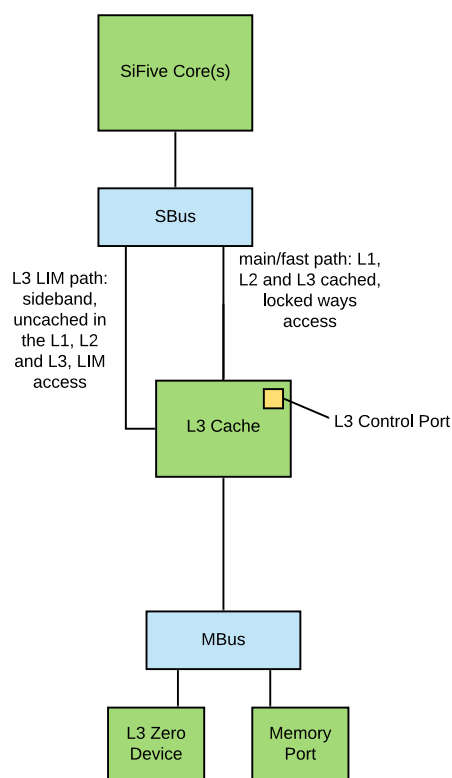


Figure 166: Difference between L3 LIM and L3 Zero Device

The recommended procedure for using the L3 Zero Device is as follows:

1. Use the `wayEnable` register to enable the desired cache ways
2. Designate a single master that will allocate into the scratchpad. For this procedure, we designate this master as Master S. All other masters (CPU and non-CPU) are denoted as Masters N.
3. Masters N: Write to the `wayMaskN` register to mask the ways that are to be used for the scratchpad. This prevents Masters N from evicting cache lines in the designated scratchpad ways.
4. Master S: Write to the `wayMaskN` register to mask all ways *except* the ways that are to be used for the scratchpad. At this point, Master S should only be able to allocate into the cache ways meant to be used as a scratchpad.
5. Master S: Write scratchpad data into the L3 Zero Device address range
6. Master S: Repeat steps 4 and 5 for each way to be used as scratchpad
7. Master S: Use the `wayMaskN` register to mask the scratchpad ways for Master S so that it is no longer able to evict cache lines from the designated scratchpad ways
8. At this point, the scratchpad ways should contain the scratchpad data, with all masters able to read, write, and execute from this address space, and no masters able to evict the scratchpad contents

14.2.4 L3 Features Access Summary

Table 187 describes the L3 features as a function of Way Enable and Way Mask.

Way Enable	Way Mask	Access Base Address	L3 Feature
0	X	0x0800_0000	LIM
1	0	0x4000_3000_0000	Way Masking and Locking — Fast Read access
1	0	0x0A00_0000	Zero Device — Fast Read access from scratchpad
1	1	0x4000_3000_0000	Way Masking and Locking — Write data mode
1	1	0x0A00_0000	Zero Device — Write data to scratchpad

Table 187: L3 Features Access Summary

14.2.5 L3 Performance Monitor

Similar to the hardware performance monitor (HPM) for the core, the L3 Cache also has an L3 performance monitoring (L3PM) facility. It consists of a set of event-programmable counters and their event selector registers. The registers are available to control the behavior of the counters. The performance event selector and other control registers are configured in machine mode, and the event-programmable counters can be read in user mode.

14.2.5.1 L3PM Event Selector Registers (pmEventSelectX)

The L3PM event selector CSRs (pmEventSelect0-5) follow the definition of mhpmeventX as defined in the RISC-V ISA. They are 64-bit bit **WARL** registers. To control the event type to count, these CSRs are used to program the corresponding event counters.

Offset	Bits	Access	Description
0x2000	64	RW	pmEventSelect0
0x2008	64	RW	pmEventSelect1
0x2010	64	RW	pmEventSelect2
0x2018	64	RW	pmEventSelect3
0x2020	64	RW	pmEventSelect4
0x2028	64	RW	pmEventSelect5

Table 188: L3 Performance Monitor Event Selectors

The event selectors are partitioned into two fields: the lower 8 bits select an event class, and the upper bits form a mask of events in that class.

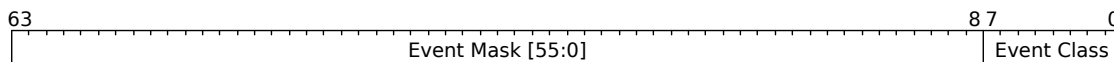


Figure 167: Event Selector Fields

The counter increments if the event corresponding to any set mask bit occurs.

14.2.5.2 L3PM Counter Client Filter CSR (pmClientFilterX)

The L3PM counter client filter register (pmClientFilter0) is a 64-bit **WARL** register that controls which client's performance events are excluded from incrementing the performance monitor counters. However, some events might not be attributed to a particular client in a specific implementation (for example, the L3 release event in SiFive's L3 Cache design).

Offset	Bits	Access	Description
0x2800	64	RW	pmClientFilter0

Table 189: L3 Performance Monitor Counter Client Filter

14.2.5.3 L3 Performance Monitor Counters (pmEventCounterX)

The L3 Performance Monitor counters (pmEventCounter0-5), follow the definition of mhpmonitorX as defined in the RISC-V ISA. They are 64-bit **WARL** registers.

Offset	Bits	Access	Description
0x3000	64	RW	pmEventCounter0
0x3008	64	RW	pmEventCounter1
0x3010	64	RW	pmEventCounter2
0x3018	64	RW	pmEventCounter3
0x3020	64	RW	pmEventCounter4
0x3028	64	RW	pmEventCounter5

Table 190: L3 Performance Monitor Counters

Note

If a counter is not implemented, such as when there is no L3PM, both the counter and its corresponding event selector are hard-wired to 0, meaning that the counter always returns 0.

14.2.5.4 Event Selector Encodings

Table 191 describes the event selector encodings available. Events are categorized into two classes based on the Event Class field encoded in pmEventSelectX[7:0]. One or more events can be programmed by setting the respective Event Mask bit for a given event class. An event selector encoding of 0 means "count nothing". Multiple events will cause the counter to increment any time any of the selected events occur.

L3 Performance Monitor Event Register		
Transaction Events, pmEventSelectX[7:0]=0x1		
Bits	Description	Can use ClientFilter?
0	PutFullData request from inner cache	Yes
1	PutPartialData request from inner cache	Yes
2	AtomicData request from inner cache	Yes
3	Get request from inner cache	Yes
4	PrefetchRead request from inner cache	Yes
5	PrefetchWrite request from inner cache	Yes
6	AcquireBlock.NtoB request from inner cache	Yes
7	AcquireBlock.NtoT request from inner cache	Yes
8	AcquireBlock.BtoT request from inner cache	Yes
9	AcquirePerm.NtoT request from inner cache	Yes
10	AcquirePerm.BtoT request from inner cache	Yes
11	Release.TtoB request from inner cache	Yes
12	Release.TtoN request from inner cache	Yes
13	Release.BtoN request from inner cache	Yes
14	ReleaseData.TtoB request from inner cache	Yes
15	ReleaseData.TtoN request from inner cache	Yes
16	ReleaseData.BtoN request from inner cache	Yes
17	ProbeBlock.toT request from outer cache	No
18	ProbeBlock.toB request from outer cache	No
19	ProbeBlock.toN request from outer cache	No
L3 Query Result Events, pmEventSelectX[7:0]=0x2		
Bits	Description	Can use ClientFilter?
0	PutFullData request from inner cache hits a valid line in L3	Yes
1	PutPartialData request from inner cache hits a valid line in L3	Yes
2	AtomicData request from inner cache hits a valid line in L3	Yes
3	Get request from inner cache hits a valid line in L3	Yes
4	Prefetch request from inner cache hits a valid line in L3	Yes
5	AcquireBlock request from inner cache hits a valid line in L3	Yes
6	AcquirePerm request from inner cache hits a valid line in L3	Yes
7	Release request from inner cache hits a valid line in L3	Yes
8	ReleaseData request from inner cache hits a valid line in L3	Yes
9	Probe request from outer cache hits a valid line in L3	No
10	PutFullData request from inner cache hits a shared line in L3	Yes

Table 191: L3PM pmEventSelect Register

11	PutPartialData request from inner cache hits a shared line in L3	yes
12	AtomicData request from inner cache hits a shared line in L3	Yes
13	Get request from inner cache hits a shared line in L3	Yes
14	Prefetch request from inner cache hits a shared line in L3	Yes
15	AcquireBlock request from inner cache hits a shared line in L3	Yes
16	AcquirePerm request from inner cache hits a shared line in L3	Yes
17	Probe request from outer cache hits a shared line in L3	No
18	Probe request from outer cache hits a dirty line in L3	No
L3 Request Events, pmEventSelectX[7:0]=0x3		
Bits	Description	Can use ClientFilter?
0	AcquireBlock.NtoB request to outer cache, miss	No
1	AcquireBlock.NtoT request to outer cache, miss	No
2	AcquireBlock.BtoT request to outer cache, miss	No
3	AcquirePerm.NtoT request to outer cache, miss	No
4	AcquirePerm.BtoT request to outer cache, miss	No
5	Release.TtoB request to outer cache, eviction	No
6	Release.TtoN request to outer cache, eviction	No
7	Release.BtoN request to outer cache, eviction	No
8	ReleaseData.TtoB request to outer cache, not applicable	No
9	ReleaseData.TtoN request to outer cache, dirty eviction	No
10	ReleaseData.BtoN request to outer cache, not applicable	No
11	ProbeBlock.toT request to inner cache, code miss hits other harts	No
12	ProbeBlock.toB request to inner cache, load miss hits other harts	No
13	ProbeBlock.toN request to inner cache, store miss hits other harts	No
Other Events, pmEventSelectX[7:0]=0x4		
Bits	Description	Can use ClientFilter?
0	Hint request from inner cache hits an inflight miss request	Yes
Note: "Inner cache" refers to the inner side of the L3 cache, i.e., transactions from the core and Front Port. "Outer cache" refers to the other side of the L3 Cache, i.e., read/write transactions from the L3 and probe requests from outside.		

Table 191: L3PM pmEventSelect Register

14.2.5.5 Setting up the pmClientFilterX Register

Table 197 shows the mapping of L3 Cache masters and their client ID.

Note that the default value of `pmClientFilterX` is 0, meaning all clients' events are counted. To disable event counting for a particular client, set its corresponding bit in `pmClientFilterX`.

14.2.5.6 Programming the `pmEventSelect` registers

The following example shows the use of 6 performance counters:

```
// directory lookup events: L2 miss
*pmEventSelect0 = 0x01 | // Event Set 1
    ( (0x01 << 6 ) |
      (0x01 << 7 ) |
      (0x01 << 8 ) |
      (0x01 << 9 ) |
      (0x01 << 10)
    ) << 8;
// directory lookup results: L2 miss hit L3
*pmEventSelect1 = 0x02 | // Event Set 2
    ( (0x01 << 5) |
      (0x01 << 6)
    ) << 8;

// L2 request misses L3
*pmEventSelect4 = 0x03 | // Event Set 3
    ( (0x01 << 0) |
      (0x01 << 1) |
      (0x01 << 2) |
      (0x01 << 3) |
      (0x01 << 4)
    ) << 8;
```

14.2.6 Error Correction Codes (ECC)

The SiFive Level 3 Cache Controller supports ECC. ECC is applied to both categories of SRAM used, the data SRAMs and the metadata SRAMs (index, tag, and directory information). The data SRAMs use Single-Error Correcting, Double-Error Detecting (SECDDED). The metadata SRAMs use Single-Error Correcting, Double-Error Detecting (SECDDED).

Whenever a correctable error is detected, the cache immediately repairs the corrupted bit and writes it back to SRAM. This corrective procedure is completely invisible to application software. However, to support diagnostics, the cache records the address of the most recently corrected metadata and data errors. Whenever a new error is corrected, a counter is increased, and an interrupt is raised. There are independent addresses, counters, and interrupts for correctable metadata and data errors.

`DirError`, `DirFail`, `DataError`, and `DataFail` signals are used to indicate that an L3 metadata, uncorrectable L3 metadata, L3 data, or uncorrectable L3 data error has occurred, respectively. These signals are connected to the PLIC, as described in Chapter 9, and are cleared upon reading their respective count registers.

14.2.7 Coherence

The SiFive L3 Cache is partially inclusive of the L1 instruction cache and is inclusive of the L1 data cache. When a block of data is allocated to the L1 cache, it is also allocated to the L3 Cache. When a block is evicted from the L1, the corresponding block in the L3 is then updated and marked dirty.

To understand how coherence is managed differently in the L3 Cache with respect to the L1 instruction and data caches, consider the following rules:

1. Only an instruction cache allocation from the Memory Port will land in the L3 Cache
2. An eviction from the L3 Cache does not cause an eviction from the instruction cache
3. An eviction from the instruction cache does not cause L3 Cache eviction either
4. A discard from the data cache does not invalidate the L3 Cache
5. Following a flush in the L3 Cache, the L3 Cache will back probe lines in L1 data cache

14.3 Memory Map

The L3 Cache Controller memory map is shown in Table 192.

Offset	Name	Description
0x0000	Config	Information about the Cache Configuration
0x0008	WayEnable	The index of the largest way which has been enabled. May only be increased.
0x0040	ECCInjectError	Inject an ECC Error
0x0100	DirECCFixLow	The low 32-bits of the most recent address to fail ECC
0x0104	DirECCFixHigh	The high 32-bits of the most recent address to fail ECC
0x0108	DirECCFixCount	Reports the number of times an ECC error occurred
0x0120	DirECCFailLow	The low 32-bits of the most recent address to fail ECC
0x0124	DirECCFailHigh	The high 32-bits of the most recent address to fail ECC
0x0128	DirECCFailCount	Reports the number of times an ECC error occurred
0x0140	DatECCFixLow	The low 32-bits of the most recent address to fail ECC
0x0144	DatECCFixHigh	The high 32-bits of the most recent address to fail ECC
0x0148	DatECCFixCount	Reports the number of times an ECC error occurred
0x0160	DatECCFailLow	The low 32-bits of the most recent address to fail ECC
0x0164	DatECCFailHigh	The high 32-bits of the most recent address to fail ECC
0x0168	DatECCFailCount	Reports the number of times an ECC error occurred
0x0200	Flush64	Flush the physical address equal to the 64-bit written data from the cache
0x0240	Flush32	Flush the physical address equal to the 32-bit written data << 4 from the cache
0x0800	WayMask0	Master 0 way enable mask register
0x0808	WayMask1	Master 1 way enable mask register
0x0810	WayMask2	Master 2 way enable mask register
0x0818	WayMask3	Master 3 way enable mask register
0x0820	WayMask4	Master 4 way enable mask register
0x0828	WayMask5	Master 5 way enable mask register
0x0830	WayMask6	Master 6 way enable mask register
0x0838	WayMask7	Master 7 way enable mask register
0x0840	WayMask8	Master 8 way enable mask register
0x0848	WayMask9	Master 9 way enable mask register
0x0850	WayMask10	Master 10 way enable mask register
0x0858	WayMask11	Master 11 way enable mask register
0x0860	WayMask12	Master 12 way enable mask register
0x0868	WayMask13	Master 13 way enable mask register
0x0870	WayMask14	Master 14 way enable mask register
0x0878	WayMask15	Master 15 way enable mask register
0x0880	WayMask16	Master 16 way enable mask register
0x0888	WayMask17	Master 17 way enable mask register
0x0890	WayMask18	Master 18 way enable mask register
0x0898	WayMask19	Master 19 way enable mask register
0x08A0	WayMask20	Master 20 way enable mask register

Table 192: L3 Cache Controller Memory Map

Offset	Name	Description
0x08A8	WayMask21	Master 21 way enable mask register
0x08B0	WayMask22	Master 22 way enable mask register
0x08B8	WayMask23	Master 23 way enable mask register
0x08C0	WayMask24	Master 24 way enable mask register
0x08C8	WayMask25	Master 25 way enable mask register
0x08D0	WayMask26	Master 26 way enable mask register
0x08D8	WayMask27	Master 27 way enable mask register
0x08E0	WayMask28	Master 28 way enable mask register
0x08E8	WayMask29	Master 29 way enable mask register
0x08F0	WayMask30	Master 30 way enable mask register
0x08F8	WayMask31	Master 31 way enable mask register
0x0900	WayMask32	Master 32 way enable mask register
0x0908	WayMask33	Master 33 way enable mask register
0x0910	WayMask34	Master 34 way enable mask register
0x0918	WayMask35	Master 35 way enable mask register
0x0920	WayMask36	Master 36 way enable mask register
0x0928	WayMask37	Master 37 way enable mask register
0x1000	FeatureDisable	Composable cache
0x2000	pmEventSelect0	Performance monitor event select 0
0x2008	pmEventSelect1	Performance monitor event select 1
0x2010	pmEventSelect2	Performance monitor event select 2
0x2018	pmEventSelect3	Performance monitor event select 3
0x2020	pmEventSelect4	Performance monitor event select 4
0x2028	pmEventSelect5	Performance monitor event select 5
0x2800	pmClientFilter0	Performance counter client disable mask 0
0x3000	pmEventCounter0	Performance monitor event counter 0
0x3008	pmEventCounter1	Performance monitor event counter 1
0x3010	pmEventCounter2	Performance monitor event counter 2
0x3018	pmEventCounter3	Performance monitor event counter 3
0x3020	pmEventCounter4	Performance monitor event counter 4
0x3028	pmEventCounter5	Performance monitor event counter 5

Table 192: L3 Cache Controller Memory Map

14.4 Register Descriptions

This section describes the functionality of the memory-mapped registers in the Level 3 Cache Controller.

14.4.1 L3 Cache Configuration Register (Config)

The Config Register can be used to programmatically determine information regarding the cache size and organization.

L3 Cache Configuration Register (Config)				
Register Offset		0x0		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	Banks	RO	0x4	Number of banks in the cache
[15:8]	Ways	RO	0x10	Number of ways per bank
[23:16]	lgSets	RO	0x9	Base-2 logarithm of the sets per bank
[31:24]	lgBlockBytes	RO	0x6	Base-2 logarithm of the bytes per cache block

Table 193: L3 Cache Configuration Register

14.4.2 Way Enable Register (WayEnable)

The wayEnable register determines which ways of the Level 3 Cache Controller are enabled as cache. Cache ways that are not enabled are mapped into the Tenstorrent X280 MC's L3 LIM (Loosely-Integrated Memory) as described in the memory map in Section 4.2.

This register is initialized to 0 on reset and may only be increased. This means that, out of reset, only a single L3 cache way is enabled, as one cache way must always remain enabled. Once a cache way is enabled, the only way to map it back into the L3 LIM address space is by a reset.

Way Enable Register (WayEnable)				
Register Offset		0x8		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	WayEnable	WARL	0x0	The index of the largest way which has been enabled. May only be increased.
[7:4]	Reserved	RO	0x0	

Table 194: Way Enable Register

14.4.3 ECC Error Injection Register (ECCInjectError)

The ECCInjectError register can be used to insert an ECC error into either the backing data or metadata SRAM. This function can be used to test error correction logic, measurement, and recovery.

ECC Error Injection Register (ECCInjectError)				
Register Offset		0x40		
Bits	Field Name	Attr.	Rst.	Description
[7:0]	ECCToggleBit	RW	0x0	Toggle (corrupt) this bit index on the next cache operation
[15:8]	Reserved			
16	ECCToggleType	RW	0x0	Toggle (corrupt) a bit in 0=data or 1=directory
[31:17]	Reserved			

Table 195: ECC Error Injection Register

14.4.4 ECC Directory Fix Registers (**DirECCFix***)

The **DirECCFixHigh** and **DirECCFixLow** registers are read-only registers that contain the address of the most recently corrected L2 metadata error. This field supplies only the portions of the address that correspond to the affected set, since all ways are corrected together.

The **DirECCFixCount** register is a read-only register that contains the number of corrected L3 metadata errors. Reading this register clears the **DirError** interrupt signal described in Section 14.2.6.

Note that these registers are volatile and may be masked in IP-XACT.

14.4.5 ECC Directory Fail Registers (**DirECCFail***)

The **DirECCFailLow** and **DirECCFailHigh** registers are read-only registers that contains the address of the most recent uncorrected L3 metadata error.

The **DirECCFailCount** register is a read-only register that contains the number of uncorrected L3 metadata errors. Reading this register clears the **DirFail** interrupt signal described in Section 14.2.6.

Note that these registers are volatile and may be masked in IP-XACT.

14.4.6 ECC Data Fix Registers (**DataECCFix***)

The **DataECCFixLow** and **DataECCFixHigh** registers are read-only registers that contain the address of the most recently corrected L3 data error.

The **DataECCFixCount** register is a read-only register that contains the number of corrected L2 data errors. Reading this register clears the **DataError** interrupt signal described in Section 14.2.6.

Note that these registers are volatile and may be masked in IP-XACT.

14.4.7 ECC Data Fail Registers (**DataECCFail***)

The **DataECCFailLow** and **DataECCFailHigh** registers are a read-only registers that contain the address of the most recent uncorrected L3 data error.

The **DataECCFailCount** register is a read-only register that contains the number of uncorrected L2 data errors. Reading this register clears the **DataFail** interrupt signal described in Section 14.2.6.

Note that these registers are volatile and may be masked in IP-XACT.

14.4.8 L3 Cache ECC Error Injection and Correction

Writes to the `ECCInjectError` register, described in Section 14.4.3, specifies a bit position within the combined data+ECC (or directoryEntry+ECC) granule to toggle or corrupt exactly 1 bit the next time an L3 entry is written.

The `ECCToggleBit` register specifies a hex value corresponding to which bit to toggle. For example, if you write a value of `0x47`, bit 71 will be toggled (consequently, for a write to an L3 data RAM that has 64 bits of data and 8 bits of ECC, writing a value $> 0x47$ will have no effect).

The error will be injected (i.e., the bit will be toggled/corrupted) on the next write. When there are multiple data+ECC granules comprising a cache line, error injection will always be applied to the first of these granules. For instance, when you have a 64+8-bit granule and a 64-byte line, it takes eight granules to make up a full cache line, and errors will be injected into the specified bit in the first of these eight granules. For metadata, this is a non-issue since there is only one directoryEntry+ECC granule for each cache line.

The error will not be reported until the corrupted entry is subsequently read, and the error is detected/corrected; that is, this will be at an arbitrary time after the `ECCInjectError` Register is written, unless a test specifically ensures that it reads the particular set and way that was corrupted due to that write.

The address reported in the fix/fail address registers (`DirECCFix` and `DirECCFail`, respectively) will always be the cache-line-size-aligned address of the start of the cache line in which the error was detected.

For error injection on the data, if the L1-L3 interface (and consequently, the width of the L3 data R/W interface) is larger than the ECC granule, the designated bit will actually be toggled on multiple granules; the number of granules being as many granules as make up the data interface. For example, if the L3 data interface is 128-bits wide and the data+ECC granule is 72-bits, two bits will get toggled—one per granule. This is, however, invisible to software since, when these bits are read, only a single corrected error will be reported, and the address will be the start of the cache line as stated above.

For the reported fix address (`DirECCFix`) on corrected errors in the metadata, only the address bits corresponding to the set (i.e., the index) are captured. In contrast, address bits corresponding to the tag are not captured and they will read out as 0.

14.4.9 Cache Flush Registers (Flush*)

The Tenstorrent X280 MC L3 Cache Controller provides two registers that can be used for flushing specific cache blocks.

`Flush64` is a 64-bit write-only register that flushes the cache block containing the address written. `Flush32` is a 32-bit write-only register that flushes a cache block containing the written address left shifted by 4 bits. In both registers, all bits must be written in a single access for the flush to take effect.

The flush operation performs a write-back and invalidate, meaning the contents are written to memory and L3, pL2, and L1 cache lines are then invalidated.

14.4.10 Way Mask Registers (wayMask*)

The wayMaskN register allows a master connected to the L3 Cache Controller to specify which L3 Cache ways can be evicted by Master N. Masters can still access memory cached in masked ways. The mapping between masters and their L3 master IDs is shown in Table 197.

At least one cache way must be enabled. It is recommended to set/clear bits in this register using atomic operations.

Way Mask 0 Register (wayMask0)				
Register Offset		0x800		
Bits	Field Name	Attr.	Rst.	Description
0	WayMask0_0	RW	0x1	Enable way 0 for Master 0
1	WayMask0_1	RW	0x1	Enable way 1 for Master 0
2	WayMask0_2	RW	0x1	Enable way 2 for Master 0
3	WayMask0_3	RW	0x1	Enable way 3 for Master 0
4	WayMask0_4	RW	0x1	Enable way 4 for Master 0
5	WayMask0_5	RW	0x1	Enable way 5 for Master 0
6	WayMask0_6	RW	0x1	Enable way 6 for Master 0
7	WayMask0_7	RW	0x1	Enable way 7 for Master 0
8	WayMask0_8	RW	0x1	Enable way 8 for Master 0
9	WayMask0_9	RW	0x1	Enable way 9 for Master 0
10	WayMask0_10	RW	0x1	Enable way 10 for Master 0
11	WayMask0_11	RW	0x1	Enable way 11 for Master 0
12	WayMask0_12	RW	0x1	Enable way 12 for Master 0
13	WayMask0_13	RW	0x1	Enable way 13 for Master 0
14	WayMask0_14	RW	0x1	Enable way 14 for Master 0
15	WayMask0_15	RW	0x1	Enable way 15 for Master 0

Table 196: Way Mask 0 Register

Master ID	Description
0	Debug
1	AXI4 Front Port ID#0 [W]
2	AXI4 Front Port ID#1 [W]
3	AXI4 Front Port ID#2 [W]
4	AXI4 Front Port ID#3 [W]
5	AXI4 Front Port ID#4 [W]
6	AXI4 Front Port ID#5 [W]
7	AXI4 Front Port ID#6 [W]
8	AXI4 Front Port ID#7 [W]
9	AXI4 Front Port ID#8 [W]
10	AXI4 Front Port ID#9 [W]
11	AXI4 Front Port ID#10 [W]
12	AXI4 Front Port ID#11 [W]
13	AXI4 Front Port ID#12 [W]
14	AXI4 Front Port ID#13 [W]
15	AXI4 Front Port ID#14 [W]
16	AXI4 Front Port ID#15 [W]
17	AXI4 Front Port ID#0 [R]
18	AXI4 Front Port ID#1 [R]
19	AXI4 Front Port ID#2 [R]
20	AXI4 Front Port ID#3 [R]
21	AXI4 Front Port ID#4 [R]
22	AXI4 Front Port ID#5 [R]
23	AXI4 Front Port ID#6 [R]
24	AXI4 Front Port ID#7 [R]
25	AXI4 Front Port ID#8 [R]
26	AXI4 Front Port ID#9 [R]
27	AXI4 Front Port ID#10 [R]
28	AXI4 Front Port ID#11 [R]
29	AXI4 Front Port ID#12 [R]
30	AXI4 Front Port ID#13 [R]
31	AXI4 Front Port ID#14 [R]
32	AXI4 Front Port ID#15 [R]
33	Trace Sink SBA
34	Hart 0 Private L2 Cache
35	Hart 1 Private L2 Cache
36	Hart 2 Private L2 Cache
37	Hart 3 Private L2 Cache

Table 197: Master IDs in the L3 Cache Controller

14.4.11 L3 Cache Feature Disable Register (FeatureDisable)

The FeatureDisable register controls the architectural clock gating functionality of the L3 Cache, which gates clock nodes when not active to reduce its clock tree and power. The FeatureDisable register controls whether trunk, regional, or "slow" regional clock gating features are enabled. The L3 cache clock gating functionality is further described in Section 15.6.

L3 Cache Feature Disable Register (FeatureDisable)				
Register Offset		0x1000		
Bits	Field Name	Attr.	Rst.	Description
0	disableTrunkClockGate	RW	0x1	When set, disables the trunk clock gating feature. When clear, dynamic gating of the L3 cache clock trunk is enabled. This clock gater incurs a 2-cycle penalty when exiting the wFI clock gated state and saves the most power. All cores in a Core Complex must be in wFI before this state is entered.
1	disableClockGate	RW	0x1	When set, disables all regional clock gating. When clear, non-aggressive clock gating is enabled. This clock gater has no penalty for exit, but less power savings than regional "slow" clock gating.
2	disableClockGateSlow	RW	0x1	When set, disables the regional clock gating "slow" feature. When clear, if regional clock gating is also enabled, the aggressive clock gating feature is enabled. This clock gater imposes a 1-cycle exit penalty from the gated state (on the first TL channel-A request), but saves more power than the no-penalty regional clock gate option.
[7:3]	Reserved			

Table 198: L3 Cache Feature Disable Register

Note

To enable the regional clock gating "slow" feature, both disableClockGate and disableClockGateSlow need to be cleared.

14.5 Procedure to Flush the L3 Cache

This section describes how to flush the L3 Cache using the Zero Device scratchpad. As the scratchpad resides in the L3 Cache, allocations made to the scratchpad can result in an eviction of a cache line that was allocated from the Memory Port. By controlling the wayMaskN register and targeted scratchpad address, it is possible to perform different flush operations.

As the scratchpad region is cacheable, it is necessary to perform a flush of the scratchpad region used to perform the L3 flush. Otherwise, if the targeted scratchpad line already exists in the L3 Cache, either through a direct access or speculation, it will prevent the desired L3 cache line from being evicted from the Memory Port when it is accessed. The steps outlined below include this necessary flush of the scratchpad region.

For flushing ranges of data, it is recommended to use the flush-by-address function in the L3 Controller space, described in Section 14.4.9.

14.5.1 Flushing a Single Index+Way

1. Write wayMaskN to allow evictions from only the desired way
2. Issue a FENCE instruction
3. Determine address A in the L3 scratchpad region that corresponds to the desired index to be flushed. For example, consider a 2 MiB 16-way cache with the Zero Device located at 0x0A00_000. To target index 8, the valid addresss would be {0x0A00_0200, 0x0A02_0200, 0x0A04_0200, ..., 0x0A1E_0200}.
4. Write address A to the L3 Controller Flush64 register
5. Issue a load from address A (or a store to address A)
6. (Optional) To force observation of the flush, write address A to the L3 Flush64 register and issue a FENCE instruction
7. Restore the wayMaskN register to the original value
8. Issue a FENCE instruction

14.5.2 Flushing the Entire L3 Cache

1. Write wayMaskN to allow evictions from only way 0
2. Issue a FENCE instruction
3. Flush the first way-size of L3 scratchpad memory using a series of Flush64 operations. For example, consider a 2 MiB 16-way cache with the Zero Device located at 0x0A00_000. 2048 lines (128 KiB) in the region from 0xA000_0000-0xA001_FFC0 must be targeted with a Flush64 operation.
4. Issue a FENCE instruction

5. Access the first way-size of L3 scratchpad memory using a series of load or store operations that correspond to each index of the L3 Cache. Considering the same cache example (2 MiB, 16-way), all 2048 lines in the region from 0xA000_0000-0xA001_FFC0 must be accessed. Only one access per 64 B cache line is required. Each scratchpad access will cause an eviction if the corresponding cache index is dirty.
6. Issue a FENCE instruction
7. Repeat steps 1-6 for the next way to be flushed until all ways have been flushed. The region flushed in step 3 needs to advance by the cache way size, as does the region accessed in step 5. Considering the same example, address range 0xA002_0000-0xA003_FFC0 will be targeted for way 1 operations.
8. (Optional) To force observation of the complete flush, repeat steps 3 and 4
9. Restore the wayMaskN register to the original value
10. Issue a FENCE instruction Memory accesses from harts not performing the flush can interfere with the flush operation. It is recommended to put other harts into WFI or a holding loop until the L3 flush is completed.

Chapter 15

Power Management

The following chapter describes power modes and establishes flows for powering up, powering down, and resetting the hardware of the Tenstorrent X280 MC.

15.1 Power Modes

Power modes include normal run mode with the Power Dial option and wait-for-interrupt clock gating mode using the `wFI` instruction. Additionally, there is a full power down mode supported via the `CEASE` instruction. These modes are covered in detail below.

15.2 Run Mode

The hart is fully operational in run mode, and SiFive designs include the option to include coarse-grained architectural clock gating. When this feature is enabled in the hart, configured instruction cache, data cache, integer pipeline, Debug Logic, and Floating-Point Unit (FPU) modules each contain their own clock gate. The clock gating feature will enable automatic clock gating of functional units when they are inactive and allow the hart to gate its own clock(s) based on activity.

15.2.1 Power Control

15.2.1.1 Core Clock

To further reduce power while in run mode, users may choose to reduce `external_source_for_core_N_clock`, which is required to be changed synchronously to the rest of the clocks in the system. It is important to note that the clock relationships with the rest of the system must still be maintained if `external_source_for_core_N_clock` is reduced. `external_source_for_core_N_clock` and all other clock signals are described in the Tenstorrent X280 MC User Guide.

15.2.1.2 Power Dial

To limit maximum power without frequency changes, Power Dial provides a method of scaling down dynamic power in a core. Power is reduced by restricting cycles allowed to advance

instructions into the execution pipeline. This feature can typically be used for throttling high CPU usage applications.

Power Dial CSR				
CSR	0x7C8			
Bits	Field Name	Attr.	Rst.	Description
[3:0]	dutycycle	RW	0x0	(1 - value/16) portion of the peak instruction throughput (i.e., value=0 is no reduction)
[7:4]	Reserved	RO	0x0	

Table 199: Power Dial CSR

The Power Dial Register may only be programmed in M-mode. When written with a non-zero value, the register restricts peak instruction throughput to the indicated rate. A value of 0 has no effect on instruction throughput. The rate is calculated per 256-cycle period, so a Power Dial value of 1 restricts instruction throughput at a common point in the pipeline to 240 cycles of each 256-cycle period. Reducing the peak rate reduces the worst-case power while minimizing the impact on performance.

15.3 WFI Clock Gate Mode

WFI clock gating mode can be entered by executing the WFI instruction. The assembly-level instruction is simply `wfi` and executing the C method using the GCC compiler can be accomplished with `asm("WFI")`.

15.3.1 WFI Wake Up

Wake up from a WFI occurs when the hart receives any interrupt. Depending on the software configuration, the hart will either immediately enter the interrupt handler, or resume execution on the instruction immediately after the WFI.

If interrupts are enabled and `mstatus.MIE=1`, then the hart will wake when an interrupt is enabled and becomes pending, and immediately enter the interrupt handler. Upon exit from the interrupt handler, program execution will resume at the instruction following the WFI.

If interrupts are enabled but `mstatus.MIE=0`, then the hart will wake when an interrupt is enabled and becomes pending but will not enter the interrupt handler. It will simply resume at the instruction immediately after the WFI in this case.

To prevent an interrupt source from waking a hart, the enable bit for that interrupt must be written to 0 prior to executing the WFI instruction. If any interrupts are pending upon executing a WFI instruction, then the WFI is effectively treated as a NOP instruction.

Refer to Chapter 7 for more detail on interrupt configuration.

15.4 CEASE Instruction for Power Down

To fully power down, follow the steps described in Section 15.11, where the last step is to execute a CEASE instruction. Once the CEASE instruction is executed, the core will not retire another instruction until reset. The CEASE opcode is 0x30500073 and can be implemented in either assembly or C. To create an assembly-level function using GCC, consider the following example.

```
.global _cease
.type      _cease, @function
_cease:
    .word 0x30500073
    ret
```

The next example demonstrates how to implement the CEASE instruction within a function in C.

```
static inline void cease()
{
    __asm__ __volatile__ (".word 0x30500073" : : : "memory"); // CEASE
}
```

15.5 Subsystem LowPower Controller (SLPC)

The Subsystem LowPower Controller (SLPC) is a unit inside the uncore that contains power management-related logic.

It includes the CorePowerState register that indicates the individual WFI and CEASE status for up to 16 Tiles in the core-complex.

It is used by core-complex power gating, but can be present when core-complex power gating is not enabled or supported.

Below are the memory map and bitfield definitions for the SLPC:

Location	Description
base + 0x000	CorePower State Register
base + 0x008	Reserved

Table 200: CorePower State Memory Map

CorePower State Register		
Bits	Field	Description
[15:0]	wfi_<x>	WFI indication from Tile-X, where x is the Tile Index. WFI status for up to 16 Tiles can be captured here.
[31:16]	cease_<x>	CEASE indication from Tile-X, where x is the Tile Index. CEASE status for up to 16 Tiles can be captured here.

Table 201: CorePower State Register

15.6 L3 Cache Clock Gating

The L3 Cache implements two levels of architectural clock gating to gate clock nodes when not active, reducing its clock tree and dynamic power.

1. The first level is a trunk clock gate on the entire L3 Cache Wrapper; that is, the clock to the entire module and all its flops is gated. This feature is disabled (i.e., the clock is always enabled) out of reset.

L3 Cache memory-mapped FeatureDisable register bit 0 (disableTrunkClockGate) enables the feature during start-up code. The clock gate only becomes active when all the cores in the Core Complex are in wFI mode and the L3 Cache is idle. This saves internal clock tree and flop's dynamic power in wFI mode. When any of the core is out of wFI mode, the clock is always enabled.

If the feature is enabled, in wFI mode, the clock is only available under the following conditions:

- Activity on the ingress memory bus initiated by any bus master
- Activity on the LIM bus
- Activity on the L3 Cache TileLink control bus

The clock remains available until all the above inflight transactions are finished at the L3 Cache Wrapper boundary.

2. The second level a regional clock gate that gates the clock of some the L3 Cache's major units. This feature is disabled (i.e., the clock is always enabled) out of reset.

FeatureDisable register bit 1 (disableClockGate) enables the feature during start-up code.

The L3 Cache FeatureDisable register is further described in Section 14.4.11.

15.7 Hardware Reset

The following list summarizes the hardware reset values required by *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.11* and applies to all SiFive designs.

1. Privilege mode is set to machine mode.
2. mstatus.MIE and mstatus.MPRV are required to be 0.
3. The misa register holds the full set of supported extensions for that implementation, and misa.MXL defaults to the widest supported ISA available, referred to as MXLEN.
4. The pc is set to the implementation specific reset vector.
5. The mcause register is set to 0x0 at reset.

6. The PMP configuration fields for address matching mode (A) and Lock (L) are set to 0, which defaults to no protection for any privilege level.

The internal state of the rest of the system should be completed by software early in the boot flow.

15.8 Early Boot Flow

For the early stages of boot, some of the first things software must consider are listed below:

- The global pointer (gp or x3) user register should be initialized to the `__global_pointer$` linker generated symbol and not changed at any point in the application program.
- The stack pointer (sp or x2) user register should be also set up as a standard part of the boot flow.
- All other user registers (x1, x4 - x31) can be written to 0 upon initial power-on.
- The `mtvec` register holds the default exception handler base address, so it is important to set up this register early in the boot flow, so it points to a properly aligned, valid exception handler location.
- Zero out the `bss` section and copy data sections into RAM areas as needed.

15.9 Interrupt State During Early Boot

Since `mstatus.MIE` defaults to 0, all interrupts are disabled globally out of reset. Prior to enabling interrupts globally through `mstatus.MIE`, consider the following:

- Ensure no timer interrupts are pending by checking the `mip.MTIP` bit. The `mtime` register is 0 out of reset and starts running immediately. However, the `mtimecmp` register does not have a reset value.

If no timer interrupt is required, leave `mie.MTIE` equal to 0 prior to enabling global interrupt with `mstatus.MIE`.

If the application requires a timer interrupt, write `mtimecmp` to a value in the future for the next timer interrupt before enabling `mstatus.MIE`.

- Write the remaining bits in the `mie` CSR to the desired value to enable interrupts based on the requirements of the system. This register is not defined to have a reset value.
- Each `msip` register in the Core-Local Interruptor (CLINT) or Core-Local Interrupt Controller (CLIC) address space is reset to 0, so no specific initialization is required for local software interrupts.

Since `msip` is memory-mapped, any hart in the system may trigger a software interrupt on another hart, so this should be considered during the boot flow on a multi-hart system.

- If a Platform-Level Interrupt Controller (PLIC) exists, check the PLIC pending status. The PLIC memory mapped pending bits are read-only, so the pending status should be cleared at the source if they reset to a non-zero status. Then, enable the PLIC interrupts as required by the system prior to enabling interrupts in the system via `mstatus.MIE`.

If an L3 Cache or Bus-Error Unit (BEU) is present, these interrupt IDs begin at 128, so the enable bits may lie in a different region of the memory map than other PLIC enable bits in the design.

- Wipe down memory if enabled with ECC. This can be done by writing `0x0` to memory with either store instructions issued by the CPU or using a DMA controller. ECC errors are reported via the Bus-Error Unit (BEU).
- Check BEU registers to ensure no errors are reported and that the enable bits reflect the requirements of the application.

15.10 Other Boot Time Considerations

- Write `0` to enable the appropriate bits in the Feature Disable CSR as described in Section 6.2.2.
- Ensure the remaining bits in the `mstatus` CSR are written to the desired application specific configuration at boot time.
- If a design includes user and supervisor privilege levels, initialize `medeleg` and `mideleg` registers to `0` until supervisor-level trap handling is set up correctly using `stvec`.
- The `mcause`, `mepc`, and `mtval` registers hold important information in the event of a synchronous exception. If the synchronous exception handler forces reset in the application, the contents of these registers can be checked to understand root cause.
- The PMP address and configuration CSRs are required to be initialized if user or supervisor privilege levels are part of the design. By default, user and supervisor modes have no permissions to the memory map unless explicitly granted by the PMP.
- The `mcycle` CSR is a 64-bit counter on both RV32 and RV64 systems, and it counts the number of cycles executed by the hart. It has an arbitrary value after reset and can be written as needed by the application.
- Instructions retired can be counted by the `minstret` register, and this also has an arbitrary value after reset. This can be written to any given value.
- The `mhpmeventX` CSR selects which hardware events to count, where the count is reflected in `mhpmpcounterX`. At any point, the `mhpmpcounterX` registers can be directly written to reset their value when the `mhpmeventX` register has the proper event selected.
- For cores with an MMU, ensure the `satp` register holds the correct configuration for address translation.
- There is no requirement for boot time initialization to any of the registers within the Debug Module, unless there is an application specific reason to do so.

- All other CSRs during boot time initialization should be considered based on system and application requirements.

15.11 Power-Down Flow

Designate one core as "primary" and all others as "secondary". For SiFive Core IP, coordination with an "External Agent" is required.

1. External Agent: Wait for communication from the primary core to initiate the following steps:
 - a. Stop sending inbound traffic (both transactions and interrupts) into the Core Complex.
 - b. Wait until all outstanding requests to the Core Complex are completed, then
 - c. Wait until `cease_from_tile_N` is high for the primary core and all secondary cores.
 - d. Once `cease_from_tile_N` is high for the primary core and all secondary cores, apply reset to the entire Core Complex.
2. Primary core:
 - a. The following sequence should be executed in machine mode and NOT out of a remote ITIM/DTIM.
 - b. Communicate with external agent to initiate cease power-down sequence.
 - c. Poll external agent until steps 1.a and 1.b are completed.
 - d. Disable all interrupts except those related to bus errors/memory corruption, and IPIs (if using enabled IPI to coordinate power-down sequence among cores).
 - i. Copy contents of any TIMs/LIMs into external memory.
 - ii. If there is an L3 cache, flush it.
 - iii. If there is no L3 cache, but there is a pL2 cache, flush it.
 - e. Inform all secondary cores to proceed.
 - f. Wait until `cease_from_tile_N` is high for all secondary cores. Examples of how this can be accomplished:
 - i. Have an off-core-complex memory-mapped register that tracks the state of the `cease_from_tile_N` signals. Primary core polls this register.
 - ii. Wire the `cease_from_tile_N` signals back into interrupt wires. Corresponding interrupts can be disabled along with all others in the first step. Primary core polls the interrupt-pending bits for those interrupts.
 - g. Disable all interrupts.
 - h. Execute CEASE instruction.
3. Secondary cores:

- a. The following sequence should be executed in machine mode and NOT out of a remote ITIM/DTIM.
- b. Execute in an idle loop for notification sent in step 2.e (could be via polling on an MMIO-accessible mailbox, polling on an IPI (disabled), or waiting on an IPI (enabled)).
- c. Disable all interrupts except those related to bus errors/memory corruption, and IPIs (if using enabled IPI to coordinate power-down sequence among cores).
- d. Copy contents of any TIMs/LIMs into external memory.
- e. If there is no L3 cache but there is a data cache, flush it using full-cache variant of CFLUSH.D.L1 if available, or per-line variant if not.
- f. Disable all interrupts.
- g. Execute CEASE instruction.

Chapter 16

Debug

This chapter describes the operation of SiFive debug hardware, which follows *The RISC-V Debug Specification, Version 1.0*. Currently, only interactive debug and hardware breakpoints are supported.

16.1 Debug Module

The Debug Module (DM) handles nearly all of the functions related to debugging. It is a slave to both the Debug Module Interface (DMI) coming from the probe, and a TileLink bus coming from the cores. From the perspective of the core, the DM appears as a 4KiB block in the memory map. The DM memory map as seen from the perspective of the core is shown in Table 203, and the register map from the perspective of the DMI is shown in Table 202.

Most of the DM is clocked by `debug_clock`. The `dmcontrol` register is accessible when `debug_clock` is not running, mainly to be able to write to `haltreq` while the core is in reset due to `ndreset`. Doing so generates a debug interrupt and will interrupt the selected core immediately once it is out of reset or during a WFI instruction.

DMI Address	Name	Description
0x04-0x0F	data0-data11	Read/Write DATA registers. 32-bit SiFive cores have 1 data register, 64-bit cores have 2. Note that these registers are volatile and may be masked in IP-XACT.
0x10	dmcontrol	Debug Module Control. See Table 216 for more information.
0x11	dmstatus	Debug Module Status. See Table 215 for more information.
0x12	hartinfo	Hart Information. See Table 217 for more information.
0x13	haltsum1	Read-only. Halt Summary 1. Only present on systems with >32 harts. Not used by SiFive.
0x14	hawindowse1	Read/Write. Select which window of up to 32 harts is visible in hawindow. Not used by SiFive since all SiFive systems have less than 32 harts.
0x15	hawindow	Read/Write. Window of 32 harts to be selected, in addition to the one selected by hartse1. Bit 0 corresponds to hart 0. A 1 will select the corresponding hart.
0x16	abstractcs	Abstract Control and Status. See Table 218 for more information.
0x17	command	Initiate abstract command. See Table 219 for more information.
0x18	abstractauto	Selects whether access to particular DATA or PROGBUF locations will re-execute the last command. Used for block transfers or other repeating commands. See Table 220 for more information.
0x20-0x2F	progbuf0-progbuf15	Read/Write PROGBUF registers.
0x32	dmcs2	Fields to set up and read back Halt Group or Resume Group configuration. Present by default on systems with more than 1 hart or with any external triggers. See Table 221 for more information.
0x38	sbcsc	System Bus Access Control and Status
0x39	sbaddress0	System Bus Address 31:0
0x3A	sbaddress1	System Bus Address 63:32.
0x3C	sbddata0	System Bus Data 31:0
0x3D	sbddata1	System Bus Data 63:32
0x40	haltsum0	Read-only. Halt Summary 0. Bit n reads 1 if hart n is halted. Note that this register is volatile and may be masked in IP-XACT.

Table 202: Debug Module Memory Map Seen from the Debug Module Interface

From the point of view of the core, the DM appears as a 4KiB block of memory. It is mapped into low memory so that memory references can use addresses relative to the \$zero register.

TL Address	Name	Attr.	Description
0x100	HALTED	WO	Written with hartid by ROM code when hart gets a debug interrupt or reenters ROM due to EBREAK. Sets halted[hartid]. If an abstract command was running, writing this also clears busy.
0x104	GOING	WO	Written by ROM code when it begins executing a command started by FLAGS[hartid].go. Clears FLAGS[hartid].go.
0x108	RESUMING	WO	Written with hartid by hart when it is about to resume. Sets resumeack[hartid] and clears halted[hartid] and FLAGS[hartid].resume.
0x10C	EXCEPTION	WO	Written by hart when it encounters an exception in debug mode. Sets cmderr to "exception".
0x300	WHERE TO	RO	JAL to ABSTRACT. This opcode is constructed by DM hardware and is needed because ABSTRACT is not a fixed address (depends on number of PROGBUF words selected in the configuration). Note that ABSTRACT is volatile and may be masked in IP-XACT.
contiguous	ABSTRACT	RO	2 words constructed by DM hardware based on abstract command written from DTM. • 0 - If transfer set, construct instruction to load/store specific register to/from DATA[0] (32 bits) or DATA[1:0] (64 bits), else NOP. • 4 - If postexec set, then NOP to fall thru and execute PROGBUF, else EBREAK to return to ROM park loop. Note that this register is volatile and may be masked in IP-XACT.
contiguous	PROGBUF	RW	Configurable number (typically 16, max 16) of R/W words to be filled in by debugger and executed by hart. Note that this register is volatile and may be masked in IP-XACT.
0x380-0x3BF	DATA	RW	Configurable number (1 for 32-bit or 2 for 64-bit, max 12) of R/W words intended for use for data transfer between debugger and hart. Since it is contiguous with PROGBUF, the debugger may use DATA as an extension of PROGBUF. Note that these registers are volatile and may be masked in IP-XACT.
0x400-0x7FF	FLAGS	RO	One byte flag per hart. • Bit 0 (go): Set by writing an abstract command, cleared by ROM write to GOING. ROM will jump to WHERE TO.

Table 203: Debug Module Memory Map from the Perspective of the Core

TL Address	Name	Attr.	Description
			Bit 1 (resume): Set by writing 1 to resumereq[hartid]. Cleared by ROM write of hartid to RESUMING. ROM restores s0 then executes dret. Note that these registers are volatile and may be masked in IP-XACT.
0x800-0xFFF	ROM	RO	Debug interrupt or EBREAK enters at 0x800, saves s0, writes hartid to HALTED, then busy-waits for <code>FLAGS[hartid] > 0</code>. If <code>FLAGS[hartid].go</code>, write 0 to GOING, then jump to WHERETO. Else write hartid to RESUMING, then execute dret to return to user program. ROM Source Code: https://github.com/chipsalliance/rocket-chip/blob/master/scripts/debug_rom/debug_rom.S

Table 203: Debug Module Memory Map from the Perspective of the Core

16.2 Debug and Trigger Registers

This section describes the per hart debug and trigger registers, which are mapped into the CSR space as follows:

CSR	Name	Allowed Access Mode	Description
0x7B0	dcsr	Debug	Debug Control and Status Register
0x7B1	dpc	Debug	Debug PC. Stores execution address just before debug exception and to return to at dret.
0x7B2	dscratch0	Debug	Debug Scratch Register 0
0x7A0	tselect	Debug, Machine	Trigger Select. Most configs implement 2, 4, or 8 triggers. Triggers are all type 6 (address/data).
0x7A1	tdata1	Debug, Machine	Trigger Data 1, mcontrol6
0x7A2	tdata2	Debug, Machine	Trigger Data 2, the address for comparison
0x7A3	tdata3	Debug, Machine	Trigger Data 3

Table 204: Debug and Trigger Registers

16.2.1 Debug Control and Status Register (dcsr)

This register gives information about debug capabilities and status. Its detailed functionality is described in *The RISC-V Debug Specification, Version 1.0*.

Debug Control and Status Register (dcsr)			
CSR	0x7B0		
Bits	Field Name	Attr.	Description
[1:0]	prv	RW	Privilege level of processor prior to debug exception and to return to at dret.
2	step	RW	Set to 0x1 to single-step.
3	nmip	RO	Non-maskable interrupt pending. Not used by SiFive.
4	mprven	WARL	Not used by SiFive.
5	Reserved		
[8:6]	cause	RO	Indicates cause of most recent debug exception.
9	stoptime	WARL	0x1 will stop timers in debug mode. Not used by SiFive (timers continue).
10	stopcount	WARL	0x1 will stop counters in debug mode. Not used by SiFive (counters continue).
11	stepie	WARL	Enable interrupts when stepping. Not used by SiFive (interrupts disabled).
12	ebreaku	RW	EBREAK instructions in U-mode enter debug mode (vs. breakpoint exception).
13	ebreaks	RW	EBREAK instructions in S-mode enter debug mode.
14	Reserved		
15	ebreakm	RW	EBREAK instructions in M-mode enter debug mode.
[27:16]	Reserved		
[31:28]	xdebugver	RO	Version

Table 205: Debug Control and Status Register

16.2.2 Debug PC (dpc)

When entering debug mode, the current PC is copied here. When leaving debug mode, execution resumes at this PC.

16.2.3 Debug Scratch (dscratch)

This register is generally reserved for use by Debug ROM in order to save registers needed by the code in Debug ROM. The debugger may use it as described in *The RISC-V Debug Specification, Version 1.0*.

16.2.4 Trigger Select Register (tselect)

To support a large and variable number of triggers for tracing and breakpoints, they are accessed through one level of indirection where the `tselect` register selects which bank of three `tdata1-3` registers are accessed via the other three addresses.

The `tselect` register has the format shown below:

Trigger Select Register (tselect)			
CSR	0x7A0		
Bits	Field Name	Attr.	Description
[63:0]	index	WARL	Selection index of triggers

Table 206: Trigger Select Register

The `index` field is a **WARL** field that does not hold indices of unimplemented triggers. Even if `index` can hold a trigger index, it does not guarantee the trigger exists. The `type` field of `tdata1` must be inspected to determine whether the trigger exists.

16.2.5 Trigger Data Registers (tdata1-3)

The `tdata1-3` registers are 64-bit read/write registers selected from a larger underlying bank of triggers by the `tselect` register.

Trigger Data Register 1 (tdata1)			
CSR	0x7A1		
Bits	Field Name	Attr.	Description
[58:0]	Trigger-Specific Data		
59	dmode	WARL	Selects between debug mode (dmode=1) and machine mode (dmode=0) views of the registers, where only debug mode code can access the debug mode view of the triggers
[63:60]	type	WARL	The type of trigger selected by <code>tselect</code> • 0x0 - No such trigger • 0x1-0x5 - Reserved • 0x6 - Address/Data Match Trigger • ≥0x7 - Reserved

Table 207: Trigger Data Register 1

Trigger Data Registers 2 and 3 (tdata2/3)			
CSR	0x7A2 - 0x7A3		
Bits	Field Name	Attr.	Description
[63:0]	Trigger-Specific Data		

Table 208: Trigger Data Registers 2 and 3

Any attempt to read/write the tdata1-3 registers in machine mode when TSELECT.dmode=1 raises an illegal-instruction exception.

16.3 Breakpoints

The Tenstorrent X280 MC supports four hardware breakpoint registers per hart, which can be flexibly shared between debug mode and machine mode.

When a breakpoint register is selected with tselect, the other CSRs access the following information for the selected breakpoint:

CSR Name	Breakpoint Alias	Description
tselect	tselect	Breakpoint selection index
tdata1	mcontrol6	Breakpoint match control
tdata2	maddress	Breakpoint match address
tdata3	N/A	Reserved

Table 209: Trigger CSRs When Used as Breakpoints

16.3.1 Breakpoint Match Control Register (mcontrol6)

Each breakpoint control register is a read/write register laid out in Table 210. This register is accessible as tdata1 when type is 0x6.

Breakpoint Match Control Register (mcontrol6)				
CSR	0x7A1			
Bits	Field Name	Attr.	Rst.	Description
0	R	WARL	0x0	Address match on load
1	W	WARL	0x0	Address match on store
2	X	WARL	0x0	Address match on instruction fetch
3	U	WARL	X	Address match on user mode
4	S	WARL	X	Address match on supervisor mode
5	Reserved	WPRI	X	
6	M	WARL	X	Address match on machine mode
[10:7]	match	WARL	X	Breakpoint Address Match type • 0x0 - Single address • 0x1 - Power-of-2 range, limited to 64 bytes in SiFive implementations • 0x2 - $\geq$ address • 0x3 - $<$ address • Others not supported by SiFive
11	chain	WARL	0x0	Chain adjacent conditions. When set, this trigger and the next must match at the same time to fire. Typically used for a range breakpoint using 2 triggers, one with match=0x2 and one with match=0x3. This is not a sequential trigger.
[15:12]	action	WARL	0x0	Breakpoint action to take
[19:16]	size	WARL	0x0	Size of the breakpoint. Fixed at 0, meaning accesses of any size that cover any part of the trigger address range will fire.
20	timing	WARL	0x0	Timing of the breakpoint. Fixed at 0, meaning breaks happen just before the event.
21	select	WARL	0x0	Perform match on address or data. Fixed at 0, meaning all triggers compare addresses only (no data value).
[58:22]	Reserved			
59	dmode	RW	0x0	Debug-only access mode
[63:60]	type	RO	0x6	Address/Data match type, always 0x6

Table 210: Breakpoint Match Control Register

The type field is a 4-bit read-only field holding the value 0x6 to indicate this is a breakpoint containing address match logic.

The action field is a 4-bit read-write **WARL** field that specifies the available actions when the address match is successful. The value 0 generates a breakpoint exception. The value 1 enters debug mode. The value 2 starts tracing. The value 3 stops tracing. The value 4 inserts a program trace sync message. The value 5 generates a trace external trigger out. Other actions are not implemented.

The R/W/X bits are individual **WARL** fields, and if set, indicate an address match should only be successful for loads, stores, and instruction fetches, respectively. All combinations of implemented bits must be supported.

The M/S/U bits are individual **WARL** fields, and if set, indicate that an address match should only be successful in the machine, supervisor and user modes, respectively. All combinations of implemented bits must be supported.

The match field is a 4-bit read-write **WARL** field that encodes the type of address range for breakpoint address matching. Three different match settings are currently supported: exact, NAPOT, and arbitrary range. A single breakpoint register supports both exact address matches and matches with address ranges that are naturally aligned powers-of-two (NAPOT) in size. Breakpoint registers can be paired to specify arbitrary exact ranges, with the lower-numbered breakpoint register giving the byte address at the bottom of the range and the higher-numbered breakpoint register giving the address 1 byte above the breakpoint range and using the chain bit to indicate both must match for the action to be taken.

NAPOT ranges make use of low-order bits of the associated breakpoint address register to encode the size of the range as follows:

address	Match Type and Size
a...aaaaaa	Exactly 1 byte
a...aaaaa0	2-byte NAPOT range
a...aaaa01	4-byte NAPOT range
a...aaa011	8-byte NAPOT range
a...aa0111	16-byte NAPOT range
a...a01111	32-byte NAPOT range
...	
a01...1111	2 ³¹ -byte NAPOT range

Table 211: NAPOT Size Encoding

maskmax6 determines the largest supported NAPOT range. The value of maskmax6 is the logarithm base 2 of the number of bytes in the largest supported NAPOT range. A value of 0 indicates that only exact address matches are supported (1-byte range). A value of 31 corresponds to the maximum NAPOT range, which is 2³¹ bytes in size. The largest range is encoded in

maddress with the 30 least-significant bits set to 1, bit 30 set to 0, and bit 31 holding the only address bit considered in the address comparison.

The value of maskmax6 is not directly observable, but can be determined via the following sequence:

1. Set match to 1 to select NAPOT mode
2. Read match. If the returned value is not 1, then NAPOT matching is not supported.
3. Write all ones to tdata2
4. Read tdata2. The value of maskmax6 is one more than the index of the most-significant zero bit. For example, if the read value is 0xFFFF_FFF7, bit 3 is zero, so maskmax6 is 4.

To provide breakpoints on an exact range, two neighboring breakpoints can be combined with the chain bit. The first breakpoint can be set to match on an address using match of 2 (greater than or equal). The second breakpoint can be set to match on address using match of 3 (less than). Setting the chain bit on the first breakpoint prevents the second breakpoint from firing unless they both match.

Note

When a chain includes both instruction trigger and data address trigger, the breakpoint does not fire. To work around this limitation, set a data trigger on any access to the data item. Then, in the GDB breakpoint command script, check whether the PC is the one you want and restart if not.

16.3.2 Breakpoint Match Address Register (maddress)

Each breakpoint match address register is a 64-bit read/write register used to hold significant address bits for address matching and also the unary-encoded address masking information for NAPOT ranges.

16.3.3 Breakpoint Execution

Breakpoint traps are taken precisely. Implementations that emulate misaligned accesses in software will generate a breakpoint trap when either half of the emulated access falls within the address range. Implementations that support misaligned accesses in hardware must trap if any byte of an access falls within the matching range.

Debug mode breakpoint traps jump to the debug trap vector without altering machine mode registers.

Machine mode breakpoint traps jump to the exception vector with "Breakpoint" set in the mcause register and with badaddr holding the instruction or data address that caused the trap.

16.3.4 Sharing Breakpoints Between Debug and Machine Mode

When debug mode uses a breakpoint register, it is no longer visible to machine mode (that is, the `tdrtype` will be 0). Typically, a debugger will leave the breakpoints alone until it needs them, either because a user explicitly requested one or because the user is debugging code in ROM.

16.4 Debug Memory Map

This section describes the Debug Module's memory map when accessed via the regular system interconnect. The Debug Module is only accessible to debug code running in debug mode on a hart (or via a Debug Transport Module). The following addresses are offsets from the base address of the Debug Module. Note that the PMP must allow M-mode access to the Debug Module address range for debugging to be possible.

16.4.1 Debug RAM and Program Buffer (0x300–0x3FF)

The Tenstorrent X280 MC has 16 32-bit words of program buffer for the debugger to direct a hart to execute arbitrary RISC-V code. Its location in memory can be determined by executing `aiupc` instructions and storing the result into the program buffer.

The Tenstorrent X280 MC has two 32-bit words of debug data RAM. Its location can be determined by reading the `DM.hartinfo` register, as described in *The RISC-V Debug Specification, Version 1.0*. This RAM space is used to pass data for the Access Register abstract command, as described in *The RISC-V Debug Specification, Version 1.0*. The Tenstorrent X280 MC supports only general-purpose register access when harts are halted. All other commands must be implemented by executing from the debug program buffer.

In the Tenstorrent X280 MC, both the program buffer and debug data RAM are general-purpose RAM and are mapped contiguously in the Core Complex memory space. Therefore, additional data can be passed in the program buffer, and additional instructions can be stored in the debug data RAM.

Debuggers must not execute program buffer programs that access any Debug Module memory except defined program buffer and debug data addresses.

16.4.2 Debug ROM (0x800–0xFFFF)

This ROM region holds the debug routines on SiFive systems. The actual total size may vary between implementations.

16.4.3 Debug Flags (0x100–0x110, 0x400–0x7FF)

The flag registers in the Debug Module are used for the Debug Module to communicate with each hart. These flags are set and read used by the debug ROM and should not be accessed by any program buffer code. The specific behavior of the flags is not further documented here.

16.4.4 Safe Address

In the Tenstorrent X280 MC, the Debug Module contains the Debug Module address range in the memory map. Memory accesses to these addresses raise access exceptions, unless the hart is in debug mode. This property allows a "safe" location for unprogrammed parts, as the default `mtvec` location is `0x0`.

16.5 Instruction Trace Interface

This section describes the interface between a SiFive core and its trace unit. The trace interface conveys information about instruction-retirement and exception events.

Table 213 lists the fields in a trace packet. The `valid` signal is 1 if and only if an instruction retires or traps (either by generating a synchronous exception or taking an interrupt). The remaining fields in the packet are only defined when `valid` is 1.

The `iaddr` field holds the address of the instruction that retired or trapped. If address translation is enabled, it is a virtual address, else it is a physical address. Virtual addresses narrower than 64 bits are sign-extended, and physical addresses narrower than 64 bits are zero-extended.

The `insn` field holds the instruction that retired or trapped. For instructions narrower than the maximum width, e.g., those in the RISC-V C extension, the unused high-order bits are zero-filled. The length of the instruction can be determined by examining the low-order bits of the instruction, as described in *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.1*. The width of the `insn` field, `ILEN`, is 32 bits for current implementations.

The `priv` field indicates the privilege mode at the time of instruction execution. On an exception, the **next** valid trace packet's `priv` field gives the privilege mode of the activated trap handler. The width of the `priv` field, `PRIVLEN`, is 3, and it is encoded as shown in Table 212.

Value*	Description
0x0	User mode
0x1	Supervisor mode
0x3	Machine mode
0x4-0x7	Debug mode
*Unspecified values are reserved.	

Table 212: Encoding of `priv`

The `exception` field is 0 if this packet corresponds to a retired instruction, or 1 if it corresponds to an exception or interrupt. In the former case, the `cause` and `interrupt` fields are undefined and the `tval` field is zero. In the latter case, the fields are set as follows:

- `interrupt` is 0 for synchronous exceptions and 1 for interrupts.
- `cause` supplies the exception or interrupt cause, as would be written to the lower `CAUSELEN` bits of the `mcause` CSR. For current implementations, `CAUSELEN` is $\log_2(64)$.

- `tval` supplies the associated trap value, e.g., the faulting virtual address for address exceptions, as would be written to the `mtval` CSR.

Future optional extensions may define `tval` to provide ancillary information in cases where it currently supplies zero.

For cores that can retire `N` instructions per clock cycle, this interface is replicated `N` times. Lower-numbered entries correspond to older instructions. If fewer than `N` instructions retire, the valid packets need not be consecutive, i.e., there may be invalid packets between two valid packets. If one of the instructions is an exception, no younger instruction will be valid.

Name	Description
<code>valid</code>	Indicates an instruction has retired or trapped.
<code>iaddr[63:0]</code>	The address of the instruction.
<code>insn[31:0]</code>	The instruction.
<code>priv[2:0]</code>	Privilege mode during execution.
<code>exception</code>	0, if the instruction retired; 1, if it trapped.
<code>interrupt</code>	0, if the exception was synchronous; 1, if interrupt.
<code>cause[5:0]</code>	Exception cause.
<code>tval[63:0]</code>	Exception data.

Table 213: SiFive Instruction Trace Interface Fields

16.6 Debug Module Interface

The SiFive Debug Module (DM) conforms to *The RISC-V Debug Specification, Version 1.0*. A debug probe or agent connects to the Debug Module through the Debug Module Interface (DMI). The following sections describe notable spec options used in the implementation and should be read in conjunction with *The RISC-V Debug Specification, Version 1.0*.

DMI is a simple read/write bus whose master is the DTM (if it exists, otherwise DMI passes through to customer logic) and whose slave is the Debug Module. The master sends a request to the slave and the slave responds with a response. A request is considered sent if `req_ready=1` indicating the master is sending a request and `req_valid=1` indicating the slave is accepting the request on this cycle. Similarly, the response is sent when both `resp_valid=1` indicating the slave is sending a response and `resp_ready=1` indicating the master is accepting it.

Note

It is the responsibility of the debugger to simulate virtual address accesses by accessing the page tables directly, then sending the translated physical address to hardware when doing the access.

Note

The Debug Module registers are not directly accessible from the core.

Group	Signal	Source	Description
System	clock	system	All signals timed to this clock. With JTAG DTM, this clock is the JTAG TCK.
	reset	system	Synchronous reset. Generated by power-on reset circuit.
Request Bus	req_ready	slave	Slave ready to receive request.
	req_valid	master	Master's request valid.
	req_addr	master	Configurable width address bus. 0x7 for SiFive.
	req_data	master	32-bit write data bus.
	req_op	master	• 0x0 = None • 0x1 = Read • 0x2 = Write • 0x3 = Reserved
Response Bus	resp_ready	master	Master is ready to receive response.
	resp_valid	slave	Slave response is valid.
	resp_data	slave	32-bit read data bus.
	resp_op	slave	• 0x0 = Success • 0x1 = Failure • 0x2 = Not used • 0x3 = Reserved

Table 214: Debug Module Interface Signals

16.6.1 Debug Module Status Register (dmstatus)

dmstatus holds the DM version number and other implementation information. Most importantly, it contains status bits that indicate the current state of the selected hart(s).

Debug Module Status Register (dmstatus)				
DMI Address		0x11		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	version	RO	0x3	Implementation version number
4	Reserved	RO	0x0	
5	hasresethaltreq	RO	0x1	1 if resethaltreq exists
[7:6]	Reserved	RO	0x0	
8	anyhalted	RO	0x0	Any currently selected hart is halted. Note that this field is volatile and may be masked in IP-XACT.
9	allhalted	RO	0x0	All currently selected harts are halted. Note that this field is volatile and may be masked in IP-XACT.
10	anyrunning	RO	0x1	Any currently selected hart is running. Note that this field is volatile and may be masked in IP-XACT.
11	allrunning	RO	0x1	All currently selected harts are running. Note that this field is volatile and may be masked in IP-XACT.
12	anyunavail	RO	0x0	Any currently selected hart is not available (i.e., is powered down). DM supports it, but not currently used by SiFive cores. Note that this field is volatile and may be masked in IP-XACT.
13	allunavail	RO	0x0	All currently selected harts are not available (i.e., is powered down). DM supports it, but not currently used by SiFive cores. Note that this field is volatile and may be masked in IP-XACT.
14	anynonexistent	RO	0x0	Any currently selected hart does not exist in the system. Note that this field is volatile and may be masked in IP-XACT.
15	allnnonexistent	RO	0x0	All currently selected harts do not exist in the system. Note that this field is volatile and may be masked in IP-XACT.
16	anyresumeack	RO	0x1	Any currently selected hart has resumed execution. Note that this field is volatile and may be masked in IP-XACT.
17	allresumeack	RO	0x1	All currently selected harts have resumed execution. Note that this field

Table 215: Debug Module Status Register

				is volatile and may be masked in IP-XACT.
18	anyhavereset	RO	0x0	Any currently selected hart has been reset, but reset has not been acknowledged. Note that this field is volatile and may be masked in IP-XACT.
19	allhavereset	RO	0x0	All currently selected harts have been reset, but reset has not been acknowledged. Note that this field is volatile and may be masked in IP-XACT.
[21:20]	Reserved	RO	0x0	
22	impebreak	RO	0x0	1 if PROGBUF is followed by implicit EBREAK. Generally, 1 for E2 cores, 0 otherwise.
[31:23]	Reserved	RO	0x0	

Table 215: Debug Module Status Register

16.6.2 Debug Module Control Register (dmcontrol)

A debugger performs most hart controls through the dmcontrol register.

Debug Module Control Register (dmcontrol)				
DMI Address		0x10		
Bits	Field Name	Attr.	Rst.	Description
0	dmactive	RW	0x0	0 disables the DM and sets DMI registers to their reset state, 1 puts the DM in operational mode. Drives dmactive output that could be used by a system power controller to maintain power to the DM while it is being used. When 1, dmcontrol should be read back until dmactive=1, which indicates that the Debug Module is fully operational. When 0, the DM TileLink clock is gated off to save power.
1	ndmreset	RW	0x0	Write 1 to reset system (assert ndreset output). Write 0 to operate normally.
2	clrresethaltreq	WO	0x0	Write 1 to clear the reset-halt-request bit
3	setresethaltreq	WO	0x0	When written to 1, the core will halt upon the next deassertion of its reset
[15:4]	Reserved	RO	0x0	
[25:16]	hartsel	RW	0x0	Selects the hart to operate on
26	hase1	RW	0x0	Selects hart(s) in the hart-array mask register (hawindow)
27	Reserved	RO	0x0	
28	ackhavereset	WO	0x0	Write 1 to acknowledge that a reset occurred on the selected hart
29	Reserved	RO	0x0	
30	resumereq	WO	0x0	Write 1 to request selected hart to resume, cleared to 0 automatically when hart resumes
31	haltreq	RW	0x0	Write 1 to request selected hart to halt. Generates debug interrupt to the core. Write 0 once halted has been set by the DM.

Table 216: Debug Module Control Register

16.6.3 Hart Info Register (hartinfo)

hartinfo contains information about the currently selected hart.

Hart Info Register (hartinfo)				
DMI Address		0x12		
Bits	Field Name	Attr.	Rst.	Description
[11:0]	dataaddr	RO	0x380	Address of DATA registers in hart memory map. 0x380 for SiFive.
[15:12]	datasize	RO	0x2	Number of DATA registers. 0x1 for 32-bit, 0x2 for 64-bit SiFive cores.
16	dataaccess	RO	0x1	DATA registers are shadowed in the hart memory map. 1 for SiFive.
[19:17]	Reserved	RO	0x0	
[23:20]	nscratch	RO	0x1	Number of dscratch registers available for debugger. 1 for SiFive.
[31:24]	Reserved	RO	0x0	

Table 217: Hart Info Register

16.6.4 Hart Array Window Register (hawindow)

This register contains a bitmap where bit 0 corresponds to hart 0, bit 1 to hart 1, etc. Any bits set in this register select the corresponding hart in addition to the hart selected by `dmcontrol.hartsel`.

16.6.5 Abstract Control and Status Register (abstractcs)

Abstract Control and Status Register (abstractcs)				
DMI Address		0x16		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	datacount	RO	0x2	Number of DATA registers. 0x1 for 32-bit, 0x2 for 64-bit SiFive cores.
[7:4]	Reserved	RO	0x0	
[10:8]	cmderr	RW1C	0x0	Non-zero value indicates an abstract command error. Remains set until cleared by writing all ones. If set, no abstract commands are accepted. • 0x0 - No error • 0x1 - Busy. Abstract command or register was accessed while command was running. • 0x2 - Not supported. Abstract command type not supported by hardware was attempted. • 0x3 - Exception. An exception occurred during execution of an abstract command. • 0x4 - Halt/resume. Abstract command attempted while hart was running or unavailable. • 0x5 - Bus. Bus error occurred during abstract command. Not used by SiFive. • 0x7 - Other. Abstract command failed for another reason. Not used by SiFive. Note that this field is volatile and may be masked in IP-XACT.
11	Reserved	RO	0x0	
12	busy	RO	0x0	Reads as 1 while Abstract command is running, 0 if not. Note that this field is volatile and may be masked in IP-XACT.
[23:13]	Reserved	RO	0x0	
[28:24]	progbufsize	RO	0x10	Number of 32-bit words in PROGBUF. Tenstorrent X280 MC has 16 words.
[31:29]	Reserved	RO	0x0	

Table 218: Abstract Control and Status Register

16.6.6 Abstract Command Register (command)

Abstract Command Register (command)			
DMI Address		0x17	
Bits	Field Name	Attr.	Description
[15:0]	regno	RW	Select which register to read/write. SiFive only supports GPRs: 0x1000-0x101F.
16	write	RW	1=write register, 0=read register. Only done if transfer=1.
17	transfer	RW	1=do the register read/write, 0=don't.
18	postexec	RW	1=execute PROGBUF after the command, 0=don't.
19	aarpostincrement	RW	Not supported by SiFive.
[22:20]	aarsize	RW	0x2, 0x3, 0x4 select 32, 64, 128 bits, respectively.
23	Reserved	RO	
[31:24]	cmdtype	RW	0=Access Register is the only type supported by SiFive.

Table 219: Abstract Command Register

Note that this register is volatile and may be masked in IP-XACT.

16.6.7 Abstract Command Autoexec Register (abstractauto)

Abstract Command Autoexec Register (abstractauto)				
DMI Address		0x18		
Bits	Field Name	Attr.	Rst.	Description
[11:0]	autoexecdata	RW	0x0	Bitmap of DATA registers [11:0]. 1 indicates DATA access initiates command.
[15:12]	Reserved	RO	0x0	
[31:16]	autoexecprogbuf	RW	0x0	Bitmap of PROGBUF words [15:0]. 1 indicates PROGBUF access initiates command.

Table 220: Abstract Command Autoexec Register**16.6.8 Debug Module Control and Status 2 Register (dmcs2)**

Table 221 describes the Debug Module Control and Status 2 Register dmcs2. If halt/resume groups are not implemented, then group will always read back as 0. The Debug Module external triggers may be allocated as needed between halt and resume groups.

Debug Module Control and Status 2 Register (dmcs2)				
DMI Address		0x32		
Bits	Field Name	Attr.	Rst.	Description
0	hgselect	RW	0x0	0=operate on harts, 1=operate on external triggers.
1	hgwrite	WO	X	When written with 1, the selected harts or external trigger is assigned to group group.
[6:2]	group	RW	0x0	Specify the halt group or resume group number that the selected harts or external triggers will be assigned to. Note that this field is volatile and may be masked in IP-XACT.
[10:7]	exttrigger	RW	0x0	Select which external trigger to act upon if hgwrite and hgselect are written to 1 in the same write. Note that this field is volatile and may be masked in IP-XACT.
11	grouptype	RW	0x0	0=operate on Halt Group configuration, 1=operate on Resume Group configuration.
[31:12]	Reserved	RO	0x0	

Table 221: Debug Module Control and Status 2 Register

16.6.9 Abstract Commands

Abstract commands provide a debugger with a path to read and write processor state and are used for extracting and modifying processor state such as registers and memory. Register `s0` is saved by the ROM and is available for use by the abstract command code. An abstract command is started by the debugger writing to `command`. In `command`, the debugger selects whether to load/store a register, execute `PROGBUF`, or both. Only GPR register transfers are supported currently. Many aspects of Abstract Commands are optional in *The RISC-V Debug Specification, Version 1.0* and are implemented as described below.

cmdtype	Feature	Support
Access Register	GPR registers	Access Register command, register number 0x1000 - 0x101F
	CSR registers	Not supported. CSRs are accessed using the Program Buffer.
	FPU registers	Not supported. FPU registers are accessed using the Program Buffer.
	Autoexec	Both autoexecprogbuf and autoexecdata are supported.
	Post-increment	Not supported.
	Core Register Access	Not supported.
Quick Access		Not supported.
Access Memory		Not supported. Memory access is accomplished using the Program Buffer.

Table 222: Debug Abstract Commands

The use of abstract commands is outlined in the following example, describing how to read a word of target memory:

1. The debugger writes opcodes to PROGBUF to accomplish the desired function.
2. The debugger writes the desired memory address to DATA[0].
3. The debugger requests an abstract command specifying to load s0 from DATA[0], then execute PROGBUF. Writing to command while hart n is selected has the side effect of setting FLAGS[n].go. Writing to command also sets busy which is readable from the debugger, and indicates that an abstract command is in progress.
4. The ROM busy-wait loop being executed by hart n sees FLAGS[n].go set.
5. ROM code writes 0 to GOING which has the effect of clearing FLAGS[n].go.
6. ROM code jumps to WHERETO, then ABSTRACT which contains the opcode lw s0, 0(DATA) to load s0 from DATA[0]. Opcodes in ABSTRACT are constructed by DM hardware from command. If command.transfer=0, no register transfer is done and instead ABSTRACT[0] reads as NOP.
7. If a register read/write is all that is needed, the debugger would set command.postexec to 0. ABSTRACT[1] would then read as EBREAK.
8. If command.postexec=1, ABSTRACT[1] reads as NOP and execution falls through to PROGBUF which will have been previously written by the debugger with the opcodes lw s0, 0(s0), then sw s0, DATA(zero), then EBREAK.
9. EBREAK reenters ROM at address 0x800. ROM writes hartid to HALTED which has the side effect of clearing busy, telling the debugger that the abstract command is finished.
10. The debugger reads the result from DATA[0].

The autoexec feature of Abstract Commands is supported by SiFive hardware (and is used by OpenOCD for memory block read and write). Once an abstract command has been completed, the debugger can read or write a particular DATA or PROGBUF location to run the command again. For example, fast download can be accomplished by setting up PROGBUF for memory write, then repeatedly writing words to DATA[0]. Each write re-executes the register transfer and PROGBUF to store the word into memory. For a 32-bit block write, the abstract command would be set up like this:

ABSTRACT	regno=s1, write=1, transfer=1, postexec=1. DM constructs the instructions <pre>lw s1,0(DATA) // load s1 from debugger NOP // fall thru to PROGBUF</pre>
PROGBUF	<pre>sw s1, 0(s0) // store s1 to memory addi s0, s0, 4 // increment memory pointer ebreak // done</pre>

Table 223: Abstract Command Example for 32-bit Block Write

16.6.10 Multi-core Synchronization

The DM is configured with one Halt Group that may be programmed to synchronize execution between harts, or between hart(s) and external logic, such as a cross-trigger matrix. The Halt Group is configured using the dmcs2 register.

16.6.10.1 Hart Array

The Hart Array is an internal bitmap that selects a subset of the harts in a system. Debug operations such as resume and halt are automatically applied to all the selected harts simultaneously.

To configure the Hart Array:

1. Set the hase1 bit in the Debug Module dmcontrol register.
2. Set bits in the hawindow register corresponding to the harts to be selected. Bit 0 = hart 0, Bit 1 = hart 1, etc.

The Hart Array covers debug operations initiated by the debugger but does not cover the case when harts halt due to other causes, such as breakpoints. This is handled with a Halt Group.

16.6.10.2 Halt Groups and External Triggers

Debug Module External Triggers consist of a pair of signals (request and acknowledgement) leading from the DM to external logic and another pair leading from external logic to the DM.

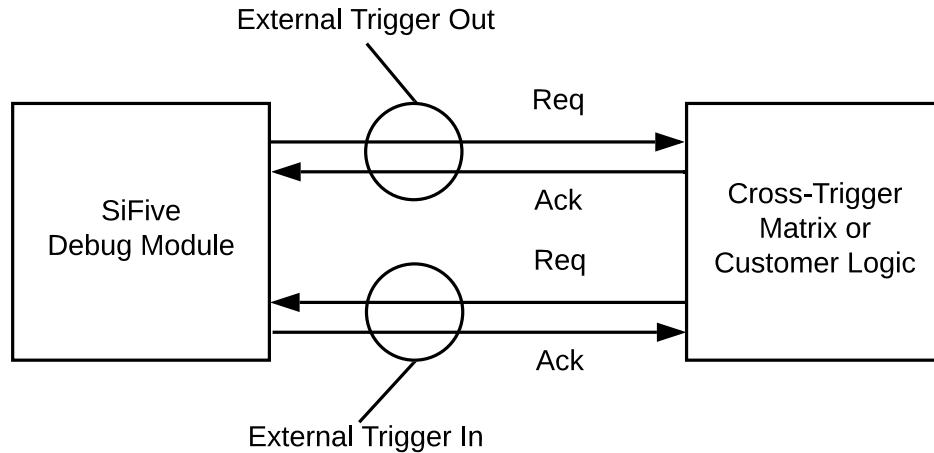


Figure 168: Debug External Triggers

When a system has more than one hart or is configured with any External Triggers, the Debug Module is configured with one Halt Group and one Resume Group. Any subset of the harts and External Triggers present in the system may be set up to participate in the Halt Group and, independently, to participate in the Resume Group.

Setting up a halt group or resume group requires a series of writes to the dmcs2 register. For example:

```
dmcs2 <- 0x0006 // Add all selected harts to halt group 1
dmcs2 <- 0x0087 // Add external trigger 1 to halt group 1
dmcs2 <- 0x0806 // Add all selected harts to resume group 1
dmcs2 <- 0x0807 // Add external trigger 0 to resume group 1
```

At this point, the user can halt and resume the selected harts by writing the `haltreq` and `resumereq` bits. If any of the harts in the halt group halts either due to writing `haltreq` or due to hitting a breakpoint, hardware in the Debug Module will generate a `haltreq` to all the other harts in the group, and it will assert `ExtTriggerOutReq[1]`. If any of the harts in the resume group resumes, hardware will resume all the other harts in the group, and it will assert `ExtTriggerOutReq[0]`. Conversely, an external agent can assert `ExtTriggerInReq[0]` to cause all harts in the resume group to resume or assert `ExtTriggerInReq[1]` to cause all harts in the halt group to halt.

The trigger request is generally synchronized to the receiver's clock domain, then acknowledged by reflecting the synchronized request back to the corresponding acknowledge.

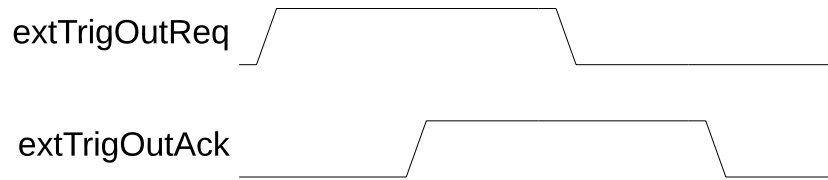


Figure 169: Debug External Trigger Handshake

When an external trigger is included in a Halt Group, any core in the group that halts for any reason causes the external trigger output to fire. Similarly, when an external trigger input that is configured into the Halt Group fires, it causes all the cores in the Halt Group to halt.

16.6.10.3 Using External Triggers

The following section provides an example of how to use the Debug Module External Triggers. For example, this code sets up trigger 0 on a store to an address `0x12345678` with action “raise breakpoint exception”:

```
csrw 0x7a0, 0x00000000 // select trigger 0
csrw 0x7a2, 0x12345678 // program trigger address comparison
csrw 0x7a1, 0x2000000000000005a // Action 0, MSU, Store
```

Debug external triggers connect to the Debug Module and can be programmed to send an event when the core halts (enters debug mode) or to cause the core to halt when an external trigger event is received.

Note

Debug external triggers cannot be generated by an M-mode trigger and cannot be programmed to cause an M-mode breakpoint.

16.6.11 System Bus Access

System Bus Access (SBA) provides an alternative method to access memory. SBA operation conforms to *The RISC-V Debug Specification, Version 1.0* and its description is not duplicated here. It implements a bus master that connects with the bus crossbar to allow access to the device’s physical address space without involving a hart to perform accesses. SBA is controlled from the DMI using registers in the range `0x37 - 0x3F`. By default, the maximum bus width supported by SBA is 64. Comparing Program Buffer memory access and SBA:

Program Buffer Memory Access	SBA Memory Access
Physical Address	Physical Address
Subject to Physical Memory Protection (PMP)	Not subject to PMP
Cache coherent	Cache coherent
Hart must be halted	Hart may be halted or running

Table 224: System Bus vs. Program Buffer Comparison

16.7 Debug Module Operational Sequences

The sections below describe the flow for entering into and exiting from debug mode. The user can halt and resume more than one hart at a time using the hart array mask.

16.7.1 Entering Debug Mode

To use debug mode, the DM must be enabled by writing `0x0000_0001` to `dmcontrol`.

The debugger can request a halt by writing `0x8000_0001` to `dmcontrol` to set `haltreq`. This generates a debug interrupt to the core.

The core enters debug mode and jumps to the debug interrupt handler located at `0x800` and serviced from the DM.

ROM code at `0x800` writes `hartid` into the `HALTED` register which has the effect of setting the halted bit for this hart. Halted bits are readable from the debugger and generally will be continually polled to check for breakpoints when a hart is running.

ROM code then busy-waits checking its hart-specific `FLAGS` register.

16.7.2 Exiting Debug Mode

The debugger writes `1` to `resumereq` in the `dmcontrol` register to restart execution. This clears `resumeack` and sets bit `1` of the `FLAGS` register for the selected hart.

The ROM busy-wait loop being executed by hart `n` sees `FLAGS[n].resume` set.

ROM code writes `hartid` to `RESUMING`, which has the effect of clearing `FLAGS[n].resume`, setting `resumeack`, and clearing `halted` for the hart.

ROM code then executes `dret` which returns to user code at the address currently in `dpc`.

The debugger sees `resumeack` and knows the resume was successful.

16.8 Debug and Core Complex Power Gating

The sections below describe the behavior of the Debug Module as it relates to core-complex clock gating.

16.8.1 Individual Core Debug Availability

As a core prepares for power gating, it may execute a software sequence terminated with the WFI instruction. Retirement of the WFI instruction causes the `wfi_from_tile_N` signal to be asserted and also transitions the core debug status to "Unavailable" in the `dmstatus` register of the Debug Module in the uncore domain that remains powered. Cores that terminate instruction execution with WFI remain available for debug until the core-complex power is removed.

16.8.2 Core Complex Debug Availability

When any number of cores, but not the entire core-complex have been powered off, the Debug Module remains powered and can access powered cores and uncore state. When the core-complex is powered off, the Debug Module is also powered off, but the `DebugTransportModule` (DTM) remains powered to respond to JTAG transactions (if JTAG is used). Accesses to state within the core-complex receive an error, and accesses to state outside the core-complex are forwarded as normal.

The core-complex debug interface offers either APB or JTAG/cJTAG options. When the core-complex is being powered off and APB is chosen, isolation clamp values terminate all APB accesses. When JTAG/cJTAG is chosen, the core-complex power domain boundary is modified so the `DebugTransportModule` (DTM) remains powered by the ungated core-complex supply. This allows JTAG operations to be forwarded while the core-complex is powered off.

Chapter 17

SiFive Trace System

17.1 Introduction

Instruction Trace is a system that collects a history of processor execution, along with other events. The trace system is set up and controlled using a TileLink interface, typically connected to the Peripheral Crossbar. Core execution activity appears on the Trace Bundle and is compressed and formatted into trace messages which are transmitted via TileLink to a Trace Sink. The system includes an SRAM sink with a 16 KiB on-chip trace buffer, an off-chip interface to a trace probe, and a System Memory sink. Each core has its own Trace Encoder and, typically, all will connect to a single Trace Funnel, which aggregates trace data from multiple sources and sends it to a single destination.

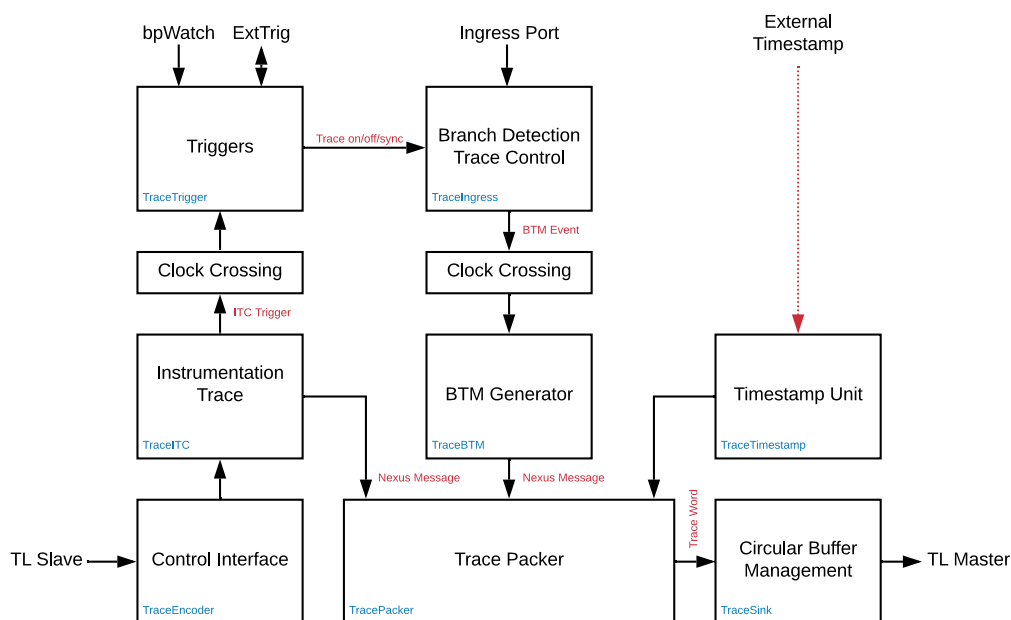


Figure 170: Trace Encoder Block Diagram

17.2 Trace Control Interface

The Trace Encoder (TE) control interface is a TileLink slave node typically connected to the CBUS crossbar, so it appears in the system physical memory space. The control interface is used to set up and control a trace session, generate messages from instrumentation writes, retrieve collected trace history and control any trace system components directly connected to this Trace Encoder.

If the Debug Module is implemented with System Bus Access (SBA), a JTAG probe can control and monitor the trace system without requiring intervention by any of the cores.

Mapping the control interface into physical memory also allows one of the cores to manage a trace session—for example, collecting crash information in the field—or to modify trace collection parameters during execution. If desired, a Physical Memory Protection (PMP) range can be configured to restrict access to the trace system from core(s). If PMP is used, the ITC stimulus registers would normally be enabled for access.

The Trace Funnel (TF) is a variant of the Trace Encoder and shares many of the same registers. The Trace Funnel occupies its own 4K block in the system memory space.

The Tenstorrent X280 MC includes multiple Trace Encoders. Each Trace Encoder feeds a single Trace Funnel.

Offset	TE Name	TF Name	Description
0x0000	teControl	tfControl	Trace Encoder/Funnel control register
0x0004	teImpl	tfImpl	Trace Encoder/Funnel implementation information
0x0010	teSinkBase	teSinkBase	Base address of circular trace buffer
0x0014	teSinkBaseHigh	teSinkBaseHigh	Bits [N:32] of the circular buffer address where N is equal to the most significant bit of the physical address.
0x0018	teSinkLimit	teSinkLimit	End address of circular trace buffer
0x001C	teSinkWP	teSinkWP	Current write location for trace data in circular buffer
0x0020	teSinkRP	teSinkRP	Access pointer for SRAM readback
0x0024	teSinkData	teSinkData	Read/write access to SRAM
0x0040	tsControl	tsControl	Timestamp control register
0x0044	tsLower	tsLower	Lower 32 bits of timestamp counter
0x0048	tsUpper	tsUpper	Upper bits of timestamp counter
0x0050	xtiControl	–	External Trigger Input control register
0x0054	xtoControl	–	External Trigger Output control register
0x0058	wpControl	–	Core watchpoint control register
0x0060	itcTraceEnable	–	ITC trace enable bits, 1 bit per virtual stimulus register
0x0064	itcTrigEnable	–	ITC trigger enable bits, 1 bit per virtual stimulus register
0x0080–0x00FF	itcStimulus	–	Instrumentation Trace stimulus 0 to 31
0x0100–0x01FF	pcSample	–	Register interface to PC Sampling Unit, if present
0x0F00–0x0FFF	pibSink	pibSink	Control registers for PIB trace sink

Table 225: Trace Encoder Memory Map

17.3 Instruction Trace Encoder

17.3.1 Ingress Port

The Instruction Trace Encoder (ITE) monitors the activity of the core and determines whether and what type of branches are taken. ITE encodes a trace message for each taken branch or exception. Trace Messages conform to the IEEE-ISTO 5001-2012 (Nexus) Standard.

Table 226 shows the interface from the core to the Trace Encoder.

Trace Spec Signal	TE Signal	Description
–	clock	Core clock
reset	reset	Core is in reset (does not reset TE)
iretire	valid	1 indicates an instruction has retired
iexception	exception	1 indicates an exception
interrupt	interrupt	1 indicates an interrupt
iaddr[63:0]	iaddr[63:0]	If iretire=1, the instruction address
instr[31:0]	insn[31:0]	If iretire=1, the instruction opcode
priv[2:0]	priv[2:0]	If iretire=1, the privilege mode: • 0x0 - User mode • 0x1 - Supervisor mode • 0x3 - Machine mode • 0x4-0x7 - Debug mode
–	bpwatch[n:0]	Core watchpoint actions, 4 bits per watchpoint: • valid: this watchpoint is matching in this cycle • action[2:0]: action field for this watchpoint
stall	teStall	Stall input to core

Table 226: Core to Trace Encoder Interface

17.3.2 In-Circuit Trace Message

In certain conditions, the Trace Encoder needs to report the address of the most recent retired instruction. The In-circuit Trace message (ICT) is used for this reporting. The address is either U-ADDR or F-ADDR and this is indicated by the TCODE (either ICT or ICT with Sync). When the message is ICT, the address is compressed and refers to the full address of the most recent message with an address and the timestamp is compressed normally. When the Sync form of ICT is used, the address is F-ADDR and, the timestamp is uncompressed. However, ICT with Sync cannot be used by the trace decoder as a starting point because there may be previous ICNT history that cannot be reported with ICT.

CKSRC	Description	Notes
8	External Trigger	Used when an external trigger with action=4 occurs at a time when no instructions are being retired or when a PTSYNC External Trigger message cannot be sent due to encoder resource limitation. Always uses ICT.
14	Watchpoint	Used to report a watchpoint with action=4 that occurs at a time when a PTSYNC Watchpoint message cannot be sent due to encoder resource limitation. The watchpoint is noted and sent as soon as possible. ICT is used for this reporting when there are no instructions being retired. Always uses ICT.
15	PC sample	Message sent when TE is in Periodic Sample mode (teInstruction=1). Normally uses ICT, but an ICT with Sync is sent at a rate determined by teSyncMaxBTM.

Table 227: CKSRC Description

17.3.3 TE Trace Packer

The Trace Ingress Port operates in the core clock domain. Minimal processing is done prior to crossing into the `external_source_for_trace_com_clock` domain, where BTMs are generated. If `external_source_for_trace_com_clock` and `external_source_for_trace_N_clock` are the same, then this async crossing is omitted.

The Trace Packer receives the BTM and packs its Nexus slices (see below) into words. Completed words are sent to the TL master interface. If a message ends in the middle of a word and no additional messages are pending, then the Packer retains the partial word until another message is received. The Packer also receives formatted Nexus messages from the ITC unit and watchpoints. BTM has priority, followed by watchpoints, then ITC. The Packer may generate more than one trace word per cycle; these are stored in a shift register until the TileLink master can consume them. When TE is disabled (`teEnable` → 0), any partial word left in the trace packer is filled with idle slices and sent.

The Trace Packer adds timestamps to the formatted Nexus messages prior to saving in the shift register. Timestamps are added at this point so that the timestamp compression thread follows through all message types, whether BTM, ITC, or watchpoints.

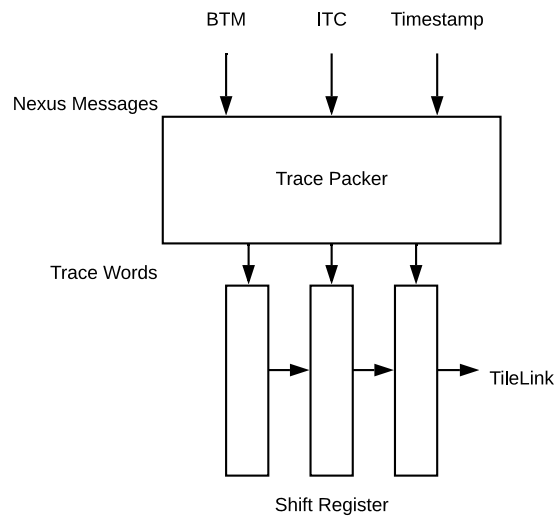


Figure 171: Trace Packer

17.3.4 Register Interface

This section describes the functionality of the Trace Encoder registers.

Trace Encoder Control Register (<code>teControl</code>)				
Address Offset		0x0000		
Bits	Field Name	Attr.	Rst.	Description
0	<code>teActive</code>	RW	0x0	Master enable for trace system. 0 resets the TE and clocks may be gated off. <code>teActive</code> must be set to 1 before any other registers (except <code>pcControl</code>), including other fields in <code>teControl</code> , are programmed.
1	<code>teEnable</code>	RW	0x0	1=TE enabled. Allows <code>teTracing</code> to turn all tracing on and off. Setting <code>teEnable</code> to 0 flushes any queued trace data to the designated sink.
2	<code>teTracing</code>	RW	0x0	1=Trace is being generated. Written from tool or controlled by triggers. Note that this field is volatile and may be masked in IP-XACT.
3	<code>teEmpty</code>	RO	0x1	Reads as 1 when all generated trace has been emitted. Note that this field is volatile and may be masked in IP-XACT.

Table 228: Trace Encoder Control Register

Trace Encoder Control Register (teControl1)				
[6:4]	teInstruction	WARL	0x0	Instruction trace generation parameters: • 0x0 - Don't generate BTM or Sync (no instruction trace) • 0x1 - Event trace mode • 0x2 - Reserved • 0x3 - Reserved • 0x4-0x5 - Reserved • 0x6 - Generate HTM and Sync without HTM optimizations • 0x7 - Generate HTM and Sync with full optimization
[8:7]	teInstrumentation	RW	0x0	ITC trace generation parameters for itcStimulus registers whose corresponding itcTraceEnable=1. • 0x0 - ITC trace disabled • 0x1 - Generate ITC messages for all itcStimulus registers • 0x2 - Generate Ownership message for itcStimulus[15, 31], disable other ITC trace • 0x3 - Generate Ownership message for itcStimulus[15, 31] and ITC messages for all other itcStimulus registers
[11:9]	Reserved	RO	0x0	
12	teStallOrOverflow	RW	0x0	Written to 1 by hardware when an overflow message is generated or when the TE requests a core stall. Debugger can write 0 to clear. Note that this field is volatile and may be masked in IP-XACT.
13	teStallEnable	RW	0x0	0=If TE cannot accept a message, an overflow is generated. 1=If TE cannot accept a message, the core is stalled until it can.
14	teStopOnWrap	RW	0x0	Disable trace (teEnable → 0) when circular buffer fills for the first time.

Table 228: Trace Encoder Control Register

Trace Encoder Control Register (teControl1)				
				Present only in systems with SRAM or System Memory sinks.
15	teInhibitSrc	RW	0x0	1=Disable SRC field in Nexus messages. Unless disabled, a SRC field of teImpl.nSrcBits is added to every Nexus message to indicate which TE generated each message.
[19:16]	teSyncMaxBTM	WARL	0x3	Maximum number of BTMs between periodic Sync messages. A Sync emitted for another reason will reset this timer. 0x0-0xB - Generate Sync after $2^{(teSyncMaxBTM + 5)}$ BTM's. Range is 32 to 65536. Default is 256. 0xC-0xE - Reserved 0xF - Off. Do not generate a Sync message due to a count of BTMs.
[23:20]	teSyncMaxInst	WARL	0xA	Maximum I - CNT between BTMs. A BTM emitted for another reason will reset this timer. 0x0-0x5 - Generate Sync when running I-CNT reaches $2^{(teSyncMaxInst + 5)}$ bytes. Range is 32 to 1024 bytes. Default is 1024. 0x6-0xE - Reserved 0xF - Off. Do not generate a Sync message due to instruction or cycle count.
[26:24]	teSliceFormat	RO	0x1	Nexus Slice Format 0x0 - Reserved 0x1 - 6 MDO + 2 MSEO 0x2-0x7 - Reserved
27	teSinkError	RW1C	0x0	Hardware sets this to 1 when a trace sink TileLink transaction results in an error. This can happen with a System Memory sink when the address or size is invalid or with PIB, ATB, and Funnel sinks when the destination device is

Table 228: Trace Encoder Control Register

Trace Encoder Control Register (teControl1)				
				disabled. Write 1 to clear. Note that this field is volatile and may be masked in IP-XACT.
[31:28]	teSink	WARL	0x8	Which sink to send trace to. If there is only one sink type, this field may read as either 0 or that sink's type number: • 0x0 - Send trace to the one sink connected • 0x1-0x3 - Reserved • 0x5 - SRAM Sink • 0x5 - ATB Sink • 0x6 - PIB Sink • 0x7 - SBA Sink • 0x8 - Funnel Sink • 0x9-0xF - Reserved

Table 228: Trace Encoder Control Register

Trace Encoder Implementation Register (teImp1)				
Address Offset		0x0004		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	version	RO	0x0	TE Version (0 is this version)
4	hasSRAMSink	RO	0x0	1 if this TE has an on-chip SRAM sink. Size of SRAM may be determined by writing all 1s to teSinkWP, then reading the value back.
5	hasATBSink	RO	0x0	1 if this TE has an ATB sink.
6	hasPIBSink	RO	0x0	1 if this TE has an off-chip trace port via a Probe Interface Block.
7	hasSBASink	RO	0x0	1 if this TE has an on-chip bus master through the Front Port.
8	hasFunnelSink	RO	0x1	1 if this TE feeds into a Trace Funnel device.
[15:9]	Reserved	RO	0x0	
[17:16]	sinkBytes	RO	0x0	Width of this TE's trace sink. • 0x0 - 32-bit • 0x1 - 64-bit • 0x2 - 128-bit • 0x3 - 256-bit
[19:18]	crossingType	RO	preset	Type of clock crossing between Ingress and BTM Generator. • 0x0 - Synchronous • 0x1 - Rational • 0x2 - Asynchronous
[23:20]	srcID	RO	preset	This TE's source ID. If nSrcBits>0 and SRC is not disabled by teInhibitSrc, then Nexus messages will all include a SRC field of nSrcBits bits. Messages from this TE will have this value in SRC.
[26:24]	nSrcBits	RO	preset	The number of bits in the Nexus SRC field, unless disabled by teInhibitSrc.
27	Reserved	RO	0x0	
[31:28]	hartid	RO	preset	The hartid of the hart this TE is connected to. Note that this field is volatile and may be masked in IP-XACT.

Table 229: Trace Encoder Implementation Register

17.4 Instrumentation Trace Component (ITC)

ITC appears as a block of 32×32 -bit stimulus registers in the Trace Encoder control interface.

A bitmap enable register selectively enables any subset of ITC registers to cause trace or trigger events. If trace is enabled, any write to an ITC stimulus register generates a Data Acquisition Message (DQM) containing the data and byte offset of the ITC register that was written. Writes of 8, 16, or 32 bits are supported. The read value of a Stimulus register reflects the number of available write slots. Before writing to an ITC Stimulus register, software must first read a Stimulus register to obtain the number of available slots. Software may then write up to that number of items to ITC Stimulus before checking again.

ITC writes may also be configured to generate external trigger outputs for communicating events to other TEs or system logic.

17.4.1 ITC Register Interface

This section describes the functionality of the Instrumentation Trace Component registers.

Instrumentation Trace Enable Register (<i>itcTraceEnable</i>)				
Address Offset		0x0060		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	<i>itcTraceEnable</i>	RW	0x0	1=write to corresponding stimulus register generates DQM or Ownership message if enabled by <i>teInstrumentation</i>

Table 230: Instrumentation Trace Enable Register

Instrumentation Trigger Enable Register (<i>itcTrigEnable</i>)				
Address Offset		0x0064		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	<i>itcTrigEnable</i>	RW	0x0	1=write to corresponding stimulus register causes an External Trigger Out. The <i>xtoEvent</i> associated with a particular External Trigger Out must also be enabled to generate a Trig Out.

Table 231: Instrumentation Trigger Enable Register

Instrumentation Trace Stimulus Registers (itcStimulus)				
Address Offset		0x0080 - 0x00FF		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	itcStimulus	RW	0x1	Writes generate Nexus Data Acquisition Message (DQM) with data value and byte offset if enabled by teInstrumentation and itcTraceEnable. Messages generated by writing to registers 0-15 (byte offset 0-63) do not include timestamps. Registers 16-31 include a timestamp if ITC timestamping is enabled by teInstrumentation. The byte offset generated by registers 16-31 will be 0-63. ITC supports full and partial word writes. To transmit a single byte through DQM, write the byte to offset 3 (bits [31:24]) in a register. To transmit 2 bytes, do a halfword write to offset 2 (bits [31:16]). The byte offset field in the DQM message will thereby indicate the size of the data value. Registers 15 and 31 may be set to generate Ownership trace message or DQM using teInstrumentation. A write to itcStimulus may also cause an External Trigger Out if enabled by xtoEvent and itcTrigEnable. Note that this field is volatile and may be masked in IP-XACT.

Table 232: Instrumentation Trace Stimulus Register

17.4.2 Using the ITC

This section provides a code example how to use the ITC. This example assumes that the trace encoder has already been enabled.

Consider a configuration with a CLINT based at address 0x1820000 and a Trace Encoder at 0x4000. If the user would like to read mtime, a memory-mapped register in the CLINT and insert the value into the trace, you can use the following sequence:

```
Store 0x4060, 0xffffffff // enable trace for all ITC stimulus
Load  x1, 0x182bff8      // read low half of memory-mapped mtime register
Load  x2, 0x182bff8      // read high half of memory-mapped mtime register
Store 0x4080, x1         // send mtime_low to ITC
Store 0x4080, x2         // send mtime_high to ITC
Store 0x4060, 0x00000000 // disable trace for all ITC stimulus
```

17.5 Trace Encoder Timestamp Unit

A Trace Encoder instance includes one of four types of timestamp unit: Internal, External, Internal with Core Clock, or Slave. The Tenstorrent X280 MC includes the following timestamp unit(s):

- Slave mode timestamp accepts a timestamp broadcast from a Trace Funnel configured with either Internal or External (not Core Clock). This one Trace Funnel is the active source that feeds all others.
- The timestamp counter or input is 48 bits.

The Timestamp Unit instantiated in the Trace Funnel is described in Section 17.10.2.

17.5.1 Trace Encoder Timestamp Register Interface

This section describes the functionality of the Timestamp Unit registers.

Trace Encoder Timestamp Control Register (tsControl)				
Address Offset		0x0040		
Bits	Field Name	Attr.	Rst.	Description
0	tsActive	RW	0x0	Master reset/enable for timestamp unit. tsActive must be set to 1 before any other fields in tsControl are programmed.
[3:1]	Reserved	RO	0x0	
[6:4]	tsType	RO	0x4	Type of Timestamp unit • 0x0 - None • 0x1 - External • 0x2 - Internal, bus clock • 0x3 - Internal, core clock • 0x4 - Slave • 0x5-0x7 - Reserved
[14:7]	Reserved	RO	0x0	
15	tsEnable	RW	0x0	Enable timestamps and add timestamp field to Sync messages

Table 233: Trace Encoder Timestamp Control Register

Trace Encoder Timestamp Control Register (tsControl)				
[17:16]	tsBranch	WARL	0x0	Add timestamp field to branch messages if tsEnable=1. 0x0 - No timestamp on branch messages 0x1 - Timestamp all indirect branch and exception messages 0x2 - Reserved 0x3 - Timestamp all branch, exception, PTCM, and Error messages
18	tsInstrumentation	RW	0x0	Add timestamp field to Instrumentation messages if tsEnable=1
19	tsOwnership	RW	0x0	Add timestamp field to Ownership messages if tsEnable=1
[23:20]	Reserved	RO	0x0	
[31:24]	tsWidth	RO	0x30	Width of timestamp in bits

Table 233: Trace Encoder Timestamp Control Register

Trace Encoder Timestamp Lower Bits Register (tsLower)				
Address Offset		0x0044		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	tsLower	RO	X	Lower 32 bits of timestamp counter. Note that this field is volatile and may be masked in IP-XACT.

Table 234: Trace Encoder Timestamp Lower Bits Register

Trace Encoder Timestamp Upper Bits Register (tsUpper)				
Address Offset		0x0048		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	tsUpper	RO	X	Upper bits of timestamp counter, zero-extended. Note that this field is volatile and may be masked in IP-XACT.

Table 235: Trace Encoder Timestamp Upper Bits Register

17.6 External Triggers

The TE is configured with 4 external trigger inputs for controlling trace. These are in addition to the external triggers present in the Debug Module when Halt Groups are configured. A trigger

input is a 2-signal pair consisting of a request from the system and an acknowledgment handshake from the TE.

The system has 4 external trigger outputs. A trigger output is a 2-signal pair consisting of a request from the TE and an acknowledgment handshake from the system.

This protocol is compatible with the Arm Cross Trigger Interface or may connect to another TE or other system logic, even in another clock domain.

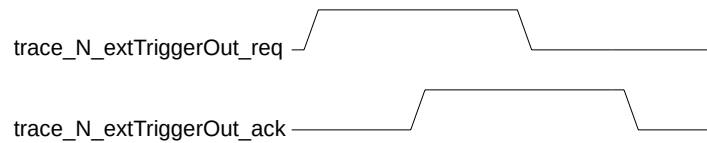


Figure 172: Trace Encoder External Trigger Handshake

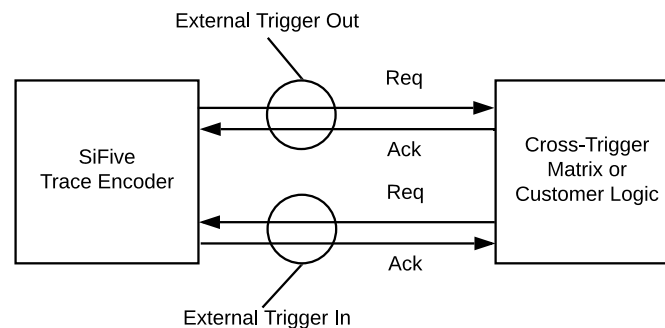


Figure 173: Trace Encoder External Triggers

`trace_N_extTriggerIn_req` is synchronized to `external_source_for_trace_N_clock` and is defined to have fired when a 0-to-1 transition occurs between two successive `external_source_for_trace_N_clock` cycles. The synchronized request is normally fed back to the system immediately on `trace_N_extTriggerIn_ack`. Note that if external triggers occur too often, some may be missed.

A trigger out may be generated by trace starting, trace stopping, or by a write to ITC.

In a multi-core system with one or more Trace Funnels, it is possible to configure the Funnels with External Triggers. This internally routes a single set of off-chip External Triggers to all the Trace Encoders. Core Complex External Trigger Inputs are broadcast to all Trace Encoders. Each Core Complex External Trigger Output is the OR-reduction of the corresponding External Trigger Outputs from each Trace Encoder. No programming is needed in the Trace Funnel to perform this function.

17.6.1 Register Descriptions

This section describes the functionality of the External Trigger registers.

External Trigger Input Control Register (xtiControl)				
Address Offset		0x0050		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	xtiAction0	WARL	0x0	Select action to perform when external trigger input 0 fires. If external trigger input 0 does not exist, then its action is fixed at 0. • 0x0 - No action • 0x1 - Reserved • 0x2 - Start trace (teTracing → 1) • 0x3 - Stop trace (teTracing → 0) • 0x4 - Record Program Trace Sync message • 0x5-0xF - Reserved
[31:4]	xtiActionN	WARL	0x0	Select actions for external triggers 1 through 7. If an external trigger input does not exist, then its action is fixed at 0.

Table 236: External Trigger Input Control Register

External Trigger Output Control Register (xtoControl)				
Address Offset		0x0054		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	xtoEvent0	RW	0x0	Bitmap to select which event(s) cause external trigger 0 output to fire. If external trigger output 0 does not exist, then all bits are fixed at 0. • 0x1 - Start trace (teTracing 0 → 1) • 0x2 - Stop trace (teTracing 1 → 0) • 0x4 - ITC write to location enabled by itcTrigEnable • 0x8 - Core watchpoint occurred with action=0x5
[31:4]	xtoEventN	RW	0x0	Select events for external trigger outputs 1 through 7. If an external trigger output does not exist, then its event bits are fixed at 0.

Table 237: External Trigger Output Register

17.6.2 Using External Trigger Inputs

This section describes how to use the external trigger inputs described above.

Each external trigger input is configured with a 4-bit field in `xtiControl` located at Trace Encoder address offset `0x50`. The value in the field defines what action to take when an external trigger input fires (i.e., the req signal transitions from 0 to 1). For example, to turn trace on when external trigger 0 fires, you would write the value `0x2` to bits `[3:0]` of `xtiControl`. To turn trace off when external trigger 1 fires, write `0x3` to bits `[7:4]`.

For the code example below, consider a design with a Trace Encoder located at `0x4000`.

```
mww 0x4000 0x01530033 // Enable trace encoder with trace off
mww 0x4050 0x00000032 // Set xtiControl as described above
```

17.6.3 Using External Trigger Outputs

External trigger outputs can be programmed to fire on any or all of 4 different conditions by writing to the 4-bit field of `xtoControl` at offset `0x54` corresponding to the trigger number.

The following code shows how to fire external trigger output 0 when trace starts and external trigger output 1 when trace stops. For the code example below, consider a design with a Trace Encoder located at `0x4000`.

```
mww 0x4054 0x00000021 // xto0=trace start, xto1=trace stop
```

17.6.4 Code Examples

This section provides code examples that utilize the External Trigger Outputs and Inputs. For the following examples, consider a configuration with a Trace Encoder at address 0x4000 in the memory map. These examples assume that the Trace Encoder has already been enabled.

1. To trigger trace on an external incoming event:

```
Store 0x4050, 0x00000020 // Set Trigger input 1 action to 2 (start trace)
```

2. To generate a trigger event when trace is enabled:

```
Store 0x4054, 0x00001000 // Trigger out 3 responds to teTracing 0->1
```

17.7 Core Watchpoints

Watchpoints are signals from the core that a breakpoint was hit, but the action associated with that breakpoint is a trace-related action. Action identifiers 2-5 are reserved for trace actions in the RISC-V Debug Spec, where breakpoints are defined. The desired action is written to the action field of the `mcontrol` CSR (0x7A1). Not all cores support trace actions; the debugger should read back `mcontrol` after setting one of these actions to verify that the option exists. Once synchronized, watchpoint actions take effect unconditionally, assuming the TE is enabled with `teEnable`.

The start and stop trace actions can operate in two modes, selected by `wpControl`.

- Edge mode: `iretire & (wpAction[n]=2)` sets `teTracing` to 1. `iretire & (wpAction[m]=3)` sets `teTracing` to 0.
- Range mode: `iretire & (wpAction[n]=2)` sets `teTracing` to 1. `iretire & (wpAction[n]≠2)` sets `teTracing` to 0. Range mode allows using only one address range watchpoint to enable tracing in a range of instruction addresses.

wpAction	Effect
0	Breakpoint exception
1	Debug exception
2	Start trace (<code>teTracing</code> → 1)
3	Stop trace (<code>teTracing</code> → 0)
4	Record Program Trace Sync message
5	Generate an External Trigger Out

Table 238: Watchpoint Action Identifiers

17.7.1 Register Descriptions

This section describes the functionality of the Core Watchpoint registers.

Core Watchpoint Control Register (wpControl)				
Address Offset		0x0058		
Bits	Field Name	Attr.	Rst.	Description
0	wpRange0	RW	0x0	Select Edge (0) or Range (1) mode for core watchpoint 0
[N:1]	wpRangeN	RW	0x0	Select Edge (0) or Range (1) mode for core watchpoints 1 to N. Fixed at 0 for any watchpoints that don't exist.

Table 239: Core Watchpoint Control Register

17.8 Watchpoint Precedence

Core watchpoints and External Trigger Inputs operate under a specific precedence when simultaneous conflicting actions occur.

- Core watchpoint with `action=5` (generate trigger out) is always executed
- For trace start and stop actions, the precedence is: core watchpoints 0 through N, then external triggers 0 through N.
 - The active watchpoint with the highest precedence that is programmed for trace start or stop is selected. All lower active watchpoints are ignored.
 - If the selected watchpoint has trace start action and trace is currently off, then trace is turned on and a Trace-On message is sent.
 - If the selected watchpoint has trace stop action and trace is currently on, then trace is turned off and a Trace-Off message is sent.
 - If a watchpoint in Range mode is active and a lower precedence watchpoint or trigger with Edge action occurs, the Edge action is ignored.
 - If a watchpoint in Range mode is active and a higher precedence Range watchpoint occurs, the lower one is disabled until it deactivates. A message is sent only if the action associated with the new Range watchpoint alters the state of tracing.
 - If a watchpoint in Range mode is active and a higher precedence Edge watchpoint occurs, the Edge action is executed, and the range trigger is disabled until it deactivates. A message is sent only if the action associated with the Edge watchpoint alters the state of tracing.
 - A disabled Range watchpoint must traverse to the inactive state (`wpAction[n]≠2`) before it can activate again.
- Core or external watchpoint with `action=4` (record Sync message) is executed if trace is enabled (`teTracing=1`) and no message was produced by above rules. If more than one watchpoint occurs simultaneously, only one Sync message is generated. If the TE can't insert the Sync due to resource limitation, the Sync message request is held until the Sync can be generated. If another Sync request occurs during this time, the second request will be ignored.

17.9 PC Sampling Unit

Since the Trace Encoder directly connects to the core's TracedInstruction output bundle, it is relatively easy to add a PC Sampling Unit that provides a read-only interface to the current execution address. In some hang scenarios, this information is useful to be able to pinpoint what the core was doing just prior to the hang. This interface can also be used for non-intrusive statistical profiling.

In systems with a watchdog timer that resets the core upon timeout, it is also useful to capture the address of the last retired instruction so that it can be read out as part of a crash analysis. The pcCapture registers capture a new sample on each occurrence of the core reset.

17.9.1 Register Descriptions

This section describes the functionality of the PC Sampling Unit registers.

PC Sample Control Register (pcControl)				
Address Offset		0x0100		
Bits	Field Name	Attr.	Rst.	Description
0	pcActive	RW	0x0	Master enable/reset for PC Sampling Unit. pcActive must be set to 1 before any other fields in pcControl are programmed. This bit is reset to 0 by the trace reset input and must be set to 0x1 in order to perform sampling. The clock to the PC Sampling Unit may be gated off when pcActive=0. Note that pcActive is not dependent on teActive, so the PC Sampling Unit can be enabled while the rest of the trace encoder is disabled.
[31:1]	Reserved	RO	0x0	

Table 240: PC Sample Control Register

PC Capture/Sample Implementation Register (pcImp1)				
Address Offset		0x0104		
Bits	Field Name	Attr.	Rst.	Description
[15:0]	pcCaptureFields	RO	preset	Bitmap indicating which core registers other than PC are present in the PC Capture register range. PC is always present. Capture registers are populated downward from 0x134 in the order listed (bit 0 to 15) with no gaps. Core registers in this list that are more than 32 bits occupy two capture register locations with the least-significant word at the higher address. • Bit 0 = MEPC • Bits [15:1] = Reserved
[31:16]	pcSampleFields	RO	preset	Bitmap indicating which core registers other than PC are present in the PC Sample register range. PC is always present. Sample registers are populated downward from 0x174 in the order listed (bit 0 to 15) with no gaps. Core registers in this list that are more than 32 bits occupy two sample register locations with the least-significant word at the higher address. • Bits [15:0] = Reserved

Table 241: PC Capture and Sample Implementation Register

PC Capture Register (pcCapture)				
Address Offset		0x013C		
Bits	Field Name	Attr.	Rst.	Description
0	pcCaptureValid	RO	0x0	The pcCapture/pcCaptureH registers contain a valid capture. This bit is reset to 0 when pcActive=0 and becomes 1 when there is an instruction retirement followed sometime later by a core reset. Note that this field is volatile and may be masked in IP-XACT.
[31:1]	pcCapture	RO	x	Address of the last instruction retired prior to the most recent core reset. Note that this field is volatile and may be masked in IP-XACT.

Table 242: PC Capture Register

PC Capture High Register (pcCaptureH)				
Address Offset		0x0138		
Bits	Field Name	Attr.	Rst.	Description
[N:0]	pcCaptureH	RO	X	If the core address is more than 32 bits, this register reflects bits [N:32] of the address of the PC Capture register. Reads as zero in systems without PC Sampling or with an address width of 32 bits or less. Note that this field is volatile and may be masked in IP-XACT.

Table 243: PC Capture High Register

PC Sample Register (pcSample)				
Address Offset		0x017C		
Bits	Field Name	Attr.	Rst.	Description
0	pcSampleValid	RO	0x0	The pcSample/pcSampleH registers contain a valid sample that has not yet been read. This bit is reset to 0 when pcActive=0 and becomes 1 when the core retires an instruction. This bit is also reset when pcSample is read. Note that this field is volatile and may be masked in IP-XACT.
[31:1]	pcSample	RO	X	Address of most recent retired instruction. Note that this field is volatile and may be masked in IP-XACT.

Table 244: PC Sample Register

PC Sample High Register (pcSampleH)				
Address Offset		0x0178		
Bits	Field Name	Attr.	Rst.	Description
[N:0]	pcSampleH	RO	X	If the core address is more than 32 bits, this register reflects bits [N:32] of the address of the most recent retired instruction. Reads as zero in systems without PC Sampling or with an address width of 32 bits or less. Since a full PC sample covers more than one register, hardware is provided to latch the full sample when any one of the sample registers below pcSample is read. In this way, reading an address range starting with a lower address and ending with pcSample will provide the values of all the fields at the time the first register was read. There is space in the address map below pcSampleH for additional fields to be added in the future. Note that this field is volatile and may be masked in IP-XACT.

Table 245: PC Sample High Register

17.10 Trace Funnel

The Trace Funnel (TF) combines messages from multiple sources into a single trace stream. The TF has the same options for sinks as a single Trace Encoder which are controlled by the same registers as described above. Trace words arriving via TileLink connections from each Trace Encoder are unpacked into full Nexus messages. Messages are chosen by a round-robin arbiter and sent to the Packer. From this point, the TF shares hardware blocks with the Trace Encoder.

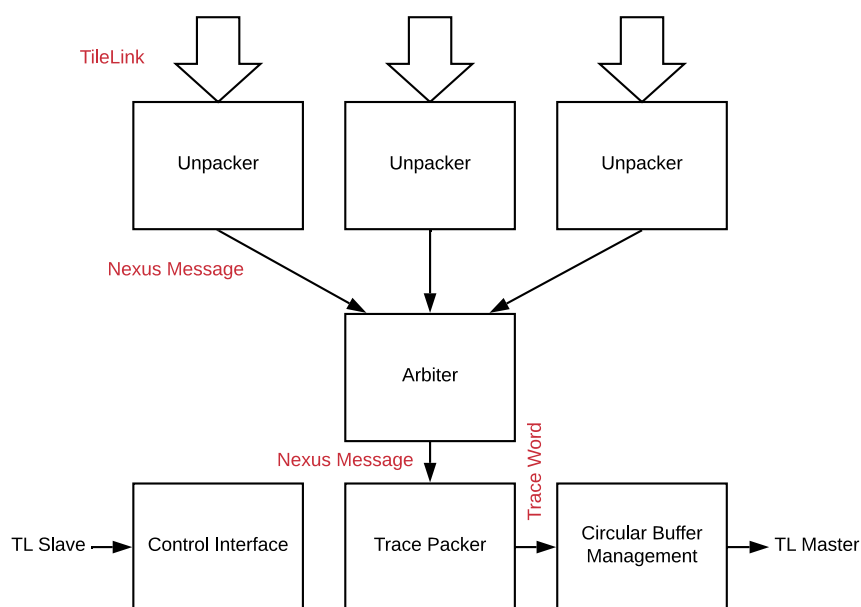


Figure 174: Trace Funnel Block Diagram

17.10.1 Register Descriptions

This section describes the functionality of the Trace Funnel registers.

Trace Funnel Control Register (tfControl)				
Address Offset		0x0000		
Bits	Field Name	Attr.	Rst.	Description
0	tfActive	RW	0x0	Master enable for trace funnel. 0 resets the TF and clocks may be gated off.
1	tfEnable	RW	0x0	1 indicates TF enabled. Messages arriving via TileLink will be accepted, combined, and sent to the designated sink. When the funnel is disabled, TileLink data will be accepted but dropped. Setting tfEnable to 0x0 will flush any partially filled trace words to the sink.
2	Reserved	RO	0x0	
3	tfEmpty	RO	0x1	Reads as 1 when all generated trace has been emitted. To stop trace, a debugger would typically write 0 to tfEnable, then read tfEmpty until it reads as 1 indicating all trace has been flushed. Note that this field is volatile and may be masked in IP-XACT.
[13:4]	Reserved	RO	0x0	
14	tfStopOnWrap	RW	0x0	Disable trace (tfEnable → 0x0) when circular buffer fills for the first time. Present only in systems with SRAM or System Memory sinks.
[26:15]	Reserved	RO	0x0	
27	tfSinkError	RW1C	0x0	Hardware sets this to 1 when a trace sink TileLink transaction results in an error. This can happen with System Memory sink when the address or size is invalid or with PIB, ATB, and Funnel sinks when the destination device is disabled. Write 1 to clear. Note that this field is volatile and may be masked in IP-XACT.
[31:28]	tfSink	WARL	0x4	0x0 - Send trace to the one sink connected 0x1-0x3 - Reserved 0x4 - SRAM Sink 0x5 - ATB Sink 0x6 - PIB Sink

Table 246: Trace Funnel Control Register

Trace Funnel Control Register (tfControl)				
				• 0x7 - SBA Sink • 0x8 - Funnel Sink • 0x9-0xF - Reserved

Table 246: Trace Funnel Control Register

Trace Funnel Implementation Register (tfImpl)				
Address Offset		0x0004		
Bits	Field Name	Attr.	Rst.	Description
[3:0]	version	RW	0x0	Version (0 is this version).
4	hasSRAMSink	RO	0x1	1 indicates TF enabled. Messages arriving via TileLink will be accepted, combined, and sent to the designated sink.
5	hasATBSink	RO	0x0	1 if this TF has an ATB sink
6	hasPIBSink	RO	0x1	1 if this TF has an off-chip trace port via a Probe Interface Block
7	hasSBASink	RO	0x1	1 if this TF has an on-chip bus master through the Front Port
8	hasFunnelSink	RO	0x0	1 if this TF is a mid-level Funnel that feeds into another Funnel
[15:9]	Reserved	RO	0x0	
[17:16]	sinkBytes	RO	0x1	Width of this TF's trace sink. 0x0 - 32 bit, 0x1 - 64 bit, 0x2 - 128 bit, 0x3 - 256 bit.
[31:18]	Reserved	RO	0x0	

Table 247: Trace Funnel Implementation Register

Trace Funnel Sink Base Register (teSinkBase)				
Address Offset		0x0010		
Bits	Field Name	Attr.	Rst.	Description
[1:0]	Reserved	RO	0x0	
[N:2]	teSinkBase	RW	0x0	Base address of trace circular buffer relative to TF TileLink master node. Fixed to the natural boundary implied by width of trace sink.

Table 248: Trace Funnel Sink Base Register

Trace Funnel Sink Base High Bits Register (teSinkBaseHigh)				
Address Offset		0x0014		
Bits	Field Name	Attr.	Rst.	Description
[N:0]	teSinkBaseH	RW	0x0	High order bits [N:32] of address of trace circular buffer in System Memory.

Table 249: Trace Funnel Sink Base High Bits Register

Trace Funnel Sink Limit Register (teSinkLimit)				
Address Offset		0x0018		
Bits	Field Name	Attr.	Rst.	Description
[1:0]	Reserved	RO	0x0	
[N:2]	teSinkLimit	RW	0x0	Highest address of trace circular buffer relative to TF TileLink master node. Fixed to natural boundary.

Table 250: Trace Funnel Sink Limit Register

Trace Funnel Sink Write Pointer Register (teSinkWP)				
Address Offset		0x001C		
Bits	Field Name	Attr.	Rst.	Description
0	teSinkWrapped	RW	0x0	Set to 1 by hardware when teSinkWP wraps. Note that this field is volatile and may be masked in IP-XACT.
1	Reserved	RO	0x0	
[N:2]	teSinkWP	RW	0x0	Address in trace sink where next trace message will be written. Fixed to natural boundary. When using a System Memory Sink and a trace word write occurs while teSinkWP=teSinkLimit, teSinkWP is set to teSinkBase. When using an SRAM sink, teSinkWP wraps back to 0 when the SRAM has been filled. Note that this field is volatile and may be masked in IP-XACT.

Table 251: Trace Funnel Sink Write Pointer Register

Trace Funnel SRAM Sink Access Pointer Register (teSinkRP)				
Address Offset		0x0020		
Bits	Field Name	Attr.	Rst.	Description
[1:0]	Reserved	RO	0x0	
[N:2]	teSinkRP	RW	0x0	Address in trace circular buffer visible through teSinkData. Auto-increments following an access to teSinkData. Note that this field is volatile and may be masked in IP-XACT.

Table 252: Trace Funnel SRAM Sink Access Pointer Register

Trace Funnel SRAM Sink Data Register (teSinkData)				
Address Offset		0x0024		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	teSinkData	RW	X	Read or write value for SRAM access. SRAM is always accessed by words through this path regardless of the width of the sink bus. Note that this field is volatile and may be masked in IP-XACT.

Table 253: Trace Funnel SRAM Sink Data Register

17.10.2 Trace Funnel Timestamp Unit

A Trace Funnel instance includes one of four types of timestamp unit: Internal, External, Internal with Core Clock, or Slave. The Tenstorrent X280 MC includes the following timestamp unit(s):

- Internal timestamp uses the `external_source_for_trace_com_clock` to increment the timestamp counter.
- Slave mode timestamp accepts a timestamp broadcast from a Trace Funnel configured with either Internal or External (not Core Clock). This one Trace Funnel is the active source that feeds all others.
- The timestamp counter or input is 48 bits.

An Internal Timestamp Unit includes a prescale divider, which can extend the range of a narrower timestamp and uses less power but has less resolution.

Since the Internal and External Timestamp clocks are system-dependent, it may be useful to measure the clock frequency for calibration. To do this, debug software can read the count value, wait a known interval, then read the count value again. $\text{Frequency} = (\text{count difference}) / (\text{time interval})$. Note when reading the count value that two reads are required to retrieve the complete value and that the counter continues to run between the two. Software can definitively determine the full count value by reading `tsUpper`, `tsLower`, then `tsUpper` again. If the two reads from `tsUpper` are the same, then $\text{count} = \{\text{tsUpper}, \text{tsLower}\}$.

If different, then examine the MSB of tsLower; if 0, then use the second tsUpper, and if 1, use the first tsUpper.

17.10.2.1 Trace Funnel Timestamp Register Interface

This section describes the functionality of the Timestamp Unit registers.

Trace Funnel Timestamp Control Register (tsControl1)				
Address Offset		0x0040		
Bits	Field Name	Attr.	Rst.	Description
0	tsActive	RW	0x0	Master reset/enable for timestamp unit. tsActive must be set to 1 before any other fields in tsControl1 are programmed.
1	tsCount	RW	0x0	1 for counter runs, 0 for counter stopped
2	tsReset	WO	X	Write 1 to reset the timestamp counter
3	Reserved	RO	0x0	
[6:4]	tsType	RO	0x2	Type of Timestamp unit • 0x0 - None • 0x1 - External • 0x2 - Internal, bus clock • 0x3 - Internal, core clock • 0x4 - Slave • 0x5-0x7 - Reserved
7	Reserved	RO	0x0	
[9:8]	tsPrescale	RW	0x0	Prescale timestamp clock by 2^{2n} (1, 4, 16, 64)
[23:10]	Reserved	RO	0x0	
[31:24]	tswidth	RO	0x30	Width of timestamp in bits

Table 254: Trace Funnel Timestamp Control Register

Trace Funnel Timestamp Lower Bits Register (tsLower)				
Address Offset		0x0044		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	tsLower	RO	X	Lower 32 bits of timestamp counter. Note that this field is volatile and may be masked in IP-XACT.

Table 255: Trace Funnel Timestamp Lower Bits Register

Trace Funnel Timestamp Upper Bits Register (tsUpper)				
Address Offset		0x0048		
Bits	Field Name	Attr.	Rst.	Description
[31:0]	tsUpper	RO	X	Upper bits of timestamp counter, zero-extended. Note that this field is volatile and may be masked in IP-XACT.

Table 256: Trace Funnel Timestamp Upper Bits Register

17.11 Nexus Messages

17.11.1 Nexus Message Formats

The following tables show the message formats generated by the Trace Encoder and are ordered with LSBs at the bottom. The TCODE in the LSBs indicates the message type.

Direct Branch Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	I - CNT	Number of instruction units (half-words) executed since the last BTM or Sync
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 3

Table 257: Direct Branch Message Description

Indirect Branch Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	U - ADDR	Compressed address of target of branch
var	I - CNT	Number of instruction units (half-words) executed since the last BTM or Sync
2	B - TYPE	Branch type, 0=indirect branch, 1=exception
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 4

Table 258: Indirect Branch Message Description

Direct Branch with Sync Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	F-ADDR	Full target address for a taken branch. High-order zeroes are not transmitted.
var	I-CNT	Number of instruction units (half-words) executed since the last BTM or Sync
4	SYNC	Reason for Sync
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 11

Table 259: Direct Branch with Sync Message Description

Indirect Branch with Sync Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	F-ADDR	Full target address for a taken branch. High-order zeroes are not transmitted.
var	I-CNT	Number of instruction units (half-words) executed since the last BTM or Sync
2	B-TYPE	Branch type, 0=indirect branch, 1=exception
4	SYNC	Reason for Sync
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 12

Table 260: Indirect Branch with Sync Message Description

Program Trace Synchronization Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	F-ADDR	Full target address of current PC. High-order zeroes are not transmitted.
var	I-CNT	Number of instruction units (half-words) executed since the last BTM or Sync. Valid only if <code>SYNC[0]=0</code> .
4	SYNC	Reason for Sync
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 9

Table 261: Program Trace Synchronization Message Description

Error Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	pad	Pad ETYPE to end of slice so timestamp begins on slice boundary
4	ETYPE	Error Type, 0=Queue Overrun
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 8

Table 262: Error Message Description

Program Trace Correlation Message		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	I - CNT	Number of instruction units (half-words) executed since the last BTM or Sync.
2	CDF	Number of CDATA fields (always 0x0)
4	EVCODE	Reason for PTCM
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 33

Table 263: Program Trace Correlation Message Description

In-Circuit Trace Message		
Bits	Name	Description
var	TSTAMP	Timestamp value
var	CKDATA	Address of most recent retired instruction, U-ADDR or F-ADDR
2	CKDF	Number of CKDATA fields (always 0, meaning 1 CKDATA follows)
4	CKSRC	Reason for ICT. Possible values described in Table 227.
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 34 (compressed) or 35 (uncompressed)

Table 264: In-Circuit Trace Message Description (Reports PC)

Data Acquisition Message (from ITC)		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	DQDATA	Write data value to ITC, zero extended.
var	IDTAG	Write data index (byte offset) in ITC, zero extended
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 7

Table 265: Data Acquisition Message (from ITC) Description

Ownership Trace Message (from ITC)		
Bit	Name	Description
var	TSTAMP	Timestamp value
var	PROCESS	Write data value to ITC, zero extended
n	SRC	Source of this message (width is <code>teImpl.nSrcBits</code>)
6	TCODE	Value = 2

Table 266: Ownership Trace Message (from ITC) Description

17.11.2 Address Compression

Address transmissions are compliant with the Nexus spec. Address fields begin with bit 1 since all execution addresses are on 2-byte boundaries. Addresses sent in U-ADDR compressed form are computed based on a reference address sent by or computed from the most recent preceding BTM or Sync message containing an address field. Starting with an F-ADDR, each U-ADDR modifies the reference address used for the next BTM. A U-ADDR is generated by XORing the full address with the reference address and sending the result starting with bit 1 and with high-order zeroes suppressed. The reverse process is used by software to recover the original full address.

Full Address	Compression			New Reference
0x003FC04		-		0x003FC04
0x003F368	ref	0000 0000 0011 1111 1100 0000 0100		0x003F368
	F-ADDR	0000 0000 0011 1111 0011 0110 1000		
	XOR	0000 0000 0000 0000 1111 0110 1100		
	U-ADDR	1111 0110 110		
0x003E100	ref	0000 0000 0011 1111 0011 0110 1000		0x003E100
	F-ADDR	0000 0000 0011 1110 0001 0000 0000		
	XOR	0000 0000 0000 0001 0010 0110 1000		
	U-ADDR	1 0010 0110 100		

Table 267: Address Compression Example

17.11.3 Timestamp Compression

If timestamp recording is enabled, Sync messages all include a full timestamp value with upper zeroes suppressed. Other message types with timestamps use a compressed value using the same XOR algorithm as in address compression. To reconstruct the full timestamp, software begins at a Sync message and stores the TSTAMP value found there, zero-extended to the full timestamp width. Software extracts the compressed TSTAMP from each message thereafter and XORs it with the previous decompressed timestamp to obtain the full timestamp value associated with each message.

Message	TSTAMP	Computed Timestamp
Sync	00 1111 1100 0001 1001	0x0000_0000FC19
BTM	11 1110	0x0000_0000FC27
BTM	0000 0100 0010	0x0000_0000FC65

Table 268: Timestamp Compression Description

17.11.4 Nexus Transmission Protocol

The Nexus standard defines a messaging protocol using a number of Message Data Out (MDO) signals and two flag signals known as MSE0[1:0] (Message Start/End Out). A Nexus message is sent or stored in slices comprised of MDO and MSE0. The first slice of a message sends the LSBs of the message and is indicated by MSE0=0x0. A variable-length field in a message always ends on a slice boundary (zero extended as needed) and the last slice of a variable field is indicated by MSE0=0x1. The last slice of a message is indicated by MSE0=0x3. Most messages have more than one variable field. TSTAMP is an optional field and may be present in some messages and not in others. When not present, the variable field just preceding TSTAMP will end with MSE0=0x3.

The Trace Encoder uses a slice format consisting of 6 MDO bits and 2 MSE0 bits. These slices are packed into words, LSB first, for storage in SRAM or sending to another destination. Following is an example of slice packing for an Indirect Branch Message with timestamp.

	MDO							MSE0		
Slice	5	4	3	2	1	0		1	0	Notes
	X	X	X	X	X	X		1	1	Idle or end of previous message
0	T5	T4	T3	T2	T1	T0		0	0	Start of message
1	I3	I2	I1	I0	B1	B0		0	0	Normal transfer
2	0	0	0	0	I5	I4		0	1	End of var field I -CNT zero-extended to slice boundary
3	0	A5	A4	A3	A2	A1		0	1	End of var field U -ADDR zero-extended to slice boundary
4	P5	P4	P3	P2	P1	P0		0	0	Normal transfer
5	0	0	P9	P8	P7	P6		1	1	End of message (implies end of var field)
Tx=TCODE, Bx=BTYPE, Ix=I -CNT, Ax=U -ADDR, Px=TSTAMP										

Table 269: Nexus Transmission Protocol Description

17.12 PIB Trace Sink

Trace data may be sent to chip pins through an interface called the Probe Interface Block (PIB). This interface typically operates at data rates of up to 100-400 MHz and can sometimes be

higher with careful constraints and board layout. PIB may consist of just one signal and in this configuration may be called SWT (Serial-wire Trace). Alternative configurations include a trace reference clock and between 1 and 8 trace data signals timed to that reference.

17.12.1 Register Descriptions

This section describes the functionality of the PIB Trace Sink registers.

PIB Sink Control Register (<code>pibControl1</code>)				
Address Offset		0x0F00		
Bits	Field Name	Attr.	Rst.	Description
0	<code>pibActive</code>	RW	0x0	Master enable/reset for PIB Sink block
1	<code>pibEnable</code>	RW	0x0	Enable PIB to produce output
2	Reserved	RO	0x0	
3	<code>pibEmpty</code>	RO	0x1	Reads 1 when PIB internal buffers are empty. Note that this field is volatile and may be masked in IP-XACT.
[7:4]	<code>pibMode</code>	WARL	0x0	Select mode for output pins. Do not change while <code>pibActive</code> = 1.
8	<code>pibRefCenter</code>	WARL	0x0	In parallel modes, adjust <code>tref</code> timing to center of bit period. This can be set only if <code>pibMode</code> selects one of the parallel protocols. Note that this field is volatile and may be masked in IP-XACT.
9	<code>pibCalibrate</code>	RW	0x0	Set this to 1 to generate a repeating calibration pattern to help tune a probe's signal delays, bit rate, etc. The calibration pattern is described in Table 272.
[15:10]	Reserved	RO	0x0	
[28:16]	<code>pibDivider</code>	RW	0x0	Timebase selection for the PIB module. The bus clock or PIB clock input is divided by <code>pibDivider</code> + 1. PIB data is sent at either this quotient rate or 1/2 of this rate, depending on <code>pibMode</code> .
[31:29]	Reserved	RO	0x0	

Table 270: PIB Sink Control Register

PIB port widths up to 9 may be selected. This width and all narrower modes may then be selected during circuit operation. Software can determine what modes are available by attempting to write each mode setting to the WARL `pibControl1.pibMode` while `pibActive`=0 and reading back to see if the value was accepted.

				Supported in PIB Port Width Configurations				
Mode	pibMode	pibREFCenter	Bit Rate	1	2	3	5	9
Off	0	X	-	✓	✓	✓	✓	✓
SWT Manchester	4	X	1/2	✓	✓	✓	✓	✓
SWT UART	5	X	1	✓	✓	✓	✓	✓
tref + 1 data	8	0	1	-	✓	✓	✓	✓
tref + 2 data	9	0	1	-	-	✓	✓	✓
tref + 4 data	10	0	1	-	-	-	✓	✓
tref + 8 data	11	0	1	-	-	-	-	✓
tref + 1 data	8	1	1/2	-	✓	✓	✓	✓
tref + 2 data	9	1	1/2	-	-	✓	✓	✓
tref + 4 data	10	1	1/2	-	-	-	✓	✓
tref + 8 data	11	1	1/2	-	-	-	-	✓

Table 271: PIB Port Width Configuration Description

17.12.2 Programming Recommendations

Since the PIB supports many different modes, it is necessary to follow a particular programming sequence.

1. Activate the PIB by setting `pibActive`.
2. Set the `pibMode`, `pibDivider`, `pibRefCenter`, and `pibCalibrate` fields. This will set the `tdata` outputs to the quiescent state (whether that is high or low depends on `pibMode`) and start `tref` running.
3. Activate the receiving device, such as a trace probe. Allow time for PLL to sync up, if using a PLL with a parallel PIB mode.
4. Set `pibEnable`. This enables the PIB to generate output either immediately (calibration mode) or when the trace encoder begins sending trace messages.

17.12.3 Calibration Mode

In calibration mode, the PIB transmits a repeating pattern. Probes can use this to automatically tune input delays due to skew on different PIB signal lines and to adjust to the transmitter's data rate. Each pattern is 4 slices in length and is transmitted as a BTM would be – LSB first, complete with start/stop framing bits. Each pattern has been chosen so that it both exercises the interface and follows Nexus message rules. In UART and Manchester modes, the pattern is repeated starting every 80 cycles of the divided clock. This leaves a 12-cycle gap in Manchester and a 40-cycle gap in UART mode. There is no gap in the parallel modes.

Mode	Calibration Slices	Wire Sequence
2-bit parallel	66 66 CC 33	2, 1, 2, 1, 2, 1, 2, 1, 0, 3, 0, 3, 3, 0, 3, 0
4-bit parallel	5A 5A F0 0F	A, 5, A, 5, 0, F, F, 0
8-bit parallel	AA 55 00 FF	AA, 55, 00, FF
UART, Manchester, 1-bit parallel	AA 55 00 FF	Alternating 1/0, then all 0, then all 1

Table 272: Calibration Mode Description

17.12.4 SWT Manchester Protocol

In this mode, the PIB outputs complete Nexus messages encapsulated between a start bit and a stop bit. Each bit period is divided into 2 phases and the sequential values of the `tdata[0]` pin during those 2 phases denote the bit value. Bits in each Nexus slice are transmitted LSB first, and the oldest slice is transmitted first. The idle state of `tdata[0]` is low in this mode.

Bit	Phase 1	Phase 2
Start	1	0
Logic 0	0	1
Logic 1	1	0
Stop/idle	0	0

Table 273: SWT Manchester Protocol Description

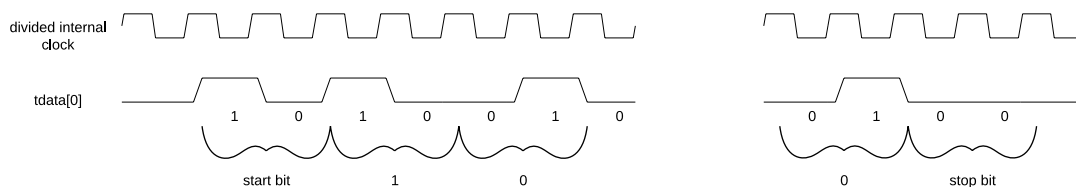


Figure 175: SWT Manchester Protocol

17.12.5 SWT UART Protocol

In UART protocol, the PIB outputs Nexus slices encapsulated in a 10-bit packet consisting of a low start bit, 8 data bits, LSB first, and a high stop bit. Another packet may begin immediately following the stop bit or there may be an idle period between packets. Slices comprising a Nexus message will be output back-to-back. When no data is being sent, `tdata[0]` is high in this mode.

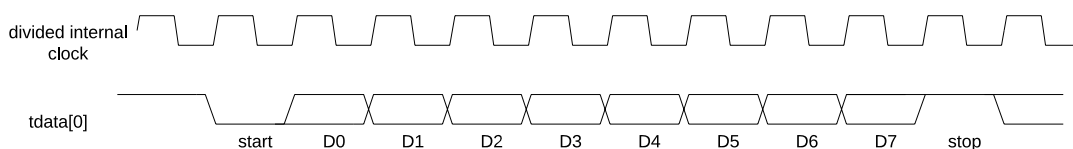


Figure 176: SWT UART Protocol

17.12.6 PIB Parallel Protocol

Traditionally, off-chip trace has used this protocol. There are a number of parallel data signals and one continuously running clock reference. The data rate of several parallel signals can be much higher than either of the serial-wire protocols.

As with SWT modes, this protocol is oriented to Nexus messages rather than fixed-width trace words. The idle state of tdata is all-ones. A transmission begins with tdata[0] driven low during a bit period. Since the first bits driven out include the MSE0 bits of the first slice, there will always be a zero starting any message. This sample and possibly the next few (depending on the width of tdata) are collected until a complete Nexus slice (8 bits) have been received. Slices are transmitted LSB first, with tdata[0] being the LSB in each beat of data. The receiver continues collecting slices until a complete slice has been collected that has MSE0=1, denoting the end of the message. The data signals may then go their idle state (all ones), or a new message may begin in the next bit period (one or more zeroes).

The clock reference, tref, normally has edges coincident with the tdata edges. Typically, a probe will use a PLL to recover a sampling clock that is twice the frequency of tref and shifted 90 degrees so that its rising edges occur near the center of each bit period. The debugger can set pibRefCenter to change the timing of tref so that there is a tref edge at the center of each bit period on tdata. Note that this option cuts the data rate in half relative to normal parallel mode and still requires the probe to sample tdata on both edges of tref.

This example shows 8-bit parallel mode with pibRefCenter=0 transmitting a 5-slice message followed by a 2-slice message.

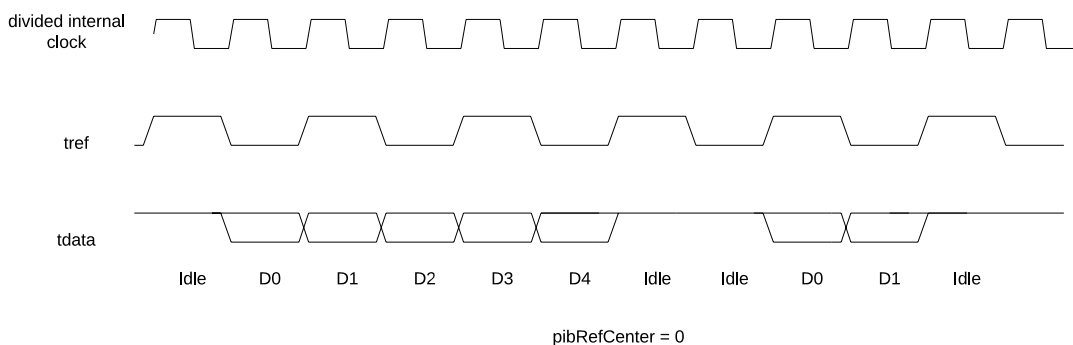


Figure 177: PIB Parallel Protocol

And an example showing 8-bit parallel mode transmitting a 4-slice packet with `pibRefCenter=1`.

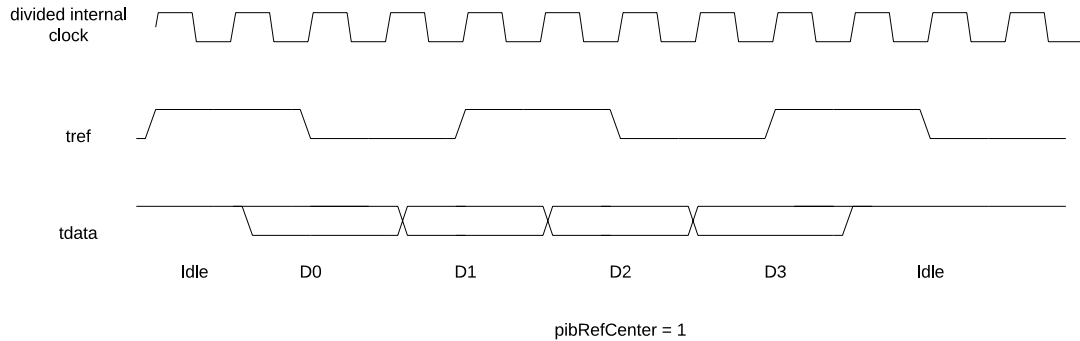


Figure 178: PIB Parallel Protocol, `pibRefCenter=1`

17.13 Nexus Branch History Trace Encoder

The Trace Encoder (TE) is a hardware block that converts execution status signals from the core to formatted Nexus trace messages. There is one TE per core and the outputs from several encoders are combined in a Trace Funnel to produce the final trace message stream.

17.13.1 History Buffer

A branch or jump is considered "inferable" if its destination can be determined solely by examining the program image. Jumps whose destination depends on processor state such as a register value are said to be "uninferable".

Rather than reporting every branch, the History Trace Mode (HTM) Trace Encoder only reports the minimal information needed by the Trace Decoder to track execution history. Unconditional inferable jumps do not generate a trace message since the decoder can follow execution flow through this type of branch by disassembling it and computing the destination address.

Conditional branches are always inferable in the RISC-V architecture.

Instead of reporting them as direct branches as in Branch Trace Mode (BTM), conditional branches are gathered into a history buffer as they are executed.

A taken branch is represented by 1 and a not-taken branch by 0. An indirect jump or sync event will generate an Indirect Branch History Message (IBHM) that includes the target address of the jump, current contents of the history buffer (HIST), and an instruction unit count (I-CNT) that includes all of the instructions executed since the previous IBHM, including unconditional jumps and all the conditional branches represented by the history buffer.

HIST is a variable length field that includes a single stop bit with value 1. Since var fields are padded to the next slice boundary, the stop bit is needed in order for the decoder to determine

where data ends and the pad begins. For example, a taken branch followed by a not-taken branch would result in a padded HIST field of 000110. Reading from the MSB (oldest instruction), three 0s are the pad, then 1 for the stop bit, 1 indicates taken branch, and 0 indicates not-taken branch.

The TE has hardware limitations on the number of conditional branches that can be conveyed in the HIST field and on the number of instructions between IBHMs. If either of these limits is exceeded, the TE will transmit a Resource Full message. The message will indicate whether it applies to HIST or I-CNT and will include the HIST field or I-CNT up to that point.

When trace turns off, a Program Trace Correlation Message (PTCM) is sent. PTCM always includes an I-CNT, but in Branch History Trace, it may also include a HIST field. Per the Nexus spec, the CDF field of the PTCM indicates the number of variable CDATE fields in the message. When CDF=1, then CDATE contains a HIST field that indicates the conditional branch history up to the point where the trace turned off. If a PTCM is sent and there is no HIST to report, then the simpler form of PTCM is used (i.e., CDF=0).

17.13.2 Return-Address Stack Optimization

Uninferable jumps are generally reported with an IBHM. However, in some cases, an uninferable jump target can be determined and uninferable jumps are not reported. The Trace Encoder will track the current depth of the return-address stack (RAS) of the core. The RAS is initialized when the Trace Encoder transmits a Sync message. Thereafter, when a call occurs, the count is incremented and when a return occurs, the count is decremented. If a return is executed and the count is non-zero, that return is considered inferable and, therefore, does not generate an IBHM. The RAS counter is "saturating"; that is, it sticks at a maximum value if too many calls are encountered and sticks at 0 if too many returns are encountered.

Consider the example below showing starting execution at 1000 and hitting a breakpoint at 1008:

Addr	Instruction	BTM	HTM	Notes
1000	c.add	PTSync, FADDR=1000	PTSync, FADDR=1000	Exit debug mode
1002	jal x1,2000	Direct, ICNT=3	—	Inferable call (no HTM message)
2000	c.sub	—	—	
2002	c.beqz	Direct, ICNT=2	—	1 (taken branch) pushed to HIST
2010	c.jr x1	Indirect, ICNT=1, UADDR=6	—	Return inferable due to RAS
1006	c.add	—	—	
1008	—	PTCM, ICNT=1	PTCM, ICNT=7, HIST=1	Enter debug mode (breakpoint)

Table 274: RAS Optimization Example

17.13.3 Repeat Branch Optimization

Another relatively easy optimization involves reporting certain cases of HIST in a compressed format using a form of the Resource Full message. A long sequence of taken branches or a long sequence of not-taken branches are common in embedded software when polling hardware registers and would normally result in multiple Resource Full messages with HIST being either all 1s or all 0s. For these cases, a vendor-defined RCODE indicates that the RDATA field of the Resource Full message contains a count rather than a bitmap of conditional branch history.

Example 1: Consider a polling loop such as `while (*uart_status & 1) { }`. Suppose the loop reads 1 (busy) 10,000 times before finally reading 0.

Addr	Instruction	BTM	HTM
1000	<code>lw x6, 0(x2)</code>	—	—
1004	<code>andi x6, x6, 1</code>	—	—
1008	<code>bnez x6, 1000</code>	Direct, ICNT=6	HIST = (HIST << 1)

Table 275: Repeat Branch Optimization Example 1

In BTM, this generates 10,000 direct branch messages occupying **20,000** bytes of trace buffer space. If the HIST buffer capacity of the HTM Encoder is 32 bits, this loop would generate 312 Resource Full messages, all indicating 32 taken branches, using **2184 bytes**. With the Repeat Branch optimization, one Resource Full message with RCODE=9 is emitted, and the final not-taken branch is loaded into the HIST buffer to be reported the next time HIST is sent. This uses only **4 bytes** of trace buffer space.

Example 2: A for-loop such as `for (i = 0; i < 10000; i++) { buf[i] = 0 }` typically compiles into an instruction sequence with a conditional branch at the top and an unconditional jump at the bottom. The conditional branch is repeatedly not taken until the loop exits.

Addr	Instruction	BTM	HTM
1000	<code>lui x6, 0</code>	—	—
1004	<code>lui x7, 10000</code>	—	—
1008	<code>bge x6, x7, 101c</code>	—	HIST = (HIST << 1) 0
100c	<code>add x28, x3, x6</code>	—	—
1010	<code>sw buf(x28), x0</code>	—	—
1014	<code>addi x6, x6, 1</code>	—	—
1018	<code>jal x0, 1008</code>	Direct, ICNT=10	Inferable jump (no message)

Table 276: Repeat Branch Optimization Example 2

Without Repeat Branch optimization, this loop would generate 312 Resource Full messages, all indicating 32 not-taken branches. With the Repeat Branch optimization, one Resource Full message with RCODE=8 is emitted, and the final taken branch is loaded into the HIST buffer to be reported the next time it is sent.

17.13.4 Sync Messages

The Program Trace Sync message (PT Sync) does not have a HIST field, so in Branch History Trace, certain Sync events generate an IBHM+Sync (IBHSync) instead of a PT Sync. The F-ADDR field can be used to determine whether an IBHSync represents an indirect branch. If an uninferable jump and sync marker event occur simultaneously, only one IBHMSync will be emitted, and SYNC will indicate the marker reason and F-ADDR will point to the discontinuous jump target. PT Sync is still used for those Sync events that indicate trace is resuming after a gap, meaning there is no previous history to transmit.

SYNC	Description	Message	Notes
0	External trigger	IBHSync	Sent when External Trigger Input occurs with xtiAction=4
1	Exit from reset	PTSync	PT Sync message sent at first iretire after reset
2	Periodic message counter	IBHSync	Include full address in every nth branch message
3	Exit from debug mode	PTSync	PT Sync Message sent at first iretire following a gap in recording
4	Sequential instruction counter	—	Not used in HTM (Resource Full is used instead)
5	Trace enable	PTSync	PT Sync message sent at first iretire following a gap in recording
6	Watchpoint	IBHSync	IBHM sent when core watchpoint occurs with action=4
7	Restart after FIFO overrun	PTSync	PT Sync message sent at first iretire following a gap in recording

Table 277: Sync Messages

17.13.5 HTM Nexus Message Formats

Bits	Name	Description
var	TSTAMP	Timestamp value
var	HIST	Conditional branch history
var	U-ADDR	Compressed address of target of branch
var	I - CNT	Number of instruction units (halfwords) executed since the last BTM or Sync.
2	B - TYPE	Branch type, 0=indirect branch, 1=exception
n	SRC	Source of this message (width is telmpl.nSrcBits)
6	TCODE	Value = 28

Table 278: Indirect Branch History Message

Note

If there is no branch history to transmit, then the Indirect Branch message is emitted instead (TCODE=4, no HIST).

Bits	Name	Description
var	TSTAMP	Timestamp value
var	HIST	Conditional branch history
var	F-ADDR	Full target address for a taken branch. High-order zeroes are not transmitted.
var	I - CNT	Number of instruction units (halfwords) executed since the last BTM or Sync.
2	B - TYPE	Branch type, 0=indirect branch, 1=exception
4	SYNC	Reason for Sync
n	SRC	Source of this message (width is telmpl.nSrcBits)
6	TCODE	Value = 29

Table 279: Indirect Branch History with Sync Message

Note

If there is no branch history to transmit, then the Indirect Branch with Sync message is emitted instead (TCODE=12, no HIST).

Bits	Name	Description
var	TSTAMP	Timestamp value
var	RDATA	I-CNT (RCODE=0), HIST (RCODE=1), or count (RCODE=8 or 9)
4	RCODE	Resource code (0=I-CNT, 1=HIST, 8=HIST_NOTTAKEN, 9=HIST_TAKEN)
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 27

Table 280: Resource Full Message

Bits	Name	Description
var	TSTAMP	Timestamp value
var	HIST	Conditional branch history (field present only if CDF=1)
var	I-CNT	Number of instruction units (halfwords) executed since the last BTM or Sync
2	CDF	Number of CDATA fields (0 or 1, 0 is backward-compatible with BTM)
4	EVCODE	Reason for PTCM
n	SRC	Source of this message (width is teImpl.nSrcBits)
6	TCODE	Value = 33

Table 281: Program Trace Correlation Message (HTM format)

Chapter 18

Error Correction Codes (ECC)

Error correction codes (ECC) are implemented on various memories within the Tenstorrent X280 MC, allowing for the detection and, in some cases, correction of memory errors. The following SRAM blocks on the Tenstorrent X280 MC support ECC: data cache, instruction cache, private L2 cache, and L3 cache.

The minimal case of an ECC error is a single-bit error that is detected, reported via interrupt handler, and corrected automatically by hardware without any software intervention. More difficult scenarios that involve double-bit errors (fatal and non-recoverable, potentially causing data loss) are still reported and tracked in hardware but are not correctable. The ECC hardware includes logic for detection and correction, in addition to 7 redundant bits per 32-bit codeword or 8 redundant bits per 64-bit codeword.

Name	Protection Type
Branch Predictor	None
L1 D-Cache Data	SECDED ECC
L1 D-Cache Tag	SECDED ECC
L1 I-Cache Data	Parity-only
L1 I-Cache Tag	Parity-only
L3 Cache Data	SECDED ECC
L3 Directory Tag	SECDED ECC
Private L2 Cache Data	SECDED ECC
Private L2 Directory Tag	SECDED ECC
UTLB	Parity-only

Table 282: Memory Protection Summary

18.1 ECC Configuration

All blocks with ECC support are enabled globally through the Bus-Error Unit (BEU) configuration registers. The BEU is used to configure ECC reporting and enable interrupt handling via the global or local interrupt controller. The global interrupt controller is the Platform-Level Interrupt Controller (PLIC). The local interrupt controller is the Core-Local Interruptor (CLINT). The BEU registers `plic_interrupt` and `local_interrupt` are used to route the errors to the respective interrupt controller. Additionally, the BEU can be used for TileLink bus errors.

18.1.1 ECC Initialization

Any SRAM block containing ECC functionality needs to be initialized prior to use. This does not include cache memory, since an internal state machine initializes data cache valid bits, and instruction cache valid bits are flops with reset. ECC will correct defective bits based on memory contents, so if memory is not first initialized to a known state, then the ECC will not operate as expected. It is recommended to use a DMA, if available, to write the entire SRAM or cache to zeros prior to enabling ECC reporting. If no DMA is present, use store instructions issued from the processor. Initializing memory with ECC from an external bus is not recommended. After initialization, ECC-related registers can be written to zero, and then ECC reporting can be enabled. 64-bit aligned writes are recommended.

The startup code in the `freedom-e-sdk/freedom-metal` directory of the IP deliverables provides a method to automatically initialize memory with ECC. This is accomplished using an assembly-level function `__metal_memory_scrub`, located in the file `freedom-metal/src/scrub.S`. The linker script provides the symbol `__metal_eccscrub_bit` as a flag to enable the startup code to initialize memory with ECC. It is important to note that this memory initialization is limited to 64 KB to support RTL simulation run times. If unexpected ECC errors occur, check the range of the startup initialization to ensure it covers the region used by the software application.

18.2 ECC Interrupt Handling and Error Injection

Single-bit errors are automatically repaired by the hardware.

BEU errors are always enabled and thus do not have a control bit in the `mie` (Machine Interrupt Enable) CSR. Likewise, there is no dedicated control bit for BEU errors in the `mideleg` (Machine Interrupt Delegation) CSR, so it cannot be delegated to a lower privilege mode than M-mode. Error injection, and thus software handling of errors, can be accomplished manually by writing the BEU accrued register. The BEU is further described in Chapter 11.

Monitoring overall ECC events can be accomplished in software via the interrupt handler.

The L3 Cache Controller contains hardware counters to track ECC events, and optionally inject ECC errors to test the software handling of ECC events. The L3 Cache Controller is further described in Chapter 14.

The exception code value is located in the `mcause` (Machine Trap Cause) CSR. When BEU interrupts are routed through the PLIC, the default exception code value will be 11 (0xB).

When ECC interrupts are routed through the CLINT, the default exception code value will be 128 (0x80). These exception codes are further detailed in Section 7.7.5.

18.3 Hardware Operation Upon ECC Error

Hardware will operate differently depending on which memory type encounters an ECC error:

- Instruction Cache: The error is corrected, and the cache line is flushed
- Data Cache: The error is corrected and the cache line is invalidated and written back to the next level of memory
- Private L2 Cache: Single-bit correction and double-bit detection for private L2 data and tag/state RAMs
- L3 Cache: Single-bit correction for L3 data and metadata (metadata includes index, tag, and directory information). Double-bit detection only on the L3 data array.

Double-bit errors are reported at the Core Complex boundary via the `halt_from_tile_N` signal that, if asserted, remains high until reset.

Appendix A

SiFive Core Complex Configuration Options

This section provides a reference of the key configuration options of the SiFive X280 Series cores and the larger Core Complex. The file `docs/core_complex_configuration.txt` lists the features and options configured in the Tenstorrent X280 MC.

A.1 X280 Series

The X280 Series comes with the following set of configuration options. Note that the configuration may be limited to a fixed set of discrete options.

Modes and ISA:

- Configurable number of Cores (1, 2, 4, or 8). In the case where more than one core is selected, all cores are configured the same.
- Configurable number of Clusters (1 or 2) **Vector Extension**
- Option between RISC-V V or Zve64f extensions
- Optional SiFive Intelligence BFloat16 MatMul Instructions extension

On-Chip Memory:

- Optional Data Local Store (DLS) with the following options:
 - Configurable size (4 KiB to 8 MiB)
 - Configurable base address
 - Configurable pipeline depth (0, 1, or 3 additional stages)
 - Configurable number of banks (1 to 64)
- Optional L3 Cache with the following options:
 - Configurable size (1, 2, 4, or 8 MiB), associativity (8- or 16-way), and banks (2, 4, 8)
- Configurable number of MMIO registers (4 to 24)

Ports:

- Memory Port, System Port, Front Port, and optional Peripheral Port, Core Local Front Port, and/or Core Local Port
 - If Multi-Cluster, optional second System Port and/or Front Port
 - Each port has a configurable base address and size (64 KB up to max. supported address)
 - Memory Port has a configurable width (128- or 256-bit)
 - If Multi-Cluster option is selected, Memory Ports also have the following options:
 - Configurable number of channels (1 or 2)
 - Configurable addressing scheme (interleaved or stacked)
 - Front Port has a configurable width (64- or 128-bit)
 - Core Local Front Port and Core Local Ports have configurable width (64- or 128-bit) and protocol (AXI4 or AHB-Lite)
- Optional Front Port Passthrough

Security:

- Optional Public key based secure Debug
- Optional Hardware Cryptographic Accelerator (HCA) with configurable base address

SiFive Insight Debug and Trace:

- Optional Nexus Trace Encoder with the following options:
 - Trace Sink (SRAM, System Memory, and/or PIB)

Interrupts:

- Platform-Level Interrupt Controller (PLIC) with configurable number of interrupts (1 to 511)

Design For Test:

- Configurable SRAM user-defined inputs (0 to 1024)
- Configurable SRAM user-defined outputs (0 to 1024)
- Optional SRAM Macro Extraction
- Optional Clock Gate Extraction
- Optional Grouping and Wrapping of extracted macros

Note that the SRAM user-defined feature is mutually exclusive to the macro extraction features.

RTL Options:

- Optional custom RTL module name prefix

Appendix B

SiFive RISC-V Implementation Registers

This section provides a reference to the SiFive RISC-V implementation version registers `marchid` and `mimpid`.

B.1 Machine Architecture ID Register (`marchid`)

Value	Core Generator
0x8000_0007	6/7/P200/X200-Series Processor

Table 283: Core Generator Encoding of `marchid`

B.2 Machine Implementation ID Register (mimpid)

Value	Generator Release Version
0x0000_0000	Pre-19.02
0x2019_0228	19.02
0x2019_0531	19.05
0x2019_0919	19.08p0p0 / 19.08.00
0x2019_1105	19.08p1p0 / 19.08.01.00
0x2019_1204	19.08p2p0 / 19.08.02.00
0x2020_0423	19.08p3p0 / 19.08.03.00
0x0120_0626	19.08p4p0 / 19.08.04.00
0x0220_0515	koala.00.00-preview and koala.01.00-preview
0x0220_0603	koala.02.00-preview
0x0220_0630	20G1.03.00 / koala.03.00-general
0x0220_0710	20G1.04.00 / koala.04.00-general
0x0220_0826	20G1.05.00 / koala.05.00-general
0x0320_0908	kiwi.00.00-preview
0x0220_1013	20G1.06.00 / koala.06.00-general
0x0220_1120	20G1.07.00 / koala.07.00-general
0x0421_0205	llama.00.00-preview
0x0421_0324	21G1.01.00 / llama.01.00-general
0x0421_0427	21G1.02.00 / llama.02.00-general
0x0521_0528	mongoose.00.00-preview
0x0521_0714	21G2.01.00 / mongoose.01.00-general
0x0521_1008	21G2.02.00 / mongoose.02.00-general
0x0621_1027	narwhal.00.00-preview
0x0621_1203	narwhal.01.00-preview
0x0621_1222	21G3.02.00 / narwhal.02.00-general
0x0622_0316	21G3.03.00 / narwhal.03.00-general
0x0622_0425	21G3.04.00 / narwhal.04.00-general

Table 284: Generator Release Encoding of mimpid

Appendix C

SiFive Custom CSRs

This section provides a reference for the custom RISC-V CSRs configured in the Tenstorrent X280 MC.

CSR	Name	Notes
0x7C0	Branch Prediction Mode CSR	See Section 6.2.1 for more information
0x7C1	SiFive Feature Disable CSR	See Section 6.2.2 for more information
0x7C8	Power Dial CSR	See Section 15.2.1 for more information

Table 285: SiFive Custom CSRs

Appendix D

Floating-Point Unit Instruction Timing

This section provides a reference for the instruction timings of the single-, double-, and half-precision floating-point unit in the Tenstorrent X280 MC.

D.1 X280 Floating-Point Instruction Timing

Single-precision floating-point unit instruction latency and repeat rates are described in Table 286.

Assembly	Operation	Latency	Repeat Rate
Sign Inject			
fabs.s rd, rs1	$f[rd] = f[rs1] $	2	1
fsgnj.s rd, rs1, rs2	$f[rd] = \{f[rs2][31], f[rs1][30:0]\}$	2	1
fsgnjn.s rd, rs1, rs2	$f[rd] = \{-f[rs2][31], f[rs1][30:0]\}$	2	1
fsgnjx.s rd, rs1, rs2	$f[rd] = \{f[rs1][31] \wedge f[rs2][31], f[rs1][30:0]\}$	2	1
Arithmetic			
fadd.s rd, rs1, rs2	$f[rd] = f[rs1] + f[rs2]$	5	1
fsub.s rd, rs1, rs2	$f[rd] = f[rs1] - f[rs2]$	5	1
fdiv.s rd, rs1, rs2	$f[rd] = f[rs1] \div f[rs2]$	9–36	8–33
fmul.s rd, rs1, rs2	$f[rd] = f[rs1] \times f[rs2]$	5	1
fsqrt.s rd, rs1	$f[rd] = \sqrt{f[rs1]}$	9–28	8–33
fmadd.s rd, rs1, rs2, rs3	$f[rd] = (f[rs1] \times f[rs2]) + f[rs3]$	5	1
fmsub.s rd, rs1, rs2, rs3	$f[rd] = (f[rs1] \times f[rs2]) - f[rs3]$	5	1
Negate Arithmetic			
fneg.s rd, rs1	$f[rd] = -f[rs1]$	2	1
fnmadd.s rd, rs1, rs2, rs3	$f[rd] = -(f[rs1] \times f[rs2]) - f[rs3]$	5	1
fnmsub.s rd, rs1, rs2, rs3	$f[rd] = -(f[rs1] \times f[rs2]) + f[rs3]$	5	1
Compare			
feq.s rd, rs1, rs2	$x[rd] = f[rs1] == f[rs2]$	4	1
fle.s rd, rs1, rs2	$x[rd] = f[rs1] \leq f[rs2]$	4	1
flt.s rd, rs1, rs2	$x[rd] = f[rs1] < f[rs2]$	4	1
fmax.s rd, rs1, rs2	$f[rd] = \max(f[rs1], f[rs2])$	2	1
fmin.s rd, rs1, rs2	$f[rd] = \min(f[rs1], f[rs2])$	2	1
Categorize			
fclass.s rd, rs1	$x[rd] = \text{classify}_s(f[rs1])$	4	1
Convert Data Type			
fcvt.w.s rd, rs1	$x[rd] = \text{sext}(s32_{f32}(f[rs1]))$	4	1
fcvt.l.s rd, rs1	$x[rd] = s64_{f32}(f[rs1])$	4	1
fcvt.s.w rd, rs1	$f[rd] = f32_{s32}(x[rs1])$	2	1
fcvt.s.l rd, rs1	$f[rd] = f32_{s64}(x[rs1])$	4	1
fcvt.wu.s rd, rs1	$x[rd] = \text{sext}(u32_{f32}(f[rs1]))$	4	1
fcvt.lu.s rd, rs1	$x[rd] = u64_{f32}(f[rs1])$	4	1
fcvt.s.wu rd, rs1	$f[rd] = f32_{u32}(x[rs1])$	2	1
fcvt.s.lu rd, rs1	$f[rd] = f32_{u64}(x[rs1])$	4	1
Move			
fmv.s rd, rs1	$f[rd] = f[rs1]$	2	1
fmv.w.x rd, rs1	$f[rd] = x[rs1][31:0]$	2	1
fmv.x.w rd, rs1	$x[rd] = \text{sext}(f[rs1][31:0])$	1	1
Load/Store			
flw rd, offset(rs1)	$f[rd] = M[x[rs1] + \text{sext}(\text{offset})][31:0]$	2	1
fsw rs2, offset(rs1)	$M[x[rs1] + \text{sext}(\text{offset})] = f[rs2][31:0]$	4	1

Table 286: X280 Single-Precision FPU Instruction Latency and Repeat Rates

*Instruction and data are in the instruction cache and data cache, respectively.

Double-precision floating-point unit latency and repeat rates are described in Table 287.

Assembly	Operation	Latency	Repeat Rate
Sign Inject			
fabs.d rd, rs1	$f[rd] = f[rs1] $	2	1
fsgnj.d rd, rs1, rs2	$f[rd] = \{f[rs2][63], f[rs1][62:0]\}$	2	1
fsgnjn.d rd, rs1, rs2	$f[rd] = \{-f[rs2][63], f[rs1][62:0]\}$	2	1
fsgnjx.d rd, rs1, rs2	$f[rd] = \{f[rs1][63] \wedge f[rs2][63], f[rs1][62:0]\}$	2	1
Arithmetic			
fadd.d rd, rs1, rs2	$f[rd] = f[rs1] + f[rs2]$	7	1
fsub.d rd, rs1, rs2	$f[rd] = f[rs1] - f[rs2]$	7	1
fdiv.d rd, rs1, rs2	$f[rd] = f[rs1] \div f[rs2]$	9–58	8–58
fmul.d rd, rs1, rs2	$f[rd] = f[rs1] \times f[rs2]$	7	1
fsqrt.d rd, rs1	$f[rd] = \sqrt{f[rs1]}$	9–57	8–58
fmadd.d rd, rs1, rs2, rs3	$f[rd] = (f[rs1] \times f[rs2]) + f[rs3]$	7	1
fmsub.d rd, rs1, rs2, rs3	$f[rd] = (f[rs1] \times f[rs2]) - f[rs3]$	7	1
Negate Arithmetic			
fneg.d rd, rs1	$f[rd] = -f[rs1]$	2	1
fnmadd.d rd, rs1, rs2, rs3	$f[rd] = -(f[rs1] \times f[rs2]) - f[rs3]$	7	1
fnmsub.d rd, rs1, rs2, rs3	$f[rd] = -(f[rs1] \times f[rs2]) + f[rs3]$	7	1
Compare			
feq.d rd, rs1, rs2	$x[rd] = f[rs1] == f[rs2]$	4	1
fle.d rd, rs1, rs2	$x[rd] = f[rs1] \leq f[rs2]$	4	1
flt.d rd, rs1, rs2	$x[rd] = f[rs1] < f[rs2]$	4	1
fmax.d rd, rs1, rs2	$f[rd] = \max(f[rs1], f[rs2])$	2	1
fmin.d rd, rs1, rs2	$f[rd] = \min(f[rs1], f[rs2])$	2	1
Categorize			
fclass.d rd, rs1	$x[rd] = \text{classify}_d(f[rs1])$	4	1
Convert Data Type			
fcvt.w.d rd, rs1	$x[rd] = \text{sext}(s32_{f64}(f[rs1]))$	4	1
fcvt.l.d rd, rs1	$x[rd] = s64_{f64}(f[rs1])$	4	1
fcvt.d.w rd, rs1	$f[rd] = f64_{s32}(x[rs1])$	2	1
fcvt.d.l rd, rs1	$f[rd] = f64_{s64}(x[rs1])$	6	1
fcvt.wu.d rd, rs1	$x[rd] = \text{sext}(u32_{f64}(f[rs1]))$	4	1
fcvt.lu.d rd, rs1	$x[rd] = u64_{f64}(f[rs1])$	4	1
fcvt.d.wu rd, rs1	$f[rd] = f64_{u32}(x[rs1])$	2	1
fcvt.d.lu rd, rs1	$f[rd] = f64_{u64}(x[rs1])$	6	1
fcvt.s.d rd, rs1	$f[rd] = f32_{f64}(f[rs1])$	2	1
fcvt.d.s rd, rs1	$f[rd] = f64_{f32}(f[rs1])$	2	1
Move			
fmv.d rd, rs1	$f[rd] = f[rs1]$	2	1
fmv.d.x rd, rs1	$f[rd] = x[rs1][63:0]$	6	1
fmv.x.d rd, rs1	$x[rd] = f[rs1][63:0]$	1	1
Load/Store			
fld rd, offset(rs1)	$f[rd] = M[x[rs1] + \text{sext}(\text{offset})][63:0]$	2	1
fsd rs2, offset(rs1)	$M[x[rs1] + \text{sext}(\text{offset})] = f[rs2][63:0]$	4	1

Table 287: X280 Double-Precision FPU Instruction Latency and Repeat Rates

*Instruction and data are in the instruction cache and data cache, respectively.

Half-precision floating-point unit latency and repeat rates are described in Table 288.

Assembly	Operation	Latency	Repeat Rate
Sign Inject			
fabs.h rd, rs1	$f[rd] = f[rs1] $	2	1
fsgnj.h rd, rs1, rs2	$f[rd] = \{f[rs2][15], f[rs1][14:0]\}$	2	1
fsgnjn.h rd, rs1, rs2	$f[rd] = \{-f[rs2][15], f[rs1][14:0]\}$	2	1
fsgnjx.h rd, rs1, rs2	$f[rd] = \{f[rs1][15] \wedge f[rs2][15], f[rs1][14:0]\}$	2	1
Arithmetic			
fadd.h rd, rs1, rs2	$f[rd] = f[rs1] + f[rs2]$	5	1
fsub.h rd, rs1, rs2	$f[rd] = f[rs1] - f[rs2]$	5	1
fdiv.h rd, rs1, rs2	$f[rd] = f[rs1] \div f[rs2]$	9–16	9–16
fmul.h rd, rs1, rs2	$f[rd] = f[rs1] \times f[rs2]$	5	1
fsqrt.h rd, rs1	$f[rd] = \sqrt{f[rs1]}$	9–15	8–15
fmadd.h rd, rs1, rs2, rs3	$f[rd] = (f[rs1] \times f[rs2]) + f[rs3]$	5	1
fmsub.h rd, rs1, rs2, rs3	$f[rd] = (f[rs1] \times f[rs2]) - f[rs3]$	5	1
Negate Arithmetic			
fneg.h rd, rs1	$f[rd] = -f[rs1]$	2	1
fnmadd.h rd, rs1, rs2, rs3	$f[rd] = -(f[rs1] \times f[rs2]) - f[rs3]$	5	1
fnmsub.h rd, rs1, rs2, rs3	$f[rd] = -(f[rs1] \times f[rs2]) + f[rs3]$	5	1
Compare			
feq.h rd, rs1, rs2	$x[rd] = f[rs1] == f[rs2]$	4	1
fle.h rd, rs1, rs2	$x[rd] = f[rs1] \leq f[rs2]$	4	1
flt.h rd, rs1, rs2	$x[rd] = f[rs1] < f[rs2]$	4	1
fmax.h rd, rs1, rs2	$f[rd] = \max(f[rs1], f[rs2])$	2	1
fmin.h rd, rs1, rs2	$f[rd] = \min(f[rs1], f[rs2])$	2	1
Categorize			
fclass.h rd, rs1	$x[rd] = \text{classify}_h(f[rs1])$	4	1
Convert Data Type			
fcvt.w.h rd, rs1	$x[rd] = \text{sext}(s32_{f16}(f[rs1]))$	4	1
fcvt.l.h rd, rs1	$x[rd] = s64_{f16}(f[rs1])$		
fcvt.h.w rd, rs1	$f[rd] = f16_{s32}(x[rs1])$	2	1
fcvt.h.l rd, rs1	$f[rd] = f16_{s64}(x[rs1])$		
fcvt.wu.h rd, rs1	$x[rd] = \text{sext}(u32_{f16}(f[rs1]))$	4	1
fcvt.lu.h rd, rs1	$x[rd] = u64_{f16}(f[rs1])$		
fcvt.h.wu rd, rs1	$f[rd] = f16_{u32}(x[rs1])$	2	1
fcvt.h.lu rd, rs1	$f[rd] = f64_{u16}(x[rs1])$		
fcvt.s.h rd, rs1	$f[rd] = f32_{f16}(f[rs1])$	2	1
fcvt.h.s rd, rs1	$f[rd] = f16_{f32}(f[rs1])$	2	1
fcvt.d.h rd, rs1	$f[rd] = f64_{f16}(f[rs1])$	2	1
fcvt.h.d rd, rs1	$f[rd] = f16_{f64}(f[rs1])$	2	1
Move			
fmv.h rd, rs1	$f[rd] = f[rs1]$	2	1
fmv.h.x rd, rs1	$f[rd] = x[rs1][15:0]$	2	1
fmv.x.h rd, rs1	$x[rd] = f[rs1][63:0]$	1	1
Load/Store			
flh rd, offset(rs1)	$f[rd] = M[x[rs1] + \text{sext}(\text{offset})][15:0]$	2	1
fsh rs2, offset(rs1)	$M[x[rs1] + \text{sext}(\text{offset})] = f[rs2][15:0]$	1	1

Table 288: X280 Half-Precision FPU Instruction Latency and Repeat Rates

Appendix E

Revision History

This section describes the changes in this document between release versions.

Version	Date	Document Changes
21G3.04.00	April 25, 2022	• <i>Table 166</i>: Added column describing BEU register attributes • <i>Section 17.10.1</i>: Added description for <code>teSink*</code> registers in the Trace Funnel memory region • <i>Table 116, Table 284</i>: Added <code>mimpid</code> value for 21G3.04.00 • Editorial fixes
21G3.03.00	March 16, 2022	• <i>Section 3.10.6</i>: Updated description of HPM counter enable registers for other privilege modes • <i>Figure 17</i>: Added Counter-Enable Register bitfield • <i>Section 3.10.4</i>: Updated description of Event Selector registers • <i>Table 15, Table 16</i>: Updated table to reflect L3 Zero Device is data cacheable • <i>Table 112</i>: Added description of Supervisor Interrupts and Exception CSRs • <i>Section 7.8</i>: Updated "Supervisor Mode Interrupts" for <code>Sscomfpmf</code> extension • <i>Table 133</i>: Updated Machine Interrupt Enable Register table for <code>Sscomfpmf</code> interrupts • <i>Table 134</i>: Updated Machine Interrupt Pending Register table for <code>Sscomfpmf</code> interrupts • <i>Table 136</i>: Added HPM Counter Overflow to <code>mcause</code> Exception Codes table • <i>Section 11.4.1</i>: Added description to clear BEU accrued bit • <i>Section 12.6</i>: Added recommended L2 prefetcher initialization values • <i>Table 184</i>: Updated Cache Flush Command bits • <i>Table 194</i>: Fixed width of <code>wayEnable</code> register • <i>Section 14.4.11</i>: Added description of L3 Cache Feature Disable register

Table 289: Tenstorrent X280 MC Manual Revision History

Version	Date	Document Changes
		• <i>Section 16.8</i>: Added "Debug and Core Complex Power Gating" section • <i>Section 17.3</i>: Removed "BTM Generator" section as design is not configured with BTM • <i>Section 17.10.2</i>: Added "Trace Funnel Timestamp Unit" section • <i>Table 16</i>: ◦ Changed "Reserved" to "Unmapped" ◦ Added Test Indicator at 0x4000 • <i>Table 116, Table 284</i>: Added mimpid value for 21G3.03.00 • <i>Section 5.12.5</i>: Added description of RISC-V status CSRs • <i>Section 5.21.2</i>: Removed <i>Linker Alignment</i> section and requirement • <i>Table 130</i>: Added <i>Encoding of RISC-V Privilege Levels</i> table • Editorial fixes
21G3.02.00	December 22, 2021	• Initial release

Table 289: Tenstorrent X280 MC Manual Revision History

Appendix F

Knowledge Base Articles

The SiFive support team provides access to an index of Knowledge Base articles in order to further assist users developing with and integrating their RISC-V IP. These articles focus on topics ranging from architectural design and software development to board bring-up and implementation.

To view more than 100 articles in the SiFive Knowledge Base, access the link below:

<https://sifive.atlassian.net/servicedesk/customer/portal/47/article/465732086>

References

Visit the SiFive forums for support and answers to frequently asked questions:
<https://forums.sifive.com>

[1] A. Waterman and K. Asanovic, Eds., The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.2, June 2019. [Online]. Available: <https://riscv.org/specifications/>

[2] —, The RISC-V Instruction Set Manual Volume II: Privileged Architecture, Version 1.11, June 2019. [Online]. Available: <https://riscv.org/specifications/privileged-isa/>

[3] SiFive TileLink Specification Version 1.8.0, August 2019. [Online]. Available: <https://sifive.com/documentation/tilelink/tilelink-spec>

[4] A. Chang, D. Barbier, and P. Dabbelt, RISC-V Platform-Level Interrupt Controller (PLIC) Specification. [Online]. Available: <https://github.com/riscv/riscv-plic-spec>

[5] E. Edgar and T. Newsome, Eds., RISC-V Debug Support, Version 1.0, May 2021. [Online]. Available: <https://github.com/riscv/riscv-debug-spec>

[6] Zfh Standard Extension for Half-Precision Floating-Point, Version 0.1, December 2020. [Online]. Available: <https://github.com/riscv/riscv-isa-manual/>

[7] RISC-V Bit-Manipulation ISA-extensions, Version 1.0, June 2021. [Online]. Available: <https://github.com/riscv/riscv-bitmanip/releases/tag/1.0.0>

[8] RISC-V "V" Vector Extension, Version 1.0-rc1, June 2021. [Online]. Available: <https://github.com/riscv/riscv-v-spec>